

ReSolveD- α : A Digital Signature Scheme from Regular Syndrome Decoding and VOLE-in-the-Head

Algorithm Specifications and Supporting Documentation

(Version 1.1)

Yu Yu^{1,6}, Kang Yang², Xiaojie Guo³, Hongrui Cui¹, Chun Guo^{4,7}, Di Yan², Yituo He¹,
Lizheng Wang¹ and Jiangxia Ge⁵

¹ Shanghai Jiao Tong University, Shanghai, China

² State Key Laboratory of Cryptology, Beijing, China

³ Shanghai Qi Zhi Institute, Shanghai, China

⁴ Shandong University, Qingdao, China

⁵ China Telecom Quantum Group Ltd., Hefei, China

⁶ East China Normal University, Shanghai, China

⁷ State Key Laboratory of Cryptography and Digital Economy Security, Qingdao, China

June 30, 2026

Table of Contents

1	Introduction	5
1.1	Features and Advantages	5
1.2	Organization	8
2	Design Rationale	8
2.1	Zero-Knowledge Proofs for RSD	8
2.1.1	Overview of Regular Syndrome Decoding	8
2.1.2	Linear Sketching	9
2.1.3	Proving Knowledge of Solutions of RSD Problems	9
2.2	VOLEitH-based Non-Interactive Zero-Knowledge Proofs	10
2.2.1	Vector Oblivious Linear Evaluation	10
2.2.2	QuickSilver	10
2.2.3	VOLE-in-the-Head	11
2.3	TCCR Hash Function	12
3	Preliminaries	13
3.1	Notation	13
3.1.1	Sampling and Assignment	13
3.1.2	Integer Ranges	13
3.1.3	Vectors and Matrices	13
3.1.4	Tuples and Bit Strings	14
3.2	Hash Function	14
3.3	Pseudorandom Generator	15
3.4	Zero-Knowledge Proof	16
3.5	Information-Theoretic Message Authentication Code	17
3.6	Digital Signature Scheme	18
3.7	Regular Syndrome Decoding	19
3.8	Linear Sketching	22
3.9	Signature Size of VOLEitH	25
4	The ReSolveD-α Signature Scheme	25
4.1	Parameter Structure	26
4.2	Top-Level APIs	26
4.2.1	Key Generation	27
4.2.2	Signing	27
4.2.3	Verification	28
4.3	Vector Oblivious Linear Evaluation	29
4.3.1	VOLE Challenge Decoding	30
4.3.2	Decoding All VOLE Challenges	30
4.3.3	VOLE Commitment	31
4.3.4	VOLE Reconstruction	32
4.3.5	VOLE Correction	32
4.3.6	Seed Conversion to VOLE	33
4.4	Batch All-but-one Vector Commitment	34
4.4.1	BAVC Commitment	35
4.4.2	BAVC Opening	35

4.4.3	BAVC Verification	36
4.4.4	Leaf Interleaving	37
4.4.5	Leaf Deinterleaving	38
4.4.6	Minimum-Span Opening	38
4.4.7	Leaf Hashing	39
4.5	RSD-based One-Way Function	39
4.5.1	RSD Matrix Sampling	39
4.5.2	RSD Noise Sampling	40
4.5.3	RSD Noise Encoding	40
4.5.4	RSD Noise Decoding	41
4.6	QuickSilver Proof System	41
4.6.1	QuickSilver Proving	42
4.6.2	QuickSilver Verification	43
4.7	Universal Hash Functions	44
4.7.1	VOLE Linear Universal Hash	44
4.7.2	ZK Linear Universal Hash	45
4.8	Utility Functions	46
4.8.1	Bits to Integer	46
4.8.2	Integer to Bits	46
4.8.3	Bits to Field	47
4.8.4	Field to Bits	47
4.9	Symmetric Primitives	47
4.9.1	Message Hash	48
4.9.2	Signing-Coin Hash	48
4.9.3	BAVC Commitment Hash	48
4.9.4	Matrix Expansion Hash	49
4.9.5	First Fiat-Shamir Hash	49
4.9.6	Second Fiat-Shamir Hash	49
4.9.7	Third Fiat-Shamir Hash	50
4.9.8	TCCR Hash Function	50
4.9.9	Pseudorandom Number Generator	50
4.9.10	Extendable-Output Function	51
4.9.11	Block-Cipher Interface	51
5	Parameter Selection	52
5.1	Classical and Quantum Security Levels	52
5.2	Selection of RSD Parameters	52
5.3	Selection Approach of VOLEitH Parameters	53
5.4	Failure Probability Analysis	54
5.5	VOLE-in-the-Head Parameters	55
6	Performance Evaluation	55
6.1	Instantiations of Standard Symmetric-key Primitives	56
6.2	Implementation Platform	57
6.3	Software Performance	58
6.4	Comparison	58
6.4.1	Comparison with SDitH	59
6.4.2	Comparison with A Variant of ReSolveD- α	60
6.5	Discussion	60

7	Concrete Security Analysis	62
7.1	Known Attacks	62
7.1.1	Brute-Force the Public Key	62
7.1.2	Brute-Force the TCCR Tree Generation	62
7.1.3	Brute-Force the PRG	63
7.1.4	Attack Soundness of the ZK Protocol	64
7.1.5	Multi-User Attacks	64
7.2	Concrete Security of the RSD Problem	64
7.3	Concrete Security of Symmetric Primitives	67
8	Security Proof	67
8.1	Correctness Analysis	67
8.2	Security Proof in the QROM and QICM	71
8.3	Security Proof in the ROM and ICM	96
8.4	Proofs of Subroutine Lemmas and Theorems	99
8.4.1	Local Emulation of Ideal Cipher	99
8.4.2	Proof of Lemma 3.1	100
8.4.3	(Quantum) Security of the CTR-based PRG	103
8.4.4	TCCR Security	110
A	Proving Knowledge of Secret Keys for NEV-KEM	129
A.1	Definition of NEV	129
A.2	Expressing Public/Seret Key Relation as Polynomials	130
A.3	Estimating the Proof Size	131

1 Introduction

This document describes and specifies the post-quantum digital signature scheme ReSolveD- α , which is based on the Regular Syndrome Decoding (RSD) assumption and the VOLE-in-the-Head (VOLEitH) framework [BBD⁺23b].¹ It provides the design rationale, algorithm specification, parameter-selection details, provable security analysis, concrete security evaluation, and performance evaluation of ReSolveD- α . ReSolveD- α is designed to provide security against quantum adversaries. Other than standard symmetric-key primitives (i.e., cryptographic hash functions and block ciphers), ReSolveD- α relies only on the conservative code-based RSD assumption over $\mathbb{F}_2$. Prior works [FJR22, LWYY24] have established that RSD is asymptotically equivalent to the standard Syndrome Decoding (SD) problem. In this document, we rigorously prove the existential unforgeability under chosen-message attacks (EUF-CMA) of ReSolveD- α in the Quantum Random Oracle Model (QROM) and Quantum Ideal Cipher Model (QICM).

1.1 Features and Advantages

Techniques. ReSolveD- α is constructed in the VOLEitH framework [BBD⁺23b]. It uses the VOLE-based zero-knowledge proof system QuickSilver [YSWW21] for concrete efficiency. The scheme builds upon ReSolveD [CLY⁺24], a code-based post-quantum signature scheme originally proposed by the design team of ReSolveD- α . As in ReSolveD, the ReSolveD- α scheme adopts the Half-Tree construction [GYW⁺23] to generate VOLE correlations. Compared with the original ReSolveD scheme, ReSolveD- α makes the following changes.

- We propose a new construction of Tweakable Circular Correlation Robust (TCCR) hash functions for instantiating the underlying hash function in Half-Tree at classical security levels from 160 bits to 512 bits. The TCCR construction achieves beyond λ -bit classical security from any n_b -bit block cipher, and is provably secure in both ICM and QICM. Previously, the state-of-the-art TCCR construction with high security was proposed by the design team in [CGW⁺24]. Compared with that construction, ours has two improvements. First, the two block-cipher calls in [CGW⁺24] use distinct keys for separation, whereas our construction uses the same key; reusing the key saves one key-schedule function call and improves computational efficiency. Second, the construction in [CGW⁺24] makes two (or multiple) calls to a linear orthomorphism σ , instantiated as $\sigma(x_L \| x_R) = x_R \oplus x_L \| x_L$, where x_L, x_R are the left and right halves of the input x . Our construction avoids these calls, saving one or two CPU cycles per hash call.
- We integrate Half-Tree with the recent large-GGM-tree construction [BBM⁺24] for batch all-but-one vector commitment (BAVC) generation. We also propose an efficient method for estimating the distribution of the BAVC opening size, from which we further estimate the distribution of the signing time.
- We replace the *random* linear sketching in ReSolveD [CLY⁺24] with the *deterministic* linear sketching used in SDitH [ABB⁺26], a code-based signature candidate in the third round of NIST’s Post-Quantum Cryptography (PQC) competition for additional post-quantum signatures [ABC⁺26]. Compared with the original ReSolveD scheme, this reduces the number of quadratic constraints and yields faster signing and verification.

Provable security. We rigorously prove the EUF-CMA security of ReSolveD- α under the RSD assumption in the QROM and QICM. To the best of our knowledge, this is the first security proof against quantum adversaries for a code-based post-quantum signature scheme in the VOLEitH framework. Our proof follows the strategy used to prove the security of the second

¹VOLE is an abbreviation of vector oblivious linear evaluation [ADI⁺17]. VOLEitH extends the design idea of the SoftSpokenOT protocol [Roy22] to generate VOLE correlations in a non-interactive way.

version of FAEST in the QROM [BBB⁺25, BBB⁺26], a symmetric-key-based candidate in the third round of the same NIST PQC competition for additional post-quantum signatures. For completeness, we also fill in proof details that are implicit or omitted in the FAEST specification. In addition, ReSolveD- α is also provably secure in the classical ROM and ICM.

Conservative assumption. The ReSolveD- α scheme relies on the hardness of the RSD problem over $\mathbb{F}_2$, which is a variant of the standard SD problem. The worst-case SD problem in certain parameter regimes is known to be NP-hard [BMvT78, Bar94], and its average-case analogue is one of the most promising assumptions for post-quantum cryptography. Asymptotically, recent work [FJR22, LWYY24] gives reductions from SD to RSD, providing theoretical support for the hardness of RSD. For the concrete parameters used by our signature scheme, however, the exact relationship between RSD and SD remains unclear [CCJ23, ES24]. Nevertheless, the best known attacks do not exploit the regular structure of the noise to gain any speed-up at these parameters [CCJ23, BØ23, ES24]. We therefore regard RSD as a conservative and promising hardness assumption for designing code-based post-quantum signatures.

Conservative security estimate. We select four parameter sets for ReSolveD- α , targeting 160-bit, 256-bit, 384-bit, and 512-bit classical security, which correspond to 80-bit, 128-bit, 192-bit, and 256-bit quantum security. The RSD parameters are selected using the most conservative estimator of [CCJ23], which we call the CCJ estimator. Under this estimator, the selected parameters achieve a security margin of 17–22 bits over the target security levels. The CCJ estimator assumes that an approximate collision search between two lists can be found in linear time. However, no such linear-time algorithm is currently known, and moreover the existence of a linear-time algorithm appears far from obvious [CCJ23]. Therefore, the parameters selected with the CCJ estimator are very conservative. In addition, some code-based post-quantum signature schemes (e.g., SDitH [ABB⁺26]) involve the cost of memory accesses when analyzing the classical/quantum security against known attacks, as recent attacks make extensive use of memory. We ignore the cost of memory accesses for the recent attacks to obtain a more conservative security estimate. Due to the conservative security of ReSolveD- α , even if new attacks are developed in the future, they have no impact on the security level of ReSolveD- α .

Table 1: **Summary of communication cost, computational cost, and concrete security of RSD for ReSolveD- α under different parameter sets.** In this table, ABP denotes the Approximate Birthday Paradox attack [CCJ23], which assumes the existence of a linear-time nearest neighbor search algorithm. This assumption does not hold for now, which makes the RSD security estimate in the column “w/ ABP” very conservative. The $|\text{pk}|$ and $|\sigma|$ denote the public key size and signature size, respectively.

Scheme	Size (Bytes)		Running time (ms)			RSD security (bits)	
	$ \text{pk} $	$ \sigma $	KeyGen	Sign	Verify	w/o ABP	w/ ABP
ReSolveD- α -160s	121	5,307	0.21	2.77	2.32	231 (≥ 115)	177 (≥ 88)
ReSolveD- α -160f	121	6,851	0.21	0.48	0.37	231 (≥ 115)	177 (≥ 88)
ReSolveD- α -256s	194	13,973	0.57	5.13	4.45	366 (≥ 183)	276 (≥ 138)
ReSolveD- α -256f	194	17,932	0.57	0.74	0.59	366 (≥ 183)	276 (≥ 138)
ReSolveD- α -384s	288	31,575	1.70	21.87	20.60	543 (≥ 271)	404 (≥ 202)
ReSolveD- α -384f	288	40,469	1.67	3.44	3.03	543 (≥ 271)	404 (≥ 202)
ReSolveD- α -512s	385	56,239	3.96	56.41	55.46	723 (≥ 361)	534 (≥ 267)
ReSolveD- α -512f	385	72,611	3.96	7.45	6.83	723 (≥ 361)	534 (≥ 267)

Competitive performance. In Table 1, we summarize the sizes of a public key and signature, running time of three algorithms, and the RSD security for the ReSolveD- α scheme under different parameter sets. The running time is taken from the optimized AVX implementation. This performance is competitive among code-based post-quantum signature schemes, even under the conservative security estimate. For the security estimate of RSD, the final two columns of Table 1 show the classical security strength and quantum security strength in parenthesis. When estimating quantum security of RSD, we consider that the Information Set Decoding (ISD) algorithm can be accelerated by a square root gain using a Grover quantum algorithm [Ber10]. We do not assume the depth constraint of quantum circuits, which allows us to obtain a conservative security.

At the 128-bit quantum security level (or the 256-bit classical security level), ReSolveD- α reduces the “public key + signature” size by 86 bytes for the small variant, but increases the total size by 454 bytes for the fast variant, compared to the state-of-the-art code-based signature scheme SDitH [ABB⁺26]. In terms of computational efficiency, compared to SDitH, ReSolveD- α improves the signing (resp., verification) time by a factor of $7.4\times$ (resp., $8.1\times$) for the small variant; it speeds up the signing (resp., verification) time by a factor of $9.4\times$ (resp., $11\times$) for the fast variant. Note that the security estimate of ReSolveD- α is more conservative than SDitH.

Flexibility. The VOLEitH framework exposes several parameters that trade signature size for running time. For each security level, we therefore specify a small variant targeting smaller signatures and a fast variant targeting faster signing and verification.

Extensibility. The Non-Interactive Zero-Knowledge (NIZK) proof framework in ReSolveD- α can be used to prove a broader range of statements beyond the RSD instances. Specifically, the underlying VOLEitH proof system excels at proving systems of low-degree constraints, and is particularly efficient when the underlying statement has a relatively short witness. Such statements arise in applications, for example, when proving knowledge of secret keys for lattice-based Key Encapsulation Mechanisms (KEMs). NIZK proofs for such statements can be applied in post-quantum Public Key Infrastructure (PKI) systems for Transport Layer Security (TLS) and Authenticated Key Exchange (AKE) protocols built from lattice-based KEMs. For instance, we can use the VOLEitH-based NIZK proof system in ReSolveD- α to prove knowledge of the secret keys on public keys for lattice-based KEMs, e.g., Kyber [SAB⁺20], NEV [ZFY23] and Aegis-enc [ZYF⁺20]. As an example, we show how to prove knowledge of the secret key on a public key for a lattice-based NEV KEM in Appendix A.

Simplicity. Other than RSD, the ReSolveD- α scheme is built entirely from symmetric-key primitives: block ciphers for Pseudo-Random Generator (PRG) and TCCR hashing, and cryptographic hash functions for Fiat-Shamir transformation and commitments. No number-theoretic operations (e.g., modular exponentiation, number-theoretic transforms, or Gaussian sampling) or floating-point arithmetic are required. This makes the ReSolveD- α design conceptually simple, easy to audit, and straightforward to implement in constant time. It also simplifies protection against side-channel attacks such as timing attacks and power-analysis attacks.

Compatibility. The symmetric-key primitives used in ReSolveD- α are block ciphers and cryptographic hash functions. We instantiate them with standardized algorithms (i.e., AES, SHA2, and SHA3), which have been widely studied. ReSolveD- α invokes these primitives only in a black-box way, so it is straightforward to replace them with future symmetric-key primitives.

1.2 Organization

The rest of this document is organized as follows. [Section 2](#) explains the design rationale of ReSolveD- α , including the overall construction approach, the use of linear sketching, the parameter-selection methodology, the failure-probability analysis, and other design considerations. [Section 3](#) introduces the notation, recalls the formal definition of digital signature schemes, and defines the Regular Syndrome Decoding problem used by the scheme. [Section 4](#) presents the full algorithm description, including the parameter structure, the top-level algorithms, the underlying VOLE, BAVC, RSD, QuickSilver, universal-hashing, utility, and symmetric-key components, as well as the selected parameter sets. [Section 6](#) reports the implementation setting and benchmark results for the current software implementations. [Section 7](#) discusses known attacks and evaluates the concrete security of the RSD assumption and the symmetric primitives used in the scheme. [Section 8](#) gives the correctness analysis and shows the formal security proof in the QROM and QICM. [Section A](#) shows an example of how to prove knowledge of secret keys in a lattice-based NEV-KEM scheme.

2 Design Rationale

ReSolveD- α is a digital signature scheme based on the Regular Syndrome Decoding (RSD) problem. A key pair in ReSolveD- α describes an instance-witness pair for the RSD problem, and a signature of ReSolveD- α is a Non-Interactive Zero-Knowledge (NIZK) proof of the knowledge of RSD witness (i.e., a noise vector). The NIZK proof system is constructed in the VOLE-in-the-Head (VOLEitH) framework.

2.1 Zero-Knowledge Proofs for RSD

Zero-knowledge proofs [[GMR85](#), [GMW91](#)] allow a prover to convince a verifier that a statement is valid, without revealing anything beyond the validity of the statement. A standard paradigm of designing signature schemes is to use an NIZK scheme to prove knowledge of the preimage of a One-Way Function (OWF), while binding the message to be signed with the proving process. Therefore, to design the ReSolveD- α signature scheme, we use an NIZK proof, which is only based on symmetric-key primitives, to prove knowledge of the solution of an RSD instance (used as an OWF).

2.1.1 Overview of Regular Syndrome Decoding

We recall the RSD problem [[AFS03](#), [AFS05](#)], which serves as a one-way function proven in zero-knowledge with the VOLEitH framework. RSD is a regular noise variant of the Syndrome Decoding (SD) problem, which asks to recover a sparse noise vector $\mathbf{e} \in \mathbb{F}_2^m$ from $(\mathbf{H}, \mathbf{y})$, where $\mathbf{H} \in \mathbb{F}_2^{(m-k) \times m}$ is a random matrix, the Hamming weight of $\mathbf{e}$ is w , and $\mathbf{y} = \mathbf{H} \cdot \mathbf{e} \in \mathbb{F}_2^{(m-k)}$. The recent works [[FJR22](#), [LWYY24](#)] have proven that RSD is asymptotically equivalent to the SD problem. RSD adopts the *regular noise distribution*: a noise vector $\mathbf{e}$ consists of w equal-length blocks $\mathbf{e}_0, \dots, \mathbf{e}_{w-1}$, where each block has Hamming weight exactly 1, with the nonzero noisy position uniformly distributed within the block. Current research indicates that, apart from Grover’s search algorithm [[Gro96](#)], there are no known efficient quantum algorithms for the RSD problem. During the process of selecting RSD parameters, we have accounted for the square root speed up of Grover’s algorithm, i.e., we do *not* limit the maximum depth of quantum circuits. This makes the parameters of ReSolveD- α conservative.

2.1.2 Linear Sketching

Due to the hardness of the RSD problem, the map $\mathbf{e} \mapsto \mathbf{H} \cdot \mathbf{e}$ is a one-way function. Given an image $\mathbf{y}$, proving knowledge of the preimage $\mathbf{e}$ needs to prove that: 1) $\mathbf{y} = \mathbf{H} \cdot \mathbf{e}$ and 2) $\mathbf{e}$ is a regular noise. Looking ahead, since VOLEitH supports free additive homomorphic operations on committed values, the first constraint can be easily proven without any communication. Then proving the regularity of $\mathbf{e}$ is reduced to proving that each block of $\mathbf{e}$ has a Hamming weight of 1. In the following, we show how to use the linear sketching to translate this constraint into a quadratic constraint, which can be efficiently proven in the VOLEitH framework.

Linear sketching was first introduced by Boyle et al. [BGI16] in the context of distributed point functions, where a *randomized* linear sketching method is proposed. The original ReSolveD signature scheme adopted the method to prove the regularity of a noise vector that has been committed. In the SDitH scheme [ABB⁺26], a *deterministic* linear sketching scheme is proposed, which has a better computational efficiency. We adopt the deterministic linear sketching scheme in ReSolveD- α and recall the basic idea as follows.

Specifically, we use the deterministic-linear-sketching technique to prove that each block $\tilde{\mathbf{e}} \in \{\mathbf{e}_0, \dots, \mathbf{e}_{w-1}\}$ of a noise vector satisfies that its Hamming weight is at most 1, i.e., $\text{HW}(\tilde{\mathbf{e}}) \leq 1$. Let $\mathbb{F}_{2^\lambda} = \mathbb{F}_2[X]/f(X)$ be an extension field of $\mathbb{F}_2$ for some monic, irreducible polynomial $f(X)$ of degree λ , where each element over $\mathbb{F}_{2^\lambda}$ is a polynomial over $\mathbb{F}_2$. We w.l.o.g. assume $n(n+1) < \lambda$ where $n = m/w$ is the length of a block vector, and define

$$x = \sum_{i=0}^{n-1} \tilde{\mathbf{e}}[i] \cdot X^i, \quad y = \sum_{i=0}^{n-1} \tilde{\mathbf{e}}[i] \cdot X^{n \cdot i}, \quad z = \sum_{i=0}^{n-1} \tilde{\mathbf{e}}[i] \cdot X^{(n+1) \cdot i}. \quad (1)$$

If $\text{HW}(\tilde{\mathbf{e}}) \leq 1$, then at most one coordinate of $\tilde{\mathbf{e}}$ contributes to the summations of x, y, z , and it holds that $x \cdot y = z$. If $\text{HW}(\tilde{\mathbf{e}}) > 1$, then both x and y are summations of multiple monomials. Moreover, since we placed enough margin in the powers of X , the cross terms in $x \cdot y$ would not cancel out, making $x \cdot y \neq z$. In Section 3.8, we give the details of the deterministic linear sketching scheme, and prove its completeness and soundness formally.

2.1.3 Proving Knowledge of Solutions of RSD Problems

Recall that for an RSD instance $(\mathbf{H}, \mathbf{y})$, we need to prove that 1) $\mathbf{H} \cdot \mathbf{e} = \mathbf{y}$ and 2) $\mathbf{e}$ consists of sub-vectors of Hamming weight 1. In ReSolveD- α , the first property is implicitly enforced by considering the systematic form of RSD, i.e., $\mathbf{H} = [\mathbf{I} \mid \mathbf{H}_B]$ where $\mathbf{I}$ is an identity sub-matrix and $\mathbf{H}_B$ is a random sub-matrix. The noise vector $\mathbf{e}$ is split into two parts accordingly, i.e., $\mathbf{e} = (\mathbf{e}_A, \mathbf{e}_B)$. Under the systematic form, the linear relation $\mathbf{y} = \mathbf{H} \cdot \mathbf{e}$ becomes $\mathbf{y} = \mathbf{e}_A + \mathbf{H}_B \cdot \mathbf{e}_B$. Then, a prover can just commit to $\mathbf{e}_B$ to let the prover and a verifier get $\llbracket \mathbf{e}_B \rrbracket$, and then both parties can reconstruct $\llbracket \mathbf{e} \rrbracket = (\mathbf{y} - \mathbf{H}_B \cdot \llbracket \mathbf{e}_B \rrbracket, \llbracket \mathbf{e}_B \rrbracket)$ based on the linear homomorphism of commitments, which satisfies the linear constraint.

As every noise block $\mathbf{e}_i$ has Hamming weight 1, each component of $\mathbf{e}_i$ XORs to 1, meaning that we can reconstruct the last coordinate of $\mathbf{e}_i$ from all but the last coordinates with only linear operations. In particular, this implies that for each noise block in $\mathbf{e}_B$, it suffices to commit to all but the last coordinates. This optimization was proposed by Carozza et al. [CCJ23] and further reduces the proof size.

For the regularity constraint, we need to show that for each sub-vector $\mathbf{e}_i \in \mathbb{F}_2^n$ of $\mathbf{e}$, its Hamming weight is 1. We break this task down into the following two sub-constraints: 1) $\mathbf{e}_i[0] \oplus \dots \oplus \mathbf{e}_i[n-1] = 1$, and 2) $\text{HW}(\mathbf{e}_i) \leq 1$. The first sub-constraint is naturally satisfied for noise blocks in $\mathbf{e}_B$. As for noise blocks in $\mathbf{e}_A$, we can use a linear constraint to describe them. For the second sub-constraint, by applying the deterministic linear sketching technique to each noise block $\mathbf{e}_i$, we can get a system of quadratic constraints over committed values, which can be efficiently proven in zero knowledge.

2.2 VOLEitH-based Non-Interactive Zero-Knowledge Proofs

As a ReSolveD- α signature consists of an NIZK transcript, the proof size affects the resulting signature size. Hence, choosing a suitable NIZK scheme is crucial for the signature scheme's efficiency. The ReSolveD- α scheme utilizes the VOLEitH framework [BBD⁺23b] to construct the underlying NIZK. The VOLEitH framework combines the VOLE-based QuickSilver proof system [YSWW21] with a publicly-verifiable scheme of producing VOLE correlations [Roy22, BBD⁺23b]. We present a bottom-up overview of the VOLEitH framework in the following.

2.2.1 Vector Oblivious Linear Evaluation

Given a binary field $\mathbb{F}_2$, its extension field $\mathbb{F}_{2^\lambda}$, and a vector length n , VOLE allows a sender to obtain a vector of messages $\mathbf{u} \in \mathbb{F}_2^n$ and MAC tags $\mathbf{v} \in \mathbb{F}_{2^\lambda}^n$ and a receiver to get the local keys $\mathbf{q} \in \mathbb{F}_{2^\lambda}^n$ and a global key $\Delta \in \mathbb{F}_{2^\lambda}$, subject to

$$\mathbf{q} = \mathbf{u} \cdot \Delta + \mathbf{v}. \quad (2)$$

VOLE can be viewed as a two-party commitment scheme. For any $i \in [\ell]$, the message $\mathbf{u}[i]$ is hidden from the receiver, as $\mathbf{v}[i]$ offers a one-time padding to mask $\mathbf{u}[i]$, which satisfies the hiding property. Moreover, the receiver can ask the sender to open $\mathbf{u}[i]$ by providing $\mathbf{v}[i]$ alongside $\mathbf{u}[i]$, while it checks $\mathbf{q}[i] = \mathbf{u}[i] \cdot \Delta + \mathbf{v}[i]$. Any attempt to provide a different pair $(\tilde{\mathbf{u}}[i], \tilde{\mathbf{v}}[i])$ while still passing the check reduces to guessing Δ , since $\Delta = (\tilde{\mathbf{u}}[i] - \mathbf{u}[i])^{-1} \cdot (\mathbf{v}[i] - \tilde{\mathbf{v}}[i])$. This happens except with negligible probability for Δ with sufficient min-entropy, so the scheme is also binding.

An alternative view of VOLE commitment is placing the committed message $\mathbf{u}[i]$ in the leading coefficient of a degree-1 polynomial $f(X) = \mathbf{u}[i] \cdot X + \mathbf{v}[i]$. The receiver's global key Δ is a random point, and its local key $\mathbf{q}[i]$ is the polynomial's evaluation on this point $f(\Delta)$. Naturally, we can generalize this pattern to higher-degree polynomials. Moreover, by relating commitments to polynomials, homomorphic operations on committed messages are also straightforward, as we are essentially operating on polynomials. This implies that VOLE commitments are also homomorphic.

VOLE can also be viewed as a vector of Information-Theoretic Message Authentication Codes (IT-MACs). We recall the IT-MAC definition in Section 3.5. For the rest of this document, we use $\llbracket x \rrbracket^d$ to denote a degree- d VOLE/IT-MAC commitment of $x \in \mathbb{F}_{2^\lambda}$ for $d \in \mathbb{N}^+$, and denote by $\llbracket \mathbf{x} \rrbracket^d$ a vector of degree- d VOLE/IT-MAC commitments for $\mathbf{x} \in \mathbb{F}_{2^\lambda}^\ell$. Below, we show that the homomorphism of VOLE/IT-MAC gives rise to efficient zero-knowledge protocols.

2.2.2 QuickSilver

QuickSilver [YSWW21] is an efficient zero-knowledge protocol for proving NP relations with IT-MACs. In the following, we recall the basic approach of QuickSilver. For our purpose, we only use QuickSilver to prove degree-2 relations.

For a vector $\mathbf{x} \in \mathbb{F}_2^n$, suppose that a prover Prv and a verifier Ver already hold $\llbracket \mathbf{x} \rrbracket^1$. Then, for a degree-2 n -variate polynomial $f(X)$, Prv can prove $f(\mathbf{x}) = 0$ by letting both parties compute $\llbracket y \rrbracket^2 := f(\llbracket \mathbf{x} \rrbracket^1)$, and then reducing the check of $f(\mathbf{x}) = 0$ to that of $y = 0$, which is accomplished via the IT-MAC opening. Based on the linear homomorphism of IT-MACs, we can verify multiple degree-2 relations in a batch. Specifically, given the authenticated witness $\llbracket \mathbf{x} \rrbracket^1$, suppose that $f_i(\mathbf{x}) = 0$ for each $i \in [\ell] = \{0, \dots, \ell - 1\}$ needs to be proven, where each f_i is a degree-2 n -variate polynomial. The QuickSilver protocol executes as follows:

- Prv and Ver compute $\llbracket y_i \rrbracket^2 := f_i(\llbracket \mathbf{x} \rrbracket^1)$ for each $i \in [\ell]$.
- Ver samples $\chi_0, \dots, \chi_{\ell-1} \leftarrow \mathbb{F}_{2^\lambda}$ uniformly at random, and then sends them to Prv.

- Prv and Ver compute $\llbracket \bar{y} \rrbracket^2 := \sum_{i=0}^{\ell-1} \chi_i \cdot \llbracket y_i \rrbracket^2$, and Ver checks $\bar{y} = 0$ via the IT-MAC opening.

Let $p_{\bar{y}}(X)$ be the degree-1 polynomial on $\llbracket \bar{e} \rrbracket^2$ as $\bar{y} = 0$. Since the coefficients of $p_{\bar{y}}(X)$ depends on the secret witness $\mathbf{x}$, to ensure the zero-knowledge property, Prv masks $\llbracket \bar{y} \rrbracket^2$ with a random IT-MAC of zero, i.e., $\llbracket 0 \rrbracket^2$ where the underlying polynomial $p_0(X)$ is a random degree-1 polynomial. The $\llbracket 0 \rrbracket^2$ can be generated by aggregating λ random degree-1 IT-MACs of elements from $\mathbb{F}_2$ into a single IT-MAC on a random element in $\mathbb{F}_{2^\lambda}$.

Since the QuickSilver zero-knowledge protocol is private-coin and thus designated-verifier (i.e., only the verifier, who knows Δ , can verify the proof), it is not directly suitable for constructing a signature scheme. In Section 2.2.3, we describe the VOLE-in-the-Head framework, which generates VOLE correlations in a public-coin fashion, thereby transforming QuickSilver into a publicly verifiable zero-knowledge proof and subsequently into a digital signature scheme.

2.2.3 VOLE-in-the-Head

Based on the approach in the SoftSpokenOT protocol [Roy22], Baum et al. [BBD⁺23b] proposed a VOLE generation scheme, which converts QuickSilver from *private coin* to *public coin*. The verifier does not need to maintain a secret internal state, and so the scheme becomes publicly verifiable. Specifically, the scheme consists of two phases.

- In the first phase, the prover commits to its VOLE output $(\mathbf{u}, \mathbf{v})$. After this phase, the prover cannot change its output.
- In the second phase, the verifier samples Δ at random and reveals it to open $\mathbf{q} = \mathbf{u} \cdot \Delta + \mathbf{v}$.

Between the two phases, the prover uses its VOLE output to prove a given NP relation via QuickSilver. After the second phase, the verifier uses $\mathbf{q}$ to check the proof. Since Δ is revealed to the prover only after the QuickSilver protocol has terminated, this information cannot compromise the security of QuickSilver. This methodology is called VOLE-in-the-Head (VOLEitH). Using the VOLEitH framework, we can instantiate a public-coin zero-knowledge proof system for the RSD relation. The prover commits only to the truncated witness $\mathbf{e}_B$, and the verifier reconstructs the commitments on the remaining portion of $\mathbf{e}$ using the systematic form of $\mathbf{H}$, and the QuickSilver protocol is used to prove the regularity constraint with deterministic linear sketching.

At a high level, the VOLEitH-based zero-knowledge proofs work as follows. Let $\text{PRG} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\ell$ be a pseudorandom generator, and suppose the construction uses τ small VOLE instances over $\mathbb{F}_{2^k}$. The prover first commits to a collection of seeds, and derives a random vector of degree-1 IT-MACs from these seeds. Then, the prover transforms random IT-MACs into chosen IT-MACs depending on the witness. Next, he runs QuickSilver on top of these IT-MACs; afterwards, the verifier obtains enough information to reconstruct its verification material. More concretely, the main steps are shown as follows:

1. Using a batch all-but-one vector commitment (BAVC) scheme, the prover commits to τ sets of 2^k seeds (using τ k -depth GGM trees), denoted by $\{\text{sd}_j^i\}_{i \in [0, \tau), j \in [0, 2^k)}$.
2. For each $i \in [0, \tau)$, the prover expands these seeds and derives a small-field VOLE correlation by computing

$$\mathbf{u}^i := \sum_{j \in \mathbb{F}_{2^k}} \text{PRG}(\text{sd}_j^i), \quad \mathbf{v}^i := \sum_{j \in \mathbb{F}_{2^k}} (-j) \cdot \text{PRG}(\text{sd}_j^i),$$

where we implicitly identify $\mathbb{F}_{2^k}$ with the index set of size 2^k .

3. To merge these τ instances into one VOLE correlation over a large field, the prover computes $\mathbf{c}^i := \mathbf{u}^0 - \mathbf{u}^i$ for each $i \in [1, \tau)$, and then sends them to the verifier. The prover defines $\mathbf{u} := \mathbf{u}^0$ and $\mathbf{v} := (\mathbf{v}^0, \dots, \mathbf{v}^{\tau-1})$. Together, $(\mathbf{u}, \mathbf{v})$ define a vector of degree-1 IT-MACs.

4. To ensure that $\{\mathbf{c}^i\}_{i \in [1, \tau)}$ are consistent with the committed seeds, the prover and verifier execute the SoftSpokenOT consistency check [Roy22].
5. Using the random IT-MACs, the prover and verifier run the QuickSilver protocol. This is possible because the verifier messages used by QuickSilver are only random challenges, and thus can be generated before the verifier has reconstructed its verification material.
6. After the QuickSilver messages are fixed, the verifier samples challenge indices $\Delta^i \in [0, 2^k)$ for all $i \in [0, \tau)$. Using the BAVC opening procedure, the prover opens all committed seeds except those at the challenged positions. From the opened seeds, the verifier reconstructs

$$\mathbf{q}^i := \sum_{j \in \mathbb{F}_{2^k}} (\Delta^i - j) \cdot \text{PRG}(\text{sd}_j^i),$$

and additionally adds the correction term $\mathbf{c}^i \cdot \Delta^i$ for each $i \in [1, \tau)$. The verifier defines $\mathbf{q} := (\mathbf{q}^0, \dots, \mathbf{q}^{\tau-1})$, which is exactly the verification material. It then uses $\mathbf{q}$ to verify the QuickSilver consistency check.

Fiat-Shamir transform. We use the Fiat-Shamir heuristic [FS87] to transform a public-coin zero-knowledge protocol into a publicly verifiable NIZK proof. In particular, since the verifier’s messages are public coins, the prover can simulate the verifier’s behavior using a cryptographic hash function modeled as a random oracle: (1) the prover generates the verifier’s messages by hashing the message to be signed together with the protocol transcript up to this point; (2) it uses the transcript as a signature on the message. For a One-Way Function (OWF), we can naturally define an NP relation based on the input and output of the OWF, with the corresponding preimage as the witness. Therefore, the final piece of ReSolveD- α is to apply the Fiat-Shamir transform to the public-coin zero-knowledge protocol for the RSD relation in the VOLE-in-the-Head framework.

2.3 TCCR Hash Function

We propose a new construction of TCCR hash functions, which is provably secure in both ICM and QICM. In particular, the TCCR hashing is constructed from a blockcipher with n_b -bit block size and n_k -bit key length, and the hashing should offer $\lambda > n_b$ bit security. This means that the hashing needs to have the input and output of $\lambda > n_b$ bits. Previous constructions like [GKWY20, GKW+20, GWYY25] focus on the $\lambda = n_b$ case. Intuitively, setting the security parameter higher than the block length requires making more than one blockcipher call per hash input.

Fortunately, we have $n_b + n_k \gg \lambda$, meaning that the λ -bit input can be fully absorbed into each blockcipher call. Specifically, for the parameters relevant to ReSolveD- α , we have $\lambda \leq 2n_b$. In this parameter regime, the design team previously proposed a TCCR hash construction which makes two blockcipher calls [CGW+24]. We sketch the basic idea below, as the new construction in this specification follows the same generic idea. To avoid various attacks, the two blockcipher calls need to employ strongly correlated yet distinct inputs. By this, the idea consists of: (i) appending the λ -bit input with the 8-bit encoding of an integer $i \in \{0, 1, \dots, 2^8 - 1\}$; (ii) padding the result with $n_b + n_k - \lambda - 8$ zeros; (iii) breaking the result into an n_b -bit part and a n_k -bit part and setting them as the input to the $(i + 1)^{\text{th}}$ blockcipher call.

Compared to [CGW+24], our new construction incorporates further optimizations by arranging the integer i in the plaintext input part of the blockcipher calls, so that the two blockcipher calls can use the same key. This design allows us to avoid one costly key schedule function evaluation. Moreover, we remove the usage of linear orthomorphism in the new design, which brings further performance optimization. The detailed construction of the new TCCR hash function is described in Section 4.9.8. We show the architecture of the new TCCR design in

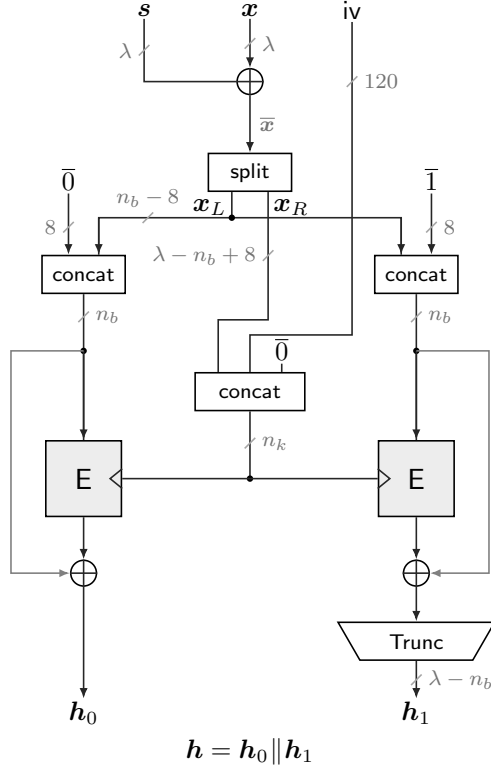


Figure 1: The new TCCR hash construction in the two-block case $n_b < \lambda \leq 2n_b$.

Figure 1. In this figure, “split” denotes splitting operation of breaking the λ -bit $\bar{x} := x \oplus s$ into the $(n_b - 8)$ -bit left part x_L and the $(\lambda - n_b + 8)$ -bit right part x_R ; “concat” denotes the concatenation operation; the shaded “E” block denotes the ideal cipher with n_b -bit block size (the vertical arrows correspond to plaintext input and ciphertext output) and n_k -bit key size (the horizontal arrow corresponds to the key input); “Trunc” denotes the truncation operation that takes the least significant $\lambda - n_b$ bits from the n_b -bit input. Note that the two ideal cipher calls share the same key, which is x_R concatenated with iv and zero-padded to n_k bits. This saves one key schedule for implementation.

3 Preliminaries

3.1 Notation

3.1.1 Sampling and Assignment

We write $x \leftarrow S$ to denote the sampling of x from S : if S is a set, then x is sampled uniformly at random from S ; if S is a distribution, then x is sampled according to S . We write $x := y$ to denote deterministic assignment.

3.1.2 Integer Ranges

For a positive integer n , we write $[n] := \{0, 1, \dots, n - 1\}$; more generally, for integers $a \leq b$, we write $[a, b) := \{a, a + 1, \dots, b - 1\}$.

3.1.3 Vectors and Matrices

Vectors are denoted by bold lowercase letters and matrices by bold uppercase letters, with entries accessed via bracket notation. For $\mathbf{v} \in \mathbb{F}^n$, we write $\mathbf{v}[i]$ for the i -th entry ($i \in [0, n)$).

and $\mathbf{v}[i, j]$ for the sub-vector from index i to $j - 1$. For $\mathbf{V} \in \mathbb{F}^{m \times n}$, we write $\mathbf{V}[i, :]$ for the i -th row ($i \in [0, m)$) and $\mathbf{V}[:, j]$ for the j -th column ($j \in [0, n)$). Concatenation uses brackets: for $\mathbf{x} \in \mathbb{F}^m$ and $\mathbf{y} \in \mathbb{F}^n$, we write $[\mathbf{x}, \mathbf{y}] \in \mathbb{F}^{m+n}$; for matrices $\mathbf{A} \in \mathbb{F}^{m \times n}$ and $\mathbf{B} \in \mathbb{F}^{m \times p}$, we write $[\mathbf{A}, \mathbf{B}] \in \mathbb{F}^{m \times (n+p)}$ for horizontal concatenation, and similarly for vertical concatenation. For $\mathbf{x}, \mathbf{y} \in \mathbb{F}^n$, their *inner product* is $\langle \mathbf{x}, \mathbf{y} \rangle := \sum_{i \in [0, n)} \mathbf{x}[i] \cdot \mathbf{y}[i] \in \mathbb{F}$.

3.1.4 Tuples and Bit Strings

We write $\{0, 1\}^n$ for a bit string of length n and $\{0, 1\}^*$ for an arbitrary-length bit string. For $\mathbf{a} \in \{0, 1\}^m$ and $\mathbf{b} \in \{0, 1\}^n$, we define $\mathbf{c} := \mathbf{a} \parallel \mathbf{b} \in \{0, 1\}^{m+n}$. We write 0^k and 1^k for all-zero and all-one strings of length k , respectively. We denote bit-wise XOR by $\oplus$ and, for $\mathbf{a} \in \{0, 1\}^n$, we write $\text{HW}(\mathbf{a})$ for the *Hamming weight* of $\mathbf{a}$, i.e., the number of nonzero entries. We use (a, b, c) to denote a tuple of elements a, b, c (not necessarily from the same set). For an indexed family of elements, we write $(x_i)_{i \in I}$ when each element x_i is a single object; when the elements are themselves tuples, we use outer braces to disambiguate, i.e., we write $\{(x_i, y_i)\}_{i \in I}$ rather than $(x_i, y_i)_{i \in I}$.

3.2 Hash Function

In our signature scheme, we require the following hash functions.

Definition 3.1 (Universal Linear Hash Function). *Let $\mathcal{H} = \{H : \mathcal{X} \rightarrow \mathcal{Y}\}$ be a family of hash functions for two Abelian groups $(\mathcal{X}, +)$ and $(\mathcal{Y}, \oplus)$. $\mathcal{H}$ is ε -universal linear if*

- **Universality.** *For any non-zero $x \in \mathcal{X}$, $\Pr_{H \leftarrow \mathcal{H}} [H(x) = 0] \leq \varepsilon$.*
- **Linearity.** *For any $x, x' \in \mathcal{X}$, $H(x + x') = H(x) \oplus H(x')$.*

Definition 3.2 (Hiding of Hash Function). *Let $\mathcal{H} = \{H : \mathbb{F}_q^{n+h} \rightarrow \mathcal{Y}\}$ be a family of hash functions. $H \in \mathcal{H}$ is $\mathbb{F}_q^n$ -hiding if the distribution of $H(x)$ is independent of $x[0, n)$ conditioned on $x[n, n+h) \leftarrow \mathbb{F}_q^h$ being uniformly random. $\mathcal{H}$ is $\mathbb{F}_q^n$ -hiding if every $H \in \mathcal{H}$ is $\mathbb{F}_q^n$ -hiding.*

Definition 3.3 (Circular Correlation Robust (CCR) Hash Function [GKWY20]). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $H^{\mathcal{O}} : \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ be a hash function using these oracles. For $x, \Delta \in \{0, 1\}^\ell$ and $b \in \{0, 1\}$, let $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}(x, b) = H^{\mathcal{O}}(x \oplus \Delta) \oplus b \cdot \Delta$ be the CCR oracle. $H^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, Q, \chi, \varepsilon)$ -CCR if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q queries to its second oracle (the CCR oracle $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}$ or a random function f),*

$$\text{AdvCCR}_{\mathcal{A}, H}^{\mathcal{O}}(Q_{\mathcal{O}}, Q, \chi) = \left| \Pr_{\Delta \leftarrow \mathcal{X}} \left[\mathcal{A}^{\mathcal{O}, \mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}}() = 1 \right] - \Pr_{f \leftarrow \mathcal{F}_{\ell+1, \ell}} \left[\mathcal{A}^{\mathcal{O}, f}() = 1 \right] \right| \leq \varepsilon,$$

where $\mathcal{A}$ cannot query $\mathcal{O}_{H, \Delta}^{\mathcal{O}, \text{CCR}}$ with both $(x, 0)$ and $(x, 1)$ for any $x \in \{0, 1\}^\ell$, χ is a distribution over $\{0, 1\}^\ell$, and $\mathcal{F}_{\ell+1, \ell} = \{f : \{0, 1\}^\ell \times \{0, 1\} \rightarrow \{0, 1\}^\ell\}$ is a function family.

Definition 3.4 (Tweakable Circular Correlation Robust Hash Function (TCCR)). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $H^{\mathcal{O}} : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$ be a hash function using these oracles, $x, T, \Delta \in \{0, 1\}^\lambda$ and $b \in \{0, 1\}$, define the TCCR oracle $\mathcal{O}_{H, \Delta}^{\text{TCCR}}(x, T, b) := H(x \oplus \Delta, T) \oplus b \cdot \Delta$. $H^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, Q, \chi, \varepsilon)$ -TCCR if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q queries to its second oracle (the TCCR oracle $\mathcal{O}_{H, \Delta}^{\text{TCCR}}$ or a random function f), and every distribution χ over $\{0, 1\}^\lambda$ with min-entropy at least ρ , it holds that*

$$\begin{aligned} & \text{AdvTCCR}_{\mathcal{A}, H}(Q, \rho) \\ &= \left| \Pr_{\Delta \leftarrow \mathcal{X}} \left[\mathcal{A}^{\mathcal{O}, \mathcal{O}_{H, \Delta}^{\text{TCCR}}}(1^\lambda) = 1 \right] - \Pr_{f \leftarrow \mathcal{F}_{\lambda+1, \lambda}} \left[\mathcal{A}^f(1^\lambda) = 1 \right] \right| \leq \text{negl}(\lambda), \end{aligned}$$

where for any $x, T \in \{0, 1\}^\lambda$, the adversary $\mathcal{A}$ is not allowed to query both $(x, T, 0)$ and $(x, T, 1)$.

The quantum analogs of CCR and TCCR are obtained by using a quantum adversary $\mathcal{A}$ in [Definition 3.3](#) and granting $\mathcal{A}$ access to $|\mathcal{O}\rangle$, i.e., oracles $\mathcal{O}$ implemented by quantum circuits (but the construction oracles $\mathcal{O}_{H,\Delta}^{\mathcal{O},\text{ccr}}$, $\mathcal{O}_{H,\Delta}^{\text{tccr}}$ and f still only appear in the non-quantum form).

Definition 3.5 (Natural and Non-trivial Query (NNQ) [[GYW+23](#)]). *Let $\mathcal{H} : \{0,1\}^\ell \rightarrow \{0,1\}^\ell$ be a hash function, $\mathcal{Q} : \{0,1\}^\ell \times \{0,1\} \rightarrow \{0,1\}^\ell$ be an oracle, and $\mathcal{T} \subseteq \{0,1\}^\ell \setminus \{0^\ell\}$ be a set of linearly independent strings. We say that a query q_i is a natural and non-trivial query if the output $x_i \in \{0,1\}^\ell$ defined by that query satisfies the following:*

- **Naturality.** x_i is defined by one of the following types of computation:
 - Type 1: $x_i := v$, where $v \in \{0^\ell\} \cup \mathcal{T}$.
 - Type 2: $x_i := x_j \oplus x_k$, where x_j, x_k were defined by two previous distinct NNQs q_j, q_k .
 - Type 3: $x_i := \mathcal{H}(x_j)$, where x_j was defined by a previous NNQ q_j .
 - Type 4: $x_i := \mathcal{Q}(x_j, b_j)$, where x_j was defined by a previous NNQ q_j and $b_j \in \{0,1\}$.
- **Non-triviality.** x_i syntactically differs from any x_j defined by another NNQ q_j .

Let $\text{Real}_{H_S,\Delta}^{\mathcal{O}}$ be the real-world oracle that, given an NNQ as input, implements $\mathcal{H} = H_S^{\mathcal{O}} : x \mapsto H^{\mathcal{O}}(x \oplus S)$ and $\mathcal{Q} = \mathcal{O}_{H_S,\Delta}^{\mathcal{O},\text{ccr}}$, executes the query according to the definition, and returns the result. Meanwhile, the ideal-world oracle $\text{Ideal}_{H_S}^{\mathcal{O}}$ is identical to $\text{Real}_{H_S,\Delta}^{\mathcal{O}}$ except that it replaces $\mathcal{Q}$ with an oracle that returns a fresh uniformly random string on each invocation.

The following lemma is an improved version of the original one of [[GYW+23](#)] by counting the ideal-model queries to capture the ideal-model-based constructions of CCR hash function. Its formal proof is provided in [Section 8.4.2](#).

Lemma 3.1 (CCR for Natural and Non-trivial Queries). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $H^{\mathcal{O}} : \{0,1\}^\ell \rightarrow \{0,1\}^\ell$ be a $(Q'_{\mathcal{O}}, Q, \chi, \varepsilon)$ -CCR hash function using these oracles, where $Q'_{\mathcal{O}} = Q_{\mathcal{O}} + (Q, \dots, Q)$ corresponds to Q more queries for each oracle in $\mathcal{O}$. For any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q NNQs to its second oracle (the real-world oracle $\text{Real}_{H_S,\Delta}^{\mathcal{O}}$ or the ideal-world oracle $\text{Ideal}_{H_S}^{\mathcal{O}}$), where each NNQ only depends on the results of the previous NNQs (with the initial NNQ only depending on an empty result),*

$$\left| \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[\mathcal{A}^{\mathcal{O}, \text{Real}_{H_S,\Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \mathcal{S}_\chi, c \leftarrow \mathcal{S}_U} \left[\mathcal{A}^{\mathcal{O}, \text{Ideal}_{H_S}^{\mathcal{O}}}() = 1 \right] \right| \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q'_{\mathcal{O}}, Q, \chi) + \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}},$$

where $\mathcal{U}$ is the uniform distribution of some proper unspecified length, and c is the random coin used for Type 4 NNQs in $\text{Ideal}_{H_S}^{\mathcal{O}}$.

3.3 Pseudorandom Generator

In our signature scheme, we use an IV-based tweakable PRG, which ensures pseudorandomness given a secret seed. For the soundness of an interactive proof system underlying the signature scheme, we also require the PRG to be almost injective.

Definition 3.6 (IV-based Tweakable PRG). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $G^{\mathcal{O}} : \mathcal{X} \times \mathcal{I} \times \mathcal{T} \rightarrow \mathcal{Y}$ be a PRG using these oracles. $G^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, \varepsilon)$ -secure if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and any $(iv, t) \in \mathcal{I} \times \mathcal{T}$,*

$$\text{AdvPRG}_{\mathcal{A}}^{\mathcal{O}, G}(Q_{\mathcal{O}}) = \left| \Pr_{x \leftarrow \mathcal{S}_\mathcal{X}} \left[\mathcal{A}^{\mathcal{O}}(G^{\mathcal{O}}(x, iv, t)) = 1 \right] - \Pr_{y \leftarrow \mathcal{S}_\mathcal{Y}} \left[\mathcal{A}^{\mathcal{O}}(y) = 1 \right] \right| \leq \varepsilon.$$

Game $\text{GameInj}_{\mathcal{A}}^{\mathcal{O},G}(Q_{\mathcal{O}}, L)$	
1 :	$(\text{iv}, \{(t_i, y_i)\}_{i \in [L]}) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}}()$
2 :	if $\exists i \in [L], x \neq x' \in \mathcal{X} : G^{\mathcal{O}}(x, \text{iv}, t_i) = G^{\mathcal{O}}(x', \text{iv}, t_i) = y_i :$
3 :	return 1
4 :	else
5 :	return 0

Figure 2: The almost injectivity game.

Definition 3.7 (Almost Injective PRG [BBB⁺25]). *Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $G^{\mathcal{O}} : \mathcal{X} \times \mathcal{I} \times \mathcal{T} \rightarrow \mathcal{Y}$ be a PRG using these oracles. $G^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, L, \varepsilon)$ -almost-injective if for any adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$,*

$$\text{AdvInj}_{\mathcal{A}}^{\mathcal{O},G}(Q_{\mathcal{O}}, L) = \Pr \left[\text{GameInj}_{\mathcal{A}}^{\mathcal{O},G}(Q_{\mathcal{O}}, L) = 1 \right] \leq \varepsilon,$$

where the almost injectivity game $\text{GameInj}_{\mathcal{A}}^{\mathcal{O},G}$ is defined in Fig. 2.

3.4 Zero-Knowledge Proof

A zero-knowledge proof (ZK) is a two-party protocol. In this document, we only consider zero-knowledge proofs for languages in the complexity class NP. In this setting, the goal of ZK is for the prover to convince the verifier that a public string $\mathbf{x}$ belongs to an NP language $\mathcal{L}$. Since $\mathcal{L}$ is an NP language, there is an efficient algorithm TM such that the following holds: if $\mathbf{x} \in \mathcal{L}$, then there exists a *witness* vector $\mathbf{w}$ with $\text{TM}(\mathbf{x}, \mathbf{w}) = 1$; conversely, if $\mathbf{x} \notin \mathcal{L}$, then $\text{TM}(\mathbf{x}, \mathbf{w}) = 0$ for all $\mathbf{w}$. For the ReSolveD- α algorithm, the NP language of interest consists of instances $(\mathbf{H}, \mathbf{y})$ where $\mathbf{y} = \mathbf{H} \cdot \mathbf{e}$ for a regular noise vector $\mathbf{e}$. Because we use the systematic form $\mathbf{H} = [\mathbf{I} \mid \mathbf{H}_B]$, the actual committed witness is the truncated blockwise representation of $\mathbf{e}_B$, from which the prover and verifier can reconstruct the full regular witness $\mathbf{e}$.

We require the interactive protocol to satisfy the following properties²:

- **Completeness:** If $\mathbf{x} \in \mathcal{L}$, then the verifier should accept the proof.
- **Soundness:** If $\mathbf{x} \notin \mathcal{L}$, then the verifier should reject with high probability.
- **Knowledge soundness:** If the verifier accepts the proof, then we can extract $\mathbf{w}$ by interacting with the prover. (This property is a strengthening of soundness.)
- **Zero-knowledge:** If $\mathbf{x} \in \mathcal{L}$, then the proof should not leak the witness $\mathbf{w}$.

In 1987, Goldreich, Micali, and Wigderson proved that every NP language admits a ZK protocol [GMW87]. Through ZK, we can construct *identification protocols*, in which the prover demonstrates to the verifier that it possesses the witness $\mathbf{w}$ corresponding to public information $\mathbf{x}$. In this process, the knowledge soundness of ZK guarantees that if the verifier accepts the proof, then the prover must hold the corresponding witness; meanwhile, the zero-knowledge property ensures that the protocol execution does not leak any information about $\mathbf{w}$.

²As a core concept of modern cryptography, the rigorous definition of zero knowledge can be found in Goldreich's textbook [Gol01, Chapter 4]. Since our focus is on the security of the signature scheme, we only provide a brief introduction here.

Proof system. We formalize the properties of an interactive proof (IP) system to be used in our security proofs.

Definition 3.8 (Soundness). Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $\Pi^{\mathcal{O}} = (\mathcal{P}^{\mathcal{O}}, \mathcal{V}^{\mathcal{O}})$ be a proof system for an NP relation $\mathcal{R}$ and its language $\mathcal{L}_{\mathcal{R}}$ using these oracles. $\Pi^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, \varepsilon)$ -sound if for any (possibly unbounded) adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$,

$$\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}, \Pi}(Q_{\mathcal{O}}) = \Pr \left[\mathbb{x} \leftarrow_s \mathcal{A}^{\mathcal{O}}(), b \leftarrow_s \langle \mathcal{A}^{\mathcal{O}}, \mathcal{V}^{\mathcal{O}} \rangle(\mathbb{x}) : \mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge b = 1 \right] \leq \varepsilon,$$

where $\langle \mathcal{A}^{\mathcal{O}}, \mathcal{V}^{\mathcal{O}} \rangle(\mathbb{x}) \in \{0, 1\}$ returns 1 if and only if the execution for the instance $\mathbb{x}$ between the prover $\mathcal{A}$ and the verifier $\mathcal{V}$ makes the verifier output **accept**.

Definition 3.9 (Interactive Proof System). For an NP relation $\mathcal{R}$ with its language $\mathcal{L}_{\mathcal{R}}$ and $k \in \mathbb{N}^+$, a k -round IP system $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ is defined as follows:

- The prover $\mathcal{P}_{\text{IP}}$ takes an instance $\mathbb{x}$ and a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ as its initial input, while the verifier $\mathcal{V}_{\text{IP}}$ only takes $\mathbb{x}$ as its initial input.
- In the i -th round, where $i \in [k]$, the prover $\mathcal{P}_{\text{IP}}$ sends a message pmsg_i to the receiver $\mathcal{V}_{\text{IP}}$, who sends back a challenge chall_i to the prover $\mathcal{P}_{\text{IP}}$.
- Upon receiving the last message pmsg_k from the prover $\mathcal{P}_{\text{IP}}$, the verifier $\mathcal{V}_{\text{IP}}$ outputs

$$\{\text{reject}, \text{accept}\} \leftarrow_s \mathcal{V}_{\text{IP}}(\mathbb{x}, \{(\text{pmsg}_i, \text{chall}_i)\}_{i \in [k]}, \text{pmsg}_k).$$

A transcript tr of IP is an ordered list of messages and has one of the following cases:

- $i \in [k] \cup \{k\}$ and the prover moves next: $\text{tr}_{2i} = (\text{pmsg}_0, \text{chall}_0, \dots, \text{pmsg}_{i-1}, \text{chall}_{i-1})$. As a special case, $\text{tr}_0 = \emptyset$ for $i = 0$ is an empty transcript.
- $i \in [k] \cup \{k\}$ and the verifier moves next: $\text{tr}_{2i+1} = (\text{pmsg}_0, \text{chall}_0, \dots, \text{pmsg}_i)$. As a special case, $\text{tr}_{2k+1} = (\text{pmsg}_0, \text{chall}_0, \dots, \text{pmsg}_{k-1}, \text{chall}_{k-1}, \text{pmsg}_k)$ is a full transcript.

Definition 3.10 (Round-by-Round Soundness [CCH⁺19]). Let $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ be a k -round IP system for some NP relation $\mathcal{R}$ with a language $\mathcal{L}_{\mathcal{R}}$. IP is ε -RBR-sound if there exists a deterministic (not necessarily polynomial-time) algorithm state , which takes an instance $\mathbb{x}$ and a transcript as input and outputs **reject** or **accept**, such that

- If $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}}$, then $\text{state}(\mathbb{x}, \emptyset) = \text{reject}$ for the empty transcript.
- In the i -th round, where $i \in [k]$, for any transcript tr_{2i} and prover's message pmsg_i , we have

$$\Pr_{\text{chall}_i} \left[\text{state}(\mathbb{x}, \text{tr}_{2i} \parallel \text{pmsg}_i \parallel \text{chall}_i) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_{2i}) = \text{reject} \right] \leq \varepsilon_i \leq \varepsilon,$$

where the probability is taken over the uniform chall_i .

- For any full transcript tr_{2k+1} , if $\text{state}(\mathbb{x}, \text{tr}_{2k+1}) = \text{reject}$, then $\mathcal{V}_{\text{IP}}(\mathbb{x}, \text{tr}_{2k+1}) = \text{reject}$.

3.5 Information-Theoretic Message Authentication Code

We adopt the generalized Information-Theoretic Message Authentication Code (IT-MAC) definition of Agarwal et al. [ABBS25]. The original presentation in [ABBS25] supports general modulus, whereas ReSolveD- α only needs modulus 2. In Definition 3.11, we simplify the notation accordingly.

Definition 3.11. For $d \in \mathbb{Z}$, a degree- d IT-MAC for $a \in \mathbb{F}_{2^\lambda}$ is defined as $\llbracket a \rrbracket^d := (p_a(x), k_a, \Delta)$, where a prover Prv holds a polynomial $p_a(x) \in \mathbb{F}_{2^\lambda}[x]$ with $\deg(p_a) = d$ and leading coefficient a ; while a verifier Ver holds the evaluation $k_a = p_a(\Delta)$, for a global key $\Delta \in \mathbb{F}_{2^\lambda}$ unknown to Prv . The degree superscript d is omitted when clear from context.

Let $\llbracket a \rrbracket^{d_a}$ and $\llbracket b \rrbracket^{d_b}$ be two IT-MACs, with corresponding polynomials $p_a(x), p_b(x)$ and evaluations k_a, k_b , and assume w.l.o.g. that $d_a \geq d_b$. IT-MAC supports the following local homomorphic operations:

- **Addition.** To compute $\llbracket a + b \rrbracket^{d_a} = \llbracket a \rrbracket^{d_a} + \llbracket b \rrbracket^{d_b}$, Prv computes $p_{a+b}(x) = p_a(x) + p_b(x) \cdot x^{d_a-d_b}$, while Ver computes $k_{a+b} = k_a + k_b \cdot \Delta^{d_a-d_b}$.
- **Multiplication.** To compute $\llbracket ab \rrbracket^{d_a \cdot d_b} = \llbracket a \rrbracket^{d_a} \cdot \llbracket b \rrbracket^{d_b}$, Prv computes $p_{ab}(x) = p_a(x) \cdot p_b(x)$, while Ver computes $k_{ab} = k_a \cdot k_b$.
- **Constants.** For any $c \in \mathbb{F}_{2^\lambda}$ and $d \in \mathbb{Z}$, define $\llbracket c \rrbracket^d$ by $p_c(x) = cx^d$ and $k_c = c\Delta^d$. Combining this with the above operations, constant addition and scalar multiplication preserve degree. Subtraction follows as $\llbracket a \rrbracket - \llbracket b \rrbracket := \llbracket a \rrbracket + \llbracket -1 \rrbracket^1 \cdot \llbracket b \rrbracket$. Moreover, to de-randomize a random IT-MAC $\llbracket r \rrbracket^1$ (where $r \leftarrow \mathbb{F}_2$) into a target value $x \in \mathbb{F}_2$, Prv publishes $\delta := x - r$, and both parties compute $\llbracket x \rrbracket^1 := \llbracket r \rrbracket^1 + \delta$.

IT-MAC satisfies the following hiding and binding properties:

- **Hiding.** Random degree-1 IT-MACs are typically generated from VOLE protocols, which inherently ensure that the committed value a is independent of Ver 's authentication material. Thus, hiding is guaranteed.
- **Binding.** To open a committed value a , Prv sends its (purported) associated polynomial $\tilde{p}_{a'}(x)$ to Ver , who checks whether $\tilde{p}_{a'}(\Delta) = k_a$. Any attempt by Prv to open a different value $a' \neq a$ while passing this check implies $\tilde{p}_{a'}(\Delta) = p_a(\Delta)$. Assuming $\Delta \leftarrow \mathbb{F}_{2^\lambda}$, the probability of such a collision is at most $\frac{\deg(p_a)}{2^\lambda}$.

3.6 Digital Signature Scheme

Definition 3.12 (Digital Signature Scheme). A digital signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ consists of three algorithms:

- $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$: A probabilistic key generation algorithm that takes as input the security parameter 1^λ and outputs a secret key sk and a public key pk .
- $\sigma \leftarrow \text{Sign}(\text{sk}, \text{msg})$: A (possibly probabilistic) signing algorithm that takes as input the secret key sk and a message $\text{msg} \in \{0, 1\}^*$, and outputs a signature σ .
- $b := \text{Verify}(\text{pk}, \text{msg}, \sigma)$: A deterministic verification algorithm that takes as input the public key pk , a message msg , and a signature σ , and outputs $b \in \{\text{accept}, \text{reject}\}$.

We say Σ is correct if for all $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda)$ and all $\text{msg} \in \{0, 1\}^*$:

$$\Pr[\text{Verify}(\text{pk}, \text{msg}, \text{Sign}(\text{sk}, \text{msg})) = \text{accept}] = 1. \quad (3)$$

We consider two standard security notions of a signature scheme: existential unforgeability under key-only attacks (EUF-KO) and existential unforgeability under chosen message attacks (EUF-CMA).

Definition 3.13 (EUF-KO and EUF-CMA Security). Let $\mathcal{O}$ be a list of ideal-model oracles, $Q_{\mathcal{O}}$ be a list of the respective numbers of the ideal-model queries, and $\Sigma^{\mathcal{O}} = (\text{KeyGen}^{\mathcal{O}}, \text{Sign}^{\mathcal{O}}, \text{Verify}^{\mathcal{O}})$ be a signature scheme using these oracles. $\Sigma^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, \varepsilon)$ -EUF-KO-secure if for any stateful adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$,

$$\text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}) = \Pr \left[\text{GameEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}) = 1 \right] \leq \varepsilon,$$

Game $\text{GameEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}})$	
1 :	$\mathcal{A}^{\mathcal{O}}()$
2 :	$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}^{\mathcal{O}}()$
3 :	$(\text{msg}, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pk})$
4 :	return $\text{Verify}^{\mathcal{O}}(\text{pk}, \text{msg}, \sigma) = \text{accept}$
Game $\text{GameEUFCMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}, Q_{\text{sig}})$	$\text{OSig}_{\text{sk}}(\text{msg})$
1 :	$\mathcal{A}^{\mathcal{O}}()$
2 :	$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}^{\mathcal{O}}()$
3 :	$M := \emptyset$
4 :	$(\text{msg}, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}, \text{OSig}_{\text{sk}}}(\text{pk})$
5 :	return $\text{Verify}^{\mathcal{O}}(\text{pk}, \text{msg}, \sigma) = \text{accept} \wedge \text{msg} \notin M$
	1 : $M := M \cup \{\text{msg}\}$
	2 : return $\text{Sign}^{\mathcal{O}}(\text{sk}, \text{msg})$

Figure 3: The EUF-KO and EUF-CMA games, where the blue parts are different.

where the EUF-KO game $\text{GameEUFKO}_{\mathcal{A}}^{\mathcal{O}, \Sigma}$ is defined in Fig. 3. Meanwhile, $\Sigma^{\mathcal{O}}$ is $(Q_{\mathcal{O}}, Q_{\text{sig}}, \varepsilon)$ -EUF-CMA-secure if for any stateful adversary $\mathcal{A}$ making at most $Q_{\mathcal{O}}$ queries respectively to the oracles in $\mathcal{O}$ and at most Q_{sig} queries to the signature oracle OSig_{sk} ,

$$\text{AdvEUFCMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}, Q_{\text{sig}}) = \Pr \left[\text{GameEUFCMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] \leq \varepsilon,$$

where the EUF-CMA game $\text{GameEUFCMA}_{\mathcal{A}}^{\mathcal{O}, \Sigma}$ is defined in Fig. 3.

3.7 Regular Syndrome Decoding

For an (m, k) -linear code over $\mathbb{F}_2$, the generator matrix $\mathbf{G} \in \mathbb{F}_2^{m \times k}$ fully describes the code. The encoding process for a message $\mathbf{s} \in \mathbb{F}_2^k$ can be expressed as

$$\mathbf{c} = \mathbf{G} \cdot \mathbf{s}.$$

The purpose of encoding is to add redundancy so that the message can be recovered even in the presence of channel noise. Here, the noise flips the bits at certain positions of the transmitted codeword, so the receiver obtains a corrupted codeword $\tilde{\mathbf{c}}$. That is, for a sparse noise vector $\mathbf{e} \in \mathbb{F}_2^m$, we have

$$\tilde{\mathbf{c}} = \mathbf{c} \oplus \mathbf{e}.$$

The parity-check matrix $\mathbf{H} \in \mathbb{F}_2^{(m-k) \times m}$ has rows that form a basis for the $(m - k)$ -dimensional space orthogonal to the column space of $\mathbf{G}$, so that $\mathbf{H}\mathbf{G} = \mathbf{0}$. We have

$$\mathbf{y} := \mathbf{H} \cdot \tilde{\mathbf{c}} = \mathbf{H} \cdot (\mathbf{G} \cdot \mathbf{s} \oplus \mathbf{e}) = \mathbf{H} \cdot \mathbf{e}.$$

We call $\mathbf{y} \in \mathbb{F}_2^{m-k}$ a *syndrome*. When the noise weight is less than half the minimum distance of the code, the noise $\mathbf{e}$ is uniquely determined by the syndrome $\mathbf{y}$. Therefore, from an information-theoretic perspective, we can recover $\mathbf{e}$ from $\mathbf{y}$, thereby recovering the original codeword $\mathbf{c}$, and subsequently finding the original message $\mathbf{s}$ via Gaussian elimination. Recovering $\mathbf{e}$ from $\mathbf{y}$ in this way is known as *syndrome decoding*.

Formally, the Syndrome Decoding (SD) problem asks to recover a sparse noise vector from its syndrome under a random parity-check matrix. The worst-case SD problem in certain parameter regimes is known to be NP-hard [BMvT78, Bar94], and its average-case analogue is one of the most promising assumptions for PQC. The SD problem is equivalent to the well-known Learning Parity with Noise (LPN) problem [BFKL94], and is also called dual LPN.

Game	$\text{GameSD}_{\mathcal{A}}^b(m, k, w)$
1 :	$\mathbf{H} \leftarrow_{\$} \mathbb{F}_2^{(m-k) \times m}$
2 :	if $b = 0$:
3 :	$\mathbf{e} \leftarrow_{\$} \mathbb{F}_2^m$ such that $\text{HW}(\mathbf{e}) = w$
4 :	$\mathbf{y} := \mathbf{H} \cdot \mathbf{e} \in \mathbb{F}_2^{m-k}$
5 :	if $b = 1$:
6 :	$\mathbf{y} \leftarrow_{\$} \mathbb{F}_2^{m-k}$
7 :	$b' \leftarrow_{\$} \mathcal{A}(\mathbf{H}, \mathbf{y})$
8 :	return b'

Figure 4: The Syndrome Decoding (SD) game.

Definition 3.14 (Syndrome Decoding). *Let m, k, w be positive integers with $m > k$ and $m > w$. The decisional SD problem $\text{SD}(m, k, w)$ is (T, ε) -hard if for any probabilistic adversary $\mathcal{A}$ running in time T ,*

$$\text{AdvSD}_{\mathcal{A}}(m, k, w) = |\Pr [\text{GameSD}_{\mathcal{A}}^0(m, k, w) = 1] - \Pr [\text{GameSD}_{\mathcal{A}}^1(m, k, w) = 1]| \leq \varepsilon,$$

where the SD game $\text{GameSD}_{\mathcal{A}}^b$ is defined in Figure 4. It is simply ε -hard if the advantage holds for any PPT adversary $\mathcal{A}$.

The search SD problem $\text{sSD}(m, k, w)$ is (T, ε) -hard if for any probabilistic adversary $\mathcal{A}$ running in time T , $\mathbf{H} \leftarrow_{\$} \mathbb{F}_2^{(m-k) \times m}$, and $\mathbf{y} = \mathbf{H} \cdot \mathbf{e}$ where $\mathbf{e} \leftarrow_{\$} \mathbb{F}_2^m$ such that $\text{HW}(\mathbf{e}) = w$,

$$\text{AdvS}_{\mathcal{A}}(m, k, w) = \Pr [\mathcal{A}(\mathbf{H}, \mathbf{y}) = \mathbf{e}] \leq \varepsilon.$$

It is simply ε -hard if the advantage holds for any PPT adversary $\mathcal{A}$.

The Regular Syndrome Decoding (RSD) problem is a variant of SD, in which a noise vector has the *regular* structure: it is divided into w consecutive blocks of size $\ell_{\text{bs}} = m/w$, and each block has Hamming weight exactly 1. The regular structure was first introduced by Augot et al. [AFS03, AFS05], and the RSD problem has been used and analyzed in a lot of recent works such as [HOSS18, BCGI18, BCG⁺19a, BCG⁺19b, YWZ20, BCG⁺22, CCJ23, LWYY24, CLY⁺24, BCC⁺24, OTX24, WWY⁺25].

Definition 3.15 (Regular Syndrome Decoding). *Let m, k, w be positive integers such that $m > k$ and $m > w$. The decisional RSD problem $\text{RSD}(m, k, w)$ is (T, ε) -hard if for any probabilistic adversary $\mathcal{A}$ running in time T ,*

$$\text{AdvRSD}_{\mathcal{A}}(m, k, w) = |\Pr [\text{GameRSD}_{\mathcal{A}}^0(m, k, w) = 1] - \Pr [\text{GameRSD}_{\mathcal{A}}^1(m, k, w) = 1]| \leq \varepsilon,$$

where the RSD game $\text{GameRSD}_{\mathcal{A}}^b$ for $b \in \{0, 1\}$ is defined in Figure 5. It is simply ε -hard if the advantage holds for any PPT adversary $\mathcal{A}$.

The search RSD problem $\text{sRSD}(m, k, w)$ is (T, ε) -hard if for any probabilistic adversary $\mathcal{A}$ running in time T , $\mathbf{H} \leftarrow_{\$} \mathbb{F}_2^{(m-k) \times m}$ and $\mathbf{y} = \mathbf{H} \cdot \mathbf{e}$ where $\mathbf{e} = [\mathbf{e}_0 \| \dots \| \mathbf{e}_{w-1}] \leftarrow_{\$} \mathbb{F}_2^m$ such that for each $i \in [0, w)$: $\mathbf{e}_i \in \mathbb{F}_2^{m/w}$ and $\text{HW}(\mathbf{e}_i) = 1$,

$$\text{AdvS}_{\mathcal{A}}(m, k, w) = \Pr [\mathcal{A}(\mathbf{H}, \mathbf{y}) = \mathbf{e}] \leq \varepsilon.$$

It is simply ε -hard if the advantage holds for any PPT adversary $\mathcal{A}$.

Game $\text{GameRSD}_{\mathcal{A}}^b(m, k, w)$	
1 :	$\mathbf{H} \leftarrow_{\$} \mathbb{F}_2^{(m-k) \times m}$
2 :	if $b = 0$:
3 :	$\mathbf{e} = [\mathbf{e}_0 \parallel \dots \parallel \mathbf{e}_{w-1}] \leftarrow_{\$} \mathbb{F}_2^m$ such that $\forall i \in [0, w) : \mathbf{e}_i \in \mathbb{F}_2^{m/w}$ and $\text{HW}(\mathbf{e}_i) = 1$
4 :	$\mathbf{y} := \mathbf{H} \cdot \mathbf{e} \in \mathbb{F}_2^{m-k}$
5 :	if $b = 1$:
6 :	$\mathbf{y} \leftarrow_{\$} \mathbb{F}_2^{m-k}$
7 :	$b' \leftarrow_{\$} \mathcal{A}(\mathbf{H}, \mathbf{y})$
8 :	return b'

Figure 5: The RSD game.

Relation between SD and RSD. The set of regular-noise vectors of weight w is a subset of all weight- w vectors, and so any algorithm that solves the SD problem can also solve the RSD problem. In the other direction, Feneuil et al. [FJR22] first showed that the exact-noise SD problem can be reduced to the RSD problem with a combinatorial security loss of $\frac{\binom{m}{w}^w}{\binom{m}{w}}$. Later, Liu et al. [LWYY24] gave a parameterized reduction from SD with an exact noise distribution to RSD with a significantly smaller security loss.

Lemma 3.2 ([LWYY24, Corollary 1]). *Let $w, m \in \mathbb{N}$ and $\alpha \geq 2$ such that $\alpha w \in \mathbb{N}$ and $(\alpha w) | m$. If the decisional SD problem $\text{SD}(m, k, w)$ is (T, ε) -hard, then the decisional RSD problem $\text{RSD}(m + \alpha w, k + \alpha w, \alpha w)$ is $(T - \text{poly}(m, k), 2^{w/\alpha} \cdot \varepsilon)$ -hard.*

This shows that RSD is not easier than SD in the asymptotical sense. From the concrete security perspective, current cryptanalysis results confirm that introducing regular noise does not significantly reduce the hardness of the underlying SD problem. In particular, the best known attacks against RSD — including information set decoding [ES24], the generalized birthday attack [CCJ23], and algebraic attacks [BØ23] — yield complexities comparable to those against SD for our parameter range. The concrete security analysis of the RSD parameters used in ReSolveD- α is given in Section 7.

Systematic form. In our construction, we work with the systematic form of the public parity-check matrix, namely

$$\mathbf{H} = [\mathbf{I} \mid \mathbf{H}_B],$$

where $\mathbf{H}_B \in \mathbb{F}_2^{(m-k) \times k}$ is sampled uniformly at random. The systematic-form SD (resp. RSD) problem is the same as the standard SD (resp. RSD) problem, except that the public matrix is sampled in this form. The following lemma shows that this restriction only incurs a constant-factor loss: for a uniformly random SD/RSD instance, the leftmost square block is invertible with exact probability

$$p_{\text{sys}} := \prod_{i=1}^{m-k} (1 - 2^{-i}),$$

and conditioned on this event, left-multiplying by its inverse yields a uniformly random systematic-form instance.

Lemma 3.3 (Reduction to systematic form). *Fix either the SD problem or the RSD problem with parameters (m, k, w) , and let*

$$p_{\text{sys}} := \prod_{i=1}^{m-k} (1 - 2^{-i}).$$

Then p_{sys} is exactly the probability that a uniformly random matrix in $\mathbb{F}_2^{(m-k) \times (m-k)}$ is invertible.

If there exists a PPT algorithm solving the systematic-form search version with success probability ε , then there exists a PPT algorithm solving the corresponding standard search version with success probability $p_{\text{sys}} \cdot \varepsilon$. Likewise, if there exists a PPT algorithm solving the systematic-form decisional version with distinguishing advantage ε , then there exists a PPT algorithm solving the corresponding standard decisional version with distinguishing advantage $p_{\text{sys}} \cdot \varepsilon$.

Consequently, if the standard SD/RSD problem is (T, ε) -hard, then its systematic-form variant is $(T - \text{poly}(m), \varepsilon/p_{\text{sys}})$ -hard.

Proof. Let $(\mathbf{H}, \mathbf{y})$ be a uniformly random SD or RSD instance, and write $\mathbf{H} = [\mathbf{L} \mid \mathbf{R}]$, where $\mathbf{L} \in \mathbb{F}_2^{(m-k) \times (m-k)}$ is the leftmost square block. The exact probability that $\mathbf{L}$ is invertible is

$$\frac{\prod_{j=0}^{m-k-1} (2^{m-k} - 2^j)}{2^{(m-k)^2}} = \prod_{i=1}^{m-k} (1 - 2^{-i}) = p_{\text{sys}}.$$

Whenever $\mathbf{L}$ is invertible, we transform the instance into

$$(\mathbf{L}^{-1}\mathbf{H}, \mathbf{L}^{-1}\mathbf{y}) = ([\mathbf{I} \mid \mathbf{L}^{-1}\mathbf{R}], \mathbf{L}^{-1}\mathbf{y}).$$

In the search case, the same noise vector $\mathbf{e}$ remains a valid solution; in particular, in the RSD case its regular structure is unchanged because the transformation acts only on $\mathbf{H}$ and $\mathbf{y}$. In the decisional case, if $\mathbf{y}$ is uniformly random in $\mathbb{F}_2^{m-k}$, then so is $\mathbf{L}^{-1}\mathbf{y}$. Moreover, conditioned on $\mathbf{L}$ being invertible, the matrix $\mathbf{L}^{-1}\mathbf{R}$ is uniformly distributed over $\mathbb{F}_2^{(m-k) \times k}$, so the transformed instance is distributed exactly as a systematic-form instance. Therefore, by aborting when $\mathbf{L}$ is singular and otherwise invoking the systematic-form solver, the success probability or distinguishing advantage is multiplied by exactly p_{sys} . $\square$

Equivalence between decisional and search versions. The decisional SD problem (aka dual LPN problem [BCG⁺19a]) has been shown to be polynomially equivalent to its search version [KSS10]. First, the dual LPN problem is equivalent to the primal version of LPN problem. The direction from a primal LPN instance to a dual one follows from the fact that the matrix in the dual LPN is the parity-check matrix of the generator matrix in the primal LPN; the other direction can be similarly proved (see, e.g., [MM11]). Second, the search-to-decision reduction for the primal LPN has been shown in [DI14].

These reductions extend naturally to the RSD setting, since regular noise is a special case of exact-weight noise. Consequently, the concrete security estimates obtained from analyzing the search version of the RSD problem (i.e., the complexity of recovering $\mathbf{e}$ from $(\mathbf{H}, \mathbf{y})$) directly translate to hardness guarantees for the decisional version used in our security proofs (Section 8).

3.8 Linear Sketching

We follow a different approach to sample the linear sketch vectors. In the original ReSolveD paper [CLY⁺24], the linear sketch is sampled by expanding a short seed into two vectors in $(\mathbb{F}_{2^\lambda})^m$ with an XOF. The design team of SDitH [ABB⁺26] showed that a deterministic mechanism also suffices for the purpose of RSD-based digital signatures. Compared with the original randomized linear sketching technique, the new sketching technique has two advantages:

- No randomness sampling is required, which reduces the number of hash/XOF calls and brings a potential performance benefit;
- For checking RSD noise vectors over $\mathbb{F}_2$, the check results of multiple noise blocks can be packed together, reducing the number of constraints, and consequently, the input length to ZKHash. This also brings performance benefit.

Therefore, we adopt this new sketching technique in ReSolveD- α . We recall its design rationales and prove the soundness of this check.

Let $\ell_{bs} \in \mathbb{N}$ denote the noise block length. Since each noise block has Hamming weight 1, we only need to commit to the first $\ell_{bs} - 1$ entries of the vector. Let

$$\mathbf{e} \in \{0, 1\}^{\ell_{bs}-1}, \quad \mathbf{e} = [e_0, \dots, e_{\ell_{bs}-2}]$$

be the vector to check. Our goal is to check that $\text{HW}(\mathbf{e}) \leq 1$.

Let X be the indeterminate of $\mathbb{F}_2[X]$. Equivalently, we can view $\mathbb{F}_2[X]$ elements with degree less than λ as finite field $\mathbb{F}_{2^\lambda}$ elements. Define the following $\mathbb{F}_2[X]$ polynomial/ $\mathbb{F}_{2^\lambda}$ element:

$$D_{\mathbf{e}} := \left(\sum_{i=0}^{\ell_{bs}-2} X^i \cdot e_i \right) \cdot \left(\sum_{i=0}^{\ell_{bs}-3} X^{(\ell_{bs}-1) \cdot i} \cdot e_i \right) - \left(\sum_{i=0}^{\ell_{bs}-3} X^{\ell_{bs} \cdot i} \cdot e_i \right) \quad (4)$$

Lemma 3.4. *Suppose $(\ell_{bs} - 2) + (\ell_{bs} - 1) \cdot (\ell_{bs} - 3) < \lambda$ and $\mathbf{e} \in \{0, 1\}^{\ell_{bs}-1}$. Then*

- *If $\text{HW}(\mathbf{e}) \leq 1$, then $D_{\mathbf{e}} = 0 \in \mathbb{F}_{2^\lambda}$,*
- *If $\text{HW}(\mathbf{e}) > 1$, then $D_{\mathbf{e}} \neq 0 \in \mathbb{F}_{2^\lambda}$.*

Proof. View $D_{\mathbf{e}} \in \mathbb{F}_{2^\lambda}$ as a $\mathbb{F}_2[X]$ polynomial $D_{\mathbf{e}}(X)$ and expand it as follows:

$$\begin{aligned} D_{\mathbf{e}}(X) &:= \left(\sum_{i=0}^{\ell_{bs}-2} X^i \cdot e_i \right) \cdot \left(\sum_{i=0}^{\ell_{bs}-3} X^{(\ell_{bs}-1) \cdot i} \cdot e_i \right) - \left(\sum_{i=0}^{\ell_{bs}-3} X^{\ell_{bs} \cdot i} \cdot e_i \right) \\ &= \underbrace{X^{\ell_{bs}-2} \cdot e_{\ell_{bs}-2} \cdot \left(\sum_{i=0}^{\ell_{bs}-3} X^{(\ell_{bs}-1) \cdot i} \cdot e_i \right)}_{E(X)} \\ &\quad + \underbrace{\left(\sum_{i=0}^{\ell_{bs}-3} X^i \cdot e_i \right) \cdot \left(\sum_{i=0}^{\ell_{bs}-3} X^{(\ell_{bs}-1) \cdot i} \cdot e_i \right) - \left(\sum_{i=0}^{\ell_{bs}-3} X^{\ell_{bs} \cdot i} \cdot e_i \right)}_{F(X)} \end{aligned}$$

For the first claim, the case of $\text{HW}(\mathbf{e}) = 0$ holds since every term in Equation (4) is zero. For $\text{HW}(\mathbf{e}) = 1$, there exists a unique index $i \in [0, \ell_{bs} - 1]$ s.t. $e[i] = 1$. Consider the following two cases:

- $i \in [0, \ell_{bs} - 2]$. In this case, $E(X) = 0$ since $e_{\ell_{bs}-2} = 0$. For $F(X)$, the three summations return X^i , $X^{(\ell_{bs}-1) \cdot i}$, and $X^{\ell_{bs} \cdot i} = X^i \cdot X^{(\ell_{bs}-1) \cdot i}$. Therefore $F(X) = 0$. In total, it holds that $D_{\mathbf{e}}(X) = 0$.
- $i = \ell_{bs} - 2$. In this case, $F(X) = 0$ since every e_i term is zero. $E(X) = 0$ since the summation returns zero. Therefore, $D_{\mathbf{e}}(X) = 0$.

For the second claim, we break down and re-assemble $E(X), F(X)$ to show that every $e_i \cdot e_j$ cross term appears in the final polynomial $D_{\mathbf{e}}(X)$. We first focus on $E(X)$.

$$\begin{aligned} E(X) &= X^{\ell_{bs}-2} \cdot e_{\ell_{bs}-2} \cdot \left(\sum_{i=0}^{\ell_{bs}-3} X^{(\ell_{bs}-1) \cdot i} \cdot e_i \right) \\ &= \sum_{i \in [0, \ell_{bs}-3]} X^{(\ell_{bs}-2) + (\ell_{bs}-1) \cdot i} \cdot (e_i \cdot e_{\ell_{bs}-2}) \end{aligned}$$

Then, we expand $F(X)$.

$$\begin{aligned} F(X) &= \left(\sum_{i=0}^{\ell_{\text{bs}}-3} X^i \cdot e_i \right) \cdot \left(\sum_{i=0}^{\ell_{\text{bs}}-3} X^{(\ell_{\text{bs}}-1) \cdot i} \cdot e_i \right) - \left(\sum_{i=0}^{\ell_{\text{bs}}-3} X^{\ell_{\text{bs}} \cdot i} \cdot e_i \right) \\ &= \sum_{i,j \in [0, \ell_{\text{bs}}-3], i \neq j} X^{j+(\ell_{\text{bs}}-1) \cdot i} e_i \cdot e_j \end{aligned}$$

Summing $E(X)$ and $F(X)$, we obtain

$$\begin{aligned} D_e(X) &= \sum_{i \in [0, \ell_{\text{bs}}-3]} X^{(\ell_{\text{bs}}-2)+(\ell_{\text{bs}}-1) \cdot i} \cdot (e_i \cdot e_{\ell_{\text{bs}}-2}) + \sum_{i,j \in [0, \ell_{\text{bs}}-3], i \neq j} X^{j+(\ell_{\text{bs}}-1) \cdot i} e_i \cdot e_j \\ &= \sum_{i \in [0, \ell_{\text{bs}}-3]} \left(\sum_{j \in [0, \ell_{\text{bs}}-2], j \neq i} X^{j+(\ell_{\text{bs}}-1) \cdot i} \cdot e_i \cdot e_j \right) \end{aligned}$$

Let α be the primitive element of $\mathbb{F}_{2^\lambda}$. Since the characteristic of $\mathbb{F}_{2^\lambda}$ is 2, and $(\ell_{\text{bs}} - 2) + (\ell_{\text{bs}} - 1) \cdot (\ell_{\text{bs}} - 3) < \lambda$, $D_e(\alpha) = 0$ is equivalent to the existence of two distinct monomials that cancel out. I.e., $\exists i, i' \in [0, \ell_{\text{bs}} - 3]$, $j, j' \in [0, \ell_{\text{bs}} - 2]$, s.t. $X^{j+(\ell_{\text{bs}}-1) \cdot i} = X^{j'+(\ell_{\text{bs}}-1) \cdot i'}$. We show that this is impossible, proving the lemma.

If $i = i'$, then we have $j = j'$, showing that the two monomials are not distinct. If $i \neq i'$, then we have $j + (\ell_{\text{bs}} - 1) \cdot i = j' + (\ell_{\text{bs}} - 1) \cdot i'$, and equivalently $j - j' = (\ell_{\text{bs}} - 1) \cdot (i' - i)$. Since $j - j' \in [-(\ell_{\text{bs}} - 2), (\ell_{\text{bs}} - 2)]$, while the right hand side takes multiples of $(\ell_{\text{bs}} - 1)$, this clearly cannot hold. $\square$

For small choices of ℓ_{bs} , the resulting D_e has bounded degree when viewed as a polynomial in $\mathbb{F}_2[X]$. Therefore, when checking multiple noise blocks as in proving the regularity of RSD noise vectors, the corresponding multiple D_e values can be packed together. For instance, for $D_{e_0}(X), D_{e_1}(X) \in \mathbb{F}_2[X]$ and $\deg(D_{e_0}(X)) < \lambda/2, \deg(D_{e_1}(X)) < \lambda/2$, the element $D_{e_0, e_1} := D_{e_0} + D_{e_1} \cdot X^{\lambda/2}$ satisfies that $D_{e_0, e_1} \neq 0 \iff D_{e_0} \neq 0 \vee D_{e_1} \neq 0$, since the coefficients in $D_{e_0, e_1}(X)$, viewed as a polynomial, correspond exactly to the original coefficients.

Using this method, we can reduce the number of constraints to check in QuickSilver. We prove the completeness and soundness of this linear sketching method in [Lemma 3.5](#).

Lemma 3.5. *For a noise vector $e \in (\mathbb{F}_2)^m$, denote its length- ℓ_{bs} sub-vectors as $[e_0, \dots, e_{w-1}] := e$. Let $d_{\text{pack}} := \ell_{\text{bs}}^2 - 3\ell_{\text{bs}} + 1$, $\ell_{\text{pack}} := \lfloor \frac{\lambda}{d_{\text{pack}}+1} \rfloor$, $\ell_{\text{qs}} := \lceil \frac{w}{\ell_{\text{pack}}} \rceil$. For $i \in [0, \ell_{\text{qs}})$, define*

$$D'_i := \sum_{j=i \cdot \ell_{\text{pack}}}^{\min(w-1, (i+1) \cdot \ell_{\text{pack}}-1)} X^{(d_{\text{pack}}+1) \cdot i} \cdot D_{e_j}. \quad (5)$$

We have the following claims:

- **Completeness:** *If $\forall j \in [0, w)$, $\text{HW}(e_j) \leq 1$, then $\forall i \in [0, \ell_{\text{qs}})$, $D'_i = 0 \in \mathbb{F}_{2^\lambda}$.*
- **Soundness:** *If $\exists j \in [0, w)$, $\text{HW}(e_j) > 1$, then $\exists i \in [0, \ell_{\text{qs}})$, $D'_i \neq 0 \in \mathbb{F}_{2^\lambda}$.*

Proof. For completeness, [Lemma 3.4](#) shows that for all $j \in [0, w)$, $D_{e_j} = 0 \in \mathbb{F}_{2^\lambda}$. Therefore, their linear combinations, which are D'_i for $i \in [0, \ell_{\text{qs}})$, are also zeros.

For soundness, by the definition of D'_i , the coefficients of $D_{e_j}(X)$ for $j \in [i \cdot \ell_{\text{pack}}, (i+1) \cdot \ell_{\text{pack}})$ are packed into the coefficients of $D'_i(X)$ without overlapping. By [Lemma 3.4](#), if $\exists j \in [0, w)$, $\text{HW}(e_j) > 1$, then $D_{e_j} \neq 0$. By the above argument, the corresponding $D'_i \neq 0 \in \mathbb{F}_{2^\lambda}$. $\square$

3.9 Signature Size of VOLEitH

We recall the signature size formula in Equation (6), as it critically informs the parameter selection process. Some messages of the VOLEitH framework can be reconstructed by the verifier locally, so after the Fiat-Shamir transform we can omit these from the protocol transcript, saving communication. The meaning of the parameters in Equation (6) is specified in Table 2. We fix the BAVC leaf-node commitment size to 2λ bits [BBB⁺26]. Also, as using linear sketching to prove RSD only yields quadratic constraints, we have $d = 2$ in ReSolveD- α .

$$\begin{aligned} \text{Size} = & \underbrace{\ell_{\text{wit}}}_{\text{Fixing witness}} + \underbrace{(\tau - 1)(\ell_{\text{wit}} + d\lambda + B)}_{c_1, \dots, c_{\tau-1}} + \underbrace{(T_{\text{open}} + 2\tau)\lambda}_{\text{BAVC Opening}} \\ & + \underbrace{(d - 1)\lambda}_{\text{QuickSilver}} + \underbrace{\lambda + B}_{\text{SoftSpoken}} + \underbrace{\lambda}_{\text{chall}_3} + \underbrace{120}_{\text{iv}} + \underbrace{32}_{\text{Counter}} \end{aligned} \quad (6)$$

4 The ReSolveD- α Signature Scheme

In this section, we specify the ReSolveD- α post-quantum digital signature scheme. We present the algorithms in a top-down manner by first defining the top-level APIs and then specifying its building blocks. As discussed in Section 2, the principle component of ReSolveD- α is an efficient zero-knowledge proof of the RSD problem in the VOLEitH framework. ReSolveD- α consists of following components.

- Section 4.2 describes the top-level APIs, including the [ReSolveD- \$\alpha\$.KeyGen](#), [ReSolveD- \$\alpha\$.Sign](#), and [ReSolveD- \$\alpha\$.Verify](#) algorithms. They offer the overall signature functionality.
- Section 4.3 describes algorithms related to vector oblivious linear evaluation relations, including [VOLE.Commit](#), [VOLE.Reconstruct](#), [VOLE.ConvertToVOLE](#), [VOLE.Fix](#), [VOLE.ChalDec](#), and [VOLE.ChalDecAll](#). They serve to generate IT-MACs (i.e., VOLE correlations) as well as providing a de-randomization process of IT-MACs.
- Section 4.4 describes Batch All-but-one Vector Commitment, including the [BAVC.Commit](#), [BAVC.Open](#), [BAVC.Verify](#), [BAVC.Interleave](#), [BAVC.Deinterleave](#), [BAVC.MinSpan](#), and [LeafHash](#) algorithms. They provide the vector commitment functionality for use of the VOLE component.
- Section 4.5 describes procedures related to RSD evaluation, including the [RSD.SampleMatrix](#), [RSD.SampleNoise](#), [RSD.EncodeNoise](#), and [RSD.DecodeNoise](#) algorithms. They serve to provide plaintext evaluation functionality of the RSD one-way function, as well as the encoding/decoding process related to the RSD one-way function for use in conjunction with the QuickSilver proof system.
- Section 4.6 describes the QuickSilver proof system, including the [QS.RSDProve](#) and [QS.RSDVerify](#) algorithms. They utilize the QuickSilver proof system and the linear sketching technique to prove that the IT-MAC-committed RSD noise vector satisfies regularity.
- Section 4.7 describes linear universal hash functions, including the [VOLEHash](#) and [ZKHash](#) algorithms. They are two linear universal hash functions that correspond to the SoftSpoken VOLE check and QuickSilver batch quadratic relation check, respectively.
- Section 4.8 describes utility functions, including the [BitsToInt](#), [IntToBits](#), [FieldToBits](#), and [BitsToField](#) algorithms. They offer data-representation conversion utilities.
- Section 4.9 describes symmetric primitives, including the fundamental symmetric-key algorithms [Enc](#) and [XOF](#), as well as the pseudorandom generator [PRG](#), the tweakable circular

correlation robust hash function [TCCR](#), and the domain-separated hash functions [MsgHash](#), [CoinHash](#), [BAVCHash](#), [MatHash](#), [FSHash₁](#), [FSHash₂](#), and [FSHash₃](#).

We begin with a detailed specification of the parameter structure in [ReSolveD- \$\alpha\$](#) , followed by the pseudocode of the eight components, and conclude with selected parameter sets of [ReSolveD- \$\alpha\$](#) . Unless stated otherwise, vectors and matrices that participate in linear-algebra operations are written over $\mathbb{F}_2$. Bitstrings that are sampled uniformly or returned by [XOF](#) or [PRG](#) are written over $\{0, 1\}$. When an object over $\mathbb{F}_2$ is used as input to a hash function or serialized into a public key or signature, we identify $\mathbb{F}_2 = \{0, 1\}$ and serialize its coordinates in the natural order.

4.1 Parameter Structure

We define the parameters of [ReSolveD- \$\alpha\$](#) in [Table 2](#). For $\text{param}_{\text{BAVC}}$, we have $k_1 = k_0 + 1$, and for $\alpha \in [0, \tau)$, if $\alpha < t_1$, then $n_\alpha = k_1$ and $N_\alpha = 2^{k_1}$; otherwise, $n_\alpha = k_0$ and $N_\alpha = 2^{k_0}$. The total number of leaves in the BAVC tree is defined to be $L = \sum_{\alpha \in [0, \tau)} N_\alpha$. For $\text{param}_{\text{RSD}}$, it holds that $m = \ell_{\text{bs}} \cdot w$ and $\ell_{\text{bs}} \mid m$. For $\text{param}_{\text{VOLE}}$, we fix $B = 16$ following the choice of [FAEST](#). Under our RSD noise encoding method, we have $\ell_{\text{wit}} = k \cdot (1 - \frac{1}{\ell_{\text{bs}}})$.

Table 2: Parameters in [ReSolveD- \$\alpha\$](#)

Group	Parameter	Description
param	λ	Security parameter
	$\text{param}_{\text{RSD}}$	RSD parameter structure
	$\text{param}_{\text{BAVC}}$	BAVC parameter structure
	$\text{param}_{\text{VOLE}}$	VOLE parameter structure
$\text{param}_{\text{BAVC}}$	τ	Number of VOLE instances
	k_0	Dimension of the first small-field type
	t_0	Number of instances of the first type
	k_1	Dimension of the second small-field type
	t_1	Number of instances of the second type
	$(N_\alpha)_{\alpha \in [0, \tau)}$	Number of leaves in each VOLE instance's binary tree
	$(n_\alpha)_{\alpha \in [0, \tau)}$	Dimension of each VOLE instance
	L	Total number of leaves in the BAVC tree
$\text{param}_{\text{RSD}}$	T_{open}	Threshold for the number of opened BAVC nodes
	m	RSD code length
	k	RSD code dimension
	w	RSD noise weight
$\text{param}_{\text{VOLE}}$	ℓ_{bs}	RSD block size
	ℓ_{wit}	Encoded length of the one-way function input
	ℓ_{vole}	VOLE commitment length
	B	Security redundancy constant for the universal hash function, fixed to be 16
	w_{grind}	Proof-of-work level

4.2 Top-Level APIs

This subsection presents the public interface of [ReSolveD- \$\alpha\$](#) . It contains the following algorithms.

- [ReSolveD- \$\alpha\$.KeyGen](#): generates an RSD public instance and the corresponding signing secret.
- [ReSolveD- \$\alpha\$.Sign](#): produces a Fiat-Shamir signature proving knowledge of a valid RSD witness.
- [ReSolveD- \$\alpha\$.Verify](#): reconstructs the public transcript and checks the signature.

4.2.1 Key Generation

The key-generation algorithm takes a parameter set **param** as input, which fixes the RSD dimensions and the security parameter λ . It samples two independent λ -bit seeds. The first seed, seed_{pk} , is public and determines the public RSD matrix through [RSD.SampleMatrix](#). The second seed, seed_{sk} , remains secret and determines the regular noise vector through [RSD.SampleNoise](#).

After expanding the two seeds, the algorithm computes the public syndrome $\mathbf{y} := \mathbf{H} \cdot \mathbf{e}$. The public key is $\text{pk} = (\text{seed}_{\text{pk}}, \mathbf{y})$, so the verifier can reconstruct the same RSD matrix but not the witness. The secret key is $\text{sk} = (\text{seed}_{\text{pk}}, \text{seed}_{\text{sk}})$, which lets the signer reconstruct both the public matrix and the regular noise when signing.

[ReSolveD- \$\alpha\$.KeyGen](#)(**param**)

Input: (**param**, **param**_{RSD})

Output: $\left(\begin{array}{l} \text{pk} := (\text{seed}_{\text{pk}} \in \{0, 1\}^\lambda, \mathbf{y} \in \mathbb{F}_2^{m-k}) \\ \text{sk} := (\text{seed}_{\text{pk}} \in \{0, 1\}^\lambda, \text{seed}_{\text{sk}} \in \{0, 1\}^\lambda) \end{array} \right)$

- 1: $\text{seed}_{\text{pk}} \leftarrow_{\$} \{0, 1\}^\lambda$
- 2: $\text{seed}_{\text{sk}} \leftarrow_{\$} \{0, 1\}^\lambda$
- 3: $\mathbf{H} := \text{RSD.SampleMatrix}(\text{seed}_{\text{pk}})$
- 4: $\mathbf{e} := \text{RSD.SampleNoise}(\text{seed}_{\text{sk}})$
- 5: $\mathbf{y} := \mathbf{H} \cdot \mathbf{e}$
- 6: $\text{pk} := (\text{seed}_{\text{pk}}, \mathbf{y})$
- 7: $\text{sk} := (\text{seed}_{\text{pk}}, \text{seed}_{\text{sk}})$
- 8: **return** (**pk**, **sk**)

4.2.2 Signing

The signing algorithm takes a secret key $\text{sk} = (\text{seed}_{\text{pk}}, \text{seed}_{\text{sk}})$ and a message **msg**. It first reconstructs the public RSD matrix from seed_{pk} and the regular noise vector from seed_{sk} . This also reconstructs the public key $\text{pk} = (\text{seed}_{\text{pk}}, \mathbf{y})$, which is hashed together with the message by [MsgHash](#) to obtain the digest μ and the salt $\mathbf{s}$ used by [VOLE.Commit](#).

The signing procedure then runs in four phases, following the VOLEitH and QuickSilver transcript structure. In the first phase, the signer samples signature randomness ρ (or sets $\rho = 0^\lambda$ in deterministic signing) and derives the BAVC root seed $\mathbf{r}$ and IV iv using [CoinHash](#). It calls [VOLE.Commit](#) to create the BAVC commitment **com**, the decommitment state **decom**, correction vectors $(\mathbf{c}_i)_{i \in [1, \tau]}$, and prover-side VOLE values $(\mathbf{u}, \mathbf{V})$. The first Fiat-Shamir challenge chall_1 is then derived from the commitment and correction vectors.

In the second phase, the signer performs the SoftSpokenOT-style VOLE consistency check and commits to the RSD witness. In particular, the signer uses a linear universal hash function [VOLEHash](#) to compress the generated IT-MACs $\llbracket \mathbf{u} \rrbracket^1$ into $\llbracket \tilde{\mathbf{u}} \rrbracket^1$ (algorithmically, the signer hashes $\mathbf{u}$ and its matrix-form tag $\mathbf{V}$ with [VOLEHash](#)), encodes the regular noise as $\mathbf{w} := \text{RSD.EncodeNoise}(\mathbf{e})$, and masks this witness as $\mathbf{d} := \mathbf{w} \oplus \mathbf{u}[0, \ell_{\text{wit}}]$. The second challenge chall_2 is obtained by hashing chall_1 , the opening of $\llbracket \tilde{\mathbf{u}} \rrbracket^1$ (namely, $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{V}}$), and the masked witness.

In the third phase, the signer restricts $\mathbf{u}$ and $\mathbf{V}$ to the rows needed by the QuickSilver check and invokes [QS.RSDProve](#). This produces $(\tilde{\mathbf{a}}_1, \tilde{\mathbf{a}}_0)$, which are the batched QuickSilver values proving that the committed witness encodes a regular RSD noise vector satisfying the public syndrome equation.

In the final phase, the signer derives the last challenge chall_3 by hashing the QuickSilver response and a counter **ctr**. The counter is increased until two conditions hold: the last w_{grind} bits of chall_3 are zero, and the BAVC opening for the decoded challenge indices fits within the threshold T_{open} . The signature consists of the correction vectors, the compressed VOLE value

$\tilde{\mathbf{u}}$, the masked witness $\mathbf{d}$, the QuickSilver value $\tilde{a}_1$, the partial BAVC opening decom_I , the final challenge, the IV, and the counter.

ReSolveD- α .Sign(sk, msg; param)

Input: $\left(\begin{array}{l} \text{sk} := (\text{seed}_{\text{pk}} \in \{0, 1\}^\lambda, \text{seed}_{\text{sk}} \in \{0, 1\}^\lambda), \\ \text{msg} \in \{0, 1\}^*; \text{param} \end{array} \right)$

Output: $\sigma := \left(\begin{array}{l} ((c_i)_{i \in [1, \tau]} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \tilde{\mathbf{u}} \in \{0, 1\}^{\lambda+B}, \mathbf{d} \in \mathbb{F}_2^{\ell_{\text{wit}}}), \\ \tilde{a}_1 \in \{0, 1\}^\lambda, \text{decom}_I \in \{0, 1\}^{(T_{\text{open}}+2\tau) \cdot \lambda}, \\ \text{chall}_3 \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}, \text{ctr} \in \{0, 1\}^{32} \end{array} \right)$

- 1: Extract $\lambda, \ell_{\text{wit}}, \tau, B, T_{\text{open}}$ from param
- 2: $\mathbf{H} := \text{RSD.SampleMatrix}(\text{seed}_{\text{pk}}; \text{param})$
- 3: $\mathbf{e} := \text{RSD.SampleNoise}(\text{seed}_{\text{sk}}; \text{param})$
- 4: $\mathbf{y} := \mathbf{H} \cdot \mathbf{e}$
- 5: $\text{pk} := (\text{seed}_{\text{pk}}, \mathbf{y})$
- 6: $(\mu, \mathbf{s}) := \text{MsgHash}(\text{pk}, \text{msg})$
- 7: $\rho \leftarrow_{\$} \{0, 1\}^\lambda$ ▷ For deterministic signing, set $\rho = 0^\lambda$
- 8: $(\mathbf{r}, \text{iv}) := \text{CoinHash}(\text{sk}, \mu, \rho)$
- 9: $(\text{com}, \text{decom}, (c_i)_{i \in [1, \tau]}, \mathbf{u}, \mathbf{V}) := \text{VOLE.Commit}(\mathbf{r}, \text{iv}, \mu, \mathbf{s}; \text{param})$
- 10: $\mathbf{V} := [\mathbf{V}, 0^{\ell_{\text{vole}}}, \dots, 0^{\ell_{\text{vole}}}]$ ▷ Extend $\mathbf{V}$ from $\lambda - w_{\text{grind}}$ to λ columns.
- 11: $\text{chall}_1 := \text{FSHash}_1(\text{com}, (c_i)_{i \in [1, \tau]})$
- 12: $\tilde{\mathbf{u}} := \text{VOLEHash}(\text{chall}_1, \mathbf{u})$
- 13: $\tilde{\mathbf{V}} := \text{VOLEHash}(\text{chall}_1, \mathbf{V})$
- 14: $\mathbf{w} := \text{RSD.EncodeNoise}(\mathbf{e}; \text{param})$
- 15: $\mathbf{d} := \mathbf{w} \oplus \mathbf{u}[0, \ell_{\text{wit}})$
- 16: $\text{chall}_2 := \text{FSHash}_2(\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$
- 17: $\mathbf{u} := \mathbf{u}[0, \ell_{\text{wit}} + \lambda)$
- 18: $\mathbf{V} := \mathbf{V}[0, \ell_{\text{wit}} + \lambda)$
- 19: $(\tilde{a}_1, \tilde{a}_0) := \text{QS.RSDProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \mathbf{H}, \mathbf{y}, \text{chall}_2; \text{param})$
- 20: $\text{ctr} := 0$
- 21: $\text{decom}_I := \perp$
- 22: **while** $\text{decom}_I = \perp$ **do**
- 23: $\text{chall}_3 := \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$
- 24: **if** $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda) \neq 0^{w_{\text{grind}}}$ **then**
- 25: $\text{ctr} := \text{ctr} + 1$
- 26: **continue**
- 27: **end if**
- 28: $I := \text{VOLE.ChalDecAll}(\text{chall}_3; \text{param})$
- 29: $\text{decom}_I := \text{BAVC.Open}(\text{decom}, I, \text{iv}; \text{param})$
- 30: **end while**
- 31: **return** $\sigma := ((c_i)_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \text{decom}_I, \text{chall}_3, \text{iv}, \text{ctr})$

Deterministic vs. randomized signing. We explicitly list the master randomness seed ρ in the signing algorithm **ReSolveD- α .Sign**. So deterministic signing can be achieved by setting it to all zeros.

4.2.3 Verification

The verification algorithm takes a public key $\text{pk} = (\text{seed}_{\text{pk}}, \mathbf{y})$, a message msg , and a signature σ . It parses the signature into the correction vectors, compressed VOLE value, masked witness,

QuickSilver response value, BAVC opening, final challenge, IV, and grinding counter. The verifier recomputes the message digest μ and salt $\mathbf{s}$ from $\mathbf{pk}$ and $\mathbf{msg}$, so the same public key is bound to the transcript checked by the verifier.

The verifier first reconstructs its IT-MAC/VOLE local keys. Using chall_3 , the partial opening decom_I , the correction vectors, and the IV, it calls **VOLE.Reconstruct** to recover the commitment com and the matrix $\mathbf{Q}$, where each row corresponds to a local key. It then recomputes chall_1 from com and the correction vectors. As in signing, it hashes the VOLE local keys with **VOLEHash**. To verify the opening of $[\tilde{u}]^1$, the verifier applies **VOLE.Fix** with $\tilde{u}$ and the final challenge as the global IT-MAC key. For a correct $\tilde{u}$ value, this reconstructs the verifier's local key to a degree-1 authentication of zero, making the fixed $\tilde{\mathbf{Q}}$ identical to the signer's $\tilde{\mathbf{V}}$.

Next, the verifier recomputes chall_2 from chall_1 , $\tilde{u}$, $\tilde{\mathbf{Q}}$, and the masked witness $\mathbf{d}$. It truncates $\mathbf{Q}$ to the rows needed by the RSD QuickSilver check and calls **QS.RSDVerify**. This routine reconstructs the missing value $\tilde{a}_0$ from the verifier-side IT-MAC keys, the masked witness, the public RSD instance, and chall_3 .

Finally, the verifier recomputes $\text{chall}'_3 := \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$. It accepts if and only if chall'_3 equals the supplied chall_3 and chall_3 satisfies the proof-of-work suffix condition.

ReSolveD- α .Verify($\mathbf{pk}, \mathbf{msg}, \sigma; \text{param}$)

Input: $\left(\begin{array}{l} \mathbf{pk} := (\text{seed}_{\mathbf{pk}} \in \{0, 1\}^\lambda, \mathbf{y} \in \mathbb{F}_2^{m-k}), \\ \mathbf{msg} \in \{0, 1\}^*, \\ \sigma := \left(\begin{array}{l} (\mathbf{c}_i)_{i \in [1, \tau]} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \tilde{\mathbf{u}} \in \{0, 1\}^{\lambda+B}, \mathbf{d} \in \mathbb{F}_2^{\ell_{\text{wit}}}, \\ \tilde{a}_1 \in \{0, 1\}^\lambda, \text{decom}_I \in \{0, 1\}^{(T_{\text{open}}+2\tau) \cdot \lambda}, \\ \text{chall}_3 \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}, \text{ctr} \in \{0, 1\}^{32} \end{array} \right) \end{array} \right)$

Output: $b \in \{0, 1\}$ $\triangleright 1 = \text{accept}, 0 = \text{reject}.$

- 1: Extract $\lambda, \tau, \ell_{\text{wit}}, B$ from param
- 2: $(\mu, \mathbf{s}) := \text{MsgHash}(\mathbf{pk}, \mathbf{msg})$
- 3: $(\text{com}, \mathbf{Q}) := \text{VOLE.Reconstruct}(\text{chall}_3, \text{decom}_I, (\mathbf{c}_i)_{i \in [1, \tau]}, \text{iv}, \mu, \mathbf{s}; \text{param})$
- 4: $\mathbf{Q} := [\mathbf{Q}, 0^{\ell_{\text{vole}}}, \dots, 0^{\ell_{\text{vole}}}]$ $\triangleright$ Extend $\mathbf{Q}$ from $\lambda - w_{\text{grind}}$ to λ columns.
- 5: $\text{chall}_1 := \text{FSHash}_1(\text{com}, (\mathbf{c}_i)_{i \in [1, \tau]})$
- 6: $\mathbf{Q}' := \text{VOLEHash}(\text{chall}_1, \mathbf{Q})$
- 7: $\tilde{\mathbf{Q}} := \text{VOLE.Fix}(\mathbf{Q}', \tilde{\mathbf{u}}, \text{chall}_3; \lambda, \lambda + B)$
- 8: $\text{chall}_2 := \text{FSHash}_2(\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{Q}}, \mathbf{d})$
- 9: $\mathbf{Q} := \begin{bmatrix} \mathbf{Q}[0, :] \\ \vdots \\ \mathbf{Q}[\ell_{\text{wit}} + \lambda - 1, :] \end{bmatrix}$ $\triangleright$ Take the first $\ell_{\text{wit}} + \lambda$ rows of $\mathbf{Q}$.
- 10: $\tilde{a}_0 := \text{QS.RSDVerify}(\mathbf{d}, \mathbf{Q}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \mathbf{H}, \mathbf{y}; \text{param})$
- 11: $\text{chall}'_3 := \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$
- 12: **if** $\text{chall}_3 = \text{chall}'_3 \wedge \text{chall}_3[\lambda - w_{\text{grind}}, \lambda) = 0^{w_{\text{grind}}}$ **then**
- 13: **return** 1
- 14: **else**
- 15: **return** 0
- 16: **end if**

4.3 Vector Oblivious Linear Evaluation

This subsection specifies the generation of VOLE correlations over $\mathbb{F}_{2^\lambda}$ in VOLEitH. The external interfaces include **VOLE.Commit**, **VOLE.Reconstruct**, which generate IT-MAC authentication materials of the prover and verifier respectively, and **VOLE.Fix**, which computes the verifier's

IT-MAC local key during the IT-MAC de-randomization process. Conceptually, the algorithms first commit to τ small VOLE instances through BAVC, then convert the committed seeds into small-field VOLE correlations, and finally use correction vectors to align all small instances so that the committed messages become identical, making a VOLE correlation over $\mathbb{F}_{2^\lambda}$. The remaining procedures in this subsection are internal. It contains the following algorithms.

- **VOLE.ChalDec** and **VOLE.ChalDecAll**: decode the final Fiat-Shamir challenge into one hidden leaf index per VOLE instance.
- **VOLE.Commit**: commits to seed trees, converts their leaves into prover-side share of the VOLE correlations (i.e., committed random messages and tags), and outputs correction vectors.
- **VOLE.Reconstruct**: verifies a partial BAVC opening and reconstructs the verifier-side VOLE local keys.
- **VOLE.Fix**: applies the correction vector to the IT-MAC local key.
- **VOLE.ConvertToVOLE**: converts a complete binary tree (resp. a binary tree with all-but-one leaf node) into prover-side (resp. verifier-side) VOLE correlations.

4.3.1 VOLE Challenge Decoding

The challenge decoding algorithm extracts the part of the final Fiat-Shamir challenge chall_3 assigned to one VOLE instance. The decoded value determines the leaf that remains hidden in the corresponding all-but-one vector commitment opening. Due to the proof-of-work condition, the last w_{grind} bits of chall_3 are fixed to zero and the remaining challenge bits are divided among the τ VOLE instances.

Because the BAVC tree depths can differ by one, not every instance receives the same number of challenge bits. For the first t_1 instances, **VOLE.ChalDec** reads a k_1 -bit substring and interprets it as an integer in $[0, 2^{k_1})$. For the remaining instances, it reads a k_0 -bit substring and obtains an integer in $[0, 2^{k_0})$. The all-instance decoder simply applies this rule to every instance and returns $I = (\Delta_0, \dots, \Delta_{\tau-1})$.

VOLE.ChalDec($\text{chall}_3, i; \text{param}$)

Input: ($\text{chall}_3 \in \{0, 1\}^\lambda, i \in [0, \tau)$) Output: $\delta \in \{0, 1\}^{k_0} \cup \{0, 1\}^{k_1}$ 1: Extract k_0, t_0, k_1, t_1 from param 2: if $i < t_1$ then 3: $\text{low} := i \cdot k_1$ 4: $\text{high} := (i + 1) \cdot k_1$ 5: $\Delta := \text{BitsToInt}(\text{chall}_3[\text{low}, \text{high}]; k_1)$ 6: else 7: $\text{low} := t_1 \cdot k_1 + (i - t_1) \cdot k_0$ 8: $\text{high} := t_1 \cdot k_1 + (i + 1 - t_1) \cdot k_0$ 9: $\Delta := \text{BitsToInt}(\text{chall}_3[\text{low}, \text{high}]; k_0)$ 10: end if 11: return Δ	▷ Due to PoW, the last w_{grind} bits of chall_3 are 0. ▷ Output is k_1 bits if $i < t_1$, otherwise k_0 bits.
---	--

4.3.2 Decoding All VOLE Challenges

The all-instance decoding algorithm applies **VOLE.ChalDec** to each of the τ VOLE instances. The resulting vector I specifies the leaves that remain hidden in the BAVC opening. Equivalently, each Δ_α corresponds to the verifier's VOLE global key for the α -th small VOLE instance.

VOLE.ChalDecAll(chall₃; param)

Input: (chall₃ ∈ {0, 1}^λ)

Output: $I := (\Delta_0, \dots, \Delta_{\tau-1}) \in [0, N_0) \times \dots \times [0, N_{\tau-1})$

- 1: Extract τ from param
- 2: **for** $\alpha \in [0, \tau)$ **do**
- 3: $\Delta_\alpha := \text{VOLE.ChalDec}(\text{chall}_3, \alpha; \text{param})$
- 4: **end for**
- 5: $I := (\Delta_0, \dots, \Delta_{\tau-1})$
- 6: **return** I

4.3.3 VOLE Commitment

The VOLE commitment algorithm is the prover-side interface of VOLE correlation generation in VOLEitH. It takes the root seed $\mathbf{r}$, the IV iv , and the hashed message μ and salt $\mathbf{s}$. It first invokes **BAVC.Commit**, which expands $\mathbf{r}$ into the large pcGGM tree and returns a global commitment com , a decommitment state decom , and the per-leaf seeds for all τ logical VOLE instances.

For each instance α , the algorithm calls **VOLE.ConvertToVOLE** on the seeds of that instance. This produces a secret committed vector $\mathbf{u}_\alpha$ and its authentication tag in matrix form $\mathbf{V}_\alpha$. In this form, the columns $\mathbf{V}_\alpha$ correspond to the bits of the small-field challenge for that instance. The matrices $\mathbf{V}_0, \dots, \mathbf{V}_{\tau-1}$ are concatenated to form the prover-side tag matrix $\mathbf{V}$ used by the later VOLE consistency check and QuickSilver proof.

Initially, the different small VOLE instances have independent committed vectors. To aggregate the small VOLE instances into VOLE over $\mathbb{F}_{2^\lambda}$, the algorithm designates $\mathbf{u}_0$ as the shared vector $\mathbf{u}$. For every $\alpha \geq 1$, it publishes the correction vector $\mathbf{c}_\alpha := \mathbf{u} \oplus \mathbf{u}_\alpha$. These correction vectors allow the verifier, after seeing the challenge-selected hidden leaves, to shift the receiver-side local keys so that all reconstructed columns are consistent with the same vector $\mathbf{u}$.

VOLE.Commit($\mathbf{r}, \text{iv}, \mu, \mathbf{s}; \text{param}$)

Input: ($\mathbf{r} \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda$)

Output: $\left(\text{com} \in \{0, 1\}^{2\lambda}, \text{decom} \in \{0, 1\}^\lambda, \right. \\ \left. (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \mathbf{u} \in \mathbb{F}_2^{\ell_{\text{vole}}}, \mathbf{V} \in \mathbb{F}_2^{\ell_{\text{vole}} \times \lambda} \right)$

- 1: Extract $\lambda, \ell_{\text{vole}}, \tau, (N_\alpha)_{\alpha \in [0, \tau)}$ from param
- 2: ($\text{com}, \text{decom}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \in N_\alpha}$) := **BAVC.Commit**($\mathbf{r}, \text{iv}, \mu, \mathbf{s}; \text{param}$)
- 3: **for** $\alpha \in [0, \tau)$ **do**
- 4: $(\mathbf{u}_\alpha, \mathbf{V}_\alpha) := \text{VOLE.ConvertToVOLE}((\text{seed}_i^\alpha)_{i \in N_\alpha}, \text{iv}; \text{param})$
- 5: **end for**
- 6: $\mathbf{V} := [\mathbf{V}_0, \dots, \mathbf{V}_{\tau-1}]$
- 7: $\mathbf{u} := \mathbf{u}_0$
- 8: **for** $\alpha \in [1, \tau)$ **do**
- 9: $\mathbf{c}_\alpha := \mathbf{u} \oplus \mathbf{u}_\alpha$
- 10: **end for**
- 11: **return** ($\text{com}, \text{decom}, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)}, \mathbf{u}, \mathbf{V}$)

4.3.4 VOLE Reconstruction

The VOLE reconstruction algorithm is the verifier-side counterpart of [VOLE.Commit](#). It takes the final challenge chall_3 , the partial BAVC opening decom_I , the correction vectors, the IV, and the hashed message μ and salt s . It first decodes chall_3 into $I = (\Delta_0, \dots, \Delta_{\tau-1})$. The value Δ_α identifies the unopened leaf in the α -th logical VOLE instance.

The algorithm then calls [BAVC.Verify](#) on the partial opening. If the opening is invalid, too large, or inconsistent with the leaf commitments, reconstruction returns $\perp$. Otherwise, [BAVC.Verify](#) returns the commitment com and all leaf seeds except the hidden seed at index Δ_α for each instance. This recovered commitment is later hashed again in the top-level verifier, which allows the verifier to produce the same transcript as the signer.

For each α , the verifier re-indexes the opened seeds by XOR-ing their indices with Δ_α . This moves the hidden seed to position 0. The verifier represents that seed by $\perp$ and converts the partially hidden list with [VOLE.ConvertToVOLE](#). The result is an uncorrected local-key matrix Q'_α . The bit decomposition of Δ_α acts as the small instance VOLE global key Δ_α . Applying [VOLE.Fix](#) with Δ_α and the published correction vector shifts the columns of Q'_α to match the unified committed message u_0 chosen by the prover. Finally, the matrices for all τ instances are concatenated into the verifier-side matrix Q .

[VOLE.Reconstruct](#)($\text{chall}_3, \text{decom}_I, (c_\alpha)_{\alpha \in [1, \tau]}, \text{iv}, \mu, s; \text{param}$)

Input: $\left(\text{chall}_3 \in \{0, 1\}^\lambda, \text{decom}_I \in \{0, 1\}^{(T_{\text{open}} + 2\tau)\lambda}, \right.$
 $\left. (c_\alpha)_{\alpha \in [1, \tau]} \in (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau-1}, \text{iv} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, s \in \{0, 1\}^\lambda \right)$

Output: $(\text{com}, Q) \in \{0, 1\}^{2\lambda} \times \mathbb{F}_2^{\ell_{\text{vole}} \times \lambda}$

```

1: Extract  $\lambda, \ell_{\text{vole}}, \tau, (n_\alpha)_{\alpha \in [0, \tau)}, (N_\alpha)_{\alpha \in [0, \tau)}, L, T_{\text{open}}$  from param
2:  $I := \text{VOLE.ChalDecAll}(\text{chall}_3; \text{param})$ 
3: Parse  $I = (\Delta_0, \dots, \Delta_{\tau-1})$ 
4:  $\text{out} := \text{BAVC.Verify}(I, \text{iv}, \mu, s, \text{decom}_I; \text{param})$ 
5: if  $\text{out} = \perp$  then
6:   return  $\perp$ 
7: else
8:   Parse  $\text{out} := (\text{com}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \neq \Delta_\alpha})$ 
9: end if
10: for  $\alpha \in [0, \tau)$  do
11:    $\text{seed}_0^\alpha := \perp$ 
12:   for  $i \in [1, N_\alpha)$  do
13:      $i' := \text{BitsToInt}(\text{IntToBits}(i; n_\alpha) \oplus \text{IntToBits}(\Delta_\alpha; n_\alpha); n_\alpha)$ 
14:      $\text{seed}_i^\alpha := \text{seed}_{i'}^\alpha$ 
15:   end for
16:    $(u'_\alpha, Q'_\alpha) := \text{VOLE.ConvertToVOLE}((\text{seed}_i^\alpha)_{i \in N_\alpha}, \text{iv}, \ell_{\text{vole}})$ 
17:    $Q_\alpha := \text{VOLE.Fix}(Q'_\alpha, c_\alpha, \Delta_\alpha; n_\alpha, \ell_{\text{vole}})$ 
18: end for
19:  $Q := [Q_0, \dots, Q_{\tau-1}]$ 
20: return  $(\text{com}, Q)$ 

```

4.3.5 VOLE Correction

The correction algorithm turns an uncorrected verifier-side matrix Q' into the corrected local-key matrix Q . It is a linear post-processing step. For every challenge bit $\Delta[i]$, the algorithm either leaves the corresponding column unchanged or XORs the correction vector c into it. Thus, if the α -th small VOLE instance was generated with committed vector u_α , the correction

vector $\mathbf{c}_\alpha = \mathbf{u}_0 \oplus \mathbf{u}_\alpha$ changes the verifier's local keys from authenticating $\mathbf{u}_\alpha$ into authenticating $\mathbf{u}_0$. The same operation is also used later by the top-level verifier when deriving the prover's IT-MAC tag for the compressed VOLE consistency checking value. This is done by correcting the verifier's local key with the compressed VOLE consistency checking value itself, essentially performing an IT-MAC opening check.

VOLE.Fix($\mathbf{Q}', \mathbf{c}, \Delta; k, \ell$)

Input: $(\mathbf{Q}' \in \mathbb{F}_2^{\ell \times k}, \mathbf{c} \in \mathbb{F}_2^\ell, \Delta \in \{0, 1\}^k)$

Output: $\mathbf{Q} \in \mathbb{F}_2^{\ell \times k}$

```

1: for  $i \in [0, k)$  do
2:    $\mathbf{q}_i := \mathbf{Q}'[:, i] \oplus \Delta[i] \cdot \mathbf{c}$   $\triangleright \Delta[i] \in \{0, 1\}$ .
3: end for
4:  $\mathbf{Q} := [\mathbf{q}_0, \dots, \mathbf{q}_{k-1}]$ 
5: return  $\mathbf{Q}$ 

```

4.3.6 Seed Conversion to VOLE

This conversion algorithm expands the leaves of a binary seed tree into VOLE correlations. It is used in two modes. During **VOLE.Commit**, all leaf seeds are known to the signer and the algorithm outputs the prover-side vector $\mathbf{u}$ and its authentication tag in matrix form $\mathbf{V}$. During **VOLE.Reconstruct**, one leaf is hidden and represented as $\perp$; the algorithm treats this missing seed as the zero vector and outputs the verifier-side IT-MAC local key in matrix form before correction.

To make the connection with the high-level expression explicit, define

$$\mathbf{r}_{0,i} := \text{PRG}(\text{seed}_i, \text{iv}, \lceil 2\lambda/128 \rceil; \ell_{\text{vole}}), \quad \mathbf{b}_i := \text{IntToBits}(i; k).$$

Here, $\mathbf{b}_i$ is the little-endian binary representation of the leaf index i . For each row position $h \in [0, \ell_{\text{vole}})$, the small-field VOLE tag can be viewed as

$$v[h] := \sum_{i \in [0, N)} \text{BitsToField}(\mathbf{b}_i; k) \cdot \mathbf{r}_{0,i}[h] \in \mathbb{F}_{2^k}.$$

Equivalently, $\mathbf{V}$ stores these tags in bit-sliced form rather than as explicit $\mathbb{F}_{2^k}$ elements. Its j -th column is

$$\mathbf{V}[:, j] = \mathbf{v}_j := \bigoplus_{i \in [0, N): \mathbf{b}_i[j]=1} \mathbf{r}_{0,i},$$

so for every row h , the row vector $\mathbf{V}[h, :]$ is exactly the k -bit binary representation of the field element $v[h]$. Applying $\text{BitsToField}(\mathbf{V}[h, :]; k)$ therefore recovers the authentication tag corresponding to the h -th committed bit. Since the field has characteristic two, the sign in the overview formula $\sum_i (-i) \cdot \text{PRG}(\text{seed}_i)$ does not change this representation.

The algorithm first expands every available seed with **PRG** into a vector in $\mathbb{F}_2^{\ell_{\text{vole}}}$. It then folds the complete binary tree upward by XORing sibling vectors. At level j , the XOR of the odd children is accumulated into the column vector $\mathbf{v}_j$. This is exactly the same as XORing all expanded leaves whose j -th index bit is 1. After all levels have been processed, the remaining root value is the committed message vector $\mathbf{u} = \bigoplus_{i \in [0, N)} \mathbf{r}_{0,i}$, while the collected odd-child sums form the columns of $\mathbf{V}$. This folding procedure is the bridge between a committed seed tree and the VOLE/IT-MAC correlations consumed by the rest of the proof system.

VOLE.ConvertToVOLE $((\text{seed}_i)_{i \in [0, N]}, \text{iv}; \text{param})$

Input: $((\text{seed}_i)_{i \in [0, N]} \in \{0, 1\}^{N \cdot \lambda}, \text{iv} \in \{0, 1\}^{120})$ $\triangleright N = 2^k$.

Output: $(\mathbf{u} \in \mathbb{F}_2^{\ell_{\text{vole}}}, \mathbf{V} \in \mathbb{F}_2^{\ell_{\text{vole}} \times k})$

```

1: Extract  $\ell_{\text{vole}}$  from param
2: if  $\text{seed}_0 = \perp$  then
3:    $\mathbf{r}_{0,0} := 0^{\ell_{\text{vole}}}$ 
4: else
5:    $\mathbf{r}_{0,0} := \text{PRG}(\text{seed}_0, \text{iv}, \lceil \frac{2\lambda}{128} \rceil; \ell_{\text{vole}})$   $\triangleright$  Increased index to avoid collision with LeafHash.
6: end if
7: for  $i \in [1, N)$  do
8:    $\mathbf{r}_{0,i} := \text{PRG}(\text{seed}_i, \text{iv}, \lceil \frac{2\lambda}{128} \rceil; \ell_{\text{vole}})$   $\triangleright$  Increased index to avoid collision with LeafHash.
9: end for
10:  $k = \log_2(N)$ 
11: for  $i \in [0, k)$  do
12:    $\mathbf{v}_i := 0^{\ell_{\text{vole}}}$ 
13: end for
14: for  $j \in [0, k)$  do
15:   for  $i \in [0, \frac{N}{2^{j+1}})$  do
16:      $\mathbf{v}_j := \mathbf{v}_j \oplus \mathbf{r}_{j,2i+1}$ 
17:      $\mathbf{r}_{j+1,i} := \mathbf{r}_{j,2i} \oplus \mathbf{r}_{j,2i+1}$ 
18:   end for
19: end for
20:  $\mathbf{u} := \mathbf{r}_{k,0}$ 
21:  $\mathbf{V} := [\mathbf{v}_0, \dots, \mathbf{v}_{k-1}]$ 
22: return  $(\mathbf{u}, \mathbf{V})$ 

```

4.4 Batch All-but-one Vector Commitment

This subsection describes the batch all-but-one vector commitment used to realize the many small VOLE instances compactly. The algorithms generate a large pcGGM tree, which is conceptually τ small trees, with leaf nodes mapped via the interleaving rule. For each logical small VOLE instance α , the challenge index Δ_α derived from **chall**₃ identifies the one leaf that remains hidden. The opening then reveals enough internal nodes to reconstruct every other leaf, while also revealing the commitments to the hidden leaves so that the verifier can recompute the global BAVC commitment. This subsection contains the following algorithms.

- **BAVC.Commit**: expands a root seed into the large pcGGM tree and commits to all leaf commitments.
- **BAVC.Open**: opens all leaves except the challenge-selected leaves, subject to the threshold T_{open} .
- **BAVC.Verify**: checks a partial opening, reconstructs the opened leaf seeds, and recomputes the commitment.
- **BAVC.Interleave** and **BAVC.Deinterleave**: translate between per-instance leaf indices and leaf positions in the large pcGGM tree.
- **BAVC.MinSpan**: compresses a set of opened leaves to a minimal set of internal tree nodes.
- **LeafHash**: derives the seed used by VOLE conversion and the corresponding leaf commitment.

In the algorithms [BAVC.Commit](#) and [BAVC.Verify](#), we use two constants $\text{const}_0, \text{const}_1 \in \{0, 1\}^\lambda$ satisfying $\text{const}_0 \neq \text{const}_1$. In our implementation, we choose $\text{const}_0 = \text{IntToBits}(0; \lambda)$ and $\text{const}_1 = \text{IntToBits}(1; \lambda)$.

4.4.1 BAVC Commitment

The BAVC commitment algorithm takes a root seed $\mathbf{r}$, the IV iv , and the hashed message μ and salt $\mathbf{s}$. It expands $\mathbf{r}$ into the large pcGGM tree with L leaves. For internal nodes in the upper part of the tree, the left child is obtained by applying [TCCR](#) to the parent seed, and the right child is derived by XOR-ing the left child with the parent seed. For the bottom internal nodes, the two children are derived by hashing the parent seed XOR-ed with the two fixed constants const_0 and const_1 . This gives a deterministic tree expansion from the root seed, the salt, and the IV.

Each leaf index $\text{id} \in [L, 2L)$ in the large pcGGM tree is mapped back to a small VOLE instance α and a leaf index i_α in its corresponding small tree by [BAVC.Deinterleave](#). The leaf value is then processed by [LeafHash](#), which outputs the seed $\text{seed}_{i_\alpha}^\alpha$ used later by [VOLE.ConvertToVOLE](#) and the corresponding leaf commitment $\text{com}_{i_\alpha}^\alpha$. After all leaves are processed, [BAVCHash](#) compresses all leaf commitments together with μ , $\mathbf{s}$, and iv into the global commitment com . The decommitment state is the root seed $\text{decom} := \mathbf{r}_1$, which allows the signer to reproduce the whole tree when opening.

[BAVC.Commit](#)($\mathbf{r}, \text{iv}, \mu, \mathbf{s}; \text{param}$)

Input: ($\mathbf{r} \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda$)

Output: $\left(\text{com} \in \{0, 1\}^{2\lambda}, \text{decom} \in \{0, 1\}^\lambda, \left(\text{seed}_i^\alpha \right)_{\alpha \in [0, \tau), i \in N_\alpha} \in (\{0, 1\}^\lambda)^{N_0} \times \dots \times (\{0, 1\}^\lambda)^{N_{\tau-1}} \right)$

```

1: Extract  $\lambda, \tau, (N_\alpha)_{\alpha \in [0, \tau)}, L$  from  $\text{param}$ 
2:  $\mathbf{r}_1 := \mathbf{r}$ 
3: for  $\text{id} \in [1, L/2)$  do
4:    $\mathbf{r}_{2:\text{id}} := \text{TCCR}(\mathbf{r}_{\text{id}}, \mathbf{s}, \text{iv})$ 
5:    $\mathbf{r}_{2:\text{id}+1} := \mathbf{r}_{2:\text{id}} \oplus \mathbf{r}_{\text{id}}$ 
6: end for
7: for  $\text{id} \in [L/2, L)$  do
8:    $\mathbf{r}_{2:\text{id}} := \text{TCCR}(\mathbf{r}_{\text{id}} \oplus \text{const}_0, \mathbf{s}, \text{iv})$ 
9:    $\mathbf{r}_{2:\text{id}+1} := \text{TCCR}(\mathbf{r}_{\text{id}} \oplus \text{const}_1, \mathbf{s}, \text{iv})$ 
10: end for
11: for  $\text{id} \in [L, 2L)$  do
12:    $(\alpha, i_\alpha) := \text{BAVC.Deinterleave}(\text{id}; \text{param})$ 
13:    $(\text{seed}_{i_\alpha}^\alpha, \text{com}_{i_\alpha}^\alpha) := \text{LeafHash}(\mathbf{r}_{\text{id}}, \text{iv})$ 
14: end for
15:  $\text{com} := \text{BAVCHash}(\mu, \mathbf{s}, (\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, \text{iv})$ 
16:  $\text{decom} := \mathbf{r}_1$ 
17: return ( $\text{com}, \text{decom}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \in N_\alpha}$ )

```

4.4.2 BAVC Opening

The BAVC opening algorithm takes the root decommitment and a challenge vector I . It recomputes the pcGGM tree from the root seed and parses $I = (\Delta_0, \dots, \Delta_{\tau-1})$. For small VOLE instance α , the leaf Δ_α is the one that must remain hidden in its corresponding small tree. The algorithm maps each pair (α, Δ_α) into its position id_α in the large pcGGM tree with [BAVC.Interleave](#) and stores the commitment to that hidden leaf in the opening.

All other leaves in the large pcGGM tree are considered opened. Rather than revealing every opened leaf seed individually, the algorithm calls **BAVC.MinSpan** on the set of opened leaves. This replaces sibling pairs by their parent whenever possible and produces a compact set of internal node seeds from which the verifier can re-expand all opened leaves. If this compact node set is larger than T_{open} , the algorithm returns $\perp$. This abort is handled by the signing algorithm via grinding (deriving a different chall_3 using an incremented counter). Otherwise, the partial opening decom_I consists of the τ hidden leaf commitments and the selected internal node seeds, padded to a fixed length.

BAVC.Open($\text{decom}, I, \text{iv}; \text{param}$)

Input: ($\text{decom} \in \{0, 1\}^\lambda, I \in [0, N_0) \times \dots \times [0, N_{\tau-1}), \text{iv} \in \{0, 1\}^{120}$)

Output: $\text{decom}_I \in \{0, 1\}^{(T_{\text{open}}+2\tau)\lambda} \cup \{\perp\}$

```

1: Extract  $\lambda, \tau, L, T_{\text{open}}, (N_\alpha)_{\alpha \in [0, \tau)}$  from param
2:  $r_1 := \text{decom}$ 
3: for  $\text{id} \in [1, L/2)$  do
4:    $r_{2:\text{id}} := \text{TCCR}(r_{\text{id}}, s, \text{iv})$ 
5:    $r_{2:\text{id}+1} := r_{2:\text{id}} \oplus r_{\text{id}}$ 
6: end for
7: for  $\text{id} \in [L/2, L)$  do
8:    $r_{2:\text{id}} := \text{TCCR}(r_{\text{id}} \oplus \text{const}_0, s, \text{iv})$ 
9:    $r_{2:\text{id}+1} := \text{TCCR}(r_{\text{id}} \oplus \text{const}_1, s, \text{iv})$ 
10: end for
11: Parse  $I = (\Delta_0, \dots, \Delta_{\tau-1})$   $\triangleright$  For  $\alpha \in [0, \tau), \Delta_\alpha \in N_\alpha$ .
12: for  $\alpha \in [0, \tau)$  do
13:    $\text{id}_\alpha := \text{BAVC.Interleave}(\alpha, \Delta_\alpha; \text{param})$ 
14:    $(\text{seed}_{\Delta_\alpha}^\alpha, \text{com}_{\Delta_\alpha}^\alpha) := \text{LeafHash}(r_{\text{id}_\alpha}, \text{iv})$ 
15: end for
16:  $\text{id-list} := \text{BAVC.MinSpan}([L, 2L) \setminus (\text{id}_0, \dots, \text{id}_{\tau-1}); \text{param})$ 
17: if  $|\text{id-list}| \leq T_{\text{open}}$  then
18:    $\text{decom}_I := ((\text{com}_{\Delta_\alpha}^\alpha)_{\alpha \in [0, \tau)}, (r_{\text{id}})_{\text{id} \in \text{id-list}}) \parallel 0^{(T_{\text{open}} - |\text{id-list}|) \cdot \lambda}$ 
19: else
20:    $\text{decom}_I := \perp$ 
21: end if
22: return  $\text{decom}_I$ 

```

4.4.3 BAVC Verification

The BAVC verification algorithm takes the challenge vector I , the IV, the hashed message μ and salt s , and the partial opening decom_I . It first recomputes the hidden leaf positions in the large pcGGM tree from I and computes the same minimum-span node set that the signer should have opened. If this set exceeds T_{open} , the verifier returns $\perp$.

The verifier then parses decom_I into the commitments of the hidden leaves and the internal node seeds for the opened part of the tree. Starting from those internal node seeds, it expands the tree downward using the same tree expansion rules as in **BAVC.Commit**. For every reconstructed opened leaf, it runs **LeafHash** to recover both the seed returned to **VOLE.Reconstruct** and the leaf commitment used for the final hash. The verifier combines the reconstructed and hidden leaf commitments, then recomputes the global BAVC hash. The output is the recomputed global commitment com together with all leaf seeds except the hidden seeds $\text{seed}_{\Delta_\alpha}^\alpha$. Thus the verifier obtains exactly the seeds needed for all-but-one reconstruction, without learning the hidden leaf seeds.

BAVC.Verify($I, iv, \mu, s, \text{decom}_I; \text{param}$)

Input: $\left(\begin{array}{l} I := (\Delta_0, \dots, \Delta_{\tau-1}) \in [0, N_0) \times \dots \times [0, N_{\tau-1}), \\ iv \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, s \in \{0, 1\}^\lambda, \\ \text{decom}_I := ((\text{com}_{i_\alpha}^\alpha)_{\alpha \in [0, \tau)} \in \{0, 1\}^{2\tau\lambda}, (r_i)_{i \in \text{id-list}} \in \{0, 1\}^{T_{\text{open}} \cdot \lambda}) \end{array} \right)$

Output: $(\text{com}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \neq \Delta_\alpha} \cup \{\perp\})$

```

1: Extract  $\lambda, \tau, L, T_{\text{open}}, (N_\alpha)_{\alpha \in [0, \tau)}$  from param
2: for  $\alpha \in [0, \tau)$  do
3:    $\text{id}_\alpha := \text{BAVC.Interleave}(\alpha, \Delta_\alpha; \text{param})$ 
4: end for
5:  $\text{id-list} := \text{BAVC.MinSpan}([L, 2L) \setminus (\text{id}_0, \dots, \text{id}_{\tau-1}); \text{param})$ 
6: if  $|\text{id-list}| > T_{\text{open}}$  then
7:   return  $\perp$ 
8: end if
9: Parse  $\text{decom}_I = ((\text{com}_{\Delta_\alpha}^\alpha)_{\alpha \in [0, \tau)}, (r_{\text{id}})_{\text{id} \in \text{id-list}})$ 
10: for  $\text{id} \in [1, L)$  do
11:   if  $\text{id} \in \text{id-list}$  then
12:     Remove  $\text{id}$  from  $\text{id-list}$ 
13:     Add  $2 \cdot \text{id}$  and  $2 \cdot \text{id} + 1$  to  $\text{id-list}$ 
14:     if  $\text{id} < L/2$  then
15:        $r_{2 \cdot \text{id}} := \text{TCCR}(r_{\text{id}}, s, iv)$ 
16:        $r_{2 \cdot \text{id} + 1} := r_{2 \cdot \text{id}} \oplus r_{\text{id}}$ 
17:     else
18:        $r_{2 \cdot \text{id}} := \text{TCCR}(r_{\text{id}} \oplus \text{const}_0, s, iv)$ 
19:        $r_{2 \cdot \text{id} + 1} := \text{TCCR}(r_{\text{id}} \oplus \text{const}_1, s, iv)$ 
20:     end if
21:   end if
22: end for
23: for  $\text{id} \in \text{id-list}$  do ▷  $\text{id-list}$  has  $L - \tau$  indices.
24:    $(\alpha, i_\alpha) := \text{BAVC.Deinterleave}(\text{id}; \text{param})$ 
25:    $(\text{seed}_{i_\alpha}^\alpha, \text{com}_{i_\alpha}^\alpha) := \text{LeafHash}(r_{\text{id}}, iv)$ 
26: end for
27:  $\text{com} := \text{BAVCHash}(\mu, s, (\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, iv)$ 
28: return  $(\text{com}, (\text{seed}_i^\alpha)_{\alpha \in [0, \tau), i \neq \Delta_\alpha})$ 

```

4.4.4 Leaf Interleaving

The interleaving algorithm maps a leaf index inside one of the τ small trees to the corresponding leaf position in the large pcGGM tree. The first 2^{k_0} leaf positions are valid for all τ small instances. So the mapping interleaves them with stride τ . The remaining leaves exist only for the t_1 instances with depth k_1 , and these extra leaves are interleaved with stride t_1 . This layout lets the implementation commit to all VOLE instances with a single large pcGGM tree while still supporting per-instance challenge indices of length k_0 or k_1 .

BAVC.Interleave($\alpha, i_\alpha; \text{param}$)

Input: $(\alpha \in [0, \tau), i_\alpha \in [0, N_\alpha))$

Output: $\text{id} \in [L, 2L)$

1: Extract $\tau, k_0, t_0, k_1, t_1, L$ from **param**

```

2: if  $i_\alpha < 2^{k_0}$  then
3:    $\text{id} = L + i_\alpha \cdot \tau + \alpha$ 
4: else
5:    $\text{id} = L + 2^{k_0} \cdot \tau + (i_\alpha - 2^{k_0}) \cdot t_1 + \alpha$ 
6: end if
7: return id

```

▷ Tree α of depth k_1 ; $\alpha \in [0, t_1)$.

4.4.5 Leaf Deinterleaving

The deinterleaving algorithm is the inverse of [BAVC.Interleave](#). Given a leaf position in the large pcGGM tree, it recovers the index α of the small VOLE instance and the leaf index i_α inside that instance. It first checks whether the position lies in the common 2^{k_0} -leaf region. If so, division by τ gives i_α , while the remainder gives α . Otherwise, the position lies in the extra region for the t_1 larger instances, so the same computation is performed with stride t_1 and offset 2^{k_0} .

[BAVC.Deinterleave](#)(id; param)

Input: $\text{id} \in [L, 2L)$

Output: $(\alpha \in [0, \tau), i_\alpha \in [0, N_\alpha))$

```

1: Extract  $\tau, k_0, t_0, k_1, t_1, L$  from param
2: if  $\text{id} < L + \tau \cdot 2^{k_0}$  then
3:    $\alpha := (\text{id} - L) \bmod \tau$ 
4:    $i_\alpha := \lfloor \frac{\text{id} - L}{\tau} \rfloor$ 
5: else
6:    $\alpha := (\text{id} - L - \tau \cdot 2^{k_0}) \bmod t_1$ 
7:    $i_\alpha := \lfloor \frac{\text{id} - L - 2^{k_0} \cdot \tau}{t_1} \rfloor + 2^{k_0}$ 
8: end if
9: return  $(\alpha, i_\alpha)$ 

```

4.4.6 Minimum-Span Opening

The minimum-span algorithm compresses a set of tree nodes by replacing sibling pairs with their parent whenever possible. In [BAVC.Open](#), the input set is the collection of leaves in the large pcGGM tree that should be opened. The algorithm scans the tree bottom-up. Whenever both children of an internal node are present, it removes the children and adds their parent. Repeating this operation produces a compact node set that spans exactly the same opened leaves. This is the node set included in decom_I , and its size determines whether the BAVC opening fits below T_{open} .

[BAVC.MinSpan](#)(id-list; param)

Input: $\text{id-list} \subseteq [1, 2L)$

Output: $\text{id-list} \subseteq [1, 2L)$

```

1: for id from  $L - 1$  down to 1 do
2:   if  $2 \cdot \text{id} \in \text{id-list} \wedge 2 \cdot \text{id} + 1 \in \text{id-list}$  then
3:     Remove  $2 \cdot \text{id}$  and  $2 \cdot \text{id} + 1$  from id-list
4:     Add id to id-list
5:   end if

```

```

6: end for
7: return id-list

```

4.4.7 Leaf Hashing

The leaf-hashing algorithm binds each expanded tree leaf to a commitment while retaining the leaf value as the seed used by the VOLE conversion step. It outputs $\text{seed} := \mathbf{x}$ and computes the leaf commitment as $\text{PRG}(\mathbf{x}, \text{iv}, 0^8; 2\lambda)$. The fixed tweak 0^8 separates leaf commitments from the PRG calls used inside [VOLE.ConvertToVOLE](#). By committing all leaf values, the later expanded VOLE/IT-MAC correlations of the prover are also committed.

[LeafHash](#)($\mathbf{x}, \text{iv}$)

Input: ($\mathbf{x} \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}$)

Output: ($\text{seed} \in \{0, 1\}^\lambda, \text{com} \in \{0, 1\}^{2\lambda}$)

```

1:  $\text{seed} := \mathbf{x}$ 
2:  $\text{com} := \text{PRG}(\mathbf{x}, \text{iv}, 0^8; 2\lambda)$ 
3: return ( $\text{seed}, \text{com}$ )

```

4.5 RSD-based One-Way Function

This subsection specifies the RSD one-way function and the witness representation used by the proof system. The public instance is a systematic RSD matrix $\mathbf{H} = [\mathbf{I} \mid \mathbf{H}_B]$ together with a syndrome $\mathbf{y}$, and the witness is a regular noise vector $\mathbf{e}$ satisfying $\mathbf{y} = \mathbf{H} \cdot \mathbf{e}$. We split the noise vector as $\mathbf{e} = (\mathbf{e}_A, \mathbf{e}_B)$ according to the systematic matrix. Because $\mathbf{e}_A$ can be recovered linearly from $\mathbf{y}$ and $\mathbf{e}_B$, the signer only commits to a compact representation of $\mathbf{e}_B$.

For the sake of conciseness, we use the IT-MAC notation in [RSD.DecodeNoise](#). When the signing algorithm invokes this function, the same affine operations are applied to the prover's IT-MAC tags; when the verification algorithm invokes it, they are applied to the verifier's IT-MAC local keys. It contains the following algorithms.

- [RSD.SampleMatrix](#): expands the public-key seed into the systematic RSD matrix.
- [RSD.SampleNoise](#): expands the secret-key seed into a block-regular noise vector.
- [RSD.EncodeNoise](#): compresses the regular noise into the witness committed by [VOLEitH](#).
- [RSD.DecodeNoise](#): reconstructs the full RSD noise, or its IT-MAC representation, from the committed witness coordinates.

4.5.1 RSD Matrix Sampling

The matrix-sampling algorithm derives the public RSD matrix from the public-key seed seed_{pk} . The matrix is stored in systematic form $\mathbf{H} = [\mathbf{I} \mid \mathbf{H}_B]$, where the identity block has $(m - k)$ rows and columns. Only the dense block $\mathbf{H}_B$ needs to be sampled, so [MatHash](#) expands seed_{pk} into $(m - k) \cdot k$ bits and interprets them as a binary matrix. This lets the public key contain only seed_{pk} and $\mathbf{y}$, while every verifier can reconstruct the same $\mathbf{H}$.

[RSD.SampleMatrix](#)($\text{seed}_{\text{pk}}; \text{param}$)

Input: $\text{seed}_{\text{pk}} \in \{0, 1\}^\lambda$

Output: $H \in \mathbb{F}_2^{(m-k) \times m}$

- 1: Extract λ, m, k from **param**
- 2: $H_B := \text{MatHash}(\text{seed}_{pk})$
- 3: **return** $H := [I \parallel H_B]$

▷ I is the identity matrix.

4.5.2 RSD Noise Sampling

The noise-sampling algorithm derives the secret RSD witness from seed_{sk} . The codeword error is partitioned into w blocks of length ℓ_{bs} . For each block, the algorithm expands seed_{sk} with a block counter, interprets the output as an integer, and reduces it modulo ℓ_{bs} to choose the unique nonzero position in that block. The resulting vector is regular by construction: every block contains exactly one 1 and all other coordinates are 0.

RSD.SampleNoise($\text{seed}_{sk}; \text{param}$)

Input: $\text{seed}_{sk} \in \{0, 1\}^\lambda$

Output: $e \in \mathbb{F}_2^m$

- 1: Extract λ, m, ℓ_{bs}, w from **param**
- 2: **for** $i \in [0, w)$ **do**
- 3: $r_i := \text{PRG}(\text{seed}_{sk}, \text{IntToBits}(i; 120), 0; \lambda)$
- 4: $r_i := \text{BitsToInt}(r_i; \lambda)$
- 5: $e_i := 0^{\ell_{bs}}$
- 6: $e_i[r_i \bmod \ell_{bs}] = 1$
- 7: **end for**
- 8: $e := [e_0, \dots, e_{w-1}]$
- 9: **return** e

4.5.3 RSD Noise Encoding

The encoding algorithm converts the full sampled noise e into the compact witness w committed by VOLEitH. The systematic relation lets the proof recover e_A from y and e_B , so w only contains information about the suffix e_B . Moreover, each e_B block has odd parity equal to 1 because the block has Hamming weight one. Therefore the last coordinate of each block is determined by the first $\ell_{bs} - 1$ coordinates, and the witness omits this last coordinate. This reduces the committed witness length from k to $k - k/\ell_{bs}$.

RSD.EncodeNoise($e; \text{param}$)

Input: $e \in \mathbb{F}_2^m$

Output: $w \in \mathbb{F}_2^{k-k/\ell_{bs}}$

▷ Our parameters ensure $\ell_{bs} \mid k$.

- 1: Extract m, k, ℓ_{bs}, w from **param**
- 2: $e_B := e[m - k, m]$
- 3: Parse $e_B := [e_{B,0}, \dots, e_{B,w_B-1}]$
- 4: **for** $i \in [0, w_B)$ **do**
- 5: $w_i := e_{B,i}[0, \ell_{bs} - 1]$
- 6: **end for**
- 7: $w := [w_0, \dots, w_{w_B-1}]$
- 8: **return** w

▷ $w_B = k/\ell_{bs}$, each block is equal-sized.

▷ Commit only the first $\ell_{bs} - 1$ bits of each block in e_B .

4.5.4 RSD Noise Decoding

The decoding algorithm expands the compact witness back into the full noise representation used by QuickSilver. It first parses the committed witness into the shortened e_B blocks. For each block, it reconstructs the omitted last coordinate as the XOR of 1 with the committed coordinates, enforcing the block-sum equation by construction. It then recovers the remaining part as $e_A := y \oplus H_B \cdot e_B$, which makes $H \cdot e = y$ hold automatically for the reconstructed vector.

This procedure does not by itself prove that every block has Hamming weight one. Instead, it provides the authenticated full vector on which the QuickSilver layer checks the remaining regularity constraints. Because all operations in [RSD.DecodeNoise](#) are affine over $\mathbb{F}_2$, the same pseudocode applies to concrete bits, prover-side IT-MAC tags, and verifier-side IT-MAC local keys.

[RSD.DecodeNoise](#)($\llbracket w \rrbracket, H, y; \text{param}$)

Input: $\llbracket w \rrbracket$ for $w \in \mathbb{F}_2^{k-k/\ell_{\text{bs}}}$, $H := [I \mid H_B] \in \mathbb{F}_2^{(m-k) \times m}$, $y \in \mathbb{F}_2^{m-k}$

Output: $\llbracket e \rrbracket$ for $e \in \mathbb{F}_2^m$

1: Extract $m, k, \ell_{\text{bs}}, w$ from **param**

2: Parse $\llbracket w \rrbracket := [\llbracket w_0 \rrbracket, \dots, \llbracket w_{w_B-1} \rrbracket]$

$\triangleright w_B := k/\ell_{\text{bs}}.$

3: **for** $i \in [0, w_B)$ **do**

4: $\llbracket e_{B,i}[0, \ell_{\text{bs}} - 1] \rrbracket := \llbracket w_i \rrbracket$

$\triangleright$ The length of each $e_{B,i}$ is ℓ_{bs} .

5: $\llbracket e_{B,i}[\ell_{\text{bs}} - 1] \rrbracket := 1 \oplus \bigoplus_{j \in [0, \ell_{\text{bs}} - 1)} \llbracket w_i[j] \rrbracket$

6: **end for**

7: $\llbracket e_B \rrbracket := [\llbracket e_{B,0} \rrbracket, \dots, \llbracket e_{B,w_B-1} \rrbracket]$

8: $\llbracket e_A \rrbracket := y \oplus H_B \cdot \llbracket e_B \rrbracket$

9: **return** $\llbracket e \rrbracket := [\llbracket e_A \rrbracket, \llbracket e_B \rrbracket]$

4.6 QuickSilver Proof System

This subsection instantiates the QuickSilver proof layer for the regularity constraints of the RSD witness. At this point in the signing transcript, the VOLE layer has already committed to the compact RSD witness and produced IT-MAC authentication material for it. QuickSilver is used to check that each reconstructed noise block has Hamming weight exactly 1. We split this regularity condition into two parts. First, the deterministic linear-sketching method of [Section 3.8](#) is applied only to the all-but-last coordinates of each block. This proves that the first $\ell_{\text{bs}} - 1$ coordinates contain either zero or one nonzero entry. Second, a linear block-sum check enforces that all ℓ_{bs} coordinates xor to 1. Together, these two conditions imply that the full block contains exactly one 1. For the e_B blocks, the xor-to-1 condition is already enforced by [RSD.DecodeNoise](#), which recovers the last coordinate from the first $\ell_{\text{bs}} - 1$ coordinates. Thus the explicit linear block-sum checks are only needed for the recovered e_A blocks.

For the sake of conciseness, we follow the convention of [Section 4.5](#) and use IT-MAC notation in the [QS.RSDProve](#) and [QS.RSDVerify](#) functions. When the signing algorithm invokes this subsection, the operations are carried out on the prover-side IT-MAC tags. When the verification algorithm invokes it, the same algebraic expressions are evaluated on the verifier-side IT-MAC local keys. It contains the following algorithms.

- [QS.RSDProve](#): computes the prover-side aggregated QuickSilver check values from the committed RSD witness.
- [QS.RSDVerify](#): reconstructs the missing QuickSilver response from the verifier-side VOLE keys and the final challenge.

4.6.1 QuickSilver Proving

The QuickSilver proving algorithm evaluates the low-degree RSD regularity constraints on IT-MAC-authenticated witness values. The input $\mathbf{w}$ is the compact RSD witness, while $\mathbf{u}$ is the committed VOLE/IT-MAC message and $\mathbf{V}$ contains the corresponding prover-side tags. The algorithm first converts each row of $\mathbf{V}$ into an element of $\mathbb{F}_{2^\lambda}$. Together with the entries of $\mathbf{w}$, the first ℓ_{wit} converted tags define degree-1 IT-MACs for the committed witness coordinates.

The prover then calls [RSD.DecodeNoise](#) on these authenticated coordinates. Since this decoding is affine, it gives authenticated values for the full reconstructed noise vector while preserving the linear syndrome relation. For every reconstructed block $\mathbf{e}_i$, the prover evaluates the deterministic sketch polynomial on the first $\ell_{\text{bs}} - 1$ coordinates. The resulting object is a degree-2 IT-MAC committing to the sketch value. When the sketch constraint is satisfied, this committed value is 0. Equivalently, the degree-2 coefficient of the IT-MAC polynomial is fixed to 0, and the prover only has to provide the remaining degree-1 and degree-0 coefficients. Several such quadratic checks are packed into one field element according to [Lemma 3.5](#), and these lower-degree coefficients are appended to $\mathbf{a}_1$ and $\mathbf{a}_0$.

The prover also checks the xor-to-1 constraints for the $\mathbf{e}_A$ blocks. Each such constraint gives a degree-1 IT-MAC of an $\mathbb{F}_2$ element that should be zero. Since one $\mathbb{F}_{2^\lambda}$ element can encode λ elements of $\mathbb{F}_2$, the algorithm packs up to λ linear checks together. To combine these linear checks with the quadratic sketch checks, the prover lifts the degree-1 zero commitment to degree 2 by multiplying its polynomial by the IT-MAC indeterminate. This is why the constant coefficient of the degree-1 check is appended to $\mathbf{a}_1$, while 0 is appended to $\mathbf{a}_0$.

Finally, the extra λ VOLE rows are aggregated into one field element. They provide the random zero IT-MAC used by [ZKHash](#) to batch and mask all QuickSilver checks. The output (a_1, a_0) is the prover-side opening information for the batched zero-checking, with the degree-2 coefficient fixed to zero.

[QS.RSDProve](#)($\mathbf{w}, \mathbf{u}, \mathbf{V}, \mathbf{H}, \mathbf{y}, \text{chall}_2; \text{param}$)

Input: $\left(\mathbf{w} \in \mathbb{F}_2^{\ell_{\text{wit}}}, \mathbf{u} \in \mathbb{F}_2^{\ell_{\text{wit}} + \lambda}, \mathbf{V} \in \mathbb{F}_2^{(\ell_{\text{wit}} + \lambda) \times \lambda}, \right.$
 $\left. \mathbf{H} := [\mathbf{I} \mid \mathbf{H}_B] \in \mathbb{F}_2^{(m-k) \times m}, \mathbf{y} \in \mathbb{F}_2^{m-k}, \text{chall}_2 \in \{0, 1\}^{3\lambda+64} \right)$

Output: $(a_1 \in \mathbb{F}_{2^\lambda}, a_0 \in \mathbb{F}_{2^\lambda})$

1: Extract $\lambda, \ell_{\text{wit}}, \tau, \ell_{\text{bs}}, m, k$ from **param**

2: **for** $i \in [0, \ell_{\text{wit}} + \lambda)$ **do**

3: $\mathbf{v}[i] := \text{BitsToField}(\mathbf{V}[i, :]; \lambda)$

▷ Convert the i -th row of $\mathbf{V}$ to $\mathbb{F}_{2^\lambda}$.

4: **end for**

5: Let $\mathbf{w}[0, \ell_{\text{wit}}), \mathbf{v}[0, \ell_{\text{wit}})$ define degree-1 IT-MACs $\llbracket \mathbf{w} \rrbracket^1$

6: $(\llbracket \mathbf{e}_0 \rrbracket^1, \dots, \llbracket \mathbf{e}_{w-1} \rrbracket^1) := \text{RSD.DecodeNoise}(\mathbf{w}, \mathbf{H}, \mathbf{y}; \text{param})$

▷ For $i \in [0, w), \mathbf{e}_i \in \mathbb{F}_2^{\ell_{\text{bs}}}$

7: Let $d_{\text{pack}} := \ell_{\text{bs}}^2 - 3\ell_{\text{bs}} + 1, \ell_{\text{pack}} := \lfloor \frac{\lambda}{d_{\text{pack}} + 1} \rfloor, \ell_{\text{qs}} := \lceil \frac{w}{\ell_{\text{pack}}} \rceil$

8: **for** $i \in [0, w)$ **do**

9:

$$\llbracket z_i \rrbracket^2 := \left(\sum_{j=0}^{\ell_{\text{bs}}-2} X^j \cdot \llbracket \mathbf{e}_i[j] \rrbracket^1 \right) \cdot \left(\sum_{j=0}^{\ell_{\text{bs}}-3} X^{(\ell_{\text{bs}}-1) \cdot j} \cdot \llbracket \mathbf{e}_i[j] \rrbracket^1 \right) - \left(\sum_{j=0}^{\ell_{\text{bs}}-3} X^{\ell_{\text{bs}} \cdot j} \cdot \llbracket \mathbf{e}_i[j] \rrbracket^1 \right)$$

10: **end for**

11: Initialize $\mathbf{a}_1, \mathbf{a}_0$ as empty vectors

12: **for** $i \in [0, \ell_{\text{qs}})$ **do**

13: $\llbracket z'_i \rrbracket^2 := \sum_{j=i \cdot d_{\text{pack}}}^{\min(m-1, (i+1) \cdot d_{\text{pack}}-1)} X^{d_{\text{pack}} \cdot j} \cdot \llbracket z_j \rrbracket^2$

14: Parse $p_{z'_i}(x) := z'_{i,0} + z'_{i,1} \cdot x$

▷ By [Lemma 3.5](#), $\deg(p_{z'_i}(x)) = 1$.

```

15:    $\mathbf{a}_1 := [\mathbf{a}_1, z'_{i,1}]$ 
16:    $\mathbf{a}_0 := [\mathbf{a}_0, z'_{i,0}]$ 
17: end for
18: Let  $\ell_{\text{lin}} := \frac{m-k}{\ell_{\text{bs}}}$ ,  $\ell'_{\text{qs}} := \lceil \frac{\ell_{\text{lin}}}{\lambda} \rceil$ 
19: for  $i \in [0, \ell'_{\text{qs}})$  do
20:    $\llbracket l_i \rrbracket^1 := \sum_{j=i \cdot \lambda}^{\min((i+1) \cdot \lambda, \ell_{\text{lin}})} X^j \cdot (\llbracket e_j[0] \rrbracket^1 + \dots + \llbracket e_j[\ell_{\text{bs}} - 1] \rrbracket^1 + \llbracket 1 \rrbracket^1)$ 
21:   Parse  $p_{l_i}(x) := l_{i,0} + l_{i,1} \cdot x$ 
22:    $\mathbf{a}_1 := [\mathbf{a}_1, l_{i,0}]$ 
23:    $\mathbf{a}_0 := [\mathbf{a}_0, 0]$ 
24: end for
25:  $\tilde{u} := \text{BitsToField}(\mathbf{u}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda]; \lambda)$ 
26:  $\tilde{v} := \sum_{i=0}^{\lambda-1} v[\ell_{\text{wit}} + i] X^i$   $\triangleright X$  is the generator of  $\mathbb{F}_{2^\lambda}$ .
27:  $a_1 := \text{ZKHash}(\text{chall}_2, [\mathbf{a}_1, \tilde{u}])$ 
28:  $a_0 := \text{ZKHash}(\text{chall}_2, [\mathbf{a}_0, \tilde{v}])$ 
29: return  $(a_1, a_0)$ 

```

4.6.2 QuickSilver Verification

The QuickSilver verification algorithm mirrors the prover's constraint computation using the verifier's local IT-MAC keys. The verifier starts from the matrix $\mathbf{Q}$ reconstructed by the VOLE layer, whose rows authenticate the committed message $\mathbf{u}$. The signature contains the public witness offset $\mathbf{d} = \mathbf{w} \oplus \mathbf{u}[0, \ell_{\text{wit}})$. After interpreting chall_3 as the global IT-MAC key Δ , the verifier applies `VOLE.Fix` with $\mathbf{d}$ to the first ℓ_{wit} rows of $\mathbf{Q}$. This changes the local keys from authenticating the random message $\mathbf{u}$ from `VOLE.Commit` into authenticating the actual witness $\mathbf{w}$.

The verifier then evaluates the same affine decoding and the same linear sketch computation as the prover, but on local keys rather than on tags. For each packed quadratic check, it records the local key contribution in $\mathbf{b}$. For the linear block-sum checks on the $\mathbf{e}_A$ blocks, it first obtains the local key of the packed degree-1 zero commitment. It then multiplies this key by Δ , matching the prover's degree-lifting step from $p_{l_i}(x)$ to $x \cdot p_{l_i}(x)$. The last λ rows of $\mathbf{Q}$ are aggregated into $\tilde{q}$, matching the prover's use of the extra VOLE rows in `QS.RSDProve`.

After hashing these verifier-side quantities with `ZKHash`, the verifier obtains the local-key evaluation of the same batched degree-2 IT-MAC opening check. Since the degree-2 coefficient is fixed to 0, the prover only supplies $\tilde{a}_1$. The verifier reconstructs the unique constant term a_0 that makes the IT-MAC opening equation hold at Δ . The top-level verifier then hashes $(\text{chall}_2, \tilde{a}_1, a_0, \text{ctr})$ to check consistency with the final challenge chall_3 .

QS.RSDVerify($\mathbf{d}, \mathbf{Q}, \text{chall}_2, \text{chall}_3, a_1, \mathbf{H}, \mathbf{y}; \text{param}$)

Input: $\left(\mathbf{d} \in \mathbb{F}_2^{\ell_{\text{wit}}}, \mathbf{Q} \in \mathbb{F}_2^{(\ell_{\text{wit}} + \lambda) \times \lambda}, \text{chall}_2 \in \{0, 1\}^{3\lambda + 64}, \text{chall}_3 \in \{0, 1\}^\lambda, \right.$
 $\left. \tilde{a}_1 \in \mathbb{F}_{2^\lambda}, \mathbf{H} := [\mathbf{I} \mid \mathbf{H}_B] \in \mathbb{F}_2^{(m-k) \times m}, \mathbf{y} \in \mathbb{F}_2^{m-k} \right)$

Output: $a_0 \in \mathbb{F}_{2^\lambda}$

1: Extract $\lambda, \ell_{\text{wit}}, \tau, m, k$ from **param**

2: $\Delta := \text{BitsToField}(\text{chall}_3; \lambda)$

3: Parse $\mathbf{Q} := \begin{bmatrix} \mathbf{Q}_0 \\ \mathbf{Q}_1 \end{bmatrix}$

$\triangleright \mathbf{Q}_0$ has ℓ_{wit} rows, while $\mathbf{Q}_1$ has λ rows.

4: $\hat{\mathbf{Q}}_0 := \text{VOLE.Fix}(\mathbf{Q}_0, \mathbf{d}, \Delta; \lambda, \ell_{\text{wit}})$

5: $\hat{\mathbf{Q}} := \begin{bmatrix} \hat{\mathbf{Q}}_0 \\ \mathbf{Q}_1 \end{bmatrix}$

```

6: for  $i \in [0, \ell_{\text{wit}} + \lambda)$  do
7:    $\mathbf{q}[i] := \text{BitsToField}(\hat{\mathbf{Q}}[i, :]; \lambda)$  ▷ Convert the  $i$ -th row of  $\mathbf{Q}$  to  $\mathbb{F}_{2^\lambda}$ .
8: end for
9: Let  $\mathbf{q}[0, \ell_{\text{wit}})$  define  $\llbracket \mathbf{w} \rrbracket^1$ 
10:  $(\llbracket \mathbf{e}_0 \rrbracket^1, \dots, \llbracket \mathbf{e}_{w-1} \rrbracket^1) := \text{RSD.DecodeNoise}(\mathbf{w}, \mathbf{H}, \mathbf{y}; \text{param})$  ▷ For  $i \in [0, w)$ ,  $\mathbf{e}_i \in \mathbb{F}_2^{\ell_{\text{bs}}}$ 
11: Let  $d_{\text{pack}} := \ell_{\text{bs}}^2 - 3\ell_{\text{bs}} + 1$ ,  $\ell_{\text{pack}} := \lfloor \frac{\lambda}{d_{\text{pack}} + 1} \rfloor$ ,  $\ell_{\text{qs}} := \lceil \frac{w}{\ell_{\text{pack}}} \rceil$ 
12: for  $i \in [0, w)$  do
13:

$$\llbracket z_i \rrbracket^2 := \left( \sum_{j=0}^{\ell_{\text{bs}}-2} X^j \cdot \llbracket \mathbf{e}_i[j] \rrbracket^1 \right) \cdot \left( \sum_{j=0}^{\ell_{\text{bs}}-3} X^{(\ell_{\text{bs}}-1) \cdot j} \cdot \llbracket \mathbf{e}_i[j] \rrbracket^1 \right) - \left( \sum_{j=0}^{\ell_{\text{bs}}-3} X^{\ell_{\text{bs}} \cdot j} \cdot \llbracket \mathbf{e}_i[j] \rrbracket^1 \right)$$

14: end for
15: Initialize  $\mathbf{b}$  as an empty vector
16: for  $i \in [0, \ell_{\text{qs}})$  do
17:    $\llbracket z'_i \rrbracket^2 := \sum_{j=i \cdot d_{\text{pack}}}^{\min(m-1, (i+1) \cdot d_{\text{pack}} - 1)} X^{d_{\text{pack}} \cdot j} \cdot \llbracket z_j \rrbracket^2$ 
18:   Let  $k_{z'_i} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key for  $\llbracket z'_i \rrbracket^1$ .
19:    $\mathbf{b} := [\mathbf{b}, k_{z'_i}]$ 
20: end for
21: Let  $\ell_{\text{lin}} := \frac{m-k}{\ell_{\text{bs}}}$ ,  $\ell'_{\text{qs}} := \lceil \frac{\ell_{\text{lin}}}{\lambda} \rceil$ 
22: for  $i \in [0, \ell'_{\text{qs}})$  do
23:    $\llbracket l_i \rrbracket^1 := \sum_{j=i \cdot \lambda}^{\min((i+1) \cdot \lambda, \ell_{\text{lin}})} X^j \cdot (\llbracket \mathbf{e}_j[0] \rrbracket^1 + \dots + \llbracket \mathbf{e}_j[\ell_{\text{bs}} - 1] \rrbracket^1 + \llbracket 1 \rrbracket^1)$ 
24:   Let  $k_{l_i} \in \mathbb{F}_{2^\lambda}$  be the IT-MAC local key for  $\llbracket l_i \rrbracket^1$ .
25:    $\mathbf{b} := [\mathbf{b}, k_{l_i} \cdot \Delta]$ 
26: end for
27:  $\tilde{\mathbf{q}} := \sum_{i=0}^{\lambda-1} \mathbf{q}[\ell_{\text{wit}} + i] X^i$  ▷  $X$  is the generator of  $\mathbb{F}_{2^\lambda}$ .
28:  $\mathbf{b} := \text{ZKHash}(\text{chall}_2, [\mathbf{b}, \tilde{\mathbf{q}}])$ 
29: return  $a_0 := b - \tilde{a}_1 \cdot \Delta$ 

```

4.7 Universal Hash Functions

This subsection defines the linear universal hash functions. Both hash functions batch IT-MAC/VOLE checks by compressing many authenticated values into a short challenge-dependent value with linear operations. The difference is the underlying linear space. **VOLEHash** is linear over $\mathbb{F}_2$ and is used for checking consistency of the VOLE correlations generated from the small VOLE instances. **ZKHash** is linear over $\mathbb{F}_{2^\lambda}$ and is used for batched zero-checking in QuickSilver. In both cases, the prover applies the hash to committed messages or prover-side tags, while the verifier applies the same linear map to local keys and then performs the corresponding IT-MAC opening check. It contains the following algorithms.

- **VOLEHash**: compresses VOLE vectors over $\mathbb{F}_2$ for the consistency check.
- **ZKHash**: compresses QuickSilver check vectors over $\mathbb{F}_{2^\lambda}$.

4.7.1 VOLE Linear Universal Hash

The **VOLEHash** algorithm batches checks on degree-1 IT-MACs over $\mathbb{F}_2$. After **VOLE.Commit**, the prover has τ small VOLE instances. After applying the published correction vectors, all instances should authenticate the same committed message $\mathbf{u}$. Rather than checking the full VOLE vectors and tag matrices, the signer computes the compressed values $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{V}}$ with

VOLEHash and binds them into the Fiat-Shamir transcript. The verifier applies the same linear map to the verifier-side local keys and then uses **VOLE.Fix** with $\tilde{\mathbf{u}}$ and the IT-MAC global key defined by chall_3 . This reconstructs the compressed local-key value that should match the signer's compressed tag in the transcript.

The input is split into a long part $\mathbf{x}_0$ and a short part $\mathbf{x}_1$. The long part is padded to a multiple of λ bits. It is parsed once as a vector over $\mathbb{F}_{2^\lambda}$ and once as a vector over $\mathbb{F}_{2^{64}}$. The algorithm evaluates a polynomial hash in each representation. The 64-bit result is embedded back into $\mathbb{F}_{2^\lambda}$, and two random linear combinations produce $\lambda + \mathbf{B}$ output bits. Finally, the short vector $\mathbf{x}_1$ is XOR-ed into the output. This final XOR gives the hiding property since the last $\lambda + \mathbf{B}$ input bits are uniformly random and are not utilized elsewhere in the signature scheme. When **VOLEHash** is applied to a matrix, it is applied column-wise with the same seed.

VOLEHash (seed, $(\mathbf{x}_0, \mathbf{x}_1)$)	(VOLEHash is linear over $\mathbb{F}_2$)
Input: $\left(\begin{array}{l} \text{seed} := \left(\begin{array}{l} \mathbf{r}_0 \in \{0, 1\}^\lambda, \mathbf{r}_1 \in \{0, 1\}^\lambda, \mathbf{r}_2 \in \{0, 1\}^\lambda, \\ \mathbf{r}_3 \in \{0, 1\}^\lambda, \mathbf{s} \in \{0, 1\}^\lambda, \mathbf{t} \in \{0, 1\}^{64} \end{array} \right), \\ (\mathbf{x}_0 \in \mathbb{F}_2^{\ell_{\text{wit}} + \lambda}, \mathbf{x}_1 \in \mathbb{F}_2^{\lambda + \mathbf{B}}) \end{array} \right)$ Output: $\mathbf{h} \in \{0, 1\}^{\lambda + \mathbf{B}}$ <pre> 1: for $i \in [0, 3]$ do 2: $r_i := \text{BitsToField}(\mathbf{r}_i; \lambda)$ 3: end for 4: $s := \text{BitsToField}(\mathbf{s}; \lambda)$ 5: $t := \text{BitsToField}(\mathbf{t}; 64)$ 6: $\ell' := \lambda \cdot \lceil \frac{\ell_{\text{wit}} + \lambda}{\lambda} \rceil$ 7: $\hat{\mathbf{y}} := \text{BitsToField}(\mathbf{x}_0; \lambda)$ 8: $\bar{\mathbf{y}} := \text{BitsToField}(\mathbf{x}_0; 64)$ 9: $h_0 := \sum_{i=0}^{\ell'/\lambda - 1} s^{\ell'/\lambda - 1 - i} \cdot \hat{\mathbf{y}}[i]$ 10: $h_1 := \sum_{i=0}^{\ell'/64 - 1} t^{\ell'/64 - 1 - i} \cdot \bar{\mathbf{y}}[i]$ 11: $h'_1 := \text{BitsToField}(\text{FieldToBits}(h_1; 64) 0^{\lambda - 64}; \lambda)$ 12: $(h_2, h_3) := (r_0 \cdot h_0 + r_1 \cdot h'_1, r_2 \cdot h_0 + r_3 \cdot h'_1)$ 13: $\mathbf{h} := (\text{FieldToBits}(h_2; \lambda) \text{FieldToBits}(h_3; \lambda)[0, \mathbf{B}]) \oplus \mathbf{x}_1$ 14: return $\mathbf{h}$ </pre>	

4.7.2 ZK Linear Universal Hash

The **ZKHash** algorithm batches checks on IT-MACs over $\mathbb{F}_{2^\lambda}$. In **QS.RSDProve**, it is applied to the IT-MAC coefficient vectors $\mathbf{a}_1$ and $\mathbf{a}_0$, together with the extra field elements $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{v}}$ for masking. In **QS.RSDVerify**, it is applied to the corresponding verifier-side local-key vector $\mathbf{b}$ and the extra local-key $\tilde{q}$ for $\tilde{\mathbf{u}}$. The result is the compressed value used in the final IT-MAC opening check. Thus **ZKHash** plays the same batching role as **VOLEHash**, but for $\mathbb{F}_{2^\lambda}$ -linear IT-MAC checks rather than $\mathbb{F}_2$ -linear VOLE consistency checks.

The seed is parsed into two random coefficients r_0, r_1 and two evaluation points s, t in $\mathbb{F}_{2^\lambda}$. The vector $\mathbf{x}_0$ is viewed as the coefficient vector of a polynomial over $\mathbb{F}_{2^\lambda}$ and evaluated twice, once at s and once at t . The two evaluations are then combined as $r_0 h_0 + r_1 h_1$. Finally, the field element x_1 is added. As in **VOLEHash**, adding x_1 provides hiding.

ZKHash (seed, $(\mathbf{x}_0, x_1)$)	(ZKHash is linear over $\mathbb{F}_{2^\lambda}$)
Input: $\left(\begin{array}{l} \text{seed} := (\mathbf{r}_0 \in \{0, 1\}^\lambda, \mathbf{r}_1 \in \{0, 1\}^\lambda, \mathbf{s} \in \{0, 1\}^\lambda, \mathbf{t} \in \{0, 1\}^{64}), \\ (\mathbf{x}_0 \in \mathbb{F}_{2^\lambda}^{\ell_{\text{wit}}}, x_1 \in \mathbb{F}_{2^\lambda}) \end{array} \right)$	

Output: $h \in \{0, 1\}^\lambda$

```

1:  $r_0 := \text{BitsToField}(r_0; \lambda)$ 
2:  $r_1 := \text{BitsToField}(r_1; \lambda)$ 
3:  $s := \text{BitsToField}(s; \lambda)$ 
4:  $t := \text{BitsToField}(t \parallel 0^{\lambda-64}; \lambda)$ 
5:  $h_0 := \sum_{i=0}^{\ell_{\text{wit}}-1} s^{\ell_{\text{wit}}-1-i} \cdot x_0[i]$ 
6:  $h_1 := \sum_{i=0}^{\ell_{\text{wit}}-1} t^{\ell_{\text{wit}}-1-i} \cdot x_0[i]$ 
7:  $h := \text{FieldToBits}(r_0 \cdot h_0 + r_1 \cdot h_1 + x_1; \lambda)$ 
8: return  $h$ 

```

4.8 Utility Functions

This subsection collects the basic conversion routines used throughout the rest of the section. Since the construction moves repeatedly between integers, bit strings, and finite-field elements, these helpers make the serialization and parsing conventions explicit and keep the interfaces of the higher-level algorithms concise.

We use three representations for data: integers, finite-field elements, and bit strings. All bit strings are indexed from left to right as $x[0], x[1], \dots$ but interpreted in little-endian order for arithmetic conversions. Thus $x[0]$ is the coefficient of 2^0 when converting to an integer, and it is the coefficient of X^0 when converting to a field element. The parameter $k \in \mathbb{N}^+$ fixes the bit length and the target field $\mathbb{F}_{2^k}$. These conventions are used for challenge decoding, counters, domain-separation bytes, and the universal-hash parsing routines above. The subsection contains the following algorithms.

- **BitsToInt**: interprets a little-endian bit string as an integer modulo 2^k .
- **IntToBits**: serializes an integer modulo 2^k as a little-endian bit string.
- **BitsToField**: interprets a bit string as a polynomial-basis element of $\mathbb{F}_{2^k}$.
- **FieldToBits**: serializes a field element by listing its polynomial-basis coefficients.

4.8.1 Bits to Integer

The **BitsToInt** algorithm interprets a length- k bit string as a little-endian integer modulo 2^k . It is used whenever a challenge substring or hash output must be read as an index, such as the BAVC leaf index in **VOLE.ChalDec**.

BitsToInt($x; k$)

Input: $x \in \{0, 1\}^k$

Output: $x \in \mathbb{Z}_{2^k}$

- ```

1: $x := \sum_{i=0}^{k-1} x[i] \cdot 2^i$
2: return x

```

### 4.8.2 Integer to Bits

The **IntToBits** algorithm is the inverse serialization map for integers modulo  $2^k$ . It outputs the little-endian bit decomposition used throughout the hash inputs, domain-separation bytes, counters, and public constants. The input is viewed modulo  $2^k$ , so the output always has exactly  $k$  bits. This fixed-length convention keeps XOF inputs unambiguous.

**IntToBits**( $x; k$ )

**Input:**  $x \in \mathbb{Z}_{2^k}$

**Output:**  $\mathbf{x} \in \{0, 1\}^k$

```

1: for $i \in [0, k)$ do
2: $\mathbf{x}[i] := (x \gg i) \pmod{2}$
3: end for
4: return $\mathbf{x}$

```

#### 4.8.3 Bits to Field

The **BitsToField** algorithm interprets a length- $k$  bit string as the polynomial-basis representation of an element of  $\mathbb{F}_{2^k}$ . The bit  $\mathbf{x}[i]$  is the coefficient of  $X^i$ , where  $X$  denotes the fixed generator used by the implementation of  $\mathbb{F}_{2^k}$ . This conversion is used to turn rows of VOLE tag matrices and local-key matrices into  $\mathbb{F}_{2^\lambda}$  elements, and also to parse universal-hash seeds as field coefficients.

**BitsToField**( $\mathbf{x}; k$ )

**Input:**  $\mathbf{x} \in \{0, 1\}^k$

**Output:**  $x \in \mathbb{F}_{2^k}$

```

1: $x := \sum_{i=0}^{k-1} \mathbf{x}[i] \cdot X^i$
2: return x

```

▷  $X$  is the generator of  $\mathbb{F}_{2^k}$ .

#### 4.8.4 Field to Bits

The **FieldToBits** algorithm serializes a field element by listing its coefficients in the same polynomial basis. It is the inverse of **BitsToField** under the fixed basis convention. All hash outputs that are computed as field elements are serialized with this routine before they are concatenated into transcripts or signatures.

**FieldToBits**( $x; k$ )

**Input:**  $x \in \mathbb{F}_{2^k}$

**Output:**  $\mathbf{x} \in \{0, 1\}^k$

```

1: Parse $x = x_0 + x_1X + \dots + x_{k-1}X^{k-1}$
2: $\mathbf{x} := [x_0, x_1, \dots, x_{k-1}]$
3: return $\mathbf{x}$

```

### 4.9 Symmetric Primitives

This subsection specifies the symmetric-key cryptographic primitives used in ReSolveD- $\alpha$ . It includes the XOF-based hash functions used to derive Fiat-Shamir challenges and commitments, the TCCR hash used in the pcGGM tree, the PRG used for seed expansion, leaf commitment, and noise sampling, and the underlying standardized interfaces for the XOF and block cipher. It contains the following algorithms.

- **MsgHash**, **CoinHash**, **BAVCHash**, and **MatHash**: domain-separated XOF calls for message binding, signing randomness, BAVC commitments, and matrix expansion.

- **FSHash<sub>1</sub>**, **FSHash<sub>2</sub>**, and **FSHash<sub>3</sub>**: the three Fiat-Shamir random-oracle calls used in the signing transcript.
- **TCCR**: the tweakable circular correlation robust hash used in the pcGGM tree of BAVC.
- **PRG**: the pseudorandom generator used for leaf commitments, noise sampling, and VOLE seed expansion.
- **XOF** and **Enc**: standardized primitive interfaces used to instantiate the random oracles and ideal ciphers.

For a hash function, whenever the security proof of [Section 8](#) requires it to be a random oracle, we realize it using **XOF** with a distinct domain separator. In contrast, **TCCR** and **PRG** are based on block ciphers.

#### 4.9.1 Message Hash

The message hash binds the public key to the signed message and also derives the salt  $\mathbf{s}$  used by the TCCR hash function. Including the public key prevents key-substitution attacks.

**MsgHash**(pk, msg)

**Input:**  $\text{msg} \in \{0, 1\}^*$ ,  $\text{pk} := (\text{seed}_{\text{pk}} \in \{0, 1\}^\lambda, \mathbf{y} \in \mathbb{F}_2^{m-k})$

**Output:**  $(\mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda)$

- 1:  $(\mu, \mathbf{s}) := \text{XOF}(\text{IntToBits}(0; 8) \parallel \text{seed}_{\text{pk}} \parallel \mathbf{y} \parallel \text{msg}; 3\lambda) \triangleright \text{IntToBits}(0; 8)$  is a domain separator.
- 2: **return**  $(\mu, \mathbf{s})$

#### 4.9.2 Signing-Coin Hash

The signing-coin hash deterministically derives the BAVC root seed and the 120-bit IV from the secret key, the message digest, and the per-signature randomness  $\rho$ . Setting  $\rho = 0^\lambda$  gives deterministic signing, while random  $\rho$  gives randomized signatures.

**CoinHash**(sk,  $\mu$ ,  $\rho$ )

**Input:**  $\text{sk} \in \{0, 1\}^{2\lambda}$ ,  $\mu \in \{0, 1\}^{2\lambda}$ ,  $\rho \in \{0, 1\}^\lambda$

**Output:**  $(\mathbf{r}, \text{iv}) \in \{0, 1\}^\lambda \times \{0, 1\}^{120}$

- 1:  $\mathbf{r} \parallel \text{iv} := \text{XOF}(\text{IntToBits}(1; 8) \parallel \text{sk} \parallel \mu \parallel \rho; \lambda + 120) \triangleright \text{IntToBits}(1; 8)$  is a domain separator.
- 2: **return**  $(\mathbf{r}, \text{iv})$   $\triangleright$  BAVC seed and salt.

#### 4.9.3 BAVC Commitment Hash

The BAVC commitment hash compresses all leaf commitments into the single commitment value  $\text{com}$  used in the Fiat-Shamir transcript. It first hashes the leaves in each small tree and then hashes the resulting per-tree digests together with  $\mu$ ,  $\mathbf{s}$ , and  $\text{iv}$ .

**BAVCHash**( $\mu, \mathbf{s}, (\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha]}, \text{iv}$ )

**Input:**  $\mu \in \{0, 1\}^{2\lambda}$ ,  $\mathbf{s} \in \{0, 1\}^\lambda$ ,  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha]} \in \{0, 1\}^{2L\lambda}$ ,  $\text{iv} \in \{0, 1\}^{120}$

**Output:**  $\text{com} \in \{0, 1\}^{2\lambda}$

- 1: **for**  $\alpha \in [0, \tau)$  **do**

```

2: comα := XOF((comiα)i∈[0,Nα]; 2λ)
3: end for
4: com := XOF(IntToBits(2; 8) || μ || s || (comα)α∈[0,τ] || iv; 2λ) ▷ IntToBits(2; 8) is a domain
 separator.
5: return com

```

#### 4.9.4 Matrix Expansion Hash

The matrix hash expands the public-key seed into the dense matrix block  $H_B$ . Together with the fixed identity block, this determines the systematic RSD matrix used by the public key.

**MatHash**(seed<sub>pk</sub>)

**Input:** seed<sub>pk</sub> ∈ {0, 1}<sup>λ</sup>

**Output:**  $H_B \in \mathbb{F}_2^{(m-k) \times k}$

```

1: h := XOF(IntToBits(3; 8) || seedpk; (m - k) · k) ▷ IntToBits(3; 8) is a domain separator.
2: for i ∈ [0, m - k) do
3: for j ∈ [0, k) do
4: HB[i, j] := h[i · k + j]
5: end for
6: end for
7: return HB

```

#### 4.9.5 First Fiat-Shamir Hash

The first Fiat-Shamir hash derives chall<sub>1</sub> from the BAVC commitment and the correction vectors. Its output serves as a seed of the VOLE consistency hash.

**FSHash<sub>1</sub>**(x)

**Input:** x ∈ {0, 1}<sup>\*</sup>

**Output:** h ∈ {0, 1}<sup>5λ+64</sup>

```

1: h := XOF(IntToBits(8 + 1; 8) || x; 5λ + 64) ▷ IntToBits(8 + 1; 8) is a domain separator.
2: return h

```

#### 4.9.6 Second Fiat-Shamir Hash

The second Fiat-Shamir hash derives chall<sub>2</sub> after the VOLE consistency values and masked witness have been fixed. This challenge seeds the QuickSilver batching hashes.

**FSHash<sub>2</sub>**(x)

**Input:** x ∈ {0, 1}<sup>\*</sup>

**Output:** h ∈ {0, 1}<sup>3λ+64</sup>

```

1: h := XOF(IntToBits(8 + 2; 8) || x; 3λ + 64) ▷ IntToBits(8 + 2; 8) is a domain separator.
2: return h

```

#### 4.9.7 Third Fiat-Shamir Hash

The third Fiat-Shamir hash derives the final challenge  $\text{chall}_3$  from the QuickSilver response and the grinding counter. The signer repeats this call until the proof-of-work suffix condition and the BAVC opening bound are both satisfied.

**FSHash<sub>3</sub>**( $x$ )

**Input:**  $x \in \{0, 1\}^*$

**Output:**  $h \in \{0, 1\}^\lambda$

- 1:  $h := \text{XOF}(\text{IntToBits}(8 + 3; 8) \| x; \lambda)$  ▷  $\text{IntToBits}(8 + 3; 8)$  is a domain separator.
- 2: **return**  $h$

#### 4.9.8 TCCR Hash Function

We define the following Tweakable Circular Correlation Robust (TCCR) hash function with 120-bit salt. The TCCR design is based on the block-cipher interface **Enc** with  $n_k$ -bit key and  $n_b$ -bit block size, satisfying  $n_k \geq \lambda$ ,  $\lceil \frac{\lambda}{n_b} \rceil \leq 2^8$ , and  $n_b \geq 128$ . The **TCCR** algorithm hashes a  $\lambda$ -bit value with a salt-derived tweak and a public mask  $s$ . For  $\lambda \leq n_b$ , one block-cipher call suffices; for larger parameters, the construction derives a fixed key from the right part of the masked input and emits enough block-cipher outputs to cover  $\lambda$  bits.

**TCCR**( $x, s, \text{iv}$ )

**Input:** ( $x \in \{0, 1\}^\lambda, s \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}$ )

**Output:**  $h \in \{0, 1\}^\lambda$

- 1:  $\bar{x} := x \oplus s$
- 2: **if**  $\lambda \leq n_b$  **then**
- 3:     **return**  $h := \text{Enc}(\text{iv} \| 0^{n_k-120}, \sigma(\bar{x} \| 0^{n_b-\lambda}); \lambda) \oplus \sigma(\bar{x} \| 0^{n_b-\lambda})$
- 4: **end if**
- 5:  $x_L := \bar{x}[0, n_b - 8)$
- 6:  $x_R := \bar{x}[n_b - 8, \lambda)$
- 7:  $N_{\text{block}} := \lceil \frac{\lambda}{n_b} \rceil$
- 8: **for**  $i \in [0, N_{\text{block}})$  **do**
- 9:      $\bar{i} := \text{IntToBits}(i; 8)$
- 10:     $h_i := \text{Enc}(x_R \| \text{iv} \| 0^{n_k+n_b-128-\lambda}, \bar{i} \| x_L; \lambda) \oplus (\bar{i} \| x_L)$
- 11: **end for**
- 12: **return**  $h := h_0 \| \dots \| h_{N_{\text{block}}-1}[0, \lambda - n_b \cdot (N_{\text{block}} - 1))$

#### 4.9.9 Pseudorandom Number Generator

Based on the off-the-shelf block cipher construction, we follow different strategies for building **PRG** under different security parameters. In particular, for  $\lambda = 160, 256, 512$ , there are suitable block ciphers with sufficient key lengths, namely AES-192, AES-256, and SHACAL-2. Therefore, the regular counter mode cipher is sufficient. For  $\lambda = 384$ , we re-use the existing **TCCR** infrastructure.

As for the counter range, **PRG** is called in **VOLE.ConvertToVOLE**, **LeafHash** and **RSD.SampleNoise**, and in all cases the total output length for a fixed seed and iv is less than  $2^8 \cdot n_b$ . Therefore, using an 8-bit counter in counter mode is sufficient. The **PRG** algorithm therefore exposes a uniform interface over all parameter sets while switching internally between counter-mode encryption and the TCCR-based construction for  $\lambda = 384$ .

**PRG**( $\mathbf{k}, \text{iv}, t; \ell$ )

**Input:** ( $\mathbf{k} \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}, t \in \mathbb{Z}_{2^8}; \ell \in \mathbb{N}^+$ )

**Output:**  $\mathbf{r} \in \{0, 1\}^\ell$

```

1: if $\lambda \in \{160, 256, 512\}$ then
2: $\bar{\mathbf{k}} := \mathbf{k} \| 0^{n_k - \lambda}$ ▷ Zero-pad the seed to n_k bits.
3: $c := \lceil \frac{\ell}{n_b} \rceil$
4: for $i \in [0, c)$ do
5: $\mathbf{m} := \text{iv} \| \text{IntToBits}(t + i; 8) \| 0^{n_b - 128}$ ▷ Zero-pad $\mathbf{m}$ to n_b bits.
6: $\mathbf{r}_i := \text{Enc}(\bar{\mathbf{k}}, \mathbf{m}; \lambda)$
7: end for
8: return $\mathbf{r} := \mathbf{r}_0 \| \mathbf{r}_1 \| \dots \| \mathbf{r}_{c-1} [0, \ell - (c - 1) \cdot n_b]$
9: else ▷ $\lambda = 384$
10: $c := \lceil \frac{\ell}{384} \rceil$
11: for $i \in [0, c)$ do
12: $\mathbf{s} := \text{IntToBits}(t + i; 8) \| 0^{\lambda - 8}$
13: $\mathbf{r}_i := \text{TCCR}(\mathbf{k}, \mathbf{s}, \text{iv})$
14: end for
15: return $\mathbf{r} := \mathbf{r}_0 \| \mathbf{r}_1 \| \dots \| \mathbf{r}_{c-1} [0, \ell - (c - 1) \cdot \lambda]$
16: end if

```

#### 4.9.10 Extendable-Output Function

The **XOF** algorithm is the common random-oracle interface used by all domain-separated hash calls. It is modeled as a random oracle in the security analysis. We provide two versions of this interface. The first one is the SM3-based “pseudoXOF” function provided by the contest, and the second one is a standardized Keccak-based instantiation. The concrete choice for each security parameter is given in Table 5.

**XOF**( $\mathbf{x}; \ell$ )

**Input:**  $\mathbf{x} \in \{0, 1\}^*$

**Output:**  $\mathbf{h} \in \{0, 1\}^\ell$

```

1: $\mathbf{h} := \text{PseudoXOF}(\mathbf{x}; \ell)$ or $\mathbf{h} := \text{Keccak}[2\lambda](\mathbf{x}; \ell)$ ▷ $\text{Keccak}[2\lambda](\cdot; \ell)$ denotes Keccak
 function with 2λ -bit capacity and ℓ -bit output.
2: return $\mathbf{h}$

```

#### 4.9.11 Block-Cipher Interface

The **Enc** algorithm abstracts the concrete block cipher used by **TCCR** and **PRG**. It has  $n_b$ -bit block size and  $n_k$ -bit key size, and it is modeled as an ideal cipher in the security analysis. In the current implementation, the selected primitive depends on the target security parameter as specified in Table 6. Future standardized symmetric cryptographic algorithms with suitable key and block lengths could serve as direct drop-in replacements for the current choices.

**Enc**( $\mathbf{k}, \mathbf{x}; \lambda$ )

**Input:** ( $\mathbf{k} \in \{0, 1\}^{n_k}, \mathbf{x} \in \{0, 1\}^{n_b}$ )

▷  $\lambda \in \{160, 256, 384, 512\}$

**Output:**  $\mathbf{c} \in \{0, 1\}^{n_b}$

```

1: if $\lambda = 160$ then

```

```

2: return $c := \text{AES-192}(k, x)$
3: else if $\lambda = 256$ then
4: return $c := \text{AES-256}(k, x)$
5: else if $\lambda = 384$ then
6: return $c := \text{Rijndael-256}(k, x)$
7: else $\triangleright \lambda = 512$
8: return $c := \text{SHACAL-2}(k, x)$
9: end if

```

## 5 Parameter Selection

### 5.1 Classical and Quantum Security Levels

The submission requirements of the NGCC competition [Ins25] define three mandatory security levels and one optional security level: 128-bit classical (80-bit quantum), 256-bit classical (128-bit quantum), 384-bit classical (192-bit quantum), and 512-bit classical (256-bit quantum) security levels. Accordingly, this document provides four parameter sets, i.e., ReSolveD- $\alpha$ -160, ReSolveD- $\alpha$ -256, ReSolveD- $\alpha$ -384, and ReSolveD- $\alpha$ -512, corresponding to the 80-bit, 128-bit, 192-bit, and 256-bit quantum security, respectively.

We consider the Grover algorithm with unlimited depth of quantum circuits to obtain a conservative security estimate. In this case, for the  $\lambda$ -bit quantum security level, one should use symmetric-key primitives with at least  $2\lambda$ -bit classical security. Similarly, we argue that the statistical security level of the proof systems should also be at least  $2\lambda$  bits, because we can imagine a trivial forgery strategy of enumerating the random coins until running into one that fails the check’s soundness. A  $2\lambda$ -bit soundness level guarantees that the density of such “bad” coins is upper-bounded by  $1/2^{2\lambda}$ . Therefore, even with Grover’s search, the expected work of such a strategy is bounded by  $2^\lambda$ , satisfying the  $\lambda$ -bit security requirement.

All required parameters satisfy the above pattern except the first one, which requires 128-bit classical security and 80-bit quantum security. Following the above argument, we conservatively target 160-bit classical security in the first parameter set, namely, ReSolveD- $\alpha$ -160.

### 5.2 Selection of RSD Parameters

Following standard practice for symmetric cryptography, we interpret a target classical security parameter  $\lambda$  as requiring at least  $2^\lambda$  symmetric-key operations, and we estimate the cost of one symmetric-key operation over a  $\lambda$ -bit key as  $\lambda^{2.3}$ . We view this estimate as conservative, as in the call for proposals of the NIST PQC project [NIS22] the circuit complexities of AES-128, AES-192, and AES-256 are estimated to be  $128^{2.14}$ ,  $192^{1.98}$ , and  $256^{2.0}$ , respectively. Accordingly, the underlying RSD instances are chosen so that the best-known classical attacks still exceed the required work factor after this overhead is taken into account.

To identify suitable RSD instances, we enumerate block sizes and noise weights within the ranges  $\ell_{\text{bs}} \in [4, 128]$ ,  $w \in [50, 999]$ . For each candidate, we choose the code dimension  $k$  in the hardness-maximizing regime identified by Carozza, Couteau, and Joux [CCJ23], namely

$$\ell_{\text{bs}}^w \leq 2^{m-k} \leq \binom{m}{w}. \quad (7)$$

Moreover, to facilitate provable security in the quantum random oracle model following the approach in [BBB<sup>+</sup>26], we additionally require the RSD-based one-way function to have a co-domain larger than its domain. In particular, we require that the number of regular noise

vectors does not exceed the number of syndromes, namely  $\ell_{\text{bs}}^w \leq 2^{m-k}$ . By the constraint in Equation (7), this constraint is satisfied automatically.

We then evaluate the concrete bit security of the resulting  $(m, k, w, \ell_{\text{bs}})$  combinations, under the conservative estimation of the RSD attacks in Section 7.2. We retain only those parameters that can offer  $(\lambda + 2.3 \cdot \log(\lambda))$ -bit security level. Among the candidates with sufficient security levels, we select parameters that minimize the effective witness length

$$\ell_{\text{vole}} = \ell_{\text{wit}} + (d - 1) \cdot \lambda, \quad (8)$$

where  $\ell_{\text{wit}}$  is the length of the extended witness in the underlying zero-knowledge proof and  $d$  is the maximum degree of the proved constraints. This quantity is a primary driver of signature size in the VOLE-in-the-Head framework, because the final signature length grows essentially linearly with  $\ell_{\text{vole}}$ . Since in ReSolveD- $\alpha$  we utilize the linear sketching-based method to prove the RSD problem, we only need to prove degree two constraints in QuickSilver, i.e.,  $d = 2$ . Following the witness length shrinking trick in [CCJ23], we have

$$\ell_{\text{wit}} = k \cdot \left(1 - \frac{1}{\ell_{\text{bs}}}\right). \quad (9)$$

Following the above method, we selected the RSD parameters for  $\lambda \in \{160, 256, 384, 512\}$ , which corresponds to classical and statistical security levels  $\text{bitsec}_c \in \{160, 256, 384, 512\}$  and quantum security levels  $\text{bitsec}_q \in \{80, 128, 192, 256\}$ .

In particular, we enumerate noise block sizes in the range 4 to 128 and noise weights in the range 50 to 999, and estimate their bit security with the conservative method in Section 7.2. Among the parameters that offer the targeted  $\lambda + 2.3 \log(\lambda)$  bit security level, we choose those that minimize signature sizes. The RSD parameters of ReSolveD- $\alpha$  is listed in Table 3. The concrete hardness estimation results is shown in Table 18.

Table 3: RSD parameters for ReSolveD- $\alpha$

| Parameter Set           | Quantum Security<br>$\text{bitsec}_q$ | Code Length<br>$m$ | Code Dimension<br>$k$ | Noise Weight<br>$w$ | Block Size<br>$\ell_{\text{bs}}$ |
|-------------------------|---------------------------------------|--------------------|-----------------------|---------------------|----------------------------------|
| ReSolveD- $\alpha$ -160 | 80                                    | 1860               | 1056                  | 310                 | 6                                |
| ReSolveD- $\alpha$ -256 | 128                                   | 2988               | 1698                  | 498                 | 6                                |
| ReSolveD- $\alpha$ -384 | 192                                   | 4452               | 2532                  | 742                 | 6                                |
| ReSolveD- $\alpha$ -512 | 256                                   | 5940               | 3378                  | 990                 | 6                                |

### 5.3 Selection Approach of VOLEitH Parameters

Once the RSD parameters are fixed, the remaining concrete efficiency trade-offs are determined by the VOLE-in-the-Head parameters, namely the repetition parameter  $\tau$ , the proof-of-work parameter  $w_{\text{grind}}$ , and the BAVC opening-size threshold  $T_{\text{open}}$ . These parameters jointly determine the signature size and the expected signing time. A larger  $\tau$  means more VOLE instances with shallower per-instance GGM trees, which reduces the total number of BAVC leaves  $L$  and thus speeds up signing (since BAVC tree operations dominate the running time), but increases the signature size due to the additional  $\tau - 1$  correction vectors  $\mathbf{c}_i$ . The proof-of-work parameter  $w_{\text{grind}}$  reduces the effective challenge space from  $\lambda$  to  $\lambda - w_{\text{grind}}$  bits, forcing the signer to perform roughly  $2^{w_{\text{grind}}}$  extra hash iterations to find a valid  $\text{chall}_3$ ; this reduces the BAVC tree size at the cost of increased signing work.  $T_{\text{open}}$  controls the space-time trade-off within each parameter band: a smaller  $T_{\text{open}}$  reduces the signature size but raises the probability that the BAVC opening exceeds the threshold, requiring the signer to retry.

Our selection methodology follows the structural patterns observed across the 12 FAEST and FAEST-EM parameter sets [BBB<sup>+</sup>26]. Analysis of these variants reveals that the GGM tree depth  $k_0 = \lceil (\lambda - w_{\text{grind}})/\tau \rceil$  consistently falls into one of two bands: fast variants use  $k_0 \in \{7, 8\}$ , while short variants use  $k_0 \in \{11, 12\}$ . We therefore adopt the same bands and, for each of the four Next Generation Commercial Cryptography (NGCC) security levels ( $\lambda = 160, 256, 384, 512$ ) [Ins25], enumerate all feasible  $(\tau, w_{\text{grind}}, T_{\text{open}})$  triples within the corresponding  $k_0$  range. From the candidates, we select those minimizing the combined size-speed cost metric  $L \cdot (T_{\text{open}} + 2\tau)$ , where  $L$  reflects the signing time (proportional to the number of BAVC leaf commitments and tree PRG calls) and  $T_{\text{open}} + 2\tau$  reflects the signature-size contribution of the BAVC component. The BAVC opening-size distribution, which determines the acceptance probability for a given  $T_{\text{open}}$ , is computed exactly using the divide-and-conquer dynamic programming method described in Section 5.4.

For the *fast* variant, we additionally require  $w_{\text{grind}} \geq 8$  and  $2^{w_{\text{grind}}} \leq L/10$ , ensuring that the proof-of-work grinding dominates the total signing attempts. We set  $T_{\text{open}}$  generously such that the BAVC rejection probability is at most  $1/3$ , so the expected number of attempts stays close to the grinding floor  $2^{w_{\text{grind}}}$ . For the *small* variant, we set  $T_{\text{open}}$  more tightly, targeting an expected number of attempts in the range  $[L/6, L/4]$ , which yields a shorter signature at the cost of more retries during signing. The resulting parameters are shown in Table 4 of Section 5.5.

#### 5.4 Failure Probability Analysis

As described in Section 5.3, the rejection sampling threshold  $T_{\text{open}}$  directly affects both the signature size and the expected signing time. Choosing  $T_{\text{open}}$  requires knowledge of the distribution of the BAVC opening size, i.e., the number of internal tree nodes that must be revealed when opening all-but- $\tau$  leaves of a binary tree with  $L$  leaves.

In the ReSolveD- $\alpha$  algorithm, the  $\tau$  small binary trees are mapped to a single large tree by interleaving their leaves: the  $j$ -th leaf of the large tree corresponds to the  $\lfloor j/\tau \rfloor$ -th leaf of the  $(j \bmod \tau)$ -th small tree. This layout is convenient for implementation. However, this interleaved layout does not decompose cleanly into independent subproblems, making it difficult to compute the opening-size distribution analytically. To address this, we consider a broader random model in which the  $\tau$  punctured leaves are chosen uniformly at random from all  $L$  leaves. This model covers the interleaved layout as a special case, and crucially, it admits a recursive decomposition: the sampling problem on a tree with  $L$  leaves and  $\tau$  punctured leaves can be split into independent subproblems on the left and right subtrees.

The following algorithm exploits this recursive structure to compute the exact distribution of the opening size. Dynamic programming with memoization on  $(L, \tau)$  avoids recomputation of subproblems.

##### RandOneTreeSample( $L, \tau$ )

**Input:**  $L, \tau$

▷ Number of leaves and number of punctured leaves.

**Output:**  $D$

▷ Distribution of opening sizes.

1: **if**  $\tau = 0$  **then**

2:     **return**  $\{1 : 0\}$

3: **else if**  $L \leq \tau$  **then**

4:     **return**  $\{0 : 1\}$

5: **end if**

6:  $D := \{\}$

▷ Format: {opening size : probability}.

7:  $L_L :=$  number of leaves in left subtree

8:  $L_R :=$  number of leaves in right subtree

9: **for**  $i \in [0, \tau]$  **do**

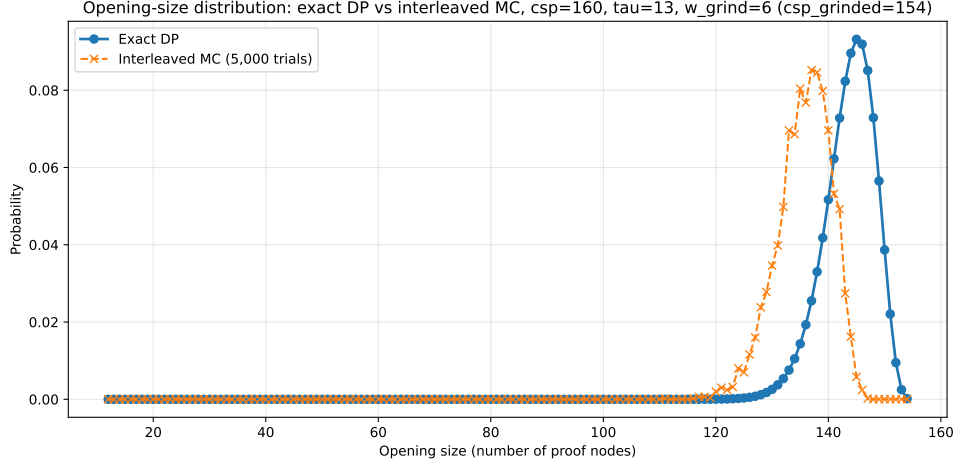


Figure 6: An example of BAVC opening size with  $\lambda = 160$ ,  $w_{\text{grind}} = 6$ ,  $\tau = 13$ , illustrating that the random sampling model is a conservative one in terms of size estimation.

```

10: $D_L := \text{RandOneTreeSample}(L_L, i)$
11: $D_R := \text{RandOneTreeSample}(L_R, \tau - i)$
12: $D' := D_L * D_R$ ▷ Cartesian product of the two distributions.
13: $D := D + \frac{\binom{L_L}{i} \cdot \binom{L_R}{\tau - i}}{\binom{L}{\tau}} \cdot D'$ ▷ Weighted sum.
14: end for
15: return D

```

Empirical validation confirms that the random sampling model used by `RandOneTreeSample` provides a conservative estimate of the opening size: it consistently yields slightly larger opening sizes than the actual interleaved layout, which leads to a lower rejection rate in practice and is therefore favorable for parameter selection. For example, in Figure 6, the opening size distribution of the random sampling model, which is exactly computed using the dynamic programming method, is juxtaposed with the distribution of the actual interleaving method, which is heuristically obtained using the Monte-Carlo method with 5,000 trials.

## 5.5 VOLE-in-the-Head Parameters

We follow the approach outlined in the previous subsections, and select two variants of parameters for each security level—a small variant and a fast variant. The rejection probability is estimated using the `RandOneTreeSample` algorithm as described in Section 5.4. The parameters, together with signature sizes, are listed in Table 4.

## 6 Performance Evaluation

We provide two implementations of `ReSolveD- $\alpha$`  as part of our proposal. The first one is a reference implementation, which is written in standard ANSI C, targeting close adherence to the specification and maximum portability. The second one is an optimized C++ implementation for x86 platforms, using platform-specific instruction sets such as AES-NI, SHA-NI, AVX2, AVX-512, and VAES. The two implementations are functionally equivalent and provide the same API. We benchmark both implementations with two XOF backends: the NGCC SM3-based pseudoXOF backend specified in the `API.PKC.zip` package and a standard Keccak backend.

Table 4: VOLE-in-the-Head parameters for ReSolveD- $\alpha$ .  $\ell_{\text{vole}}$ ,  $\tau$ ,  $w_{\text{grind}}$ , and  $T_{\text{open}}$  denote the VOLE commitment length, the number of VOLE instances, the proof-of-work level, and the threshold for the number of opened BAVC nodes, respectively.  $|\text{pk}|$ ,  $|\text{sk}|$ , and  $|\sigma|$  denote the sizes of public key, secret key, and signature, respectively.

| Parameter set            | VOLEitH Parameters   |        |                    |                   | Sizes (Bytes) |               |            |                          |
|--------------------------|----------------------|--------|--------------------|-------------------|---------------|---------------|------------|--------------------------|
|                          | $\ell_{\text{vole}}$ | $\tau$ | $w_{\text{grind}}$ | $T_{\text{open}}$ | $ \text{pk} $ | $ \text{sk} $ | $ \sigma $ | $ \text{pk}  +  \sigma $ |
| ReSolveD- $\alpha$ -160s | 1056                 | 14     | 6                  | 129               | 121           | 40            | 5307       | 5428                     |
| ReSolveD- $\alpha$ -160f | 1056                 | 21     | 8                  | 139               | 121           | 40            | 6851       | 6972                     |
| ReSolveD- $\alpha$ -256s | 1687                 | 22     | 12                 | 224               | 194           | 64            | 13973      | 14167                    |
| ReSolveD- $\alpha$ -256f | 1687                 | 35     | 8                  | 223               | 194           | 64            | 17932      | 18126                    |
| ReSolveD- $\alpha$ -384s | 2510                 | 34     | 10                 | 332               | 288           | 96            | 31575      | 31863                    |
| ReSolveD- $\alpha$ -384f | 2510                 | 53     | 9                  | 336               | 288           | 96            | 40469      | 40757                    |
| ReSolveD- $\alpha$ -512s | 3343                 | 46     | 6                  | 439               | 385           | 128           | 56239      | 56624                    |
| ReSolveD- $\alpha$ -512f | 3343                 | 72     | 8                  | 447               | 385           | 128           | 72611      | 72996                    |

## 6.1 Instantiations of Standard Symmetric-key Primitives

Except for RSD, the ReSolveD- $\alpha$  signature scheme depends on the standard symmetric-key primitives: *block ciphers* for PRG and TCCR, and *cryptographic hash functions* for Fiat-Shamir and sampling public randomness. Below, we show how to instantiate these symmetric-key primitives with AES [Nat01], SHACAL-2 [KKL<sup>+</sup>05] and Keccak [ND15] whose security has been extensively studied over a long period of time. In the future, we are able to replace these symmetric-key primitives with future standardized primitives.

**Cryptographic hash functions.** The NGCC competition provides an SM3-based pseudoXOF function (in the API\_PKC.zip package) for the evaluation of the submitted algorithms’ correctness and functionality. To comply with this evaluation framework, we include a version of ReSolveD- $\alpha$  that uses pseudoXOF as its underlying XOF. For a standardized alternative, we also offer a version that employs Keccak, which is the winner of the NIST SHA-3 cryptographic hash competition [ND15] and has withstood extensive public scrutiny, providing high confidence in its security. In this version, for a given security parameter  $\lambda$ , we instantiate the XOF as Keccak[ $2\lambda$ ] with a capacity of  $2\lambda$  bits, offering  $\lambda$ -bit collision resistant, which is indeed a SHAKE variant. The XOF choices for each parameter set are summarized in Table 5.

Table 5: XOF used in each parameter set

| Parameter Set           | Quantum security | pseudoXOF backend | Keccak backend | Reference |
|-------------------------|------------------|-------------------|----------------|-----------|
| ReSolveD- $\alpha$ -160 | 80               | pseudoXOF         | Keccak[320]    | [ND15]    |
| ReSolveD- $\alpha$ -256 | 128              | pseudoXOF         | Keccak[512]    | [ND15]    |
| ReSolveD- $\alpha$ -384 | 192              | pseudoXOF         | Keccak[768]    | [ND15]    |
| ReSolveD- $\alpha$ -512 | 256              | pseudoXOF         | Keccak[1024]   | [ND15]    |

**Symmetric-key primitives from block ciphers.** Aside from random oracles, we utilize the following standardized primitives in various components of ReSolveD- $\alpha$ :

- The TCCR algorithm requires a suitable block cipher modeled as an ideal cipher;
- The LeafHash algorithm in BAVC.Commit and BAVC.Verify invokes a PRG;
- The VOLE.ConvertToVOLE algorithm utilizes a PRG;

- The [RSD.SampleNoise](#) algorithm utilizes a [PRG](#) to sample an RSD noise.

We use the standardized block ciphers to instantiate the above components, with the concrete choice depending on the target security level. The general principles are that:

- For the [TCCR](#) construction in [Section 2.3](#), the combined key length and block length  $n_k + n_b$  should be no less than  $\lambda + 128$ , where  $\lambda$  is the security parameter;
- To instantiate [PRG](#) with a block cipher in the counter mode, we require  $n_k \geq \lambda$ .

The design choices are summarized in [Table 6](#). For the 160-bit and 256-bit parameter sets, we use AES-192 and AES-256, with 192/256-bit key and 128-bit block, to instantiate both [TCCR](#) and [PRG](#). For the 384-bit parameter set, we use Rijndael-256 with 256-bit key and 256-bit block to construct a TCCR hash function following the approach outlined in [Section 2.3](#), and then use this TCCR construction to build the [PRG](#). For the 512-bit parameter set, we use SHACAL-2 with 512-bit key and 256-bit block to instantiate the TCCR hash function, and use its counter mode to realize the [PRG](#).

Table 6: Symmetric-key primitives based on block ciphers for each parameter set

| Parameter Set           | Quantum security | TCCR Hash    | PRG                     | Reference                           |
|-------------------------|------------------|--------------|-------------------------|-------------------------------------|
| ReSolveD- $\alpha$ -160 | 80               | AES-192      | AES-192                 | <a href="#">[Nat01]</a>             |
| ReSolveD- $\alpha$ -256 | 128              | AES-256      | AES-256                 | <a href="#">[Nat01]</a>             |
| ReSolveD- $\alpha$ -384 | 192              | Rijndael-256 | Rijndael-256-based TCCR | <a href="#">[DR00]</a>              |
| ReSolveD- $\alpha$ -512 | 256              | SHACAL-2     | SHACAL-2                | <a href="#">[KKL<sup>+</sup>05]</a> |

The block ciphers AES-128, AES-192, and AES-256 are the Rijndael variants selected by NIST as the Advanced Encryption Standard (AES) [\[Nat01\]](#). The Rijndael-256 is now promoted as an officially recognized new standard variant by NIST [\[Nat24\]](#). Modern processors provide dedicated instruction sets such as AES-NI, which offer substantial acceleration for AES or Rijndael computations. SHACAL-2 reuses the compression function of SHA-256 standard [\[Nat15\]](#). It was chosen as a standard block cipher by the NESSIE project [\[Eur03\]](#) in 2003 and remains secure for more than 20 years. Moreover, the evaluation of SHACAL-2 can be accelerated using the SHA-NI instruction set.

## 6.2 Implementation Platform

We tested both implementations on a laptop computer equipped with an AMD Ryzen 9 9955HX3D CPU (16 cores, 32 threads, x86\_64) and 30 GiB memory. The CPU clock frequency ranges from 1.22 GHz to 5.46 GHz. The compiler is GCC 15.2.0 and the operating system is Ubuntu 26.04 LTS, Linux 7.0.0-15-generic. [Table 7](#) summarizes the implementation platforms used for benchmarking.

Table 7: Benchmark platforms for the reference and optimized implementations.

| Item                  | Reference implementation | Optimized implementation                        |
|-----------------------|--------------------------|-------------------------------------------------|
| Language              | ANSI C                   | C++                                             |
| Instruction set       | Portable C only          | AES-NI, AVX2, SHA-NI, AVX-512, VAES, VPCLMULQDQ |
| Cryptographic library | XKCP for Keccak          | XKCP for Keccak                                 |

### 6.3 Software Performance

Tables 8 to 11 report the wall-clock time as well as cycle counts for key generation, signing, and verification across all parameter sets. Each entry is the mean over 100 trials. For the optimized AVX tables, cycle counts are the median of 100 cycle-counter samples after 10 warm-up invocations. We report separate tables for each combination of implementation and XOF backend. In all tables, time is reported in ms and cycle counts are reported as processor cycles, with commas used as thousands separators.

Table 8: Benchmarks for the reference implementation using the NGCC pseudoXOF backend.

| Parameter set            | KeyGen |            | Sign    |               | Verify  |               |
|--------------------------|--------|------------|---------|---------------|---------|---------------|
|                          | ms     | cycles     | ms      | cycles        | ms      | cycles        |
| ReSolveD- $\alpha$ -160s | 1.588  | 3,963,209  | 133.459 | 333,017,046   | 127.867 | 319,063,191   |
| ReSolveD- $\alpha$ -160f | 1.600  | 3,994,012  | 24.771  | 61,811,652    | 21.872  | 54,576,924    |
| ReSolveD- $\alpha$ -256s | 4.010  | 10,006,987 | 358.171 | 893,732,579   | 343.452 | 857,004,888   |
| ReSolveD- $\alpha$ -256f | 4.046  | 10,096,658 | 50.152  | 125,142,311   | 44.173  | 110,225,498   |
| ReSolveD- $\alpha$ -384s | 9.308  | 23,226,718 | 977.054 | 2,438,010,082 | 923.580 | 2,304,577,976 |
| ReSolveD- $\alpha$ -384f | 9.527  | 23,773,328 | 137.443 | 342,957,045   | 121.954 | 304,308,796   |
| ReSolveD- $\alpha$ -512s | 23.710 | 59,163,496 | 659.060 | 1,644,528,999 | 568.416 | 1,418,349,272 |
| ReSolveD- $\alpha$ -512f | 24.084 | 60,096,685 | 140.210 | 349,861,554   | 110.093 | 274,711,947   |

Table 9: Benchmarks for the reference implementation using the Keccak backend.

| Parameter set            | KeyGen |            | Sign    |               | Verify  |               |
|--------------------------|--------|------------|---------|---------------|---------|---------------|
|                          | ms     | cycles     | ms      | cycles        | ms      | cycles        |
| ReSolveD- $\alpha$ -160s | 0.814  | 2,032,308  | 123.602 | 308,420,439   | 120.478 | 300,623,412   |
| ReSolveD- $\alpha$ -160f | 0.947  | 2,363,586  | 22.670  | 56,569,314    | 20.376  | 50,845,244    |
| ReSolveD- $\alpha$ -256s | 1.992  | 4,971,751  | 328.132 | 818,777,851   | 322.292 | 804,204,490   |
| ReSolveD- $\alpha$ -256f | 2.016  | 5,030,334  | 43.785  | 109,255,154   | 39.950  | 99,687,140    |
| ReSolveD- $\alpha$ -384s | 5.075  | 12,664,333 | 870.154 | 2,171,264,265 | 858.382 | 2,141,891,088 |
| ReSolveD- $\alpha$ -384f | 5.070  | 12,651,140 | 119.243 | 297,542,672   | 110.232 | 275,058,955   |
| ReSolveD- $\alpha$ -512s | 8.295  | 20,699,043 | 422.351 | 1,053,878,445 | 403.273 | 1,006,273,612 |
| ReSolveD- $\alpha$ -512f | 8.293  | 20,695,033 | 90.675  | 226,258,413   | 77.573  | 193,565,584   |

### 6.4 Comparison

We first compare the performance of ReSolveD- $\alpha$  with a state-of-the-art RSD-based post-quantum digital signature scheme SDitH [ABB<sup>+</sup>26], which has proceeded into the third round of the NIST PQC competition for additional signatures [NIS22]. As discussed in Sections 1.1 and 7, during selecting RSD parameters, we assumed a linear-time approximate collision search algorithm, which greatly accelerated RSD attack algorithms. Nevertheless, existing approximate collision search algorithms are strictly super-linear [MO15, ES24]. This renders the selected RSD parameters in ReSolveD- $\alpha$  particularly conservative with respect to security margins. Therefore, we also evaluate the overhead with the *recommended* conservative parameters, by comparing ReSolveD- $\alpha$  with a variant of ReSolveD- $\alpha$  (called ReSolveD- $\alpha'$ ), where ReSolveD- $\alpha'$  is the same as ReSolveD- $\alpha$  except for adopting an *alternative* set of somewhat more aggressive parameters that are estimated via the best-known RSD attacks (ignoring the linear-time approximate collision search attacks).

Table 10: Benchmarks for the optimized AVX implementation using the NGCC pseudoXOF backend.

| Parameter set            | KeyGen |            | Sign    |             | Verify  |             |
|--------------------------|--------|------------|---------|-------------|---------|-------------|
|                          | ms     | cycles     | ms      | cycles      | ms      | cycles      |
| ReSolveD- $\alpha$ -160s | 0.922  | 2,301,300  | 13.488  | 33,532,362  | 10.646  | 26,499,312  |
| ReSolveD- $\alpha$ -160f | 0.921  | 2,301,462  | 1.712   | 4,186,050   | 1.440   | 3,599,587   |
| ReSolveD- $\alpha$ -256s | 2.395  | 5,985,275  | 37.558  | 83,875,600  | 27.370  | 68,300,150  |
| ReSolveD- $\alpha$ -256f | 2.400  | 5,988,350  | 3.550   | 8,680,762   | 3.159   | 7,895,962   |
| ReSolveD- $\alpha$ -384s | 5.435  | 13,503,650 | 134.735 | 299,571,875 | 95.539  | 237,810,587 |
| ReSolveD- $\alpha$ -384f | 5.424  | 13,525,912 | 14.322  | 33,839,787  | 11.776  | 29,365,562  |
| ReSolveD- $\alpha$ -512s | 18.201 | 44,799,762 | 290.054 | 714,007,712 | 234.144 | 583,250,600 |
| ReSolveD- $\alpha$ -512f | 17.974 | 44,794,937 | 28.939  | 70,727,225  | 26.761  | 66,863,762  |

Table 11: Benchmarks for the optimized AVX implementation using the Keccak backend.

| Parameter set            | KeyGen |           | Sign   |             | Verify |             |
|--------------------------|--------|-----------|--------|-------------|--------|-------------|
|                          | ms     | cycles    | ms     | cycles      | ms     | cycles      |
| ReSolveD- $\alpha$ -160s | 0.213  | 530,787   | 2.772  | 6,542,162   | 2.318  | 5,778,512   |
| ReSolveD- $\alpha$ -160f | 0.213  | 529,550   | 0.482  | 1,178,187   | 0.372  | 917,237     |
| ReSolveD- $\alpha$ -256s | 0.574  | 1,432,400 | 5.126  | 12,152,362  | 4.446  | 11,095,425  |
| ReSolveD- $\alpha$ -256f | 0.574  | 1,432,362 | 0.742  | 1,843,525   | 0.587  | 1,461,162   |
| ReSolveD- $\alpha$ -384s | 1.697  | 4,160,262 | 21.867 | 53,723,987  | 20.604 | 51,265,962  |
| ReSolveD- $\alpha$ -384f | 1.665  | 4,153,012 | 3.438  | 8,534,750   | 3.031  | 7,591,400   |
| ReSolveD- $\alpha$ -512s | 3.958  | 9,884,300 | 56.414 | 139,763,400 | 55.464 | 134,346,762 |
| ReSolveD- $\alpha$ -512f | 3.961  | 9,889,825 | 7.455  | 18,568,062  | 6.826  | 17,001,450  |

#### 6.4.1 Comparison with SDitH

We compare the 256-bit security instances of ReSolveD- $\alpha$  with the SDitH instance at NIST level 5 security, as it is the only matching security level among the parameter sets for both schemes. We benchmarked the optimized implementation from the submission package of SDitH on the same testing platform as described in [Section 6.2](#). For ReSolveD- $\alpha$ , we use the optimized AVX implementation with Keccak backend from [Table 11](#). The results are listed in [Table 12](#).

Table 12: Performance comparison between ReSolveD- $\alpha$  and SDitH.

| Scheme                   | Running time ( <i>ms</i> ) |              | Communication (Bytes) |           |
|--------------------------|----------------------------|--------------|-----------------------|-----------|
|                          | Signing                    | Verification | Public key            | Signature |
| ReSolveD- $\alpha$ -256s | 5.126                      | 4.446        | 194                   | 13,973    |
| ReSolveD- $\alpha$ -256f | 0.742                      | 0.587        | 194                   | 17,932    |
| SDitH-cat5-short         | 38.11                      | 36.16        | 132                   | 14,121    |
| SDitH-cat5-fast          | 6.973                      | 6.410        | 132                   | 17,540    |

### 6.4.2 Comparison with A Variant of ReSolveD- $\alpha$

**The variant of ReSolveD- $\alpha$  under an alternative set of RSD parameters.** We select a set of RSD parameters with bit security evaluated using the best-known RSD attacks without the linear-time approximate collision search attacks (where the underlying heuristic assumption does not hold for now). To be consistent with our previous parameter selection process, for a security parameter of  $\lambda$  bits (targeting  $\lambda$ -bit classical security), we also choose a target  $(\lambda + 2.3 \cdot \log(\lambda))$  bit RSD security level. To the best of our knowledge, the LPN estimator [YYW<sup>+</sup>25] and RSD estimator [ES24] already covers existing RSD attacks, including the linearization and ISD-type attack in [CCJ23], so we only use these two estimators to derive bit security values.

Table 13: The alternative set of RSD parameters for ReSolveD- $\alpha'$

| Parameter Set            | Quantum Security<br>$\text{bitsec}_q$ | Code Length<br>$m$ | Code Dimension<br>$k$ | Noise Weight<br>$w$ | Block Size<br>$\ell_{\text{bs}}$ |
|--------------------------|---------------------------------------|--------------------|-----------------------|---------------------|----------------------------------|
| ReSolveD- $\alpha'$ -160 | 80                                    | 1460               | 780                   | 292                 | 5                                |
| ReSolveD- $\alpha'$ -256 | 128                                   | 2320               | 1240                  | 464                 | 5                                |
| ReSolveD- $\alpha'$ -384 | 192                                   | 3455               | 1850                  | 691                 | 5                                |
| ReSolveD- $\alpha'$ -512 | 256                                   | 4560               | 2440                  | 912                 | 5                                |

We follow the same approach as in Section 5.1 to minimize  $\ell_{\text{wit}} = k \cdot (1 - \frac{1}{\ell_{\text{bs}}})$  among parameters satisfying the target security level. The selected parameters are listed in Table 13. We stress that this set of parameters only serve as a comparison object showing the performance potential if a more accurate RSD security model (without considering the non-existing linear-time approximate collision algorithm) were to be adopted. The parameters in Table 3 remains the recommended parameters. The complexity estimate of existing RSD attacks on the new parameters are listed in Tables 14 and 15.

Table 14: Complete results returned by the RSD estimator [ES24] for the alternative RSD parameters in Table 13.

| Algorithm      | ReSolveD- $\alpha'$ -160 | ReSolveD- $\alpha'$ -256 | ReSolveD- $\alpha'$ -384 | ReSolveD- $\alpha'$ -512 |
|----------------|--------------------------|--------------------------|--------------------------|--------------------------|
| RegularISDPerm | 201.7                    | 304.9                    | 441.5                    | 572.2                    |
| RegularISDEnum | 180.9                    | 281.1                    | 414.1                    | 541.7                    |
| RegularISDRep  | 178.8                    | 276.0                    | 406.2                    | 533.3                    |
| CCJ-Enum       | 211.6                    | 330.1                    | 487.3                    | 638.6                    |
| CCJ-Lin        | 222.2                    | 332.5                    | 478.3                    | 617.7                    |
| SDAttack       | 387.6                    | 611.0                    | 909.5                    | 1197.5                   |

Under the same VOLEitH parameters, we instantiated ReSolveD- $\alpha$  using the alternative set of RSD parameters in Table 13; we denote this alternative parameterization by ReSolveD- $\alpha'$ . In Table 16 and Table 17, we demonstrate its communication and computation compared to ReSolveD- $\alpha$ , respectively. The percentage shown next to each ReSolveD- $\alpha'$  entry is the relative *reduction* compared to the corresponding ReSolveD- $\alpha$  entry.

## 6.5 Discussion

The performance bottlenecks of ReSolveD- $\alpha$  can be read off directly from the pseudocode in Sections 4.2, 4.4 and 4.9. The dominant costs are not isolated expensive primitives, but rather a few symmetric-key functions that are executed a very large number of times.

**Key generation.** The main extra cost of ReSolveD- $\alpha$  compared with FAEST appears already in ReSolveD- $\alpha$ .KeyGen. The routine RSD.SampleMatrix expands  $\text{seed}_{\text{pk}}$  mininto the full systematic

Table 15: Complete results returned by the current LPN estimator [YYW<sup>+</sup>25] for the alternative RSD parameters in Table 13.

| Algorithm | ReSolveD- $\alpha'$ -160 | ReSolveD- $\alpha'$ -256 | ReSolveD- $\alpha'$ -384 | ReSolveD- $\alpha'$ -512 |
|-----------|--------------------------|--------------------------|--------------------------|--------------------------|
| Gauss     | 302.2                    | 467.9                    | 688.1                    | 898.9                    |
| SD        | 696.0                    | 1102.0                   | 1643.9                   | 2162.9                   |
| SD2       | 209.6                    | 345.3                    | 523.7                    | 698.9                    |
| SDISD     | 285.9                    | 448.2                    | 664.6                    | 872.0                    |
| BJMM      | 284.8                    | 445.8                    | 661.4                    | 867.7                    |
| ISD       | 209.6                    | 345.3                    | 523.7                    | 698.9                    |
| HYB       | 183.7                    | 285.0                    | 420.3                    | 550.6                    |
| PER       | 201.7                    | 304.9                    | 441.5                    | 572.2                    |
| ENU       | 180.9                    | 281.1                    | 414.1                    | 541.7                    |
| REP       | 180.2                    | 280.3                    | 413.5                    | 541.6                    |
| AGB2      | 230.3                    | 336.9                    | 477.4                    | 612.1                    |

matrix part  $\mathbf{H}_B \in \mathbb{F}_2^{(m-k) \times k}$  via [MatHash](#), i.e., one long [XOF](#) call producing  $(m-k)k$  bits. This is substantially heavier than FAEST’s key generation, where the public key is obtained from a small number of forward AES evaluations.

**BAVC tree generation and leaf processing.** The first major hotspot is [BAVC.Commit](#). The loops over  $\text{id} \in [1, L/2)$  and  $\text{id} \in [L/2, L)$  evaluate [TCCR](#) for essentially every internal tree node, while the loop over  $\text{id} \in [L, 2L)$  calls [LeafHash](#) for every leaf. Since [LeafHash](#) itself expands each leaf by [PRG](#)( $\mathbf{x}, \text{iv}, 0^8; 2\lambda$ ), the total cost scales with the whole interleaved BAVC forest size as reflected by  $L$ . A higher  $\tau$  value results in a smaller  $L$  value. This is the most computation-heavy part of signing, and it is also mirrored during verification in [BAVC.Verify](#), where all opened internal nodes are expanded again and the remaining leaves are recomputed.

**VOLE expansion.** The second hotspot is [VOLE.ConvertToVOLE](#). In [VOLE.Commit](#), this conversion is executed once for every  $\alpha \in [0, \tau)$ , and each call first expands all seeds with [PRG](#)( $\text{seed}_i, \text{iv}, \cdot; \ell_{\text{vole}}$ ) before performing the XOR folding that derives  $(\mathbf{u}, \mathbf{V})$ . Thus the cost is driven by repeated seed expansion over all leaves of all VOLE instances. For the 160/256/512-bit parameter sets, this directly translates into a large number of block-cipher invocations in counter mode; for the 384-bit set, the same step is implemented through repeated [TCCR](#) calls.

**Fiat-Shamir hashing, rejection sampling, and proof-of-work.** The top-level signing algorithm computes [MsgHash](#), [CoinHash](#), [FSHash<sub>1</sub>](#), and [FSHash<sub>2</sub>](#) once, but the real hashing bottleneck is the while-loop that repeatedly computes [FSHash<sub>3</sub>](#)( $\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr}$ ) until both the proof-of-work condition and the opening-size condition are met. This loop combines proof-of-work with rejection sampling, so the cost grows with the expected number of signing attempts. Consequently, the wall-clock signing time is much more sensitive to  $(\tau, w_{\text{grind}}, T_{\text{open}})$  than the verification time.

**Benefit of instruction-set acceleration.** The optimized implementation should benefit most from accelerating exactly the symmetric-key primitives above. AES-NI directly speeds up both the counter-mode [PRG](#) and the block-cipher calls inside [TCCR](#) at the 160, 256, and 384 security levels, as they use ciphers in the Rijndael family. This affects tree expansion, leaf-node commitments, and VOLE seed expansion simultaneously, so one low-level optimization improves several top-level algorithms at once. For the 512-bit security level, since we use the SHA256-based SHACAL-2 cipher, the SHA-NI instructions can be used for acceleration.

Table 16: VOLE-in-the-Head parameters and sizes for ReSolveD- $\alpha$  with the recommended and alternative RSD parameters. Compared to ReSolveD- $\alpha$ , the improvement ratio of ReSolveD- $\alpha'$  is shown in parentheses.

| Parameter Set             | VOLEitH Parameters   |        |                    |                   | Sizes (Bytes) |               |             |                          |
|---------------------------|----------------------|--------|--------------------|-------------------|---------------|---------------|-------------|--------------------------|
|                           | $\ell_{\text{vole}}$ | $\tau$ | $w_{\text{grind}}$ | $T_{\text{open}}$ | $ \text{pk} $ | $ \text{sk} $ | $ \sigma $  | $ \text{pk}  +  \sigma $ |
| ReSolveD- $\alpha$ -160s  | 1056                 | 14     | 6                  | 129               | 121           | 40            | 5307        | 5428                     |
| ReSolveD- $\alpha$ -160f  | 1056                 | 21     | 8                  | 139               | 121           | 40            | 6851        | 6972                     |
| ReSolveD- $\alpha$ -256s  | 1687                 | 22     | 12                 | 224               | 194           | 64            | 13973       | 14167                    |
| ReSolveD- $\alpha$ -256f  | 1687                 | 35     | 8                  | 223               | 194           | 64            | 17932       | 18126                    |
| ReSolveD- $\alpha$ -384s  | 2510                 | 34     | 10                 | 332               | 288           | 96            | 31575       | 31863                    |
| ReSolveD- $\alpha$ -384f  | 2510                 | 53     | 9                  | 336               | 288           | 96            | 40469       | 40757                    |
| ReSolveD- $\alpha$ -512s  | 3343                 | 46     | 6                  | 439               | 385           | 128           | 56239       | 56624                    |
| ReSolveD- $\alpha$ -512f  | 3343                 | 72     | 8                  | 447               | 385           | 128           | 72611       | 72996                    |
| ReSolveD- $\alpha'$ -160s | 800 (24%)            | 14     | 6                  | 129               | 105 (13%)     | 40            | 4859 (8%)   | 4964 (8%)                |
| ReSolveD- $\alpha'$ -160f | 800 (24%)            | 21     | 8                  | 139               | 105 (13%)     | 40            | 6179 (10%)  | 6284 (10%)               |
| ReSolveD- $\alpha'$ -256s | 1264 (25%)           | 22     | 12                 | 224               | 167 (14%)     | 64            | 12807 (8%)  | 12974 (8%)               |
| ReSolveD- $\alpha'$ -256f | 1264 (25%)           | 35     | 8                  | 223               | 167 (14%)     | 64            | 16077 (10%) | 16244 (10%)              |
| ReSolveD- $\alpha'$ -384s | 1880 (25%)           | 34     | 10                 | 332               | 249 (14%)     | 96            | 28889 (9%)  | 29138 (9%)               |
| ReSolveD- $\alpha'$ -384f | 1880 (25%)           | 53     | 9                  | 336               | 249 (14%)     | 96            | 36282 (10%) | 36531 (10%)              |
| ReSolveD- $\alpha'$ -512s | 2480 (26%)           | 46     | 6                  | 439               | 329 (15%)     | 128           | 51271 (9%)  | 51600 (9%)               |
| ReSolveD- $\alpha'$ -512f | 2480 (26%)           | 72     | 8                  | 447               | 329 (15%)     | 128           | 64835 (11%) | 65164 (11%)              |

## 7 Concrete Security Analysis

We analyze the security of ReSolveD- $\alpha$  with respect to known attacks in [Section 7.1](#). In [Section 7.2](#), we give a detailed analysis of the concrete security of the RSD one-way function. The cryptographic primitives used in ReSolveD- $\alpha$  are the one-way function (OWF) based on the Regular Syndrome Decoding problem, the block-cipher-based PRG and TCCR, and the hash functions (XOF) instantiated as random oracles. The symmetric primitives used in ReSolveD- $\alpha$  are instantiated from well-studied block ciphers and standardized hash-function interfaces; their concrete security is discussed in [Section 7.3](#). In our security proof, we model the hash functions as random oracles. This is a standard practice in security analysis of cryptography.

### 7.1 Known Attacks

#### 7.1.1 Brute-Force the Public Key

The simplest attack strategy is to recover the signing key by inverting the OWF output contained in the public key. In ReSolveD- $\alpha$ , this is exactly the search version of the RSD problem: given  $(\mathbf{H}, \mathbf{y})$ , recover a regular noise vector  $\mathbf{e}$  such that  $\mathbf{H} \cdot \mathbf{e} = \mathbf{y}$ . This attack is therefore captured by the concrete RSD analysis in [Section 7.2](#).

#### 7.1.2 Brute-Force the TCCR Tree Generation

Another way to try to recover the signing key is to attack the TCCR construction used in [BAVC.Commit](#) and [BAVC.Verify](#) to generate the correlated GGM tree. This is the analogue of the PRG attack discussed in FAEST, except that our tree expansion in [BAVC.Commit](#) is driven by a tweakable circular-correlation-robust hash rather than a length-doubling PRG. At each internal node of the tree, a signature reveals all-but-one of the relevant descendants along the

Table 17: Benchmarks for the optimized AVX implementation using the Keccak backend with the recommended and alternative RSD parameters. Compared to ReSolveD- $\alpha$ , the improvement ratio of ReSolveD- $\alpha'$  is shown in parentheses.

| Parameter set             | KeyGen      |                 | Sign         |                   | Verify       |                   |
|---------------------------|-------------|-----------------|--------------|-------------------|--------------|-------------------|
|                           | ms          | cycles          | ms           | cycles            | ms           | cycles            |
| ReSolveD- $\alpha$ -160s  | 0.213       | 530,787         | 2.772        | 6,542,162         | 2.318        | 5,778,512         |
| ReSolveD- $\alpha$ -160f  | 0.213       | 529,550         | 0.482        | 1,178,187         | 0.372        | 917,237           |
| ReSolveD- $\alpha$ -256s  | 0.574       | 1,432,400       | 5.126        | 12,152,362        | 4.446        | 11,095,425        |
| ReSolveD- $\alpha$ -256f  | 0.574       | 1,432,362       | 0.742        | 1,843,525         | 0.587        | 1,461,162         |
| ReSolveD- $\alpha$ -384s  | 1.697       | 4,160,262       | 21.867       | 53,723,987        | 20.604       | 51,265,962        |
| ReSolveD- $\alpha$ -384f  | 1.665       | 4,153,012       | 3.438        | 8,534,750         | 3.031        | 7,591,400         |
| ReSolveD- $\alpha$ -512s  | 3.958       | 9,884,300       | 56.414       | 139,763,400       | 55.464       | 134,346,762       |
| ReSolveD- $\alpha$ -512f  | 3.961       | 9,889,825       | 7.455        | 18,568,062        | 6.826        | 17,001,450        |
| ReSolveD- $\alpha'$ -160s | 0.142 (33%) | 357,100 (33%)   | 2.567 (7%)   | 6,014,500 (8%)    | 2.141 (8%)   | 5,323,737 (8%)    |
| ReSolveD- $\alpha'$ -160f | 0.143 (33%) | 358,337 (32%)   | 0.388 (20%)  | 965,075 (18%)     | 0.341 (8%)   | 831,162 (9%)      |
| ReSolveD- $\alpha'$ -256s | 0.324 (44%) | 814,250 (43%)   | 4.843 (6%)   | 11,342,950 (7%)   | 4.158 (7%)   | 10,371,337 (7%)   |
| ReSolveD- $\alpha'$ -256f | 0.325 (43%) | 811,312 (43%)   | 0.581 (22%)  | 1,438,300 (22%)   | 0.505 (14%)  | 1,262,162 (14%)   |
| ReSolveD- $\alpha'$ -384s | 1.025 (40%) | 2,566,362 (38%) | 18.930 (13%) | 45,778,862 (15%)  | 17.745 (14%) | 44,014,475 (14%)  |
| ReSolveD- $\alpha'$ -384f | 1.026 (38%) | 2,568,025 (38%) | 2.762 (20%)  | 6,828,200 (20%)   | 2.469 (19%)  | 6,186,800 (19%)   |
| ReSolveD- $\alpha'$ -512s | 2.296 (42%) | 5,728,400 (42%) | 49.801 (12%) | 122,824,762 (12%) | 47.243 (15%) | 117,742,262 (12%) |
| ReSolveD- $\alpha'$ -512f | 2.289 (42%) | 5,722,750 (42%) | 6.156 (17%)  | 15,062,025 (19%)  | 5.752 (16%)  | 14,556,162 (14%)  |

opened paths. An attacker may therefore try to guess the hidden node value and test the guess by evaluating [TCCR](#) and checking consistency with the revealed tree structure. This is a multi-target attack surface, because many tree nodes across one or more signatures are expanded under the same construction. However, the relevant security notion here is precisely the multi-target TCCR security of the block-cipher-based construction. Since the per-signature salt  $iv$  is chosen uniformly at random and independently for every signature, and the tree computations are domain-separated by their explicit inputs (including  $\text{const}_0, \text{const}_1$  in the lower half of the tree), we are not aware of any attack that performs better than attacking a single target of the underlying block-cipher-based TCCR primitive.

Moreover, the proof in [Section 8.4.2](#) guarantees information-theoretic security in the ideal cipher model. Therefore, the advantage of any attack against TCCR translates to a distinguishing attack against the underlying block cipher.

### 7.1.3 Brute-Force the PRG

The [PRG](#) in ReSolveD- $\alpha$  is used in [LeafHash](#), in [VOLE.ConvertToVOLE](#), and in [RSD.SampleNoise](#). The first two uses are most relevant to signature forgery. In [LeafHash](#), a hidden leaf seed can be tested by expanding it to a candidate commitment. In [VOLE.ConvertToVOLE](#), all-but-one seeds are known after opening, so an attacker may try to guess the missing seed and verify the resulting VOLE correlation. This is similar in spirit to the corresponding FAEST attack on the PRG used in leaf commitments and VOLE conversion. Nevertheless, the calls are separated by the random per-signature  $iv$  and by distinct counter values, so we are not aware of any effective multi-target attack that combines these invocations in a way that is meaningfully better than attacking a single target of the underlying block-cipher-based PRG.

**Given multiple signatures.** Suppose the attacker is given up to  $Q_{\text{sig}}$  signatures. Each signature uses an independent random  $iv \in \{0, 1\}^{120}$ , so IV collisions are expected to remain negligible as long as  $Q_{\text{sig}} \ll 2^{60}$ . If the IVs are unique, then both the TCCR-based tree generation and the PRG-based expansions are independent across signatures, and the best known attacks reduce to the corresponding single-signature attacks described above. Even if

a small number of IV collisions were to occur accidentally, they are not under the control of the attacker and do not appear to create a practical attack vector that competes with directly attacking the RSD public key.

#### 7.1.4 Attack Soundness of the ZK Protocol

Instead of recovering the signing key, an attacker may attempt to forge a signature by violating the soundness of the underlying VOLE-in-the-Head proof system. This possibility is covered by the EUF-KO security proof in [Theorem 8.4](#). As in FAEST, the concrete meaning of this bound is that an attacker must either break the random-oracle layer, violate the binding and hiding guarantees of the commitment layer, or produce a false witness that passes the QuickSilver checks. We are not aware of any attack strategy on these components that is more effective than exhaustive search against the underlying primitives.

#### 7.1.5 Multi-User Attacks

We also consider the setting where the attacker is given many public keys and signatures under independently generated keys. Since each public key uses an independently sampled RSD instance, there is no obvious way to aggregate users into a single multi-target inversion attack on the OWF. Moreover, the fresh per-signature IV randomizes the TCCR and PRG instances used inside the commitment and VOLE layers, so the auxiliary symmetric-key computations are also isolated across users. Finally, key-substitution attacks [\[MS04\]](#) are prevented because the public key is hashed together with the message in [MsgHash](#), so each signature is bound to its originating public key.

### 7.2 Concrete Security of the RSD Problem

We provide the concrete security analysis of the RSD problem. Our parameter selection is based on the latest publicly available cryptanalysis results [\[CCJ23, ES24, LWYY24\]](#). Below we review the different attack algorithms and their complexities.

**Linearization attack (Lin).** The linearization attack was first proposed by Saarinen [\[Saa07\]](#). The basic idea is to exploit the regularity of the noise to reduce the number of columns in the parity-check matrix  $\mathbf{H}$ . Within each block of  $\ell_{\text{bs}}$  columns, the first column is XORed with all remaining columns and removed. The same operation is applied to the syndrome  $\mathbf{y}$ . After this transformation, one obtains a reduced system over a matrix  $\mathbf{H}' \in \mathbb{F}_2^{(m-k) \times (w(\ell_{\text{bs}}-1))}$ . The system is then solved by random sub-matrix inversion: a square sub-matrix is selected and inverted; if the resulting solution vector is sparse, it is accepted. The expected cost of this attack is  $(m-k)^\omega/P$ , where  $\omega = 2.3$  and  $P$  is the success probability per iteration, given by

$$P = \left[ \frac{\binom{\ell_{\text{bs}}-2}{k/w-1}}{\binom{\ell_{\text{bs}}-1}{k/w-1}} \right]^{(1-1/\ell_{\text{bs}})w}.$$

The implementation in the *RSD Estimator* artifact [\[ES24\]](#) uses instead the smoothed closed-form model derived in [\[ES24\]](#) from the linearization analysis of [\[CCJ23\]](#). More precisely, the artifact evaluates the inverse success probability through the continuous approximation

$$\left( \frac{m-w}{m-k} \right)^{(1-w/m)w},$$

and models the linear-algebra cost as  $(m-k)^2m$ , yielding the complexity expression

$$\log_2((m-k)^2m) + (1-w/m)w \log_2 \left( \frac{m-w}{m-k} \right). \quad (10)$$

This is exactly the quantity reported in the “CCJ-Lin” row of Table 20. We follow the same approximation method in our estimation. Notice that in Equation (10) the matrix operation’s complexity is estimated to be  $(m - k)^2 m$  whereas we estimate this quantity as  $(m - k)^{2.3}$ . Therefore, we conclude that our estimation is a more conservative one.

**Information Set Decoding attack (ISD).** Information Set Decoding is a family of algorithms for solving the syndrome decoding problem, originally proposed by Prange [Pra62] and subsequently improved by Lee–Brickell [LB88], Stern [Ste88], BJMM [BJMM12], May–Ozerov [MO15], and others. For regular-noise instances, there are two complementary ways to use this line of work. One may first reduce the regular instance to an exact-noise LPN/SD instance by using the per-block regularity constraint, and then apply the pooled-Gauss, statistical-decoding, Stern–Dumer, and BJMM-style estimators analyzed in [LWYY24]. Alternatively, one may exploit the regular structure directly. Esser and Santini [ES24] developed regular-ISD variants, including permutation-based, enumeration-based, and representation-based algorithms; the representation-based variant is usually the strongest of these attacks for our parameter range.

The ISD attack for RSD operates on the systematic form of  $\mathbf{H}$  and exploits the regularity of the noise to split the target vector into two halves, each with weight  $w/2$ , enabling an efficient birthday-type search. The approximate total cost is

$$\left( q \cdot \frac{k + q}{\ell_{\text{bs}}} \cdot \ell_{\text{bs}}^{\frac{k+q}{2\ell_{\text{bs}}}} \right) + \left( \frac{\ell_{\text{bs}}^{\frac{k+q}{2\ell_{\text{bs}}}}}{2^q} \cdot (m - k - q) \cdot \frac{k + q}{\ell_{\text{bs}}} \right), \quad (11)$$

where  $q$  is an optimization parameter balancing the two terms.

The current LPN Estimator [YYW+25] combines both viewpoints: it reports the reduced exact-noise attacks under the rows Gauss, SD, SD2, SDISD, and BJMM, and it also reports the regular-ISD rows PER, ENU, and REP. It further includes the recent hybrid algebraic attack of Wang et al. [WWY+25], shown as HYB in Table 21. This hybrid attack constructs a reduced quadratic system from the regular instance and replaces the meet-in-the-middle enumeration step by a hybrid procedure that enumerates selected block positions and solves the remaining system.

**Nearest Neighbor search attack (NN).** Carozza et al. [CCJ23] proposed a variant of ISD that applies a nearest neighbor search between two lists instead of an exact collision search. The attack augments  $\mathbf{H}$  with  $w$  additional rows encoding the odd-weight constraint on each noise block, then performs the nearest neighbor search. A conservative lower bound on the cost (assuming linear-time nearest neighbor search) is

$$\text{Cost} \geq (\ell_{\text{bs}})^{(k-w)/(2\ell_{\text{bs}})-1} \cdot (k - w) \cdot (m - k). \quad (12)$$

For clarity, this estimate corresponds directly to Eq. (6) of [CCJ23]. By contrast, the *RSD Estimator* artifact of Esser and Santini [ES24] does not implement Eq. (6) verbatim in its “CCJ-Enum” entry. Instead, it follows the concrete formulas from [ES24], where the CCJ-style enumeration algorithm and the linearization algorithm are analyzed separately.

**Algebraic attacks.** Briaud and Oygarden [BØ23] proposed an algebraic attack against the RSD assumption in the setting of secure multi-party computation. The attack transforms the regularity constraints into a polynomial system and estimates the cost of solving the resulting system. Liu, Wang, Yang, Yin, and Yu subsequently refined this approach in AGB 2.0 [LWY+25], improving the concrete algebraic estimator for RSD parameters used in PCG applications. The current LPN Estimator includes this AGB 2.0 attack as the AGB2 row in Table 21. For our

selected parameters, AGB 2.0 does not dominate the best regular-ISD or hybrid estimates, but we include it in the final comparison.

**Overall assessment of RSD hardness.** Table 18 summarizes the best attack complexities obtained from the three estimators that we considered. The “CCJ23 Estimator (best)” column records the minimum among the linearization, ISD, and NN values produced by our implementation of the attack methods listed in [CCJ23]. The “RSD Estimator (best)” column records the minimum among the algorithms returned by the artifact of Esser and Santini [ES24], and the “LPN Estimator” column reports the values returned by the current LPN Estimator [YYW<sup>+</sup>25], which integrates the exact-noise LPN attacks of [LWYY24], the regular-ISD attacks of [ES24], the hybrid attack of [WWY<sup>+</sup>25], and the AGB 2.0 algebraic attack of [LWY<sup>+</sup>25]. We further provide complete breakdown tables for our estimator, the RSD estimator, and the LPN estimator in Tables 19 to 21.

From Table 18, we observe that the NN estimate is indeed a very conservative one as it assumes a linear-time nearest neighbor search algorithm and all currently known algorithms have strictly super-linear running time. Consequently, it out-performs all other existing cryptanalysis, confirming our claim that the RSD parameter choice in ReSolveD- $\alpha$  is conservative. For the LPN Estimator results, the strongest attack in our parameter range is the representation-based regular-ISD attack, shown as REP in Table 21.

Table 18: Concrete security estimates for the RSD parameters of ReSolveD- $\alpha$

| Parameter Set           | CCJ23 Estimator<br>[CCJ23] | RSD Estimator<br>[ES24] | LPN Estimator<br>[YYW <sup>+</sup> 25] |
|-------------------------|----------------------------|-------------------------|----------------------------------------|
| ReSolveD- $\alpha$ -160 | 177.3                      | 231.7                   | 235.9                                  |
| ReSolveD- $\alpha$ -256 | 276.5                      | 366.1                   | 373.1                                  |
| ReSolveD- $\alpha$ -384 | 404.7                      | 543.2                   | 550.4                                  |
| ReSolveD- $\alpha$ -512 | 534.4                      | 723.0                   | 731.0                                  |

Table 19: Complete results returned by our implementation of [CCJ23]

| Algorithm | ReSolveD- $\alpha$ -160 | ReSolveD- $\alpha$ -256 | ReSolveD- $\alpha$ -384 | ReSolveD- $\alpha$ -512 |
|-----------|-------------------------|-------------------------|-------------------------|-------------------------|
| Lin       | 266.8                   | 417.5                   | 612.7                   | 809.9                   |
| ISD       | 306.2                   | 483.5                   | 714.2                   | 947.9                   |
| NN        | 177.3                   | 276.5                   | 404.7                   | 534.4                   |

Table 20: Complete results returned by the RSD estimator [ES24]

| Algorithm      | ReSolveD- $\alpha$ -160 | ReSolveD- $\alpha$ -256 | ReSolveD- $\alpha$ -384 | ReSolveD- $\alpha$ -512 |
|----------------|-------------------------|-------------------------|-------------------------|-------------------------|
| RegularISDPerm | 260.4                   | 402.1                   | 585.4                   | 770.5                   |
| RegularISDEnum | 237.4                   | 374.9                   | 553.4                   | 734.3                   |
| RegularISDRep  | 231.7                   | 366.1                   | 543.2                   | 723.0                   |
| CCJ-Enum       | 272.5                   | 431.5                   | 637.7                   | 846.6                   |
| CCJ-Lin        | 285.7                   | 437.5                   | 633.7                   | 831.6                   |
| SDAttack       | 430.9                   | 690.8                   | 1030.9                  | 1376.6                  |

Table 21: Complete results returned by the current LPN estimator [YYW<sup>+</sup>25]

| Algorithm | ReSolveD- $\alpha$ -160 | ReSolveD- $\alpha$ -256 | ReSolveD- $\alpha$ -384 | ReSolveD- $\alpha$ -512 |
|-----------|-------------------------|-------------------------|-------------------------|-------------------------|
| Gauss     | 372.2                   | 584.8                   | 860.5                   | 1139.0                  |
| SD        | 833.5                   | 1336.3                  | 1989.9                  | 2651.1                  |
| SD2       | 330.4                   | 541.6                   | 815.4                   | 1094.5                  |
| SDISD     | 351.1                   | 557.9                   | 826.6                   | $\geq 999$              |
| BJMM      | 349.0                   | 553.9                   | 819.9                   | $\geq 999$              |
| ISD       | 330.4                   | 541.6                   | 815.4                   | $\geq 999$              |
| HYB       | 240.0                   | 379.7                   | 561.2                   | 746.1                   |
| PER       | 260.4                   | 402.1                   | 585.4                   | 770.5                   |
| ENU       | 237.4                   | 374.9                   | 553.4                   | 734.6                   |
| REP       | 235.9                   | 373.1                   | 550.4                   | 731.0                   |
| AGB2      | 298.5                   | 449.3                   | 644.2                   | 842.1                   |

### 7.3 Concrete Security of Symmetric Primitives

We construct **PRG** and **TCCR** in the ideal cipher model and provide information-theoretic security proofs in Section 8. As for concrete security, it is a common practice to assume that a well-tested encryption scheme (resp. XOF) offers concrete security on the same level as an ideal cipher (resp. RO). We stress that the implementation of ReSolveD- $\alpha$  utilizes currently standardized symmetric cryptographic schemes or calls APIs provided by the contest’s specification. In the future, we are able to replace these functions with future standardized schemes.

## 8 Security Proof

We first prove functional correctness and then present the security proofs against any quantum/classical adversary. For simplicity, we omit the public parameters (e.g., **param**) of functions and leave them implicit in this section.

### 8.1 Correctness Analysis

**Correctness of VOLE commitment.** We first prove the correctness of the subroutine **VOLE.ConvertToVOLE** used in the VOLE commitment scheme, and then derive the correctness of the full VOLE commitment scheme from it.

**Lemma 8.1** (Correctness of **VOLE.ConvertToVOLE**). *Given any  $k \in \mathbb{N}^+$ ,  $N = 2^k$ , and  $\Delta \in [0, N)$ , let  $(\text{seed}_0, \dots, \text{seed}_{N-1})$  and  $(\overline{\text{seed}}_0, \dots, \overline{\text{seed}}_{N-1})$  satisfy*

$$\begin{aligned} \overline{\text{seed}}_0 &= \perp, \\ \forall i \in [1, N) : \overline{\text{seed}}_i &= \text{seed}_{i \oplus \Delta}. \end{aligned}$$

*For any  $\text{iv}$ , it holds that*

$$\Pr \left[ \begin{array}{l} (\delta_0, \dots, \delta_{k-1}) := (\Delta[0], \dots, \Delta[k-1]), \\ (\mathbf{u}, [\mathbf{v}_0, \dots, \mathbf{v}_{k-1}]) := \text{VOLE.ConvertToVOLE}((\text{seed}_i)_{i \in [0, N)}, \text{iv}), \\ (\mathbf{u}', [\mathbf{q}_0, \dots, \mathbf{q}_{k-1}]) := \text{VOLE.ConvertToVOLE}((\overline{\text{seed}}_i)_{i \in [0, N)}, \text{iv}) \\ \quad : \forall j \in [0, k) : \mathbf{q}_j = \mathbf{v}_j \oplus \delta_j \cdot \mathbf{u} \end{array} \right] = 1.$$

*Proof.* Since **PRG** is deterministic, the **PRG** outputs  $(\mathbf{r}_{0,i})_{i \in [0, N)}$  computed from  $(\text{seed}_i)_{i \in [0, N)}$  (i.e., the outputs of the prover) and the **PRG** outputs  $(\overline{\mathbf{r}}_{0,i})_{i \in [0, N)}$  computed from  $(\overline{\text{seed}}_i)_{i \in [0, N)}$

(i.e., the outputs of the verifier) satisfy the basic equality:

$$\forall i \in [1, N) : \bar{\mathbf{r}}_{0,i} = \mathbf{r}_{0,i \oplus \Delta}. \quad (13)$$

According to [VOLE.ConvertToVOLE](#), it holds for the prover that

$$\begin{aligned} \mathbf{r}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \mathbf{r}_{0,d}, \\ \mathbf{u} = \mathbf{r}_{k,0} &= \sum_{d \in [0, N)} \mathbf{r}_{0,d} \\ \mathbf{v}_j &= \sum_{i \in [0, \frac{N}{2^{j+1}})} \mathbf{r}_{j,2i+1}. \end{aligned} \quad (14)$$

Meanwhile, it holds for the verifier that

$$\begin{aligned} \bar{\mathbf{r}}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \bar{\mathbf{r}}_{0,d}, \\ \mathbf{q}_j &= \sum_{i \in [0, \frac{N}{2^{j+1}})} \bar{\mathbf{r}}_{j,2i+1}. \end{aligned} \quad (15)$$

We use an induction to prove  $\bar{\mathbf{r}}_{j,i} = \mathbf{r}_{j,i \oplus \Delta_{[j:k-1]}}$  for any  $j \in [0, k)$ ,  $i \in [1, \frac{N}{2^j})$ , and the  $(k-j)$ -bit prefix  $\Delta_{[j:k-1]} = (\Delta[j], \dots, \Delta[k-1]) = (\delta_j, \dots, \delta_{k-1})$ . For the base case  $j = 0$ , this target equality holds due to [Eq. \(13\)](#). Then, we assume that the target equality holds for  $j-1 \in [0, k-1)$  and consider the case  $j \in [1, k)$  for induction. Following from [Eq. \(14\)](#) and [Eq. \(15\)](#), it holds that

$$\begin{aligned} \mathbf{r}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \mathbf{r}_{0,d} = \sum_{d=(2i) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \mathbf{r}_{0,d} \\ &= \left( \sum_{d=(2i) \cdot 2^{j-1}}^{(2i+1) \cdot 2^{j-1} - 1} \mathbf{r}_{0,d} \right) \oplus \left( \sum_{d=(2i+1) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \mathbf{r}_{0,d} \right) = \mathbf{r}_{j-1,2i} \oplus \mathbf{r}_{j-1,2i+1}, \end{aligned} \quad (16)$$

$$\begin{aligned} \bar{\mathbf{r}}_{j,i} &= \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \bar{\mathbf{r}}_{0,d} = \sum_{d=(2i) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \bar{\mathbf{r}}_{0,d} \\ &= \left( \sum_{d=(2i) \cdot 2^{j-1}}^{(2i+1) \cdot 2^{j-1} - 1} \bar{\mathbf{r}}_{0,d} \right) \oplus \left( \sum_{d=(2i+1) \cdot 2^{j-1}}^{(2i+2) \cdot 2^{j-1} - 1} \bar{\mathbf{r}}_{0,d} \right) = \bar{\mathbf{r}}_{j-1,2i} \oplus \bar{\mathbf{r}}_{j-1,2i+1}. \end{aligned} \quad (17)$$

Combining [Eq. \(16\)](#), [Eq. \(17\)](#), and the induction assumption, we have for any  $i \in [1, \frac{N}{2^j})$ ,

$$\begin{aligned} \bar{\mathbf{r}}_{j,i} &= \bar{\mathbf{r}}_{j-1,2i} \oplus \bar{\mathbf{r}}_{j-1,2i+1} && \text{(By Eq. (17))} \\ &= \mathbf{r}_{j-1,(2i) \oplus \Delta_{[j-1:k-1]}} \oplus \mathbf{r}_{j-1,(2i+1) \oplus \Delta_{[j-1:k-1]}} && \text{(By the induction assumption)} \\ &= \mathbf{r}_{j-1,2(i \oplus \Delta_{[j:k-1]})} \oplus \mathbf{r}_{j-1,2(i \oplus \Delta_{[j:k-1]})+1} && \text{(For any } \delta_{j-1} \in \{0, 1\}) \\ &= \mathbf{r}_{j,i \oplus \Delta_{[j:k-1]}} && \text{(By Eq. (16))} \end{aligned}$$

showing that the target equality holds and concluding the induction.

For any  $j \in [0, k)$ , if  $\delta_j = 0$  (i.e.,  $\Delta_{[j:k-1]}$  is even), it holds that  $\mathbf{q}_j = \mathbf{v}_j$  since the permutation  $i \mapsto (i \oplus \Delta_{[j:k-1]})$  does not change the oddness of the second subscripts of the terms summed in

$\mathbf{q}_j$  and  $\mathbf{v}_j$  so that the two sums are equivalent. Otherwise, if  $\delta_j = 1$  (i.e.,  $\Delta_{[j:k-1]}$  is odd), the permutation will flip the parity of the second subscripts of the summed terms. By using the proved target equality for all  $\bar{\mathbf{r}}_{j,i}$ 's with odd  $i$ 's (thus,  $i \in [1, \frac{N}{2^j})$ ) in  $\mathbf{q}_j$ , we have that

$$\begin{aligned}\mathbf{q}_j \oplus \mathbf{v}_j &= \left( \sum_{i \in [0, \frac{N}{2^j+1})} \bar{\mathbf{r}}_{j,2i+1} \right) \oplus \left( \sum_{i \in [0, \frac{N}{2^j+1})} \mathbf{r}_{j,2i+1} \right) \\ &= \left( \sum_{i \in [0, \frac{N}{2^j+1})} \mathbf{r}_{j,2i} \right) \oplus \left( \sum_{i \in [0, \frac{N}{2^j+1})} \mathbf{r}_{j,2i+1} \right) \\ &= \sum_{i \in [0, \frac{N}{2^j})} \mathbf{r}_{j,i} = \sum_{i \in [0, \frac{N}{2^j})} \sum_{d=i \cdot 2^j}^{(i+1) \cdot 2^j - 1} \mathbf{r}_{0,d} = \sum_{d \in [0, N)} \mathbf{r}_{0,d} = \mathbf{u}.\end{aligned}$$

Putting the two cases together, we have that  $\mathbf{q}_j = \mathbf{v}_j \oplus \delta_j \cdot \mathbf{u}$  for any  $j \in [0, k)$ .  $\square$

**Definition 8.1** (Correctness of VOLE Commitment). *The correctness of the VOLE commitment scheme  $\text{VOLE} = (\text{VOLE.Commit}, \text{VOLE.Reconstruct})$  holds if:*

- $\forall \mathbf{r} \in \{0, 1\}^\lambda, \text{iv} \in \{0, 1\}^{120}, \mu \in \{0, 1\}^{2\lambda}, \mathbf{s} \in \{0, 1\}^\lambda$ .
- $(\text{com}, \text{decom}, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLE.Commit}(\mathbf{r}, \text{iv}, \mu, \mathbf{s})$ .
- $\forall \text{chall} \in \{0, 1\}^\lambda, I = (I_0, \dots, I_{\tau-1}) \leftarrow \text{VOLE.ChalDecAll}(\text{chall})$ .
- $\text{decom}_I \leftarrow \text{BAVC.Open}(\text{decom}, I, \text{iv})$ .
- One of the following cases holds:
  - $\text{VOLE.Reconstruct}$  outputs  $\perp$ , or
  - $(\overline{\text{com}}, \mathbf{Q}) \leftarrow \text{VOLE.Reconstruct}(\text{chall}, \text{decom}_I, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau)}, \text{iv}, \mu, \mathbf{s})$  such that  $\overline{\text{com}} = \text{com}$  and  $\mathbf{Q} = \mathbf{V} \oplus \text{chall} \cdot \mathbf{u}$ .

**Lemma 8.2** (Correctness of VOLE Commitment). *The VOLE commitment scheme  $\text{VOLE} = (\text{VOLE.Commit}, \text{VOLE.Reconstruct})$  satisfies the correctness defined in Definition 8.1.*

*Proof.*  $\text{VOLE.Reconstruct}$  returns  $\perp$  if and only if  $\text{BAVC.Verify}$  returns  $\perp$ , and the latter happens if and only if  $|\text{id-list}| > T_{\text{open}}$ .

Otherwise, the partial decommitment  $\text{decom}_I$  gives the binary-tree nodes in  $\text{BAVC.Commit}$  indexed by  $\text{id-list}$  that span all leaves except the  $\tau$  punctured leaves in the set  $I$ , together with the commitment values corresponding to these punctured leaves. Using the binary-tree nodes in  $\text{id-list}$ ,  $\text{BAVC.Verify}$  reconstructs all seed values and commitment values except those at the punctured leaves. It then uses these commitment values together with the commitment values of the punctured leaves to recover the same  $\text{com}$  as computed by  $\text{BAVC.Commit}$ .

Moreover, for each  $\alpha \in [0, \tau)$ , the seed values returned by  $\text{BAVC.Verify}$  agree with those computed by  $\text{BAVC.Commit}$  at every position in the block of size  $N_\alpha$  except at the punctured index  $\Delta_\alpha$ . By Lemma 8.1, we obtain at step 16 of  $\text{VOLE.Reconstruct}$  that

$$\mathbf{Q}'_\alpha = \mathbf{V}_\alpha \oplus [\delta_{\alpha,0} \cdot \mathbf{u}_\alpha, \dots, \delta_{\alpha,n_\alpha-1} \cdot \mathbf{u}_\alpha],$$

where  $(\delta_{\alpha,0}, \dots, \delta_{\alpha,n_\alpha-1}) = (\Delta_\alpha[0], \dots, \Delta_\alpha[n_\alpha-1])$ . Since  $\mathbf{c}_\alpha = \mathbf{u} \oplus \mathbf{u}_\alpha$  for each  $\alpha \in [1, \tau)$ , the final output of  $\text{VOLE.Reconstruct}$  satisfies

$$\begin{aligned}\forall \alpha \in [0, \tau) : \mathbf{Q}_\alpha &= \mathbf{V}_\alpha \oplus [\delta_{\alpha,0} \cdot \mathbf{u}, \dots, \delta_{\alpha,n_\alpha-1} \cdot \mathbf{u}], \\ \mathbf{Q} &= \mathbf{V} \oplus [\delta_{0,0} \cdot \mathbf{u}, \dots, \delta_{0,n_0-1} \cdot \mathbf{u}, \dots, \delta_{\tau-1,0} \cdot \mathbf{u}, \dots, \delta_{\tau-1,n_{\tau-1}-1} \cdot \mathbf{u}].\end{aligned}$$

This proves correctness.  $\square$

**Correctness of QuickSilver.** Given correctly generated VOLE correlations, we prove the correctness of QuickSilver, the zero-knowledge proof protocol used in our signature scheme.

**Lemma 8.3** (Correctness of QuickSilver). *For any  $\text{chall}_3 \in \{0, 1\}^\lambda$ , let a set of VOLE correlations  $(\mathbf{u}, \mathbf{V}, \mathbf{Q}, \Delta) \in \mathbb{F}_2^{\ell_{\text{wit}}+\lambda} \times \mathbb{F}_2^{(\ell_{\text{wit}}+\lambda) \times \lambda} \times \mathbb{F}_2^{(\ell_{\text{wit}}+\lambda) \times \lambda} \times \mathbb{F}_2^\lambda$  satisfy*

$$\mathbf{Q} = \mathbf{V} \oplus [\delta_{0,0} \cdot \mathbf{u}, \dots, \delta_{0,n_0-1} \cdot \mathbf{u}, \dots, \delta_{\tau-1,0} \cdot \mathbf{u}, \dots, \delta_{\tau-1,n_{\tau-1}-1} \cdot \mathbf{u}],$$

where  $\Delta = (\Delta_0, \dots, \Delta_{\tau-1}) := \text{VOLE.ChalDecAll}(\text{chall}_3)$  and for each  $i \in [0, \tau)$ , it holds that  $(\delta_{i,0}, \dots, \delta_{i,n_i-1}) := (\Delta_i[0], \dots, \Delta_i[n_i-1])$ . For any  $\text{chall}_2$ , we have

$$\Pr \left[ \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{ReSolveD-}\alpha.\text{KeyGen}(), \\ \text{pk} = (\text{seed}_{\text{pk}}, \mathbf{y}), \\ \text{sk} = (\text{seed}_{\text{pk}}, \text{seed}_{\text{sk}}), \\ \mathbf{H} := \text{RSD.SampleMatrix}(\text{seed}_{\text{pk}}), \\ \mathbf{e} := \text{RSD.SampleNoise}(\text{seed}_{\text{sk}}), : a_0 = \tilde{a}_0 \\ \mathbf{w} := \text{RSD.EncodeNoise}(\mathbf{e}), \\ \mathbf{d} := \mathbf{w} \oplus \mathbf{u}[0, \ell_{\text{wit}}), \\ (a_1, a_0) := \text{QS.RSDProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \mathbf{H}, \mathbf{y}, \text{chall}_2), \\ \tilde{a}_0 := \text{QS.RSDVerify}(\mathbf{d}, \mathbf{Q}, \text{chall}_2, \text{chall}_3, a_1, \mathbf{H}, \mathbf{y}) \end{array} \right] = 1.$$

*Proof.* In [QS.RSDProve](#) and [QS.RSDVerify](#), the given VOLE correlations are converted into a new VOLE relation  $\mathbf{q} = \mathbf{v} \oplus \Delta \cdot \mathbf{u}$ , where  $\mathbf{q}, \mathbf{v} \in \mathbb{F}_2^{\ell_{\text{wit}}+\lambda}$ . This new VOLE relation is used to verify that the extended witness  $\mathbf{w}$  derived from  $\mathbf{e}$  satisfies the constraints corresponding to  $\mathbf{H} \cdot \mathbf{e} = \mathbf{y}$ , where  $\mathbf{H}$  and  $\mathbf{y}$  are the public values given to [QS.RSDProve](#) and [QS.RSDVerify](#). Since the extended witness  $\mathbf{w}$  is valid, all these constraints are satisfied. Therefore, the value  $b$  computed by [QS.RSDVerify](#) is derived from a quadratic polynomial whose constant coefficient is exactly  $\tilde{a}_0 = a_0$ . This proves the lemma.  $\square$

**Correctness of ReSolveD- $\alpha$ .** Based on the correctness of QuickSilver, we can prove the correctness of our signature scheme.

**Theorem 8.1** (Correctness of ReSolveD- $\alpha$ ). *The signature scheme ReSolveD- $\alpha$  satisfies that, for any message  $\text{msg} \in \{0, 1\}^*$ ,*

$$\Pr \left[ \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{ReSolveD-}\alpha.\text{KeyGen}(), \\ \sigma \leftarrow \text{ReSolveD-}\alpha.\text{Sign}(\text{sk}, \text{msg}), : b = 1 \\ b := \text{ReSolveD-}\alpha.\text{Verify}(\text{pk}, \text{msg}, \sigma) \end{array} \right] = 1.$$

*Proof.* For a given signature  $\sigma$ , up to Step 3 of [ReSolveD- \$\alpha\$ .Verify](#), the correctness of the VOLE commitment scheme ([Lemma 8.2](#)) implies that the value  $\text{com}$  computed in Step 9 of [ReSolveD- \$\alpha\$ .Sign](#) equals the value computed in Step 3 of [ReSolveD- \$\alpha\$ .Verify](#), and that

$$\begin{aligned} \mathbf{Q} &= \mathbf{V} \oplus [\delta_{0,0} \cdot \mathbf{u}, \dots, \delta_{0,n_0-1} \cdot \mathbf{u}, \dots, \delta_{\tau-1,0} \cdot \mathbf{u}, \dots, \delta_{\tau-1,n_{\tau-1}-1} \cdot \mathbf{u}], \\ \tilde{\mathbf{Q}} &= \tilde{\mathbf{V}}, \end{aligned}$$

where  $(\Delta_0, \dots, \Delta_{\tau-1}) := \text{VOLE.ChalDecAll}(\text{chall}_3)$  and, for each  $i \in [0, \tau)$ ,  $(\delta_{i,0}, \dots, \delta_{i,n_i-1}) := (\Delta_i[0], \dots, \Delta_i[n_i-1])$ . These equalities imply that [ReSolveD- \$\alpha\$ .Sign](#) and [ReSolveD- \$\alpha\$ .Verify](#) compute the same values of  $\text{chall}_1$  and  $\text{chall}_2$ .

By the correctness of the QuickSilver proof system ([Lemma 8.3](#)), the value  $\tilde{a}_0$  computed in [ReSolveD- \$\alpha\$ .Verify](#) equals the value  $\tilde{a}_0$  implicitly determined by the signer through the call to [QS.RSDProve](#) in [ReSolveD- \$\alpha\$ .Sign](#). Together with the above equalities and the definition of  $\text{ctr}$  in a valid signature, it follows that the value  $\text{chall}'_3$  computed by [ReSolveD- \$\alpha\$ .Verify](#) satisfies the check in Step 12, so [ReSolveD- \$\alpha\$ .Verify](#) outputs 1. Hence the theorem follows.  $\square$

## 8.2 Security Proof in the QROM and QICM

**Proof overview.** Our proof uses a similar blueprint to FAESTv2 [BBB<sup>+</sup>26] in the round 2 NIST submission. In the QROM, we would like to reduce the EUF-KO security of ReSolveD- $\alpha$  to the soundness of its underlying (public-coin) IP system. To do so, we first view the random-oracle calls in ReSolveD- $\alpha$  (including those at very low-level subroutine BAVC.Commit) as the generation of Fiat-Shamir challenges. From the view of Fiat-Shamir transformation, we can reversely define the underlying IP system and analyze its RBR soundness. Then, using the implication lemma of [BBB<sup>+</sup>25] from RBR soundness to soundness, we prove the soundness of the IP system. Finally, by switching a valid public key in the EUF-KO security game into a random one, which would be invalid with a non-negligible probability (depending on the concrete parameter choices), we obtain a lossy-key argument [KW03] to show that an efficient EUF-KO adversary can break the soundness of the IP system when the public key is invalid. This proof strategy does not require the knowledge soundness of the IP system since the witness extraction is difficult in the QROM.

Moreover, we can reduce the EUF-CMA security to the EUF-KO security. The high-level idea is that we use a hybrid argument to replace the signature oracle  $\text{OSig}_{\text{sk}}$  by a simulated oracle that never requires the secret key  $\text{sk}$ . In this hybrid argument, we use the quantum unpredictability of the commitment scheme and the adaptive reprogramming lemma [LGX25] to replace some Fiat-Shamir challenges  $(\text{chall}_i)_{i \in \{1,2,3\}}$  by uniform values in the QROM. Based on these uniform challenges and the hiding property of the commitment scheme, we simulate the secret key  $\text{sk}$  committed in the signature by a random value, removing the required  $\text{sk}$ .

**Interactive proof system  $\mathcal{R}\alpha\text{-IP}$ .** We construct the following public-coin interactive proof system  $\mathcal{R}\alpha\text{-IP}$ . In  $\mathcal{R}\alpha\text{-IP}$ , the prover  $\mathcal{P}_{\mathcal{R}\alpha\text{-IP}}$  takes the instance  $\mathbf{x} = \text{pk}' = (\mathbf{H}, \mathbf{y})$  and the witness  $\mathbf{w} = \text{sk}' = \mathbf{e}$  as input while the verifier  $\mathcal{V}_{\mathcal{R}\alpha\text{-IP}}$  only includes the instance  $\mathbf{x}$  in its input. The interaction between the prover and the verifier is shown in Fig. 7.

The prover  $\mathcal{P}_{\mathcal{R}\alpha\text{-IP}}$  runs **ReSolveD- $\alpha$ .Sign** except for the following modifications:

1. It does not call **RSD.SampleMatrix** and **RSD.SampleNoise** to compute  $\mathbf{H}$  and  $\mathbf{e}$ , respectively, but uses these values in  $\text{pk}'$  and  $\text{sk}'$  instead. It never uses  $\text{seed}_{\text{pk}}$  or  $\text{seed}_{\text{sk}}$ .
2. It replaces  $\text{pk}$  by  $\text{pk}'$  in **MsgHash** at the step 6.
3. It does not call **CoinHash** at the step 8 but uses uniformly sampled  $(r, \text{iv})$ .
4. It ignores the requirement that  $\text{chall}_3$  should include  $w_{\text{grind}} > 0$  trailing zero bits, but uses a uniformly sampled  $\text{ctr}$  for only one trial.
5. It does not query any random oracle but instead interacts with the verifier  $\mathcal{V}_{\mathcal{R}\alpha\text{-IP}}$  to receive the challenges (see Fig. 7) as the outputs of random oracle.

The verifier  $\mathcal{V}_{\mathcal{R}\alpha\text{-IP}}$  runs **ReSolveD- $\alpha$ .Verify** but has the following modifications:

1. It does not call **RSD.SampleMatrix** to compute  $\mathbf{H}$  but uses the value in  $\text{pk}'$  instead. It never uses  $\text{seed}_{\text{pk}}$ .
2. It replaces  $\text{pk}$  by  $\text{pk}'$  in **MsgHash** at the step 2.
3. It ignores the computation of challenges at the step 2 of **ReSolveD- $\alpha$ .Verify**, the step 27 of **BAVC.Verify** and the step 5, 8, and 11 of **ReSolveD- $\alpha$ .Verify**, but sends uniformly sampled challenges (with their specified lengths) to the prover  $\mathcal{P}_{\mathcal{R}\alpha\text{-IP}}$ .
4. Given  $\text{pmsg}_0 = ((\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha)}, \text{iv})$ , it outputs reject finally if there exists  $\alpha \in [0, \tau)$  such that (i) there exists  $i \in [0, N_\alpha)$  such that  $\text{com}_i^\alpha$  has at least two pre-images, or (ii) there



4.  $\text{chall}_2 = \text{FSHash}_2(\text{chall}_1, \text{pmsg}_2) = \text{FSHash}_2(\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$ .
5.  $\text{chall}_3 = \text{FSHash}_3(\text{chall}_2, \text{pmsg}_3) = \text{FSHash}_3(\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$ .

Finally, the prover sends the message  $\text{msg} = \text{pmsg}_{-1}$  and its signature  $\sigma'$  to the verifier:

$$\begin{aligned} \sigma' &= (\text{pmsg}_0, \text{pmsg}_1, \text{pmsg}_2, \text{pmsg}_3, \text{pmsg}_4) \\ &= ((\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}, \text{iv}, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau]}, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d}, \tilde{a}_1, \tilde{a}_0, \text{ctr}, \text{decom}_I). \end{aligned} \quad (18)$$

- The verifier (i.e., the verification algorithm) will use  $\mathbb{x}$ ,  $\sigma'$ , and the given random oracles to reproduce the above challenges if needed.

Note that the signature scheme  $\text{FS}[\text{R}\alpha\text{-IP}]$  uses uniformly sampled  $(r, \text{iv})$  due to [Item 3](#).

**RBR soundness of  $\text{R}\alpha\text{-IP}$ .**  $\text{R}\alpha\text{-IP}$  does not use any random oracle. The following theorem comes from pure probability analysis based on the uniformly random challenges.

**Theorem 8.2** (RBR Soundness of  $\text{R}\alpha\text{-IP}$ ). *In  $\text{R}\alpha\text{-IP}$ , consider the following ideal-model oracles and their respective numbers of queries:*

- $\mathcal{E}$  is an ideal cipher realizing [PRG](#) in [LeafHash](#), with  $Q_{\mathcal{E}}$  queries,
- $\mathcal{E}'$  is an ideal cipher realizing [TCCR](#) in [BAVC.Commit](#), with  $Q_{\mathcal{E}'}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. Let  $\mathcal{R}$  be the RSD relation and  $\mathcal{L}_{\mathcal{R}}$  be its language. For the language  $\mathcal{L}_{\mathcal{R}}$ ,  $\text{R}\alpha\text{-IP}$  is  $\varepsilon_{\text{RBR}}$ -RBR-sound in the QICM parameterized by  $(\mathcal{O}, Q_{\mathcal{O}})$ , where

$$\begin{aligned} \varepsilon_{\text{RBR}} &= \max(\varepsilon_{-1}, \varepsilon_0, \varepsilon_1, \varepsilon_2, \varepsilon_3), \\ \varepsilon_{-1} &= \varepsilon_0 = 0, \\ \varepsilon_1 &= \varepsilon_{\text{vole}} \cdot \binom{\tau}{2}, \varepsilon_2 = \varepsilon_{\text{zk}}, \varepsilon_3 = \frac{2}{2^\lambda}. \end{aligned}$$

*Proof.* The RBR soundness is taken over the random challenges and independent of the ideal ciphers. We construct the following *non-polynomial-time* algorithm `state`.

- Initial case. On input an instance  $\mathbb{x}$  and an empty transcript  $\text{tr}_0 = \emptyset$ , `state` enumerates all possible witness to check whether  $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$ . If and only if  $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}}$ , `state` outputs `reject`.
- On input  $\mathbb{x}$  and  $\text{tr}_2 = (\text{msg}, \mu, \mathbf{s})$ , `state` outputs `state`( $\mathbb{x}, \text{tr}_0$ ). We have

$$\Pr_{(\mu, \mathbf{s})} [\text{state}(\mathbb{x}, \text{tr}_2) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_0) = \text{reject}] = 0 = \varepsilon_{-1}.$$

- On input  $\mathbb{x}$  and  $\text{tr}_4 = (\text{tr}_2, (\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}, \text{iv}, \text{com})$ , `state` outputs `state`( $\mathbb{x}, \text{tr}_2$ ).

$$\Pr_{\text{com}} [\text{state}(\mathbb{x}, \text{tr}_4) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_2) = \text{reject}] = 0 = \varepsilon_0.$$

- On input  $\mathbb{x}$  and  $\text{tr}_6 = (\text{tr}_4, (\mathbf{c}_\alpha)_{\alpha \in [1, \tau]}, \text{chall}_1)$ , if `state`( $\mathbb{x}, \text{tr}_4$ ) = `accept`, `state` outputs `accept`; otherwise, `state` enumerates all possible pre-images and distinguishes the following cases:
  - Case `BadPre`: There exists  $\alpha \in [0, \tau)$  such that (i) there exists  $i \in [0, N_\alpha)$  such that  $\text{com}_i^\alpha$  has at least two pre-images, or (ii) there exist two distinct  $i, i' \in [0, N_\alpha)$  such that both  $\text{com}_i^\alpha$  and  $\text{com}_{i'}^\alpha$  have no pre-images. In this case, `state` outputs `reject`.  
Conditioned on that `BadPre` does not happen (denoted by an event  $\neg \text{BadPre}$ ), there must be one of the following two cases for each  $\alpha \in [0, \tau)$ .

- Case  $\text{OneNoPre}_\alpha$ : There exists only one  $i \in [0, N_\alpha)$  such that  $\text{com}_i^\alpha$  has no pre-image (i.e., for any  $i' \neq i$ ,  $\text{com}_{i'}^\alpha$  has a unique pre-image  $\text{seed}_{i'}^\alpha$ ). state defines  $\Delta_\alpha = i$  and extracts

$$\begin{aligned} (\mathbf{u}'_\alpha, \mathbf{Q}'_\alpha) &= \text{VOLE.ConvertToVOLE}((\perp, \text{seed}_{1 \oplus \Delta_\alpha}^\alpha, \dots, \text{seed}_{(N_\alpha-1) \oplus \Delta_\alpha}^\alpha), \text{iv}), \\ (\mathbf{u}_\alpha, \mathbf{V}_\alpha) &= \begin{cases} (\mathbf{u}'_\alpha, \text{VOLE.Fix}(\mathbf{Q}'_\alpha, \mathbf{u}'_\alpha, \Delta_\alpha)), & \alpha = 0 \\ (\mathbf{u}'_\alpha \oplus \mathbf{c}_\alpha, \text{VOLE.Fix}(\mathbf{Q}'_\alpha, \mathbf{u}'_\alpha, \Delta_\alpha)), & \alpha \in [1, \tau) \end{cases} \end{aligned} \quad (19)$$

- Case  $\text{UniquePre}_\alpha$ : For any  $i \in [0, N_\alpha)$ ,  $\text{com}_i^\alpha$  has a unique pre-image  $\text{seed}_i^\alpha$ . state extracts

$$(\mathbf{u}_\alpha, \mathbf{V}_\alpha) = \text{VOLE.ConvertToVOLE}((\text{seed}_i^\alpha)_{i \in [0, N_\alpha)}, \text{iv}). \quad (20)$$

That is, conditioned on  $\neg \text{BadPre}$ , state can extract  $(\mathbf{u}_\alpha, \mathbf{V}_\alpha)$  for each  $\alpha \in [0, \tau)$ . We define, conditioned on  $\neg \text{BadPre}$ , an event  $\text{BadCh}_1$  that there exist two distinct  $\alpha, \alpha' \in [0, \tau)$  such that  $\mathbf{u}_\alpha \neq \mathbf{u}_{\alpha'}$  and  $\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) = \text{VOLEHash}(\text{chall}_1, \mathbf{u}_{\alpha'})$ . If and only if  $\text{BadCh}_1$  occurs, state additionally outputs `accept`. We use the following lemma to bound the probability.

**Lemma 8.4.** *Given an  $\varepsilon_{\text{vole}}$ -universal linear family  $\text{VOLEHash}$  of hash functions, we have*

$$\Pr_{\text{chall}_1} \left[ \text{BadCh}_1 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg \text{BadPre} \end{array} \right] \leq \varepsilon_{\text{vole}} \cdot \binom{\tau}{2}.$$

*Proof.* Conditioned on  $\neg \text{BadPre}$ , either  $\text{OneNoPre}_\alpha$  or  $\text{UniquePre}_\alpha$  occurs for each  $\alpha \in [0, \tau)$ . All  $(\mathbf{u}_\alpha)_{\alpha \in [0, \tau)}$ 's are well-defined (more precisely, syntactically identical due to Lemma 8.1). For any two *fixed* distinct  $\alpha, \alpha' \in [0, \tau)$ , as the randomness  $\text{chall}_1$  is uniform and independent of  $(\mathbf{u}_\alpha)_{\alpha \in [0, \tau)}$ , the event that  $\mathbf{u}_\alpha \neq \mathbf{u}_{\alpha'}$  and  $\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) = \text{VOLEHash}(\text{chall}_1, \mathbf{u}_{\alpha'})$  can only happen with a probability at most  $\varepsilon_{\text{vole}}$  due to the universality and linearity. The target bound follows from a simple union bound over all possible distinct  $\alpha, \alpha' \in [0, \tau)$ . This lemma is a special case of the Theorem 2 in [BBD<sup>+</sup>23b].  $\square$

As a result, we have that

$$\begin{aligned} & \Pr_{\text{chall}_1} [\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject}] \\ &= \Pr_{\text{chall}_1} \left[ \text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \text{BadPre} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_1} [\text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject}] \\ & \quad + \Pr_{\text{chall}_1} \left[ \text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg \text{BadPre} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_1} [\neg \text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject}] \\ &= \Pr_{\text{chall}_1} \left[ \text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg \text{BadPre} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_1} [\neg \text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject}] \\ &\leq \Pr_{\text{chall}_1} \left[ \text{state}(\mathbb{x}, \text{tr}_6) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg \text{BadPre} \end{array} \right] \\ &= \Pr_{\text{chall}_1} \left[ \text{BadCh}_1 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_4) = \text{reject} \\ \wedge \neg \text{BadPre} \end{array} \right] \leq \varepsilon_{\text{vole}} \cdot \binom{\tau}{2} = \varepsilon_1. \end{aligned}$$

- On input  $\mathbb{x}$  and  $\text{tr}_8 = (\text{tr}_6, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d}, \text{chall}_2)$ , if  $\text{state}(\mathbb{x}, \text{tr}_6) = \text{accept}$ , state outputs accept; otherwise, the condition  $\text{state}(\mathbb{x}, \text{tr}_6) = \text{reject}$  happens if and only if  $\text{BadPre}$  or  $\neg \text{BadPre} \wedge \neg \text{BadCh}_1$  occurs, where the latter event is that all  $\{(\mathbf{u}_\alpha, \mathbf{V}_\alpha)\}_{\alpha \in [0, \tau]}$  can be extracted and, for any two distinct  $\alpha, \alpha' \in [0, \tau]$ ,  $\mathbf{u}_\alpha = \mathbf{u}_{\alpha'}$  or  $\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) \neq \text{VOLEHash}(\text{chall}_1, \mathbf{u}_{\alpha'})$ . state distinguishes the following mutually exclusive and exhaustive cases of the condition:
  - Case  $\text{BadPre}$ . In this case, state still outputs reject.
  - Case  $\neg \text{BadPre} \wedge \neg \text{BadCh}_1$ . In this case, state defines  $\mathbf{u} := \mathbf{u}_0$  and  $\mathbf{w} := \mathbf{d} \oplus \mathbf{u}[0, \ell_{\text{wit}}]$ , and extracts the witness  $\mathbf{e} := \text{RSD.DecodeNoise}(\mathbf{w}, \mathbf{H}, \mathbf{y})$ . In QuickSilver, the relation to be proved is parsed as degree-2 polynomials. Let  $\mathbf{a}$  be a vector of the degree-2 terms of the polynomials evaluated on  $\mathbf{e}$ . Note that the witness  $\mathbf{e}$  satisfies the RSD relation if and only if  $\mathbf{a} = \mathbf{0}$  according to Lemma 3.5. We define an event  $\text{BadCh}_2$  that  $\mathbf{a} \neq \mathbf{0}$  and  $\text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0$ . If and only if  $\text{BadCh}_2$  happens, state additionally outputs accept.

**Lemma 8.5.** *Given an  $\varepsilon_{\text{zk}}$ -universal linear family  $\text{ZKHash}$  of hash functions, we have*

$$\Pr_{\text{chall}_2} \left[ \text{BadCh}_2 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \leq \varepsilon_{\text{zk}}.$$

*Proof.* From the law of total probability, we have

$$\begin{aligned} & \Pr_{\text{chall}_2} \left[ \mathbf{a} \neq \mathbf{0} \wedge \text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\ &= \Pr_{\text{chall}_2} \left[ \text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \\ \wedge \mathbf{a} \neq \mathbf{0} \end{array} \right] \\ & \quad \cdot \Pr_{\text{chall}_2} \left[ \mathbf{a} \neq \mathbf{0} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\ &\leq \Pr_{\text{chall}_2} \left[ \text{ZKHash}(\text{chall}_2, \mathbf{a}) = 0 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \\ \wedge \mathbf{a} \neq \mathbf{0} \end{array} \right] \\ &\leq \varepsilon_{\text{zk}}, \quad \text{(By uniform } \text{chall}_2 \text{ and the universality)} \end{aligned}$$

arriving at the target bound.  $\square$

From this lemma and the cases, we can prove that

$$\begin{aligned}
& \Pr_{\text{chall}_2} \left[ \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\
&= \Pr_{\text{chall}_2} \left[ \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge \text{BadPre} \end{array} \right] \\
&\quad \cdot \Pr_{\text{chall}_2} \left[ \text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\
&\quad + \Pr_{\text{chall}_2} \left[ \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\
&\quad \cdot \Pr_{\text{chall}_2} \left[ \neg \text{BadPre} \wedge \neg \text{BadCh}_1 \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\
&= \Pr_{\text{chall}_2} \left[ \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\
&\quad \cdot \Pr_{\text{chall}_2} \left[ \neg \text{BadPre} \wedge \neg \text{BadCh}_1 \mid \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \right] \\
&\leq \Pr_{\text{chall}_2} \left[ \text{state}(\mathbb{x}, \text{tr}_8) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \\
&= \Pr_{\text{chall}_2} \left[ \text{BadCh}_2 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_6) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1) \end{array} \right] \leq \varepsilon_{\text{zk}} = \varepsilon_2.
\end{aligned}$$

- On input  $\mathbb{x}$  and  $\text{tr}_{10} = (\text{tr}_8, \tilde{a}_1, \tilde{a}_0, \text{ctr}, \text{chall}_3)$ , if  $\text{state}(\mathbb{x}, \text{tr}_8) = \text{accept}$ , state outputs accept; otherwise, the condition  $\text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}$  happens if and only if  $\text{BadPre}$  or  $\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2$  occurs. state distinguishes the mutually exclusive and exhaustive cases of the condition:

- Case  $\text{BadPre}$ . In this case, state still outputs reject.
- Case  $\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2$ . In this case, state will use the previously extracted  $\{(\mathbf{u}_\alpha, \mathbf{V}_\alpha)\}_{\alpha \in [0, \tau]}$  and  $\text{chall}_3 \in \{0, 1\}^\lambda$  to compute

$$\begin{aligned}
\Delta &= (\Delta_0, \dots, \Delta_{\tau-1}) := \text{VOLE.ChalDecAll}(\text{chall}_3) \in \mathbb{F}_2^\lambda, \\
\forall \alpha \in [0, \tau) : (\delta_{\alpha,0}, \dots, \delta_{\alpha, k_\alpha-1}) &:= (\Delta_\alpha[0], \dots, \Delta_\alpha[k_\alpha-1]) \in \mathbb{F}_2^{k_\alpha}, \\
\forall \alpha \in [0, \tau) : \mathbf{Q}_\alpha &:= \mathbf{V}_\alpha \oplus [\delta_{\alpha,0} \cdot \mathbf{u}_\alpha, \dots, \delta_{\alpha, k_\alpha-1} \cdot \mathbf{u}_\alpha] \in \mathbb{F}_2^{\ell_{\text{vole}} \times k_\alpha}, \\
\mathbf{Q} &:= [\mathbf{Q}_0, \dots, \mathbf{Q}_{\tau-1}] \in \mathbb{F}_2^{\ell_{\text{vole}} \times \lambda}.
\end{aligned} \tag{21}$$

Then, state uses  $\mathbf{Q}$  to compute  $\tilde{\mathbf{Q}}$  (until the step 7) and  $\tilde{a}_0$  (until the step 10) according to [ReSolveD- \$\alpha\$ .Verify](#). We define an event  $\text{BadQS}$  that the extracted witness  $e$  does not satisfy the RSD relation and  $\tilde{a}_0 = \tilde{a}_0$ , and an event  $\text{BadCh}_3 := (\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}) \wedge \text{BadQS}$ . If and only if  $\text{BadCh}_3$  happens, state additionally outputs accept.

**Lemma 8.6.** *We have*

$$\Pr_{\text{chall}_3} \left[ \text{BadCh}_3 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \leq \frac{2}{2^\lambda}.$$

*Proof.* Conditioned on  $\text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2)$ , we have

- For  $\tilde{\mathbf{V}} = [\tilde{\mathbf{V}}_0, \dots, \tilde{\mathbf{V}}_{\tau-1}] \in \mathbb{F}_2^{(\lambda+B) \times \lambda}$ , the checked equality  $\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}$  is equivalent to

$$\begin{aligned}
& \tilde{\mathbf{Q}} = \tilde{\mathbf{V}} \in \mathbb{F}_2^{(\lambda+B) \times \lambda} \\
& \Leftrightarrow \forall \alpha \in [0, \tau) : \text{VOLEHash}(\text{chall}_1, \mathbf{Q}_\alpha) \oplus [\delta_{\alpha,0} \cdot \tilde{\mathbf{u}}, \dots, \delta_{\alpha,k_\alpha-1} \cdot \tilde{\mathbf{u}}] = \tilde{\mathbf{V}}_\alpha \in \mathbb{F}_2^{(\lambda+B) \times k_\alpha} \\
& \Leftrightarrow \forall \alpha \in [0, \tau) : \\
& \quad [\delta_{\alpha,0} \cdot (\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) \oplus \tilde{\mathbf{u}}), \dots, \delta_{\alpha,k_\alpha-1} \cdot (\text{VOLEHash}(\text{chall}_1, \mathbf{u}_\alpha) \oplus \tilde{\mathbf{u}})] \\
& \quad = \text{VOLEHash}(\text{chall}_1, \mathbf{V}_\alpha) \oplus \tilde{\mathbf{V}}_\alpha \in \mathbb{F}_2^{(\lambda+B) \times k_\alpha}. \tag{22}
\end{aligned}$$

Conditioned on  $\neg \text{BadPre} \wedge \neg \text{BadCh}_1$ , we define a subset from  $(\{(\mathbf{u}_\alpha, \mathbf{V}_\alpha)\}_{\alpha \in [0, \tau)}, \tilde{\mathbf{u}}, \tilde{\mathbf{V}})$ :

$$S = \left\{ \Delta = [\delta_{0,0}, \dots, \delta_{0,k_0-1}, \dots, \delta_{\tau-1,0}, \dots, \delta_{\tau-1,k_{\tau-1}-1}] \in \{0, 1\}^\lambda \mid \text{such that Eq. (22) holds} \right\} \subseteq \{0, 1\}^\lambda,$$

which corresponds to the prover's guess on  $\text{chall}_3$  (i.e.,  $\Delta$ ) by the time of its sending  $\text{pmsg}_2$ . It is clear that  $\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}$  if and only if the uniformly sampled  $\Delta \in S \neq \emptyset$ . Therefore,

$$\Pr_{\text{chall}_3} \left[ \tilde{\mathbf{Q}} = \tilde{\mathbf{V}} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \leq \frac{|S|}{2^\lambda}.$$

- Conditioned on  $\text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \wedge \neg \text{BadCh}_2$ , no valid extended witness can satisfy the RSD relation (otherwise,  $\text{state}(\mathbb{x}, \text{tr}_0) = \text{accept}$  further guarantees  $\text{state}(\mathbb{x}, \text{tr}_8) = \text{accept}$ ). Also,  $\text{ZKHash}(\text{chall}_2, \mathbf{a}) \neq 0$  for the vector of degree-2 coefficients  $\mathbf{a} \neq 0$  evaluated on any invalid extended witness. Additionally conditioned on the prover's correct guess on  $\Delta$  (i.e.,  $\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}$ ), the  $\text{QS.RSDVerify}$  algorithm ensures that  $\tilde{a}_0 - \tilde{a}_0$  is a degree-2 non-zero polynomial of  $\Delta$ . With this additional condition,  $\Delta$  is uniformly sampled from  $S$  and independent of the polynomial. We can use the Schwartz-Zippel lemma to show that

$$\Pr_{\text{chall}_3} \left[ \text{BadQS} \mid \begin{array}{l} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \\ \wedge (\tilde{\mathbf{Q}} = \tilde{\mathbf{V}}) \end{array} \right] \leq \frac{2}{|S|}.$$

This lemma follows from the above two upper bounds.  $\square$

Using the law of total probability, we have

$$\begin{aligned}
& \Pr_{\text{chall}_3} [\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\
&= \Pr_{\text{chall}_3} \left[ \text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge \text{BadPre} \end{array} \right] \\
&\quad \cdot \Pr_{\text{chall}_3} [\text{BadPre} \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\
&\quad + \Pr_{\text{chall}_3} \left[ \text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \\
&\quad \cdot \Pr_{\text{chall}_3} [\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\
&= \Pr_{\text{chall}_3} \left[ \text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \\
&\quad \cdot \Pr_{\text{chall}_3} [\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \mid \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject}] \\
&\leq \Pr_{\text{chall}_3} \left[ \text{state}(\mathbb{x}, \text{tr}_{10}) = \text{accept} \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \\
&= \Pr_{\text{chall}_3} \left[ \text{BadCh}_3 \mid \begin{array}{c} \text{state}(\mathbb{x}, \text{tr}_8) = \text{reject} \\ \wedge (\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2) \end{array} \right] \leq \frac{2}{2^\lambda} = \varepsilon_3.
\end{aligned}$$

- Finally, on input  $\mathbb{x}$  and a full transcript  $\text{tr}_{11} = (\text{tr}_{10}, \text{decom}_I)$ ,  $\text{state}$  outputs **reject** if and only if  $\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{reject}$  or the commitments  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}$  reconstructed from  $\text{decom}_I$  (as per [BAVC.Verify](#)) do not equal  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}$ .  $\text{state}(\mathbb{x}, \text{tr}_{10}) = \text{reject}$  occurs if and only if  $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}}$ , and either  $\text{BadPre}$  or  $\neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \wedge \neg \text{BadCh}_3$  occurs.
  - If  $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \text{BadPre}$  occurs, the verifier  $\mathcal{V}_{\text{R}\alpha\text{-IP}}$  outputs **reject** given  $\text{pmsg}_0$ .
  - If  $\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \neg \text{BadPre} \wedge \neg \text{BadCh}_1 \wedge \neg \text{BadCh}_2 \wedge \neg \text{BadCh}_3$  occurs, the sub-event

$$\begin{aligned}
\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \neg \text{BadCh}_3 &= \mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge ((\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee \neg \text{BadQS}) \\
&= (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge (\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}})) \vee (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge \neg \text{BadQS}) \\
&= (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge (\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}})) \vee (\mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge (\tilde{a}_0 \neq \tilde{a}_0)) \\
&= \mathbb{x} \notin \mathcal{L}_{\mathcal{R}} \wedge ((\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee (\tilde{a}_0 \neq \tilde{a}_0))
\end{aligned}$$

must hold, which further implies that the event  $(\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee (\tilde{a}_0 \neq \tilde{a}_0)$  occurs. According to [Eq. \(19\)](#), [Eq. \(20\)](#), [Eq. \(21\)](#), and [Lemma 8.1](#), the value of  $\mathbf{Q}$  computed in  $\text{state}$  is identically defined as its counterpart in [ReSolveD- \$\alpha\$ .Verify](#) run by the verifier  $\mathcal{V}_{\text{R}\alpha\text{-IP}}$ . Thus, so is the  $(\tilde{\mathbf{Q}}, \tilde{a}_0)$  identically computed from the same  $\mathbf{Q}$ . This indicates that the event  $(\tilde{\mathbf{Q}} \neq \tilde{\mathbf{V}}) \vee (\tilde{a}_0 \neq \tilde{a}_0)$  is identically defined for  $\text{state}$  and [ReSolveD- \$\alpha\$ .Verify](#) run by the verifier. If this event occurs, the verifier  $\mathcal{V}_{\text{R}\alpha\text{-IP}}$  outputs **reject** given  $\text{pmsg}_2$  or  $\text{pmsg}_3$ .

- If the commitments  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}$  reconstructed from  $\text{decom}_I$  (as per [BAVC.Verify](#)) do not equal  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}$ , the verifier  $\mathcal{V}_{\text{R}\alpha\text{-IP}}$  outputs **reject** given  $\text{pmsg}_4$ .

Therefore, for the constructed algorithm  $\text{state}$ , if  $\text{state}(\mathbb{x}, \text{tr}_{11}) = \text{reject}$  for any full transcript  $\text{tr}_{11}$ , then  $\mathcal{V}_{\text{R}\alpha\text{-IP}}(\mathbb{x}, \text{tr}_{11}) = \text{reject}$  as required.

The above round-by-round analysis concludes this proof.  $\square$

**RBR soundness of  $R\alpha\text{-IP} \Rightarrow$  soundness of  $\text{FS}[R\alpha\text{-IP}]$ .** To achieve this reduction, we use the lemma of [BBB<sup>+</sup>25], which compiles RBR soundness into soundness in the QROM.

**Definition 8.2** (Collision-Resistance-Preserving and Sensitive Function [BBB<sup>+</sup>25]). *A function  $f^H : \mathcal{X} \rightarrow \mathcal{Y}$  that can be computed using  $N$  queries to a random oracle  $H : \mathcal{X}' \rightarrow \mathcal{Y}'$  is collision-resistance-preserving if, for any  $x_0, x_1 \in \mathcal{X}$  such that  $x_0 \neq x_1$  and  $f^H(x_0) = f^H(x_1)$ , there exist  $x'_0, x'_1 \in \mathcal{X}'$  such that  $x'_0 \neq x'_1$  and*

- $H$  is queried on input  $x'_i$  when  $f^H(x_i)$  is computed for each  $i \in \{0, 1\}$ .
- $H(x'_0) = H(x'_1)$ .

Moreover, the function  $f^H$  is sensitive if (i)  $|\{f^{H^{x' \mapsto u}}(x) \mid u \in \mathcal{Y}'\}| \in \{1, |\mathcal{Y}'|\}$ , and (ii) for any  $x, x' \in \mathcal{X}$  such that  $x \neq x'$ ,  $f^H(x)$  does not make the same queries to  $H$  as  $f^H(x')$ .

**Lemma 8.7** (RBR Soundness of IP  $\Rightarrow$  Soundness of IP [BBB<sup>+</sup>25]). *Let IP be an IP system with  $\ell$  challenges  $(\text{chall}_i)_{i \in [\ell]}$ . Let  $S \subset [\ell]$  and  $|S| = \ell_1$  such that IP has the RBR soundness with an error 0 for the challenges  $(\text{chall}_i)_{i \in S}$  and an error  $\varepsilon$  else. Let  $f^{H_1}$  be a collision-resistance-preserving and sensitive function making  $N$  queries to the random oracle  $H_1$  and  $\text{FS}[\text{IP}]$  be the Fiat-Shamir transformation of IP, where the challenges  $(\text{chall}_i)_{i \in S}$  are derived using  $f^{H_1}$  and the challenge  $\text{chall}_i$  is derived using the random oracle  $H_2^i$  for each  $i \in [\ell] \setminus S$ . Let  $Q_{H_1}$  be the number of queries to  $H_1$  and  $Q_{H_2^i}$  be the number of queries to  $H_2^i$  for each  $i \in [\ell] \setminus S$ . Let  $\mathcal{O} = (H_1, (H_2^i)_{i \in [\ell] \setminus S})$  and  $Q_{\mathcal{O}} = (Q_{H_1}, (Q_{H_2^i})_{i \in [\ell] \setminus S})$ . For any quantum adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  quantum queries respectively to the oracles in  $\mathcal{O}$ ,*

$$\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}, \text{FS}[\text{IP}]}(Q_{\mathcal{O}}) \leq 30 \cdot Q^3 \cdot 2^{-\ell_{\text{out}}} + 10 \cdot Q^2 \cdot \varepsilon$$

where  $\ell_{\text{out}}$  is the output length of  $H_1$  and  $Q = (Q_{H_1} + \sum_{i \in [\ell] \setminus S} Q_{H_2^i}) + 2\ell + 2(N - 1)\ell_1$ .

[BBB<sup>+</sup>25] has argued that  $f^H(x_0, \dots, x_{N-1}) = (H(x_0), \dots, H(x_{N-1}))$  meets Definition 8.2. Using this result, we have the following theorem.

**Theorem 8.3** (Soundness of  $\text{FS}[R\alpha\text{-IP}]$ ). *In  $\text{FS}[R\alpha\text{-IP}]$ , consider the following ideal-model oracles and their respective numbers of queries:*

- $E$  is an ideal cipher realizing PRG in LeafHash, with  $Q_E$  queries,
- $E'$  is an ideal cipher realizing TCCR in BAVC.Commit, with  $Q_{E'}$  queries,
- MsgHash is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{MsgHash}}$  queries,
- BAVCHash is a random oracle in BAVC.Commit, with  $Q_{\text{BAVCHash}}$  queries,
- FSHash<sub>1</sub> is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{FSHash}_1}$  queries,
- FSHash<sub>2</sub> is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{FSHash}_2}$  queries,
- FSHash<sub>3</sub> is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{FSHash}_3}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles (where  $E$  and  $E'$  are inherited from  $R\alpha\text{-IP}$  and do not affect RBR soundness) and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. For any quantum adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  quantum queries respectively to the oracles in  $\mathcal{O}$ ,

$$\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}, \text{FS}[R\alpha\text{-IP}]}(Q_{\mathcal{O}}) \leq 30 \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot \varepsilon_{\text{RBR}},$$

where  $Q = Q_{\text{MsgHash}} + Q_{\text{BAVCHash}} + Q_{\text{FSHash}_1} + Q_{\text{FSHash}_2} + Q_{\text{FSHash}_3} + 10$  and  $\varepsilon_{\text{RBR}}$  is from Theorem 8.2.

*Proof.* Due to Theorem 8.2, in  $\text{FS}[R\alpha\text{-IP}]$  (where the underlying IP system  $R\alpha\text{-IP}$  proceeds in the QICM), we can define  $S = \{0\}$  and  $\ell = 5$  (where we slightly abuse the notation since the subscript of challenges starts from  $-1$ ). For this  $S$ ,  $\text{chall}_0$  is derived with an RBR soundness error 0 from one call to the random oracle BAVCHash. Therefore, this theorem follows from Lemma 8.7 by using  $S = \{0\}$ ,  $\ell = 5$ ,  $\ell_1 = 1$ ,  $N = 1$ ,  $\ell_{\text{out}} = 2\lambda$ , and the given  $Q$ .  $\square$

**Soundness of FS[R $\alpha$ -IP]  $\Rightarrow$  EUF-KO security of ReSolveD- $\alpha$ .** We recall a lemma used in the lossy-key argument and provide the proof of the EUF-KO security.

**Lemma 8.8** ([BBB<sup>+</sup>25]). *Let  $G : \mathcal{X} \rightarrow \mathcal{Y}$  be a PRG. For any (potentially unbounded) distinguisher  $\mathcal{A}$ ,*

$$\Pr_{y \leftarrow \text{im}(G)} [\mathcal{A}(G, y) = 1] \leq \frac{|\text{dom}(G)|}{|\text{im}(G)|} \cdot \Pr_{x \leftarrow \text{dom}(G)} [\mathcal{A}(G, G(x)) = 1].$$

**Theorem 8.4** (EUF-KO Security of ReSolveD- $\alpha$  against Quantum Adversary). *In ReSolveD- $\alpha$ , consider the following ideal-model oracles and their respective numbers of queries:*

- $\mathbf{E}$  is an ideal cipher realizing PRG in LeafHash, with  $Q_{\mathbf{E}}$  queries,
- $\mathbf{E}'$  is an ideal cipher realizing TCCR in BAVC.Commit, with  $Q_{\mathbf{E}'}$  queries,
- $\mathbf{E}''$  is an ideal cipher realizing PRG in RSD.SampleNoise, with  $Q_{\mathbf{E}''}$  queries,
- MatHash is a random oracle in RSD.SampleMatrix, with  $Q_{\text{MatHash}}$  queries,
- MsgHash is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{MsgHash}}$  queries,
- CoinHash is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{CoinHash}}$  queries,
- BAVCHash is a random oracle in BAVC.Commit, with  $Q_{\text{BAVCHash}}$  queries,
- FSHash<sub>1</sub> is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{FSHash}_1}$  queries,
- FSHash<sub>2</sub> is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{FSHash}_2}$  queries,
- FSHash<sub>3</sub> is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{FSHash}_3}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. Let  $\mathcal{O}' = \mathcal{O} \setminus \{\mathbf{E}'', \text{MatHash}, \text{CoinHash}\}$  and  $Q_{\mathcal{O}'} = Q_{\mathcal{O}} \setminus \{Q_{\mathbf{E}''}, Q_{\text{MatHash}}, Q_{\text{CoinHash}}\}$ . For any quantum adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  quantum queries respectively to the oracles in  $\mathcal{O}$ ,

$$\begin{aligned} & \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}) \\ & \leq \left(1 - \frac{\ell_{\text{bs}}^w}{2^{m-k}}\right)^{-1} \cdot \left( \text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}', \text{FS}[\text{R}\alpha\text{-IP}]}(Q_{\mathcal{O}'}) + \text{AdvInj}_{\mathcal{A}_{\text{Inj}}}^{\mathbf{E}, \text{PRG}'}(Q_{\mathbf{E}} + L, L) + \text{AdvRSD}_{\mathcal{A}_{\text{RSD}}}(m, k, w) \right) \\ & \quad + \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\mathbf{E}'', \text{PRG}''}(Q_{\mathbf{E}''}) + \left( \frac{\lfloor \frac{\lambda}{\ell_{\text{bs}}} \rfloor + 1}{\lambda} - \frac{1}{\ell_{\text{bs}}} \right) + \left( 2 + 5 \cdot \left(1 - \frac{\ell_{\text{bs}}^w}{2^{m-k}}\right)^{-1} \right) \cdot \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right), \end{aligned}$$

where  $\text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}', \text{FS}[\text{R}\alpha\text{-IP}]}(Q_{\mathcal{O}'})$  is from Theorem 8.3,  $\varepsilon_{\text{SR}}(\frac{\lambda}{2})$  is from Lemma 8.12,

- $\text{PRG}' : (x, \text{iv}, t) \in \{0, 1\}^{\lambda} \times \{0, 1\}^{120} \times \{0, 1\}^8 \mapsto \text{PRG}(x, \text{iv}, t; 2\lambda) \in \{0, 1\}^{2\lambda}$  is an  $\mathbf{E}$ -based almost injective PRG (Conjecture 8.1), and
- $\text{PRG}'' : x \in \{0, 1\}^{\lambda} \mapsto \text{PRG}(x, 0^{120}, 0^8; m \cdot \lambda) \in \{0, 1\}^{m \cdot \lambda}$  is an  $\mathbf{E}''$ -based PRG (Theorem 8.8).

*Proof.* We prove this theorem using the following hybrid argument.

- $G_0$ . This is the real EUF-KO security game.

$$\Pr[G_0(Q_{\mathcal{O}}) = 1] = \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}). \quad (23)$$

- $G_1$ . This game is identical to the previous one, except that at the very beginning, it samples uniform  $(\text{seed}_{\text{pk}}, \mathbf{H}_B)$ , uses the compressed oracle technique to locally emulate the random oracle [MatHash](#) in [RSD.SampleMatrix](#) for  $\mathcal{A}$  at the only position  $\text{seed}_{\text{pk}}$  so that

$$\mathbf{H} = [\mathbf{I} \parallel \mathbf{H}_B] = \text{RSD.SampleMatrix}(\text{seed}_{\text{pk}}),$$

and sends  $\text{pk} = (\text{seed}_{\text{pk}}, \mathbf{H} \cdot \mathbf{e})$  to  $\mathcal{A}$ . These two games are perfectly indistinguishable due to the compressed oracle technique.

$$\Pr [G_0(Q_{\mathcal{O}}) = 1] = \Pr [G_1(Q_{\mathcal{O}}) = 1]. \quad (24)$$

- $G_2$ . This game is identical to the previous one, except that it uses uniformly regular noise  $\mathbf{e}$  rather than computing it from  $\text{seed}_{\text{sk}}$ . We use a reduction to the PRG security of [PRG''](#) (called in [RSD.SampleNoise](#)) against an adversary  $\mathcal{A}_{\text{PRG}}$  to prove that

$$|\Pr [G_1(Q_{\mathcal{O}}) = 1] - \Pr [G_2(Q_{\mathcal{O}}) = 1]| \leq \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{E'', \text{PRG}''}(Q_{E''}) + \left( \frac{\lfloor \frac{\lambda}{\ell_{\text{bs}}} \rfloor + 1}{\lambda} - \frac{1}{\ell_{\text{bs}}} \right) + 2 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \quad (25)$$

Assume that [Eq. \(25\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{PRG}}$  that internally runs  $\mathcal{A}$  to break the PRG security:

1. Given the quantum oracle access to  $E''$ ,  $\mathcal{A}_{\text{PRG}}$  uses  $E''$  for  $\mathcal{A}$ 's quantum queries to  $E''$ .
2.  $\mathcal{A}_{\text{PRG}}$  locally emulates the oracles in  $\mathcal{O} \setminus \{E''\}$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3. Given some PRG challenge  $y$ ,  $\mathcal{A}_{\text{PRG}}$  uses  $y$  as  $\mathbf{r}$  in [RSD.SampleNoise](#) to proceed as in  $G_1$ .
4.  $\mathcal{A}_{\text{PRG}}$  receives  $(\text{msg}, \sigma)$  from  $\mathcal{A}$  and outputs [ReSolveD- \$\alpha\$ .Verify](#)( $\text{pk}, \text{msg}, \sigma$ )  $\stackrel{?}{=} \text{accept}$ .

$\mathcal{A}_{\text{PRG}}$  makes at most  $Q_{E''}$  queries to  $E''$ . If  $\mathcal{A}_{\text{PRG}}$  is given  $y = \text{PRG}''(\text{seed}_{\text{sk}})$  for some uniform  $\text{seed}_{\text{sk}}$  as in the real PRG game, the execution is  $G_1$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr_{\text{seed}_{\text{sk}}} \left[ \mathcal{A}_{\text{PRG}}^{E''}(\text{PRG}''(\text{seed}_{\text{sk}})) = 1 \mid \neg \text{BadOra} \right] = \Pr [G_1(Q_{\mathcal{O}}) = 1].$$

If  $\mathcal{A}_{\text{PRG}}$  is given some uniform  $y$ , the execution is  $G_2$  conditioned on  $\neg \text{BadOra}$  with statistical distance  $\frac{\lfloor \frac{\lambda}{\ell_{\text{bs}}} \rfloor + 1}{\lambda} - \frac{1}{\ell_{\text{bs}}}$  due to the modulo  $\ell_{\text{bs}}$  in [RSD.SampleNoise](#):

$$\Pr_y \left[ \mathcal{A}_{\text{PRG}}^{E''}(y) = 1 \mid \neg \text{BadOra} \right] = \Pr [G_2(Q_{\mathcal{O}}) = 1] + \left( \frac{\lfloor \frac{\lambda}{\ell_{\text{bs}}} \rfloor + 1}{\lambda} - \frac{1}{\ell_{\text{bs}}} \right).$$

From the emulation loss of [Lemma 8.12](#) for  $E$  and  $E'$  (each of which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have  $\Pr [\text{BadOra}] \leq 2 \cdot \varepsilon_{\text{SR}} (\frac{\lambda}{2})$ . Using the contradiction assumption and the above results, we have that  $\mathcal{A}_{\text{PRG}}$  breaks the PRG security conditioned on  $\neg \text{BadOra}$  so that [Eq. \(25\)](#) holds.

- $G_3$ . This game is identical to the previous one, except that it sends uniform  $\text{pk}$  to  $\mathcal{A}$ . We use a reduction to the RSD security against an adversary  $\mathcal{A}_{\text{RSD}}$  to prove

$$|\Pr [G_2(Q_{\mathcal{O}}) = 1] - \Pr [G_3(Q_{\mathcal{O}}) = 1]| \leq \text{AdvRSD}_{\mathcal{A}_{\text{RSD}}}(m, k, w) + 3 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \quad (26)$$

Assume that [Eq. \(26\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{RSD}}$  that internally runs  $\mathcal{A}$  to break the RSD security:

1. Given some RSD challenge  $(\mathbf{H}_B, \mathbf{y})$ ,  $\mathcal{A}_{\text{RSD}}$  samples uniform  $\text{seed}_{\text{pk}}$ , uses the compressed oracle technique to locally emulate [MatHash](#) for  $\mathcal{A}$  at the only position  $\text{seed}_{\text{pk}}$  so that

$$\mathbf{H} = [\mathbf{I} \parallel \mathbf{H}_B] = \text{RSD.SampleMatrix}(\text{seed}_{\text{pk}}),$$

and sends  $\text{pk} = (\text{seed}_{\text{pk}}, \mathbf{H} \cdot \mathbf{e})$  to  $\mathcal{A}$ .

2.  $\mathcal{A}_{\text{RSD}}$  locally emulates the oracles in  $\mathcal{O}'$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3.  $\mathcal{A}_{\text{RSD}}$  receives  $(\text{msg}, \sigma)$  from  $\mathcal{A}$  and outputs  $\text{ReSolveD-}\alpha.\text{Verify}(\text{pk}, \text{msg}, \sigma) \stackrel{?}{=} \text{accept}$ .

If  $\mathcal{A}_{\text{RSD}}$  is given  $\mathbf{y} = \mathbf{H} \cdot \mathbf{e}$ , the execution is  $\text{G}_2$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \text{GameRSD}_{\mathcal{A}_{\text{RSD}}}^0(m, k, w) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ \text{G}_2(Q_{\mathcal{O}}) = 1 \right].$$

Otherwise (i.e.,  $\mathbf{y}$  is uniform), the execution is  $\text{G}_3$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \text{GameRSD}_{\mathcal{A}_{\text{RSD}}}^1(m, k, w) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ \text{G}_3(Q_{\mathcal{O}}) = 1 \right].$$

From the emulation loss of [Lemma 8.12](#) for  $\text{E}$ ,  $\text{E}'$ , and  $\text{E}''$  (each of which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have  $\Pr [\text{BadOra}] \leq 3 \cdot \varepsilon_{\text{SR}}(\frac{\lambda}{2})$ . Using the contradiction assumption and the above results, we have that  $\mathcal{A}_{\text{RSD}}$  breaks the RSD security conditioned on  $\neg \text{BadOra}$ , leading to a contradiction and proving [Eq. \(26\)](#).

- $\text{G}_4$ . This game is identical to the previous one, except that it in addition returns 0 if  $\text{pk} = (\text{seed}_{\text{pk}}, \mathbf{y}) \in \mathcal{L}_{\mathcal{R}}$  for the language  $\mathcal{L}_{\mathcal{R}}$  of the relation

$$\mathcal{R} = \left\{ (\mathbf{y}, \mathbf{e}) : \mathbf{y} = \mathbf{H} \cdot \mathbf{e} \mid \begin{array}{l} \mathbf{H} = \text{RSD.SampleMatrix}(\text{seed}_{\text{pk}}), \\ \mathbf{e} \text{ is regular} \end{array} \right\}.$$

By definition, we have

$$\Pr [\text{G}_4(Q_{\mathcal{O}}) = 1] = \Pr [\text{G}_3(Q_{\mathcal{O}}) = 1 \wedge \text{pk} \notin \mathcal{L}_{\mathcal{R}}]. \quad (27)$$

As the RSD encoding  $G_{\mathbf{H}} : \mathbf{e} \mapsto \mathbf{H} \cdot \mathbf{e}$  is a PRG for any uniform, regular  $\mathbf{e}$  and any public, fixed  $\mathbf{H} = [\mathbf{I} \parallel \mathbf{H}_B]$  where  $\mathbf{H}_B$  is uniform, we construct a PRG adversary  $\mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, \mathbf{y}) =$

$\mathcal{A}_{\text{RSD}}(\mathbf{H}, \mathbf{y})$  from  $\mathcal{A}_{\text{RSD}}$ , which distinguishes  $G_2$  and  $G_3$ . Using this adversary, we have

$$\begin{aligned}
& \Pr [ G_3(Q_{\mathcal{O}}) = 1 ] \\
&= \Pr_{\mathbf{y} \leftarrow \text{codom}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, \mathbf{y}) = 1 \mid \neg \text{BadOra} ] \quad (\text{By the definitions of } G_3 \text{ and } \mathcal{A}_{\text{PRG}}) \\
&= \Pr_{\mathbf{y} \leftarrow \text{codom}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, \mathbf{y}) = 1 \wedge \text{pk} \in \mathcal{L}_{\mathcal{R}} \mid \neg \text{BadOra} ] \\
&\quad + \Pr_{\mathbf{y} \leftarrow \text{codom}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, \mathbf{y}) = 1 \wedge \text{pk} \notin \mathcal{L}_{\mathcal{R}} \mid \neg \text{BadOra} ] \\
&\quad (\text{By the law of total probability}) \\
&= \Pr_{\mathbf{y} \leftarrow \text{codom}(G_{\mathbf{H}})} [ \text{pk} \in \mathcal{L}_{\mathcal{R}} ] \cdot \Pr_{\mathbf{y} \leftarrow \text{codom}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, \mathbf{y}) = 1 \mid \neg \text{BadOra} \wedge \text{pk} \in \mathcal{L}_{\mathcal{R}} ] \\
&\quad (\text{By the independence between } \neg \text{BadOra} \text{ and } \text{pk} \in \mathcal{L}_{\mathcal{R}}) \\
&\quad + \Pr [ G_3(Q_{\mathcal{O}}) = 1 \wedge \text{pk} \notin \mathcal{L}_{\mathcal{R}} ] \quad (\text{By the definitions of } G_3 \text{ and } \mathcal{A}_{\text{PRG}}) \\
&= \frac{|\text{im}(G_{\mathbf{H}})|}{|\text{codom}(G_{\mathbf{H}})|} \cdot \Pr_{\mathbf{y} \leftarrow \text{codom}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, \mathbf{y}) = 1 \mid \neg \text{BadOra} \wedge \text{pk} \in \mathcal{L}_{\mathcal{R}} ] \\
&\quad + \Pr [ G_4(Q_{\mathcal{O}}) = 1 ] \quad (\text{By Eq. (27)}) \\
&= \frac{|\text{im}(G_{\mathbf{H}})|}{|\text{codom}(G_{\mathbf{H}})|} \cdot \Pr_{\mathbf{y} \leftarrow \text{im}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, \mathbf{y}) = 1 \mid \neg \text{BadOra} ] + \Pr [ G_4(Q_{\mathcal{O}}) = 1 ] \\
&\leq \frac{|\text{im}(G_{\mathbf{H}})|}{|\text{codom}(G_{\mathbf{H}})|} \cdot \frac{|\text{dom}(G_{\mathbf{H}})|}{|\text{im}(G_{\mathbf{H}})|} \cdot \Pr_{e \leftarrow \text{dom}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, G_{\mathbf{H}}(e)) = 1 \mid \neg \text{BadOra} ] \\
&\quad (\text{By Lemma 8.8 for such a PRG adversary that } \neg \text{BadOra} \text{ must hold}) \\
&\quad + \Pr [ G_4(Q_{\mathcal{O}}) = 1 ] \\
&= \frac{\ell_{\text{bs}}^w}{2^{m-k}} \cdot \Pr_{e \leftarrow \text{dom}(G_{\mathbf{H}})} [ \mathcal{A}_{\text{PRG}}(G_{\mathbf{H}}, G_{\mathbf{H}}(e)) = 1 \mid \neg \text{BadOra} ] + \Pr [ G_4(Q_{\mathcal{O}}) = 1 ] \\
&= \frac{\ell_{\text{bs}}^w}{2^{m-k}} \cdot \Pr [ G_2(Q_{\mathcal{O}}) = 1 ] + \Pr [ G_4(Q_{\mathcal{O}}) = 1 ]. \quad (\text{By the definitions of } G_2 \text{ and } \mathcal{A}_{\text{PRG}})
\end{aligned}$$

Combining the above inequality with Eq. (26), we have an upper bound

$$\begin{aligned}
& \Pr [ G_2(Q_{\mathcal{O}}) = 1 ] \\
&\leq \left( 1 - \frac{\ell_{\text{bs}}^w}{2^{m-k}} \right)^{-1} \cdot \left( \Pr [ G_4(Q_{\mathcal{O}}) = 1 ] + \text{AdvRSD}_{\mathcal{A}_{\text{RSD}}}(m, k, w) + 3 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right) \right). \quad (28)
\end{aligned}$$

- $G_5$ . This game is identical to the previous one, except that it receives  $(\text{msg}, \sigma)$  from  $\mathcal{A}$ , runs  $\text{ReSolveD-}\alpha.\text{Verify}(\text{pk}, \text{msg}, \sigma)$  but uses its sampled  $(\mathbf{H}, \mathbf{y})$  therein without using  $\text{pk}$ , uses the internal results in this verification to construct a signature  $\sigma'$  of the form (as per Eq. (18)) in  $\text{FS}[\text{R}\alpha\text{-IP}]$ , and outputs whatever the following modified verifier of  $\text{FS}[\text{R}\alpha\text{-IP}]$  outputs on  $\text{pk}' = (\mathbf{H}, \mathbf{y})$ ,  $\text{msg}$ , and  $\sigma'$ . This game works as follows:

– It parses the signature of  $\mathcal{A}$  as:

$$\sigma = (\text{iv}, (\mathbf{c}_{\alpha})_{\alpha \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \text{ctr}, \text{decom}_I, \text{chall}_3)$$

and uses these values in the constructed  $\sigma'$ .

- It uses  $(\text{com}_i^{\alpha})_{\alpha \in [0, \tau], i \in [0, N_{\alpha}]}$  reconstructed from  $\text{decom}_I$  (as per  $\text{BAVC.Verify}$ ) for the commitments  $(\text{com}_i^{\alpha})_{\alpha \in [0, \tau], i \in [0, N_{\alpha}]}$  in the constructed  $\sigma'$ .
- It uses the value  $\tilde{\mathbf{Q}}$  computed at the step 7 of  $\text{ReSolveD-}\alpha.\text{Verify}$  for the value  $\tilde{\mathbf{V}}$  in the constructed  $\sigma'$ .
- It uses the value  $\tilde{a}_0$  computed at the step 10 of  $\text{ReSolveD-}\alpha.\text{Verify}$  for the value  $\tilde{a}_0$  in the constructed  $\sigma'$ .

- The modified verifier is identical to the verifier of  $\text{FS}[\text{R}\alpha\text{-IP}]$ , except that it will not output **reject** for the two cases for the message  $\text{pmsg}_0$  (see [Item 4](#)).

By the above  $((\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha]}, \tilde{V}, \tilde{a}_0)$ , it is trivial that this modified verifier never outputs **reject** for  $\text{pmsg}_2$ ,  $\text{pmsg}_3$ , and  $\text{pmsg}_4$ . As a result, this game (i.e., the modified verifier with the given  $(\text{pk}', \text{msg}, \sigma')$ ) outputs 1 if and only if  $\text{ReSolveD-}\alpha.\text{Verify}(\text{pk}, \text{msg}, \sigma) \stackrel{?}{=} \text{accept}$  (i.e., the previous game outputs 1). We have

$$\Pr [G_4(Q_{\mathcal{O}}) = 1] = \Pr [G_5(Q_{\mathcal{O}}) = 1]. \quad (29)$$

We stress that, in the games subsequent to (including) this game, the modifier verifier need not call [MatHash](#) in [RSD.SampleMatrix](#) to spawn  $\mathbf{H}$ , which is locally sampled by the games.

- $G_6$ . This game is identical to the previous one, except that its modified verifier additionally outputs **reject** for the case (i) in the message  $\text{pmsg}_0$  (see [Item 4](#)). This modification makes this game return 0 for the case (i) in  $\text{pmsg}_0$ . We use a reduction to the almost injectivity of  $\text{PRG}'$  (in [LeafHash](#) to compute  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha]}$ ) against an adversary  $\mathcal{A}_{\text{Inj}}$  to prove

$$|\Pr [G_5(Q_{\mathcal{O}}) = 1] - \Pr [G_6(Q_{\mathcal{O}}) = 1]| \leq \text{AdvInj}_{\mathcal{A}_{\text{Inj}}}^{\text{E}, \text{PRG}'}(Q_{\text{E}} + L, L) + 2 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \quad (30)$$

Due to the definition of the case (i), we define an event **CollPre** that there exist  $\alpha \in [0, \tau)$  and  $i \in [0, N_\alpha)$  such that  $\text{com}_i^\alpha$  has at least two pre-images. We observe that  $G_5$  and  $G_6$  are in the same probability space, and the two games proceed identically conditioned on the event  $\neg \text{CollPre}$ . Using the well-known difference lemma [[Sho04](#)], we have

$$|\Pr [G_5(Q_{\mathcal{O}}) = 1] - \Pr [G_6(Q_{\mathcal{O}}) = 1]| \leq \Pr [\text{CollPre}].$$

Assume that [Eq. \(30\)](#) does not hold for contradiction and construct the following quantum adversary  $\mathcal{A}_{\text{Inj}}$  that internally runs  $\mathcal{A}$  to break the almost injectivity:

1. Given the quantum oracle access to  $\text{E}$ ,  $\mathcal{A}_{\text{Inj}}$  uses  $\text{E}$  for  $\mathcal{A}$ 's quantum queries to  $\text{E}$ .
2.  $\mathcal{A}_{\text{Inj}}$  locally emulates the oracles in  $\mathcal{O} \setminus \{\text{E}\}$  for  $\mathcal{A}$ . Let **BadOra** be the event that this emulation is not perfect.
3.  $\mathcal{A}_{\text{Inj}}$  follows  $G_6$  with the adversary  $\mathcal{A}$  to reconstruct  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha]}$  and sends

$$(\text{iv}, \{(0, \text{com}_i^\alpha)\}_{\alpha \in [0, \tau), i \in [0, N_\alpha]}),$$

where  $\text{iv}$  is included in  $\mathcal{A}$ 's signature  $\sigma$ , to the challenger of the almost injectivity game.

During this reconstruction,  $\mathcal{A}_{\text{Inj}}$  can make at most  $L$  more quantum queries to  $\text{E}$  so that there are at most  $Q_{\text{E}} + L$  quantum queries to  $\text{E}$ . Conditioned on  $\neg \text{BadOra}$ , such a game is the almost injective game so that  $\Pr [\text{CollPre} \mid \neg \text{BadOra}] = \text{AdvInj}_{\mathcal{A}_{\text{Inj}}}^{\text{E}, \text{PRG}'}(Q_{\text{E}} + L, L)$ . From the emulation loss of [Lemma 8.12](#) for  $\text{E}'$  and  $\text{E}''$  (each of which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have  $\Pr [\text{BadOra}] \leq 2 \cdot \varepsilon_{\text{SR}} (\frac{\lambda}{2})$ . Using the above results, we prove a contradiction so that [Eq. \(30\)](#) holds.

- $G_7$ . This game is identical to the previous one, except that its modified verifier additionally outputs **reject** for the case (ii) in the message  $\text{pmsg}_0$  (see [Item 4](#)). Note that this hybrid is perfectly indistinguishable from the previous one as the reconstructed  $(\text{com}_i^\alpha)_{\alpha \in [0, \tau), i \in [0, N_\alpha]}$  follow [BAVC.Verify](#) and have at most one  $\text{com}_{i_\alpha}^\alpha$  (which is given in the adversary's  $\text{decom}_I$ ) with no pre-image for each  $\alpha \in [0, \tau)$ . Therefore, the case (ii) never happens so that

$$\Pr [G_6(Q_{\mathcal{O}}) = 1] = \Pr [G_7(Q_{\mathcal{O}}) = 1]. \quad (31)$$

| Game $\text{GameRP}_{\mathcal{A}}^b(Q, R)$                                          | $\text{Orac}(y')$                                               |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 1 : $\mathcal{O}_0 \leftarrow \$ \mathcal{F}_{X,Y}, \mathcal{O}_1 := \mathcal{O}_0$ | 1 : Parse $y'$ as a distribution $p$ over $X \times \{0, 1\}^*$ |
| 2 : $y' \leftarrow \$ \mathcal{A}^{\mathcal{O}_b}()$                                | 2 : $(x, x') \leftarrow \$ p$                                   |
| 3 : <b>for</b> $i \in [R]$ :                                                        | 3 : $y \leftarrow \$ Y$                                         |
| 4 : $(z, x, y) \leftarrow \$ \text{Orac}(y')$                                       | 4 : <b>return</b> $(x, x'), x, y$                               |
| 5 : $\mathcal{O}_1 \leftarrow \$ \mathcal{O}_1[x \mapsto y]$                        |                                                                 |
| 6 : $y' \leftarrow \$ \mathcal{A}^{\mathcal{O}_b}(z)$                               |                                                                 |
| 7 : <b>return</b> $b' := (y' = 1)$                                                  |                                                                 |

Figure 8: The adaptive reprogramming game in QROM.

- **G<sub>8</sub>**. This game is identical to the previous one, except that it locally emulates the random oracle **CoinHash** (i.e., returning uniform  $(r, \text{iv})$ ) using the compressed oracle technique for  $\mathcal{A}$ . Since the modified verifier does not invoke **CoinHash**, this game is perfectly indistinguishable from the previous one. Notice that the verifier run by this game is identical to the  $\text{FS}[\text{R}\alpha\text{-IP}]$  verifier, which is explicitly given  $\mathbf{H}$  and does not call **MatHash** in **RSD.SampleMatrix**. This game is the soundness game of  $\text{FS}[\text{R}\alpha\text{-IP}]$  given the oracles in  $\mathcal{O}'$ . Therefore, we have

$$\Pr [\text{G}_7(Q_{\mathcal{O}}) = 1] = \Pr [\text{G}_8(Q_{\mathcal{O}}) = 1] = \text{AdvSnd}_{\mathcal{A}}^{\mathcal{O}', \text{FS}[\text{R}\alpha\text{-IP}]}(Q_{\mathcal{O}'}). \quad (32)$$

Based on Eq. (23), Eq. (24), Eq. (25), Eq. (28), Eq. (29), Eq. (30), Eq. (31), and Eq. (32), we can prove this theorem.  $\square$

**Quantum-list unpredictability of **VOLE.Commit**.** This quantum-list unpredictability is to ensure that the quantum adversary, given the access to QROMs, cannot predict the honestly generated commitment, based on which the Fiat-Shamir challenge  $\text{chall}_1$  will be computed. It will be used to prove the EUF-CMA security.

**Lemma 8.9** (Adaptive Reprogramming in QROM [LGX25]). *Let  $X$  and  $Y$  be two finite sets. Let  $\mathcal{A}$  be any stateful quantum adversary making at most  $Q$  quantum queries to  $\mathcal{O}_b$  as per the adaptive reprogramming game  $\text{GameRP}_{\mathcal{A}}^b(Q, R)$  in Fig. 8 for  $b \in \{0, 1\}$ . Let  $p_X^i$  be the  $i$ -th marginal distribution over  $X$  parsed by  $\text{Orac}$ ,  $p_{X, \max}^i = \mathbb{E}(\max_x p_X^i(x))$  be the expectation taken over  $\mathcal{A}$ 's behavior until the  $i$ -th query to  $\text{Orac}$ , and  $p_{X, \max} \geq p_{X, \max}^i$  for any  $i \in [R]$ . We have*

$$|\Pr [\text{GameRP}_{\mathcal{A}}^0(Q, R) = 1] - \Pr [\text{GameRP}_{\mathcal{A}}^1(Q, R) = 1]| \leq 2\sqrt{2} \cdot \sqrt{R \cdot Q \cdot p_{X, \max}}.$$

**Definition 8.3** (Quantum-List Unpredictability [BBB<sup>+</sup>25]). *Let  $\mathcal{O}$  be a list of ideal-model oracles,  $Q_{\mathcal{O}}$  be a list of the respective numbers of the ideal-model queries, and  $\mathcal{B}^{\mathcal{O}}$  be an algorithm using these oracles.  $\mathcal{B}^{\mathcal{O}}$  is  $(Q_{\mathcal{O}}, Q, \varepsilon)$ -quantum-list-unpredictable if for any stateful quantum adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  queries respectively to the oracles in  $\mathcal{O}$ ,*

$$\text{AdvUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}}(Q_{\mathcal{O}}, Q) = \left| \Pr [\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 0}(Q_{\mathcal{O}}, Q) = 1] - \Pr [\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 1}(Q_{\mathcal{O}}, Q) = 1] \right| \leq \varepsilon,$$

where the quantum-list unpredictability game  $\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, b}$  for  $b \in \{0, 1\}$  is defined in Fig. 9.

**Lemma 8.10** (Quantum-List Unpredictability of **VOLE.Commit**). *Consider the following ideal-model oracles and their respective numbers of queries:*

- $\mathbf{E}$  is an ideal cipher realizing **PRG** in **LeafHash**, with  $Q_{\mathbf{E}}$  queries,

| Game $\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, b}(Q_{\mathcal{O}}, Q)$ |                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1:                                                                                   | $ x\rangle \leftarrow \mathcal{A}^{\mathcal{O}}()$ , where $ x\rangle$ is a quantum register holding a superposition of a $Q$ -element list $x = (x_i)_{i \in [Q]}$ , which guesses the output of $\mathcal{B}$ |
| 2:                                                                                   | $c \leftarrow \mathcal{B}^{\mathcal{O}}()$                                                                                                                                                                      |
| 3:                                                                                   | $b' := 1$                                                                                                                                                                                                       |
| 4:                                                                                   | <b>if</b> $b = 1$ :                                                                                                                                                                                             |
| 5:                                                                                   | Apply a binary measurement to $ x\rangle$ to test the membership $b'' := (c \in x)$ in superposition                                                                                                            |
| 6:                                                                                   | <b>if</b> $b'' = 1$ : $b' := 0$                                                                                                                                                                                 |
| 7:                                                                                   | <b>if</b> $b' = 1$ : $b' \leftarrow \mathcal{A}^{\mathcal{O}}( x\rangle, c)$                                                                                                                                    |
| 8:                                                                                   | <b>return</b> $b'$                                                                                                                                                                                              |

Figure 9: The quantum-list unpredictability game.

- $E'$  is an ideal cipher realizing **TCCR** in **BAVC.Commit**, with  $Q_{E'}$  queries,
- **BAVCHash** is a random oracle in **BAVC.Commit**, with  $Q_{\text{BAVCHash}}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. Let  $\chi$  be a uniform distribution over  $\{0, 1\}^\lambda$ . For any quantum adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  quantum queries respectively to the oracles in  $\mathcal{O}$  and having  $Q$  guesses in  $|x\rangle$ ,

$$\begin{aligned} & \text{AdvUP}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}}(Q_{\mathcal{O}}, Q) \\ & \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{E', \text{CCR}} \left( Q_{E'} + \frac{3L}{2} + 2\lceil \log L \rceil + 5, 2\lceil \log L \rceil + 5, \chi \right) + \frac{(2\lceil \log L \rceil + 5)^2}{2^{\lambda-1}} \\ & \quad + \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{E, \text{PRG}'}(Q_E) + 2\sqrt{2} \cdot \frac{\sqrt{Q_{\text{BAVCHash}} + 1}}{2^\lambda} + \frac{2\sqrt{Q}}{2^\lambda} + \frac{Q}{2^{2\lambda}} + 4 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right), \end{aligned}$$

where  $\varepsilon_{\text{SR}}(\frac{\lambda}{2})$  is from [Lemma 8.12](#),

- **PRG'** :  $x \in \{0, 1\}^\lambda \mapsto \text{PRG}(x, \text{iv}, 0^8; 2\lambda) \in \{0, 1\}^{2\lambda}$  is an  $E$ -based PRG for some fixed  $\text{iv}$  ([Theorem 8.8](#)), and
- **CCR** :  $x \in \{0, 1\}^\lambda \mapsto \text{TCCR}(x, s, \text{iv}) \in \{0, 1\}^\lambda$  is an  $E'$ -based CCR hash function for some fixed  $s$  and (uniform)  $\text{iv}$  ([Theorem 8.13](#)).

*Proof.* We introduce the following hybrid argument.

- $G_0$ . This game is the real quantum-list unpredictability game.

$$\Pr [ G_0(Q_{\mathcal{O}}, Q) = 1 ] = \Pr [ \text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 0}(Q_{\mathcal{O}}, Q) = 1 ]. \quad (33)$$

- $G_1$ . This game is identical to the previous one, except that in **VOLE.Commit**, it replaces the leftmost leaf seed  $\text{seed}_0^0$  (which lies in the leftmost punctured path  $0^h$  in a possibly not full pseudorandom correlated GGM tree [[GYW<sup>+</sup>23](#)] of the height  $h = \lceil \log L \rceil$ ) and the at most  $h$  off-path nodes adjacent to the path  $0^h$  by random values in  $\{0, 1\}^\lambda$ . We use a reduction to the CCR for NNQs (see [Lemma 3.1](#)) against an adversary  $\mathcal{A}_{\text{NNQ}}$  to prove

$$\begin{aligned} & |\Pr [ G_0(Q_{\mathcal{O}}, Q) = 1 ] - \Pr [ G_1(Q_{\mathcal{O}}, Q) = 1 ]| \\ & \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{E', \text{CCR}} \left( Q_{E'} + \frac{3L}{2} + 2\lceil \log L \rceil + 5, 2\lceil \log L \rceil + 5, \chi \right) + \frac{(2\lceil \log L \rceil + 5)^2}{2^{\lambda-1}} + \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \end{aligned} \quad (34)$$

Assume that [Eq. \(34\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{NNQ}}$  that internally runs  $\mathcal{A}$  to break the CCR for NNQs:

1. Given the quantum oracle access to  $E'$ ,  $\mathcal{A}_{\text{NNQ}}$  uses  $E'$  for  $\mathcal{A}$ 's quantum queries to  $E'$ .
2.  $\mathcal{A}_{\text{NNQ}}$  locally emulates the oracles in  $\{E, \text{BAVCHash}\}$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3. Given the access to the NNQ oracle  $\mathcal{O}_{\text{NNQ}}$ ,  $\mathcal{A}_{\text{NNQ}}$  proceeds as follows:
  - $\mathcal{A}_{\text{NNQ}}$  queries  $\mathcal{O}_{\text{NNQ}}$  with  $q_0 : \text{sum}_0 := 0^\lambda$ ,  $q'_0 : l := \text{const}_0$ , and  $q''_0 : r := \text{const}_1$ .
  - For each  $i \in [h-1]$ ,  $\mathcal{A}_{\text{NNQ}}$  queries  $\mathcal{O}_{\text{NNQ}}$  with  $q_{i+1} : \text{sum}_{i+1} := \mathcal{Q}(\text{sum}_i, 1)$  and  $q'_{i+1} ::= \text{off}_{i+1} := \text{sum}_{i+1} \oplus \text{sum}_i$ .
  - $\mathcal{A}_{\text{NNQ}}$  queries  $\mathcal{O}_{\text{NNQ}}$  with  $q_h : x := \text{sum}_{h-1} \oplus l$  and  $q'_h : x' := \text{sum}_{h-1} \oplus r$ .
  - $\mathcal{A}_{\text{NNQ}}$  queries  $\mathcal{O}_{\text{NNQ}}$  with  $q_{h+1} : \text{seed}_0^0 := \mathcal{Q}(x, 0)$  and  $q'_{h+1} : \text{off}_h := \mathcal{Q}(x', 0)$ .
4.  $\mathcal{A}_{\text{NNQ}}$  uses the returned  $\text{seed}_0^0$  and  $\{\text{off}_i\}_{i \in [1, h]}$  to reconstruct all  $(\text{seed}_i^\alpha)_{i \in [0, N_\alpha]}$  as per the tree expansion in [VOLE.Commit](#) and honestly follows the rest of computation to obtain  $c$  as the output of [VOLE.Commit](#).
5.  $\mathcal{A}_{\text{NNQ}}$  outputs whatever the quantum-list unpredictability game outputs.

$\mathcal{A}_{\text{NNQ}}$  makes  $2h + 5$  NNQs to its given NNQ oracle and, in the step 4 as above, at most  $\frac{3L}{2}$  more queries to  $E'$  to run [CCR](#) to obtain the leaf seeds  $(\text{seed}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]} \setminus \{\text{seed}_0^0\}$  rooted with the off-path nodes  $\{\text{off}_i\}_{i \in [1, h]}$  adjacent to the path  $0^h$ .

If  $\mathcal{O}_{\text{NNQ}} = \text{Real}_{\text{CCR}, \Delta}^{E'}$ , the distribution of  $c$  (for  $\mathbf{r} = \Delta \leftarrow \chi$ ) is that in the previous game conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \mathcal{A}_{\text{NNQ}}^{E', \text{Real}_{\text{CCR}, \Delta}^{E'}}() = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ G_0(Q_{\mathcal{O}}, Q) = 1 \right].$$

If  $\mathcal{O}_{\text{NNQ}} = \text{Ideal}_{\text{CCR}}^{E'}$ , the leftmost leaf seed and the at most  $h$  off-path nodes are as uniform as in this game, leading to the distribution of  $c$  in this game conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \mathcal{A}_{\text{NNQ}}^{E', \text{Ideal}_{\text{CCR}}^{E'}}() = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ G_1(Q_{\mathcal{O}}, Q) = 1 \right].$$

From the emulation loss of [Lemma 8.12](#) for  $E$  (which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of [BAVCHash](#) from the compressed oracle technique, we have  $\Pr[\text{BadOra}] \leq \varepsilon_{\text{SR}}(\frac{\lambda}{2})$ . Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 3.1](#) for the specified  $\mathcal{A}_{\text{NNQ}}$ , showing that [Eq. \(34\)](#) holds.

- $G_2$ . This game is identical to the previous one, except that in [VOLE.Commit](#), it replaces the leftmost commitment  $\text{com}_0^0$  by a random value in  $\{0, 1\}^{2\lambda}$ . We use a reduction to the PRG security of [PRG'](#) (called in [LeafHash](#)) against an adversary  $\mathcal{A}_{\text{PRG}}$  to prove that

$$|\Pr[G_1(Q_{\mathcal{O}}, Q) = 1] - \Pr[G_2(Q_{\mathcal{O}}, Q) = 1]| \leq \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{E, \text{PRG}'}(Q_E) + \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right). \quad (35)$$

Assume that [Eq. \(35\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{PRG}}$  that invokes  $\mathcal{A}$  to break the PRG security:

1. Given the quantum oracle access to  $E$ ,  $\mathcal{A}_{\text{PRG}}$  uses  $E$  for  $\mathcal{A}$ 's quantum queries to  $E$ .
2.  $\mathcal{A}_{\text{PRG}}$  locally emulates the oracles in  $\{E', \text{BAVCHash}\}$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3. Given some PRG challenge  $y$ ,  $\mathcal{A}_{\text{PRG}}$  uses  $y$  as  $\text{com}_0^0$  to proceed as in  $G_1$ .
4.  $\mathcal{A}_{\text{PRG}}$  outputs whatever the quantum-list unpredictability game outputs.

$\mathcal{A}_{\text{PRG}}$  makes at most  $Q_E$  queries to  $E$ . If  $\mathcal{A}_{\text{PRG}}$  is given  $y = \text{PRG}'(\text{seed}_0^0)$  for some uniform  $\text{seed}_0^0$  as in the real PRG game, the execution is  $G_1$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr_{\text{seed}_0^0} \left[ \mathcal{A}_{\text{PRG}}^E(\text{PRG}'(\text{seed}_0^0)) = 1 \mid \neg \text{BadOra} \right] = \Pr [ G_1(Q_{\mathcal{O}}, Q) = 1 ].$$

If  $\mathcal{A}_{\text{PRG}}$  is given some uniform  $y$ , the execution is  $G_2$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr_y \left[ \mathcal{A}_{\text{PRG}}^E(y) = 1 \mid \neg \text{BadOra} \right] = \Pr [ G_2(Q_{\mathcal{O}}, Q) = 1 ].$$

From the emulation loss of [Lemma 8.12](#) for  $E'$  (which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of [BAVCHash](#) from the compressed oracle technique, we have  $\Pr [ \text{BadOra} ] \leq \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right)$ . Using the contradiction assumption and the above results, we have that  $\mathcal{A}_{\text{PRG}}$  breaks the PRG security conditioned on  $\neg \text{BadOra}$  so that [Eq. \(35\)](#) holds.

- $G_3$ . This game is identical to the previous one, except that in [VOLE.Commit](#), it replaces the output commitment  $\text{com}$  by a random value in  $\{0, 1\}^{2\lambda}$ . We use a reduction to the adaptive reprogramming game ([Fig. 8](#)) against an adversary  $\mathcal{A}_{\text{RP}}$  to prove

$$|\Pr [ G_2(Q_{\mathcal{O}}, Q) = 1 ] - \Pr [ G_3(Q_{\mathcal{O}}, Q) = 1 ]| \leq 2\sqrt{2} \cdot \frac{\sqrt{Q_{\text{BAVCHash}} + 1}}{2^\lambda} + 2 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \quad (36)$$

Assume that [Eq. \(36\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{RP}}$  that internally runs  $\mathcal{A}$  as a subroutine (where  $\mathcal{O}_0 = \text{BAVCHash}$  in the game):

1. Given the quantum oracle access to  $\mathcal{O}_b$ ,  $\mathcal{A}_{\text{RP}}$  uses  $\mathcal{O}_b$  for  $\mathcal{A}$ 's quantum queries to [BAVCHash](#).
2.  $\mathcal{A}_{\text{RP}}$  locally emulates the oracles in  $\{E, E'\}$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3. When it is time to compute the output  $c$  of [VOLE.Commit](#),  $\mathcal{A}_{\text{RP}}$  defines a distribution  $y'$  over  $X = \{0, 1\}^{2\lambda L} \times \{0, 1\}^{120}$  such that the first  $2\lambda$  bits (i.e.,  $\text{com}_0^0$ ) is uniform while the other part is fixed to  $((\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]} \setminus \{\text{com}_0^0\}, \text{iv})$ , receives

$$x = ((\text{com}_i^\alpha)_{\alpha \in [0, \tau], i \in [0, N_\alpha]}, \text{iv}),$$

and queries  $\mathcal{O}_b$  with  $x$  to obtain  $\text{com}$  as a part of  $c$ .

4.  $\mathcal{A}_{\text{RP}}$  outputs whatever the quantum-list unpredictability game outputs.

$\mathcal{A}_{\text{RP}}$  makes at most  $Q_{\text{BAVCHash}} + 1$  adaptive quantum queries to  $\mathcal{O}_b$ . If  $b = 0$ ,  $\mathcal{O}_0 = \text{BAVCHash}$  implies that the view of  $\mathcal{A}$  is identically distributed as in  $G_2$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \text{GameRP}_{\mathcal{A}_{\text{RP}}}^0(Q_{\text{BAVCHash}} + 1, 1) = 1 \mid \neg \text{BadOra} \right] = \Pr [ G_2(Q_{\mathcal{O}}, Q) = 1 ].$$

If  $b = 1$ ,  $\mathcal{O}_1$  is identically distributed as in  $G_3$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \text{GameRP}_{\mathcal{A}_{\text{RP}}}^1(Q_{\text{BAVCHash}} + 1, 1) = 1 \mid \neg \text{BadOra} \right] = \Pr [ G_3(Q_{\mathcal{O}}, Q) = 1 ].$$

From the emulation loss of [Lemma 8.12](#) for  $E$  and  $E'$  (each of which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation), we have  $\Pr [ \text{BadOra} ] \leq 2 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right)$ . Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 8.9](#) for  $Q = Q_{\text{BAVCHash}} + 1$ ,  $R = 1$ , and  $p_{X, \max} \leq \frac{1}{2^{2\lambda}}$ , showing that [Eq. \(36\)](#) holds.

- $G_4$ . This game is identical to the previous one, except that it applies the binary measurement at the step 5 to test the membership  $b'' := (\text{VOLE.Commit}(r, s, iv) \in x)$  in superposition but does not perform the step 6. Let  $\varepsilon$  be the probability that this measurement returns  $b'' = 1$  (which will be analyzed in the end of this proof). From the gentle measurement lemma, the original state and post-measurement state of the quantum register  $|x\rangle$  have a disturbance at most  $2\sqrt{\varepsilon}$ , which upper bounds the advantage of  $\mathcal{A}$ . As a result, we have

$$|\Pr[G_3(Q_{\mathcal{O}}, Q) = 1] - \Pr[G_4(Q_{\mathcal{O}}, Q) = 1]| \leq 2\sqrt{\varepsilon}. \quad (37)$$

- $G_5$ . This game is identical to the previous one, except that it further performs the step 6. It is the ideal quantum-list unpredictability game. Moreover, this game proceeds in the same probability space as the previous game so that  $\varepsilon$  is equivalently defined and quantitatively identical. Since this game and the previous game are identical unless the binary measurement returns  $b'' = 1$ , the difference lemma [Sho04] gives

$$\begin{aligned} & |\Pr[G_4(Q_{\mathcal{O}}, Q) = 1] - \Pr[G_5(Q_{\mathcal{O}}, Q) = 1]| \\ &= \left| \Pr[G_4(Q_{\mathcal{O}}, Q) = 1] - \Pr[\text{GameUP}_{\mathcal{A}}^{\mathcal{O}, \mathcal{B}, 1}(Q_{\mathcal{O}}, Q) = 1] \right| \leq \varepsilon. \end{aligned} \quad (38)$$

Finally, we consider how to bound  $\varepsilon$  in the probability space underlying  $G_5$ . To maximize  $\varepsilon$ , we without loss of generality assume that each of the  $Q$  guesses in  $x$  is stored in its own quantum sub-register so that  $|x\rangle = \otimes_{i \in [Q]} |x_i\rangle$  (where each  $|x_i\rangle$  is a computational basis state). For any fixed output  $c$  of  $\text{VOLE.Commit}$ , the projector  $P_c^{\text{No}}$  meaning  $c$  not in any sub-register is

$$P_c^{\text{No}} = \bigotimes_{i \in [Q]} (I - |c\rangle\langle c|)_i$$

and the projector  $P_c^{\text{Yes}} = I^{\otimes Q} - P_c^{\text{No}}$  corresponds to  $c$  in some sub-register. Applying the binary measurement to  $|x\rangle$ , we have for any fixed  $c$ ,

$$\begin{aligned} \Pr[b'' = 1 \mid c] &= \langle x | P_c^{\text{Yes}} | x \rangle \\ &= 1 - \langle x | P_c^{\text{No}} | x \rangle \\ &= 1 - \prod_{i \in [Q]} \langle x_i | (I - |c\rangle\langle c|) | x_i \rangle \\ &= 1 - \prod_{i \in [Q]} (1 - |\langle x_i | c \rangle|^2) \\ &= 1 - \prod_{i \in [Q]} (1 - \delta_{x_i, c}) \quad (\text{Since } |x_i\rangle \text{ and } |c\rangle \text{ are computational basis states}) \\ &= \mathbb{1}(c \in x) \end{aligned}$$

where the Kronecker delta  $\delta_{x_i, c} = 1$  if  $x_i = c$  and  $\delta_{x_i, c} = 0$  otherwise. Therefore,

$$\begin{aligned} \varepsilon = \Pr[b'' = 1] &= \sum_c \Pr[b'' = 1 \mid c] \cdot \Pr[c] \\ &\leq \sum_c \Pr[b'' = 1 \mid c] \cdot \frac{1}{2^{2\lambda}} \quad (\text{Since the uniform } \text{com} \in \{0, 1\}^{2\lambda} \text{ is a part of } c) \\ &= \frac{1}{2^{2\lambda}} \cdot \sum_c \mathbb{1}(c \in x) \\ &\leq \frac{Q}{2^{2\lambda}}. \quad (\text{Since the sum counts the distinct elements in } x) \end{aligned}$$

Using this bound with Eq. (33), Eq. (34), Eq. (35), Eq. (36), Eq. (37), and Eq. (38), we can prove the target bound.  $\square$

| Game $\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, b}(Q_{\mathcal{O}}, Q)$ |                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 :                                                                                         | <b>for</b> $i \in [Q]$ :                                                                                                                                                                |
| 2 :                                                                                         | $(\mu^i, \mathbf{s}^i) \leftarrow \{0, 1\}^{2\lambda} \times \{0, 1\}^{\lambda}$                                                                                                        |
| 3 :                                                                                         | $(r^i, \text{iv}^i) \leftarrow \{0, 1\}^{\lambda} \times \{0, 1\}^{120}$                                                                                                                |
| 4 :                                                                                         | $(\text{com}^i, \text{decom}^i, (\mathbf{c}_{\alpha}^i)_{\alpha \in [1, \tau]}, \mathbf{u}^i, \mathbf{V}^i) \leftarrow \text{VOLE.Commit}(r^i, \mu^i, \mathbf{s}^i, \text{iv}^i)$       |
| 5 :                                                                                         | $\text{chall}_3^i \leftarrow \{0, 1\}^{\lambda - w_{\text{grind}}} \  0_{\text{grind}}^w$ such that $I^i := \text{VOLE.ChalDecAll}(\text{chall}_3^i)$ does not abort $\text{BAVC.Open}$ |
| 6 :                                                                                         | $\text{decom}_{I^i}^i := \text{BAVC.Open}(\text{decom}^i, I^i)$                                                                                                                         |
| 7 :                                                                                         | <b>if</b> $b = 1$ :                                                                                                                                                                     |
| 8 :                                                                                         | $((\mathbf{c}_{\alpha}^i)_{\alpha \in [1, \tau]}, \mathbf{u}^i) \leftarrow (\mathbb{F}_2^{\ell_{\text{vole}}})^{\tau}$                                                                  |
| 9 :                                                                                         | $\mathcal{A}^{\mathcal{O}}(\text{com}^i, (\mathbf{c}_{\alpha}^i)_{\alpha \in [1, \tau]}, \mathbf{u}^i, \text{chall}_3^i, \text{decom}_{I^i}^i)$                                         |
| 10 :                                                                                        | $b' \leftarrow \mathcal{A}^{\mathcal{O}}()$                                                                                                                                             |
| 11 :                                                                                        | <b>return</b> $b'$                                                                                                                                                                      |

Figure 10: The multi-hiding game of  $\text{VOLE.Commit}$ .

**EUF-KO security of ReSolveD- $\alpha \Rightarrow$  EUF-CMA security of ReSolveD- $\alpha$ .** We finish the picture by proving the multi-hiding property of the commitment scheme and putting all things established before together to prove the EUF-CMA security.

**Lemma 8.11** (Multi-Hiding of  $\text{VOLE.Commit}$ ). *Consider the following ideal-model oracles and their respective numbers of queries:*

- $\mathbf{E}$  is an ideal cipher realizing  $\text{PRG}$  in  $\text{LeafHash}$  and  $\text{VOLE.ConvertToVOLE}$ , with  $Q_{\mathbf{E}}$  queries,
- $\mathbf{E}'$  is an ideal cipher realizing  $\text{TCCR}$  in  $\text{BAVC.Commit}$ , with  $Q_{\mathbf{E}'}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. Let  $\chi$  be a uniform distribution over  $\{0, 1\}^{\lambda}$ . For any stateful quantum adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  quantum queries respectively to the oracles in  $\mathcal{O}$ ,

$$\begin{aligned}
& \text{AdvMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}}(Q_{\mathcal{O}}, Q) \\
&= \left| \Pr \left[ \text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 0}(Q_{\mathcal{O}}, Q) = 1 \right] - \Pr \left[ \text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 1}(Q_{\mathcal{O}}, Q) = 1 \right] \right| \\
&\leq Q \cdot \left( 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathbf{E}', \text{CCR}} \left( Q_{\mathbf{E}'} + \frac{3L}{2} + (2\lceil \log L \rceil + 5) \cdot \tau, (2\lceil \log L \rceil + 5) \cdot \tau, \chi \right) \right. \\
&\quad \left. + \frac{(2\lceil \log L \rceil + 5)^2 \cdot \tau^2}{2^{\lambda-1}} + \tau \cdot \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\mathbf{E}, \text{PRG}'}(Q_{\mathbf{E}}) + 2 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right) \right),
\end{aligned}$$

where the multi-hiding game  $\text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, b}(Q_{\mathcal{O}}, Q)$  for  $b \in \{0, 1\}$  is defined in Fig. 10,  $\varepsilon_{\text{SR}}(\frac{\lambda}{2})$  is from Lemma 8.12,

- $\text{PRG}' : \mathbf{x} \in \{0, 1\}^{\lambda} \mapsto \text{PRG}(\mathbf{x}, \text{iv}, 0^8; 2\lambda) \| \text{PRG}(\mathbf{x}, \text{iv}, t; \ell_{\text{vole}}) \in \{0, 1\}^{2\lambda} \times \mathbb{F}_2^{\ell_{\text{vole}}}$  is an  $\mathbf{E}$ -based PRG for some fixed  $\text{iv}$  and  $t$  (Theorem 8.8), and
- $\text{CCR} : \mathbf{x} \in \{0, 1\}^{\lambda} \mapsto \text{TCCR}(\mathbf{x}, \mathbf{s}, \text{iv}) \in \{0, 1\}^{\lambda}$  is an  $\mathbf{E}'$ -based CCR hash function for some fixed  $\mathbf{s}$  and (uniform)  $\text{iv}$  (Theorem 8.13).

*Proof.* For clarity, we introduce the following hybrid argument for  $Q = 1$  and omit the superscript  $i$  in these hybrids. We will see how to generalize it for any  $Q \geq 1$  in the end.

- $G_0$ . This is the real multi-hiding game.

$$\Pr [ G_0(Q_{\mathcal{O}}, Q) = 1 ] = \Pr \left[ \text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 0}(Q_{\mathcal{O}}, Q) = 1 \right]. \quad (39)$$

- $G_1$ . This game is identical to the previous one, except that for each  $\alpha \in [0, \tau)$ , it replaces the punctured leaf seed  $\text{seed}_{\Delta_\alpha}^\alpha$  by a random value in  $\{0, 1\}^\lambda$ . Similar to the game hop from  $G_0$  to  $G_1$  in the proof of Lemma 8.10, we use a reduction to the CCR for NNQs against an adversary  $\mathcal{A}_{\text{NNQ}}$ , which makes at most  $(2\lceil \log L \rceil + 5) \cdot \tau$  NNQs to its given NNQ oracle and at most  $\frac{3L}{2}$  more queries to  $E'$  to emulate the output of  $\text{VOLE.Commit}$  for its internally used  $\mathcal{A}$  against the multi-hiding game. As a result,

$$\begin{aligned} & |\Pr [ G_0(Q_{\mathcal{O}}, Q) = 1 ] - \Pr [ G_1(Q_{\mathcal{O}}, Q) = 1 ]| \\ & \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{E', \text{CCR}} \left( Q_{E'} + \frac{3L}{2} + (2\lceil \log L \rceil + 5) \cdot \tau, (2\lceil \log L \rceil + 5) \cdot \tau, \chi \right) \\ & \quad + \frac{(2\lceil \log L \rceil + 5)^2 \cdot \tau^2}{2^{\lambda-1}} + \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \end{aligned} \quad (40)$$

- $G_2$ . This game is identical to the previous one, except that for each  $\alpha \in [0, \tau)$ , it replaces the punctured commitment  $\text{com}_{\Delta_\alpha}^\alpha$  along with the VOLE payload  $\mathbf{u}_\alpha$  by a random value in  $\{0, 1\}^{2\lambda} \times \mathbb{F}_2^{\ell_{\text{vole}}}$ . As the pre-image  $\text{seed}_{\Delta_\alpha}^\alpha$  of  $\text{com}_{\Delta_\alpha}^\alpha$  (modulo  $\text{PRG}$  in  $\text{LeafHash}$ ) and the  $i = \Delta_\alpha$  additive term (modulo  $\text{PRG}$  in  $\text{VOLE.ConvertToVOLE}$ ) in  $\mathbf{u}_\alpha = \sum_{i \in [N_\alpha]} \text{PRG}(\text{seed}_i^\alpha, \text{iv}, i)$ , which follows from  $\text{VOLE.ConvertToVOLE}$ , is hidden from the adversary and independent of other pre-images, a reduction to the PRG security of  $\text{PRG}'$  shows that

$$\begin{aligned} & |\Pr [ G_1(Q_{\mathcal{O}}, Q) = 1 ] - \Pr [ G_2(Q_{\mathcal{O}}, Q) = 1 ]| \\ & = \left| \Pr [ G_1(Q_{\mathcal{O}}, Q) = 1 ] - \Pr \left[ \text{GameMH}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE.Commit}, 1}(Q_{\mathcal{O}}, Q) = 1 \right] \right| \\ & \leq \tau \cdot \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{E, \text{PRG}'}(Q_E) + \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right), \end{aligned} \quad (41)$$

where the first equality comes from that  $\mathbf{c}_\alpha = \mathbf{u}_0 \oplus \mathbf{u}_\alpha$  for each  $\alpha \in [1, \tau)$  is uniform in this game so that  $((\mathbf{c}_\alpha^i)_{\alpha \in [1, \tau)}, \mathbf{u}^i)$  is as uniform as in the ideal multi-hiding game when  $Q = 1$ .

To generalize the above hybrid argument for any  $Q \geq 1$ , we can use it for each  $i \in [Q]$  as each execution has independent randomness, arriving at the target bound by a factor of  $Q$ .  $\square$

**Theorem 8.5** (EUF-CMA Security of ReSolveD- $\alpha$  against Quantum Adversary). *In ReSolveD- $\alpha$ , consider the following ideal-model oracles and their respective numbers of queries:*

- $E$  is an ideal cipher realizing  $\text{PRG}$  in  $\text{LeafHash}$  and  $\text{VOLE.ConvertToVOLE}$ , with  $Q_E$  queries,
- $E'$  is an ideal cipher realizing  $\text{TCCR}$  in  $\text{BAVC.Commit}$ , with  $Q_{E'}$  queries,
- $E''$  is an ideal cipher realizing  $\text{PRG}$  in  $\text{RSD.SampleNoise}$ , with  $Q_{E''}$  queries,
- $\text{MatHash}$  is a random oracle in  $\text{RSD.SampleMatrix}$ , with  $Q_{\text{MatHash}}$  queries,
- $\text{MsgHash}$  is a random oracle in  $\text{ReSolveD-}\alpha.\text{Sign}$ , with  $Q_{\text{MsgHash}}$  queries,
- $\text{CoinHash}$  is a random oracle in  $\text{ReSolveD-}\alpha.\text{Sign}$ , with  $Q_{\text{CoinHash}}$  queries,
- $\text{BAVCHash}$  is a random oracle in  $\text{BAVC.Commit}$ , with  $Q_{\text{BAVCHash}}$  queries,
- $\text{FSHash}_1$  is a random oracle in  $\text{ReSolveD-}\alpha.\text{Sign}$ , with  $Q_{\text{FSHash}_1}$  queries,
- $\text{FSHash}_2$  is a random oracle in  $\text{ReSolveD-}\alpha.\text{Sign}$ , with  $Q_{\text{FSHash}_2}$  queries,

- **FSHash<sub>3</sub>** is a random oracle in **ReSolveD- $\alpha$ .Sign**, with  $Q_{\text{FSHash}_3}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. Let  $\mathcal{O}' = \{E, E', \text{BAVCHash}\}$  and  $Q_{\mathcal{O}'} = \{Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil, Q_{E'} + \frac{Q_{\text{sig}} \cdot 3L}{2}, Q_{\text{BAVCHash}} + Q_{\text{sig}}\}$ . Let  $\mathcal{O}'' = \{E, E'\}$  and  $Q_{\mathcal{O}''} = \{Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil, Q_{E'} + \frac{Q_{\text{sig}} \cdot 3L}{2}\}$ . For any quantum adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  quantum queries respectively to the oracles in  $\mathcal{O}$  and at most  $Q_{\text{sig}}$  queries to the signature oracle  $\text{OSig}_{\text{sk}}$ ,

$$\begin{aligned} & \text{AdvEUFMA}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}, Q_{\text{sig}}) \\ & \leq \frac{Q_{\text{sig}}^2}{2^\lambda} + Q_{\text{sig}} \cdot \text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FSHash}_1}) + 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FSHash}_2} + Q_{\text{sig}})}{2^{5\lambda+64}}} \\ & \quad + 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FSHash}_3} + Q_{\text{sig}})}{2^{3\lambda+64}}} + \text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}}(Q_{\mathcal{O}''}, Q_{\text{sig}}) \\ & \quad + (Q_{\text{sig}} + 7) \cdot \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right) + \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}), \end{aligned}$$

where  $\text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}})$  is from [Theorem 8.4](#),  $\text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FSHash}_1})$  is from [Lemma 8.10](#),  $\text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}}(Q_{\mathcal{O}''}, Q_{\text{sig}})$  is from [Lemma 8.11](#), and  $\varepsilon_{\text{SR}}(\frac{\lambda}{2})$  is from [Lemma 8.12](#).

*Proof.* We introduce the following hybrid argument.

- $G_0$ . This is the real EUF-CMA security game.

$$\Pr [G_0(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] = \text{AdvEUFMA}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}, Q_{\text{sig}}). \quad (42)$$

- $G_1$ . This game is identical to the previous one, except that the signature oracle  $\text{OSig}_{\text{sk}}$  uses random  $(r, iv)$  instead of those from **CoinHash**. The indistinguishability between this game and the previous game follows from the random oracle **CoinHash** for independently uniform  $\rho$ 's. Taking a union bound over all possible pairwise collision of  $\rho \in \{0, 1\}^\lambda$ , we have

$$|\Pr [G_0(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_1(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1]| \leq \frac{Q_{\text{sig}}^2}{2^\lambda}. \quad (43)$$

- $G_2$ . This game is identical to the previous one, except that
  - All random oracles are emulated by the compressed oracle technique [[Zha19](#)].
  - When  $\text{OSig}_{\text{sk}}$  queries the random oracle **FSHash<sub>1</sub>** on  $(\text{com}, (c_\alpha)_{\alpha \in [1, \tau]})$ , this game will apply a binary measurement to the compressed oracle database to test whether this query has been recorded in the database. If true, this game outputs 0; otherwise, it programs **FSHash<sub>1</sub>** to output a uniformly sampled  $\text{chall}_1$ .

We consider the following subroutine hybrid argument for the games  $\{G_{1,i}\}_{i \in [Q_{\text{sig}}+1]}$ , where the game  $G_{1,i}$  applies the binary measurement and runs according to the measurement result as in  $G_2$  for the first  $i$  queries to  $\text{OSig}_{\text{sk}}$ . It is obvious that  $G_{1,0} = G_1$  and  $G_{1,Q_{\text{sig}}} = G_2$ . Thus,  $G_{1,i}$  and  $G_{1,i+1}$  differ only for the  $(i+1)$ -th query. We use a reduction to the quantum-list unpredictability game ([Fig. 9](#)) against an adversary  $\mathcal{A}_{\text{UP}}$  to prove

$$\begin{aligned} & |\Pr [G_{1,i}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_{1,i+1}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1]| \\ & \leq \text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FSHash}_1}) + \varepsilon_{\text{SR}}\left(\frac{\lambda}{2}\right). \end{aligned} \quad (44)$$

Assume that [Eq. \(44\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{UP}}$  that uses  $\mathcal{A}$  to break the quantum-list unpredictability:

1. Given the quantum oracle access to  $\mathcal{O}'$ ,  $\mathcal{A}_{\text{UP}}$  uses  $\mathcal{O}'$  for  $\mathcal{A}$ 's quantum queries to  $\mathcal{O}'$ .
2.  $\mathcal{A}_{\text{UP}}$  locally emulates the oracles in  $\mathcal{O} \setminus \mathcal{O}'$  (which includes  $\text{FHash}_1$  using the compressed oracle technique) for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3.  $\mathcal{A}_{\text{UP}}$  emulates the execution of  $G_{1,i}$  for  $\mathcal{A}$  except that for the  $(i+1)$ -th query to  $\text{OSig}_{\text{sk}}$ , it outputs the quantum register  $|x\rangle$  consisting of all sub-registers for the compressed oracle database inputs of  $\text{FHash}_1$ . When  $\mathcal{A}_{\text{UP}}$  is given  $c$ , it uses this value for the  $(i+1)$ -th signature output by  $\text{OSig}_{\text{sk}}$ , and honestly computes the remaining  $Q_{\text{sig}} - i - 1$  signatures for  $\mathcal{A}$  as in  $G_{1,i}$ .
4.  $\mathcal{A}_{\text{UP}}$  outputs whatever its emulated game against  $\mathcal{A}$  outputs.

Here,  $\mathcal{A}_{\text{UP}}$  makes at most  $Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil$  queries to  $E$  (by the construction of  $\text{PRG}$ ), at most  $Q_{E'} + Q_{\text{sig}} \cdot \frac{3L}{2}$  queries to  $E'$ , and at most  $Q_{\text{BAVCHash}} + Q_{\text{sig}}$  queries to  $\text{BAVCHash}$ . Moreover, the number of sub-registers in  $|x\rangle$  is at most  $Q_{\text{FHash}_1}$ . If  $b = 0$ ,  $\mathcal{A}_{\text{UP}}$  proceeds in the game  $\text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 0}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1})$  and  $\mathcal{A}$  works as in the game  $G_{1,i}$  (where no binary measurement or programming for the  $(i+1)$ -th query is needed in either the emulated game of  $\mathcal{A}_{\text{UP}}$  or the game  $G_{1,i}$ ) conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 0}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ G_{1,i}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

If  $b = 1$ ,  $\mathcal{A}_{\text{UP}}$  is in the game  $\text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 1}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1})$ . Conditioned on  $b'' = 0$ , the programming in  $G_{1,i+1}$  must succeed and result in  $\text{chall}_1$  as uniform as the one from the compressed oracle technique in the emulated game of  $\mathcal{A}_{\text{UP}}$ . That is, the emulated game of  $\mathcal{A}_{\text{UP}}$  is identically distributed as  $G_{1,i+1}$  under this condition as long as  $\neg \text{BadOra}$  occurs. Conditioned on  $b'' = 1$ , both the quantum-list unpredictability game and the game  $G_{1,i+1}$  output 0 as long as  $\neg \text{BadOra}$  occurs. Thus,

$$\Pr \left[ \text{GameUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}, 1}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ G_{1,i+1}(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

From the emulation loss of [Lemma 8.12](#) for  $E''$  (which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have  $\Pr [\text{BadOra}] \leq \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right)$ . Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 8.10](#) parameterized by  $(Q_{\mathcal{O}'}, Q_{\text{FHash}_1})$  conditioned on  $\neg \text{BadOra}$  so that [Eq. \(44\)](#) holds. Taking a union bound over all  $i$ 's, it follows from  $G_{1,0} = G_1$  and  $G_{1,Q_{\text{sig}}} = G_2$  that

$$\begin{aligned} & \left| \Pr [G_1(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_2(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] \right| \\ & \leq Q_{\text{sig}} \cdot \text{AdvUP}_{\mathcal{A}_{\text{UP}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{FHash}_1}) + Q_{\text{sig}} \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \end{aligned} \quad (45)$$

- $G_3$ . This game is identical to the previous one, except that when  $\text{OSig}_{\text{sk}}$  queries the random oracle  $\text{FHash}_2$ , it is reprogrammed to output a uniform  $\text{chall}_2$ . We use a reduction to the adaptive reprogramming game ([Fig. 8](#)) against an adversary  $\mathcal{A}_{\text{RP}}$  to prove

$$\left| \Pr [G_2(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] - \Pr [G_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] \right| \leq 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FHash}_2} + Q_{\text{sig}})}{2^{5\lambda+64}}} + 3 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \quad (46)$$

Assume that [Eq. \(46\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{RP}}$  that internally runs  $\mathcal{A}$  as a subroutine (where  $\mathcal{O}_0 = \text{FHash}_2$ ):

1. Given the quantum oracle access to  $\mathcal{O}_b$ ,  $\mathcal{A}_{\text{RP}}$  directly uses  $\mathcal{O}_b$  for  $\mathcal{A}$ 's quantum queries to  $\text{FHash}_2$ .

2.  $\mathcal{A}_{\text{RP}}$  locally emulates the oracles in  $\mathcal{O} \setminus \{\text{FSH}_{\text{Hash}_2}\}$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3. In each of the  $\text{OSig}_{\text{sk}}$  calls, when it is time to compute  $\text{chall}_2$ ,  $\mathcal{A}_{\text{RP}}$  defines a distribution  $y'$  over  $X = \{0, 1\}^{5\lambda+64} \times \{0, 1\}^{\lambda+B} \times \{0, 1\}^{(\lambda+B)\lambda} \times \mathbb{F}_2^{\ell_{\text{wit}}}$  and with the parameter  $(\mathbf{u}, \mathbf{V}, \mathbf{d})$  according to  $\text{VOLE.Commit}$ , such that the first  $5\lambda + 64$  bits (i.e.,  $\text{chall}_1$ ) is uniform while the other part is fixed to

$$\begin{aligned}\tilde{\mathbf{u}} &:= \text{VOLEHash}(\text{chall}_1, \mathbf{u}), \\ \tilde{\mathbf{V}} &:= \text{VOLEHash}(\text{chall}_1, \mathbf{V}), \\ \mathbf{d} &:= \mathbf{d},\end{aligned}$$

receives  $x = (\text{chall}_1, \tilde{\mathbf{u}}, \tilde{\mathbf{V}}, \mathbf{d})$ , and queries  $\mathcal{O}_b$  with  $x$  to obtain  $\text{chall}_2$ .

4.  $\mathcal{A}_{\text{RP}}$  outputs whatever its emulated game against  $\mathcal{A}$  outputs.

$\mathcal{A}_{\text{RP}}$  can make at most  $Q_{\text{FSH}_{\text{Hash}_2}} + Q_{\text{sig}}$  adaptive quantum queries to  $\mathcal{O}_b$ . If  $b = 0$ , the oracle  $\mathcal{O}_0 = \text{FSH}_{\text{Hash}_2}$  shows that the view of  $\mathcal{A}$  is identically distributed as in  $\text{G}_2$  conditioned on  $\neg \text{BadOra}$

$$\Pr \left[ \text{GameRP}_{\mathcal{A}_{\text{RP}}}^0(Q_{\text{FSH}_{\text{Hash}_2}} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ \text{G}_2(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

If  $b = 1$ ,  $\mathcal{O}_1$  is reprogrammed as in  $\text{G}_3$  conditioned on  $\neg \text{BadOra}$ :

$$\Pr \left[ \text{GameRP}_{\mathcal{A}_{\text{RP}}}^1(Q_{\text{FSH}_{\text{Hash}_2}} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg \text{BadOra} \right] = \Pr \left[ \text{G}_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

From the emulation loss of [Lemma 8.12](#) for  $\text{E}$ ,  $\text{E}'$ , and  $\text{E}''$  (each of which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have  $\Pr[\text{BadOra}] \leq 3 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right)$ . Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 8.9](#) for  $Q = Q_{\text{FSH}_{\text{Hash}_2}} + Q_{\text{sig}}$ ,  $R = Q_{\text{sig}}$ , and  $p_{X, \max} \leq \frac{1}{2^{5\lambda+64}}$  conditioned on  $\neg \text{BadOra}$ .

- $\text{G}_4$ . This game is identical to the previous one, except that when  $\text{OSig}_{\text{sk}}$  queries the random oracle  $\text{FSH}_{\text{Hash}_3}$ , it is reprogrammed to output a uniform  $\text{chall}_3$ . We use a reduction to the adaptive reprogramming game ([Fig. 8](#)) against an adversary  $\mathcal{A}_{\text{RP}}$  to prove

$$\left| \Pr \left[ \text{G}_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] - \Pr \left[ \text{G}_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] \right| \leq 2\sqrt{2} \cdot \sqrt{\frac{Q_{\text{sig}} \cdot (Q_{\text{FSH}_{\text{Hash}_3}} + Q_{\text{sig}})}{2^{3\lambda+64}}} + 3 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \quad (47)$$

Assume that [Eq. \(47\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{RP}}$  that internally runs  $\mathcal{A}$  as a subroutine (where  $\mathcal{O}_0 = \text{FSH}_{\text{Hash}_3}$ ):

1. Given the quantum oracle access to  $\mathcal{O}_b$ ,  $\mathcal{A}_{\text{RP}}$  directly uses  $\mathcal{O}_b$  for  $\mathcal{A}$ 's quantum queries to  $\text{FSH}_{\text{Hash}_3}$ .
2.  $\mathcal{A}_{\text{RP}}$  locally emulates the oracles in  $\mathcal{O} \setminus \{\text{FSH}_{\text{Hash}_3}\}$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3. In each of the  $\text{OSig}_{\text{sk}}$  calls, when it is time to compute  $\text{chall}_3$ ,  $\mathcal{A}_{\text{RP}}$  defines a distribution  $y'$  over  $X = \{0, 1\}^{3\lambda+64} \times \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda} \times \{0, 1\}^{32}$  and with the parameter  $(\mathbf{w}, \mathbf{u}, \mathbf{V}, \mathbf{H}, \mathbf{y}, \text{ctr})$  such that the first  $3\lambda + 64$  bits (i.e.,  $\text{chall}_2$ ) is uniform, the value of  $(\tilde{a}_1, \tilde{a}_0)$  is distributed as in  $\text{QS.RSDProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \mathbf{H}, \mathbf{y}, \text{chall}_2)$ , and  $\text{ctr} := \text{ctr}$ , receives  $x = (\text{chall}_2, \tilde{a}_1, \tilde{a}_0, \text{ctr})$ , and queries  $\mathcal{O}_b$  with  $x$  to obtain  $\text{chall}_3$ .
4.  $\mathcal{A}_{\text{RP}}$  outputs whatever its emulated game against  $\mathcal{A}$  outputs.

$\mathcal{A}_{\text{RP}}$  can make at most  $Q_{\text{FHash}_3} + Q_{\text{sig}}$  adaptive quantum queries to  $\mathcal{O}_b$ . If  $b = 0$ , the oracle  $\mathcal{O}_0 = \text{FHash}_3$  shows that the view of  $\mathcal{A}$  is identically distributed as in  $\text{G}_3$  conditioned on  $\neg\text{BadOra}$ :

$$\Pr \left[ \text{GameRP}_{\mathcal{A}_{\text{RP}}}^0(Q_{\text{FHash}_3} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg\text{BadOra} \right] = \Pr \left[ \text{G}_3(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

If  $b = 1$ ,  $\mathcal{O}_1$  is reprogrammed as in  $\text{G}_4$  conditioned on  $\neg\text{BadOra}$ :

$$\Pr \left[ \text{GameRP}_{\mathcal{A}_{\text{RP}}}^1(Q_{\text{FHash}_3} + Q_{\text{sig}}, Q_{\text{sig}}) = 1 \mid \neg\text{BadOra} \right] = \Pr \left[ \text{G}_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

From the emulation loss of [Lemma 8.12](#) for  $\text{E}$ ,  $\text{E}'$ , and  $\text{E}''$  (each of which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have  $\Pr[\text{BadOra}] \leq 3 \cdot \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right)$ . Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 8.9](#) for  $Q = Q_{\text{FHash}_3} + Q_{\text{sig}}$ ,  $R = Q_{\text{sig}}$ , and  $p_{X,\text{max}} \leq \frac{1}{2^{3\lambda+64}}$  conditioned on  $\neg\text{BadOra}$ .

- $\text{G}_5$ . This game is identical to the previous one, except that for each  $\text{OSig}_{\text{sk}}$  query, it samples uniform  $((c_\alpha)_{\alpha \in [1,\tau]}, \mathbf{u})$ , reconstructs  $(\Delta, \mathbf{Q})$  from uniform  $\text{chall}_3$  (until  $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda] = 0_{\text{grind}}^w$ ) and  $\text{decom}_I$ , and reversely computes  $\mathbf{V} := \mathbf{Q} \oplus \Delta \cdot \mathbf{u}$  for the output signature. We use a reduction to the multi-hiding game ([Fig. 10](#)) against an adversary  $\mathcal{A}_{\text{MH}}$  to prove that

$$\left| \Pr \left[ \text{G}_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] - \Pr \left[ \text{G}_5(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right] \right| \leq \text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{sig}}) + \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right). \quad (48)$$

Assume that [Eq. \(48\)](#) does not hold for contradiction. We can construct the quantum adversary  $\mathcal{A}_{\text{MH}}$  that uses  $\mathcal{A}$  to break the multi-hiding of [VOLE.Commit](#):

1. Given the quantum oracle access to  $\mathcal{O}''$ ,  $\mathcal{A}_{\text{MH}}$  uses  $\mathcal{O}''$  for  $\mathcal{A}$ 's quantum queries to  $\mathcal{O}''$ .
2.  $\mathcal{A}_{\text{MH}}$  locally emulates the oracles in  $\mathcal{O} \setminus \mathcal{O}''$  for  $\mathcal{A}$ . Let  $\text{BadOra}$  be the event that this emulation is not perfect.
3. For each  $\text{OSig}_{\text{sk}}$  query,  $\mathcal{A}_{\text{MH}}$  receives  $(\text{com}, (c_\alpha)_{\alpha \in [1,\tau]}, \mathbf{u}, \text{chall}_3, \text{decom}_I)$  from the multi-hiding game, reconstructs  $(\Delta, \mathbf{Q})$  using  $\text{chall}_3$  and  $\text{decom}_I$ , and reversely computes  $\mathbf{V} := \mathbf{Q} \oplus \Delta \cdot \mathbf{u}$  for the output signature.
4.  $\mathcal{A}_{\text{MH}}$  outputs whatever its emulated game against  $\mathcal{A}$  outputs.

$\mathcal{A}_{\text{MH}}$  makes at most  $Q_{\text{E}} + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil$  queries to  $\text{E}$  (by the construction of [PRG](#)) and at most  $Q_{\text{E}'} + Q_{\text{sig}} \cdot \frac{3L}{2}$  queries to  $\text{E}'$ . If  $b = 0$ , the emulated game of  $\mathcal{A}_{\text{MH}}$  runs as  $\text{G}_4$  conditioned on  $\neg\text{BadOra}$  since in each  $\text{OSig}_{\text{sk}}$  query, (i)  $(\mu, \mathbf{s})$  is independently uniform over  $\{0, 1\}^{2\lambda} \times \{0, 1\}^\lambda$  for the unique signed message (without loss of generality), (ii)  $\text{chall}_3$  is uniformly sampled until  $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda] = 0_{\text{grind}}^w$ , and (iii)  $\mathbf{V}$  is correctly defined for the VOLE correlation by [Lemma 8.1](#) for each of its columns. So,

$$\Pr \left[ \text{GameMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}, 0}(Q_{\mathcal{O}'}, Q_{\text{sig}}) = 1 \mid \neg\text{BadOra} \right] = \Pr \left[ \text{G}_4(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

If  $b = 1$ , the ideal multi-hiding game leads to the same distribution of  $\text{G}_5$  conditioned on  $\neg\text{BadOra}$ :

$$\Pr \left[ \text{GameMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}'', \text{VOLE.Commit}, 1}(Q_{\mathcal{O}'}, Q_{\text{sig}}) = 1 \mid \neg\text{BadOra} \right] = \Pr \left[ \text{G}_5(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1 \right].$$

From the emulation loss of [Lemma 8.12](#) for  $\text{E}''$  (which has  $|\mathcal{X}| \geq \frac{\lambda}{2}$  due to the implementation) and the perfect emulation of random oracles from the compressed oracle technique, we have  $\Pr[\text{BadOra}] \leq \varepsilon_{\text{SR}} \left( \frac{\lambda}{2} \right)$ . Using the contradiction assumption and the above results, we prove a contradiction against [Lemma 8.11](#) for  $Q_{\mathcal{O}'}$  and  $Q = Q_{\text{sig}}$  conditioned on  $\neg\text{BadOra}$ .

- $G_6$ . This game is identical to the previous one, except that for each  $\text{OSig}_{\text{sk}}$  query, it samples uniform  $\tilde{\mathbf{u}}$ , computes  $\tilde{\mathbf{V}} := \tilde{\mathbf{Q}} \oplus \Delta \cdot \tilde{\mathbf{u}}$  for the reconstructed  $(\Delta, \mathbf{Q})$ , and uniformly resamples  $\mathbf{u}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \mathbf{B})$  and  $\mathbf{V}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \mathbf{B})$  to be consistent with  $\tilde{\mathbf{u}} = \text{VOLEHash}(\text{chall}_1, \mathbf{u})$  and  $\tilde{\mathbf{V}} = \text{VOLEHash}(\text{chall}_1, \mathbf{V})$ .

This game has the same distribution as the previous one since (i)  $\mathbf{u}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \mathbf{B})$  and  $\mathbf{V}[\ell_{\text{wit}} + \lambda, \ell_{\text{wit}} + 2\lambda + \mathbf{B})$  in both games are uniform, and (ii)  $\text{VOLEHash}$  is  $\mathbb{F}_2^{\ell_{\text{wit}} + \lambda}$ -hiding.

$$\Pr [G_5(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] = \Pr [G_6(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1]. \quad (49)$$

- $G_7$ . This game is identical to the previous one, except that for each  $\text{OSig}_{\text{sk}}$  query, it samples uniform  $\tilde{a}_1$  and  $\tilde{a}_0$ , and uniformly resamples  $\mathbf{u}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$  and  $\mathbf{V}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$  to be consistent with  $\tilde{\mathbf{u}} = \text{VOLEHash}(\text{chall}_1, \mathbf{u})$ ,  $\tilde{\mathbf{V}} = \text{VOLEHash}(\text{chall}_1, \mathbf{V})$ ,  $\tilde{a}_1 = \text{ZKHash}(\text{chall}_2, [\mathbf{a}_1, \tilde{\mathbf{u}}])$ , and  $\tilde{a}_0 = \text{ZKHash}(\text{chall}_2, [\mathbf{a}_0, \tilde{\mathbf{v}}])$ .

This game has the same distribution as the previous one since (i)  $\mathbf{u}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$  and  $\mathbf{V}[\ell_{\text{wit}}, \ell_{\text{wit}} + \lambda)$  in both games are uniform, and (ii)  $\text{ZKHash}$  is  $\mathbb{F}_2^{\ell_{\text{wit}}}$ -hiding.

$$\Pr [G_6(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] = \Pr [G_7(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1]. \quad (50)$$

- $G_8$ . This game is identical to the previous one, except that for each  $\text{OSig}_{\text{sk}}$  query, it samples uniform  $\mathbf{d}$ . This game has the same distribution as the previous one since  $\mathbf{u}[0, \ell_{\text{wit}})$  is already uniform in  $G_5$  and independent of any other value in the signature computation. Thus,

$$\begin{aligned} \Pr [G_7(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] &= \Pr [G_8(Q_{\mathcal{O}}, Q_{\text{sig}}) = 1] \\ &= \text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}), \end{aligned} \quad (51)$$

where the last equality follows from the fact that in  $G_8$ , for each  $\text{OSig}_{\text{sk}}$  query, the output signature can be simulated without  $\text{sk}$  as in the EUF-KO security game.

According to Eq. (42), Eq. (43), Eq. (45), Eq. (46), Eq. (47), Eq. (48), Eq. (49), Eq. (50), and Eq. (51), we obtain this theorem.  $\square$

### 8.3 Security Proof in the ROM and ICM

Although the classical security is trivially implied by the quantum security, we can have more dedicated proofs against a classical adversary. In this setting, we use the following proof ideas of [BBD<sup>+</sup>23a], the version 1.1 of FAEST, which is not based on the lossy-key argument.

**Theorem 8.6** (EUF-KO Security of ReSolveD- $\alpha$  against Classical Adversary). *In ReSolveD- $\alpha$ , consider the following ideal-model oracles and their respective numbers of queries:*

- $E$  is an ideal cipher realizing PRG in LeafHash, with  $Q_E$  queries,
- $E'$  is an ideal cipher realizing TCCR in BAVC.Commit, with  $Q_{E'}$  queries,
- $E''$  is an ideal cipher realizing PRG in RSD.SampleNoise, with  $Q_{E''}$  queries,
- MatHash is a random oracle in RSD.SampleMatrix, with  $Q_{\text{MatHash}}$  queries,
- MsgHash is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{MsgHash}}$  queries,
- CoinHash is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{CoinHash}}$  queries,
- BAVCHash is a random oracle in BAVC.Commit, with  $Q_{\text{BAVCHash}}$  queries,
- FSHash<sub>1</sub> is a random oracle in ReSolveD- $\alpha$ .Sign, with  $Q_{\text{FSHash}_1}$  queries,

| Game | $\text{GameVOLEExtBind}_{\mathcal{A}}^{\mathcal{O}, \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}})$                                                                                                                                                                                                 |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 :  | $(\text{com}, (\mathbf{c}_{\alpha})_{\alpha \in [1, \tau]}, \text{iv}) \leftarrow \$ \mathcal{A}^{\mathcal{O}}()$                                                                                                                                                                              |
| 2 :  | $(\mathbf{u}_{\alpha}^*, \mathbf{V}_{\alpha}^*)_{\alpha \in [0, \tau)} \leftarrow \$ \text{Ext}(\mathcal{L}_{\mathcal{O}}, \text{com}, (\mathbf{c}_{\alpha})_{\alpha \in [1, \tau)}, \text{iv})$                                                                                               |
| 3 :  | $(\text{chall}_3, \text{decom}_I) \leftarrow \$ \mathcal{A}^{\mathcal{O}}()$ such that $I := \text{VOLE.ChalDecAll}(\text{chall}_3)$                                                                                                                                                           |
| 4 :  | $\forall \alpha \in [0, \tau) : (\delta_{\alpha,0}, \dots, \delta_{\alpha, n_{\alpha}-1}) := (\Delta_{\alpha}[0], \dots, \Delta_{\alpha}[n_{\alpha}-1])$                                                                                                                                       |
| 5 :  | $(\overline{\text{com}}, \mathbf{Q}_0, \dots, \mathbf{Q}_{\tau-1}) \leftarrow \$ \text{VOLE.Reconstruct}(\text{chall}, \text{decom}_I, (\mathbf{c}_{\alpha})_{\alpha \in [1, \tau)}, \text{iv}, \mu, \mathbf{s})$                                                                              |
| 6 :  | <b>return</b> 0 <b>if</b>                                                                                                                                                                                                                                                                      |
| 7 :  | (a) $\text{VOLE.Reconstruct}$ returns $\perp$ or $\overline{\text{com}} \neq \text{com}$ , or                                                                                                                                                                                                  |
| 8 :  | (b) $\forall \alpha \in [0, \tau) : \mathbf{Q}_{\alpha} = \mathbf{V}_{\alpha}^* \oplus [\delta_{\alpha,0} \cdot (\mathbf{u}_{\alpha}^* \oplus \mathbf{c}_{\alpha}), \dots, \delta_{\alpha, n_{\alpha}-1} \cdot (\mathbf{u}_{\alpha}^* \oplus \mathbf{c}_{\alpha})]$ , where $\mathbf{c}_0 = 0$ |
| 9 :  | <b>return</b> 1                                                                                                                                                                                                                                                                                |

Figure 11: The extractable binding game of  $\text{VOLE} = (\text{VOLE.Commit}, \text{VOLE.Reconstruct})$ .

- $\text{FSHash}_2$  is a random oracle in  $\text{ReSolveD-}\alpha.\text{Sign}$ , with  $Q_{\text{FSHash}_2}$  queries,
- $\text{FSHash}_3$  is a random oracle in  $\text{ReSolveD-}\alpha.\text{Sign}$ , with  $Q_{\text{FSHash}_3}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. Let  $\mathcal{O}' = \{\mathbf{E}, \mathbf{E}', \text{BAVCHash}\}$  and  $Q_{\mathcal{O}'} = \{Q_{\mathbf{E}}, Q_{\mathbf{E}'}, Q_{\text{BAVCHash}}\}$ . For any classical adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  classical queries respectively to the oracles in  $\mathcal{O}$ ,

$$\begin{aligned}
\text{AdvEUFKO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}) &\leq \frac{(Q_{\text{BAVCHash}} + Q_{\text{FSHash}_1})^2}{2^{2\lambda}} + \frac{(Q_{\text{FSHash}_1} + Q_{\text{FSHash}_2})^2}{2^{5\lambda+64}} \\
&\quad + \frac{(Q_{\text{FSHash}_2} + Q_{\text{FSHash}_3})^2}{2^{3\lambda+64}} + \frac{Q_{\text{FSHash}_3}^2}{2^{\lambda}} \\
&\quad + Q_{\text{FSHash}_1} \cdot \text{AdvVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'}) \\
&\quad + Q_{\text{FSHash}_1} \cdot Q_{\text{FSHash}_2} \cdot \varepsilon_{\text{vole}} \cdot \binom{\tau}{2} + Q_{\text{FSHash}_2} \cdot \varepsilon_{\text{zk}} + Q_{\text{FSHash}_3} \cdot \frac{2}{2^{\lambda}} \\
&\quad + \text{AdvRSD}_{\mathcal{A}_{\text{RSD}}}(m, k, w) \\
&\quad + \text{AdvPRG}_{\mathcal{A}_{\text{PRG}}}^{\mathbf{E}'', \text{PRG}''}(Q_{\mathbf{E}''}) + \left( \frac{\lfloor \frac{\lambda}{\ell_{\text{bs}}} \rfloor + 1}{\lambda} - \frac{1}{\ell_{\text{bs}}} \right),
\end{aligned}$$

where the extractable binding game  $\text{GameVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'})$  is defined in Fig. 11, and the advantage

$$\text{AdvVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'}) = \Pr \left[ \text{GameVOLEExtBind}_{\mathcal{A}_{\text{VOLE}}}^{\mathcal{O}', \text{VOLE}, \text{Ext}}(Q_{\mathcal{O}'} ) = 1 \right].$$

**Theorem 8.7** (EUF-CMA Security of  $\text{ReSolveD-}\alpha$  against Classical Adversary). *In  $\text{ReSolveD-}\alpha$ , consider the following ideal-model oracles and their respective numbers of queries:*

- $\mathbf{E}$  is an ideal cipher realizing  $\text{PRG}$  in  $\text{LeafHash}$  and  $\text{VOLE.ConvertToVOLE}$ , with  $Q_{\mathbf{E}}$  queries,
- $\mathbf{E}'$  is an ideal cipher realizing  $\text{TCCR}$  in  $\text{BAVC.Commit}$ , with  $Q_{\mathbf{E}'}$  queries,
- $\mathbf{E}''$  is an ideal cipher realizing  $\text{PRG}$  in  $\text{RSD.SampleNoise}$ , with  $Q_{\mathbf{E}''}$  queries,
- $\text{MatHash}$  is a random oracle in  $\text{RSD.SampleMatrix}$ , with  $Q_{\text{MatHash}}$  queries,
- $\text{MsgHash}$  is a random oracle in  $\text{ReSolveD-}\alpha.\text{Sign}$ , with  $Q_{\text{MsgHash}}$  queries,

- **CoinHash** is a random oracle in **ReSolveD- $\alpha$ .Sign**, with  $Q_{\text{CoinHash}}$  queries,
- **BAVCHash** is a random oracle in **BAVC.Commit**, with  $Q_{\text{BAVCHash}}$  queries,
- **FSHash<sub>1</sub>** is a random oracle in **ReSolveD- $\alpha$ .Sign**, with  $Q_{\text{FSHash}_1}$  queries,
- **FSHash<sub>2</sub>** is a random oracle in **ReSolveD- $\alpha$ .Sign**, with  $Q_{\text{FSHash}_2}$  queries,
- **FSHash<sub>3</sub>** is a random oracle in **ReSolveD- $\alpha$ .Sign**, with  $Q_{\text{FSHash}_3}$  queries.

Let  $\mathcal{O}$  be a list of the above ideal-model oracles and  $Q_{\mathcal{O}}$  be a list of their respective numbers of queries. Let  $\mathcal{O}' = \{E, E'\}$  and  $Q_{\mathcal{O}'} = \{Q_E + Q_{\text{sig}} \cdot 2L \cdot \lceil \frac{2\lambda + \ell_{\text{vole}}}{\lambda} \rceil, Q_{E'} + \frac{Q_{\text{sig}} \cdot 3L}{2}\}$ . For any classical adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  queries respectively to the oracles in  $\mathcal{O}$  and at most  $Q_{\text{sig}}$  queries to the signature oracle  $\text{OSig}_{\text{sk}}$ ,

$$\begin{aligned}
& \text{AdvEUF-CMA}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}, Q_{\text{sig}}) \\
& \leq Q_{\text{sig}} \cdot \left( \frac{Q_{\text{sig}} + Q_{\text{CoinHash}}}{2^\lambda} + \frac{Q_{\text{sig}} + Q_{\text{FSHash}_1}}{2^{2\lambda+128}} + \frac{Q_{\text{sig}} + Q_{\text{BAVCHash}}}{2^{2\lambda}} \right. \\
& \quad + 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{E', \text{CCR}} \left( Q_{E'} + \frac{3L}{2} + 2\lceil \log L \rceil + 5, 2\lceil \log L \rceil + 5, \chi \right) + \frac{(2\lceil \log L \rceil + 5)^2}{2^{\lambda-1}} \\
& \quad + \frac{Q_{\text{sig}} + Q_{\text{FSHash}_2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{\text{FSHash}_3}}{2^{3\lambda+64}} \Bigg) \\
& \quad + \text{AdvMH}_{\mathcal{A}_{\text{MH}}}^{\mathcal{O}', \text{VOLE.Commit}}(Q_{\mathcal{O}'}, Q_{\text{sig}}) \\
& \quad + \text{AdvEUF-KO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}}),
\end{aligned}$$

where  $\text{AdvEUF-KO}_{\mathcal{A}}^{\mathcal{O}, \text{ReSolveD-}\alpha}(Q_{\mathcal{O}})$  is from [Theorem 8.6](#), and

- **CCR** :  $x \in \{0, 1\}^\lambda \mapsto \text{TCCR}(x, s, \text{iv}) \in \{0, 1\}^\lambda$  is an  $E'$ -based CCR hash function for some fixed  $s$  and (uniform)  $\text{iv}$  ([Theorem 8.12](#)).

**Proof sketch.** For the EUF-KO bound in [Theorem 8.6](#), we need the analysis of the collision between random-oracle inputs and the extractable binding property of the VOLE commitment ([Fig. 11](#)). This extractable binding property can only exist in the classical setting, where it is feasible to observe all oracle queries. Assuming that all random-oracle inputs have no collision, we can prove that (**VOLE.Commit**, **VOLE.Reconstruct**) satisfies the property by defining **Ext** to use these non-colluding inputs to reconstruct the VOLE outputs of the prover as in our proof against the quantum adversary, but no longer use the inefficient enumeration of all pre-images. Then, we have the same probability analysis of the RBR soundness in the quantum proof since these bad events occur regardless of the classical or quantum adversary. However, we do not use the reduction from RBR soundness to soundness but take a simple union bound over all random-oracle queries for these bad events. Putting these arguments with the underlying RSD security and the PRG security for secret  $\text{seed}_{\text{sk}}$ , we arrive at the EUF-KO bound against the classical adversary.

For the EUF-CMA bound in [Theorem 8.7](#), the proof is similar to its counterpart in the QROM and QICM in that it replaces the real signature oracle by a simulated oracle without the secret key to reduce the EUF-CMA security to the EUF-KO security. The main difference between the quantum and classical security is whether the local emulation of (Q)ICM can be perfect or not due to the quantum/classical access. Furthermore, we can directly analyze the successful programming and do not rely on the reprogramming lemma. Based on the above strategies, we have the following security theorems in the classical setting. We stress that, in the classical setting, we do not rely on [Conjecture 8.1](#) as in the quantum setting but can provably construct an almost injective PRG in the ideal cipher model as in [Theorem 8.9](#).

## 8.4 Proofs of Subroutine Lemmas and Theorems

In this subsection, we provide formal proofs for the lemmas and theorems used in the previous proofs for completeness.

### 8.4.1 Local Emulation of Ideal Cipher

**Lemma 8.12** (Simulating a Quantum SPRP). *Let  $\mathcal{X}$  and  $\mathcal{K}$  be finite sets, and let  $E : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$  be an ideal cipher. Let  $q$  be a query bound. Then there exists a construction  $\widehat{E}_r$  built from a random string  $r$  and  $\Theta(\log_2^2 |\mathcal{X}|)$  rounds of the Morris-Rogaway sometimes-recurse (SR) shuffle [MR14], each round using a  $2q$ -wise independent hash function  $\mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$ , such that:*

1. (Resource.) *Implementing the  $\Theta(\log_2^2 |\mathcal{X}|)$  round functions requires a random string  $r$  of length*

$$\Theta(q \log_2^2 |\mathcal{X}| \cdot \log_2(|\mathcal{X}||\mathcal{K}|)) \text{ bits.}$$

2. (Classical security.) *For any (classical)  $(p, q)$ -bounded distinguisher  $D$ ,*

$$\left| \Pr[D^{E, E^{-1}} \Rightarrow 1] - \Pr[D^{\widehat{E}_r, \widehat{E}_r^{-1}} \Rightarrow 1] \right| \leq \frac{1}{|\mathcal{X}|}.$$

3. (Quantum security.) *For any quantum distinguisher  $D$  making  $q$  superposition queries,*

$$\left| \Pr[D^{|E\rangle} \Rightarrow 1] - \Pr[D^{\widehat{E}_r} \Rightarrow 1] \right| \leq \varepsilon_{\text{SR}}(|\mathcal{X}|),$$

where  $\varepsilon_{\text{SR}}(|\mathcal{X}|)$  is the quantum strong PRP (SPRP) advantage of the SR shuffle of Morris and Rogaway [MR14], as lifted to the quantum setting by [Zha25].

*Proof. Step 1: SR construction and classical gap.* By [MR14, Theorem 4], the sometimes-recurse (SR) shuffle can convert  $t = \Theta(\log_2^2 |\mathcal{X}| - \log_2 |\mathcal{X}| \cdot \log_2 \varepsilon)$  independent random functions  $F_1, \dots, F_t : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$  and  $t$  round-keys into a  $t$ -round block cipher construction  $\widehat{E}_r$  such that the classical SPRP advantage against  $E$  is at most  $\varepsilon$ , independently of the adversary's computational power. Setting  $\varepsilon = 1/|\mathcal{X}|$  yields  $t = \Theta(\log_2^2 |\mathcal{X}|)$  rounds. The total number of round-key bits is  $\Theta(\log_2^3 |\mathcal{X}|)$ .

**Step 2: Implementing the round functions.** In the classical setting, each  $F_i : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$  is implemented as a  $q$ -wise independent hash function (sufficient to fool any  $q$ -query distinguisher [Zha21, Sect. 5.4]). In the quantum setting, each  $F_i$  must be a  $2q$ -wise independent hash function instead [Zha12]. In both cases, the change in degree has no asymptotic effect on the  $\Theta(\cdot)$  notation for the resource count.

Storing  $\Theta(\log_2^2 |\mathcal{X}|)$  such functions, each requiring  $\Theta(q \cdot (\log_2 |\mathcal{K}| + \log_2 |\mathcal{X}|))$  random bits to describe, the total length of the shared random string is

$$|r| = \Theta(q \log_2^2 |\mathcal{X}| \cdot \log_2(|\mathcal{X}||\mathcal{K}|)).$$

**Step 3: Classical security (Item 2).** By the SR theorem [MR14, Theorem 4], with  $\varepsilon = 1/|\mathcal{X}|$ , the classical SPRP advantage between  $\widehat{E}_r$  and the ideal cipher  $E$  is at most  $1/|\mathcal{X}|$ , establishing Item 2.

**Step 4: Quantum security (Item 3).** By [Zha25], a classically secure full-domain SPRP (such as the SR construction with quantum-degree-doubled round functions from Step 2) can be lifted to a quantum-secure SPRP. Concretely, [Zha25] guarantees the existence of a quantum SPRP advantage bound, which we denote  $\varepsilon_{\text{SR}}(|\mathcal{X}|)$ , for the SR construction. As noted in [Zha25], the explicit form of  $\varepsilon_{\text{SR}}(|\mathcal{X}|)$  cannot be extracted from the current version of [MR14]; we therefore leave it as a named quantity. This establishes Item 3.  $\square$

### 8.4.2 Proof of Lemma 3.1

**Lemma 3.1** (CCR for Natural and Non-trivial Queries). *Let  $\mathcal{O}$  be a list of ideal-model oracles,  $Q_{\mathcal{O}}$  be a list of the respective numbers of the ideal-model queries, and  $H^{\mathcal{O}} : \{0, 1\}^{\ell} \rightarrow \{0, 1\}^{\ell}$  be a  $(Q'_{\mathcal{O}}, Q, \chi, \varepsilon)$ -CCR hash function using these oracles, where  $Q'_{\mathcal{O}} = Q_{\mathcal{O}} + (Q, \dots, Q)$  corresponds to  $Q$  more queries for each oracle in  $\mathcal{O}$ . For any adversary  $\mathcal{A}$  making at most  $Q_{\mathcal{O}}$  queries respectively to the oracles in  $\mathcal{O}$  and at most  $Q$  NNQs to its second oracle (the real-world oracle  $\text{Real}_{H_S, \Delta}^{\mathcal{O}}$  or the ideal-world oracle  $\text{Ideal}_{H_S}^{\mathcal{O}}$ ), where each NNQ only depends on the results of the previous NNQs (with the initial NNQ only depending on an empty result),*

$$\left| \Pr_{\Delta, S \leftarrow \mathcal{S}_{\chi}} \left[ \mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \mathcal{S}_{\chi}, c \leftarrow \mathcal{U}} \left[ \mathcal{A}^{\mathcal{O}, \text{Ideal}_{H_S}^{\mathcal{O}}}() = 1 \right] \right| \leq 2 \cdot \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q'_{\mathcal{O}}, Q, \chi) + \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}},$$

where  $\mathcal{U}$  is the uniform distribution of some proper unspecified length, and  $c$  is the random coin used for Type 4 NNQs in  $\text{Ideal}_{H_S}^{\mathcal{O}}$ .

*Proof.* We prove this lemma from the following two lemmas.

**Lemma 8.13.** *Let  $\text{coll}_{\text{real}}$  be the event that, during the interaction between  $\mathcal{A}$  and its given oracles, there exists two defined results  $x, x' \in \{0, 1\}^{\ell}$  such that  $x = x'$ . It holds that*

$$\Pr_{\Delta, S \leftarrow \mathcal{S}_{\chi}} [\text{coll}_{\text{real}}] \leq \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q_{\mathcal{O}}, Q, \chi) + \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}}.$$

*Proof.* For some  $f' \leftarrow \mathcal{F}_{\ell, \ell}$ , define a hybrid oracle  $\text{Hyb}_{f', \Delta}^{\mathcal{O}}$  that is identical to  $\text{Real}_{H_S, \Delta}^{\mathcal{O}}$  except that it executes a Type 3 NNQ by defining  $x_i := f'(x_j)$  and executes a Type 4 NNQ by defining  $x_i := f'(x_j \oplus \Delta) \oplus b_j \cdot \Delta$ . In other words,  $\text{Hyb}_{f', \Delta}^{\mathcal{O}}$  replaces the  $H_S^{\mathcal{O}}$  calls in  $\text{Real}_{H_S, \Delta}^{\mathcal{O}}$  by  $f'$  calls. Let  $\text{coll}_{\text{hyb}}$  be the event that, during the interaction between  $\mathcal{A}$  and  $(\mathcal{O}, \text{Hyb}_{f', \Delta}^{\mathcal{O}})$ , there exist two defined results  $x, x' \in \{0, 1\}^{\ell}$  such that  $x = x'$ .

First, we proceed to prove that

$$\left| \Pr_{\Delta, S \leftarrow \mathcal{S}_{\chi}} [\text{coll}_{\text{real}}] - \Pr_{\Delta \leftarrow \mathcal{S}_{\chi}, f' \leftarrow \mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}] \right| \leq \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q_{\mathcal{O}}, Q, \chi). \quad (52)$$

For contradiction, we assume that Eq. (52) does not hold for a deterministic polynomial-time  $\mathcal{A}$ . We show that the following adversary  $\mathcal{A}_{\text{CCR}}$  can use  $\mathcal{A}$  to break the CCR security of  $H^{\mathcal{O}}$ :

1. Given the quantum oracle access to  $\mathcal{O}$ ,  $\mathcal{A}_{\text{CCR}}$  uses  $\mathcal{O}$  for  $\mathcal{A}$ 's quantum queries to  $\mathcal{O}$ .
2.  $\mathcal{A}_{\text{CCR}}$  samples  $\Delta \leftarrow \mathcal{S}_{\chi}$ . Upon receiving an NNQ  $q_i$  from  $\mathcal{A}$ ,  $\mathcal{A}_{\text{CCR}}$  proceeds as follows:
  - If  $q_i$  is a Type 1 or Type 2 NNQ,  $\mathcal{A}_{\text{CCR}}$  executes it as required.
  - If  $q_i$  is a Type 3 NNQ with input  $x_j$ ,  $\mathcal{A}_{\text{CCR}}$  queries the CCR oracle with  $(x_j, 0)$ , receives a response  $r$ , and defines the result of  $q_i$  to  $r$ .
  - If  $q_i$  is a Type 4 NNQ with input  $(x_j, b_j)$ ,  $\mathcal{A}_{\text{CCR}}$  queries the CCR oracle with  $(x_j \oplus \Delta, 0)$ , receives a response  $r$ , and defines the result of  $q_i$  to  $r \oplus b_j \cdot \Delta$ .
3.  $\mathcal{A}_{\text{CCR}}$  terminates when  $\mathcal{A}$  terminates. It outputs 1 if there exist two defined results  $x, x'$  such that  $x = x'$ ; otherwise it outputs 0.

$\mathcal{A}_{\text{CCR}}$  makes at most  $Q_{\mathcal{O}}$  queries respectively to the oracles in  $\mathcal{O}$  due to at most  $Q_{\mathcal{O}}$  queries of  $\mathcal{A}$ , and at most  $Q$  queries to the CCR oracle due to at most  $Q$  NNQs. Furthermore,  $\mathcal{A}_{\text{CCR}}$  is polynomial-time since  $\mathcal{A}$  is polynomial-time.

If  $\mathcal{A}_{\text{CCR}}$  is given  $\mathcal{O}_{H,S}^{\mathcal{O},\text{ccr}}$  as the second oracle for  $S \leftarrow \mathbb{S} \chi$ ,  $\mathcal{A}$  works as if it is given  $\text{Real}_{H,S,\Delta}^{\mathcal{O}}$  as the second oracle. Thus, during this emulated interaction between  $\mathcal{A}$  and its given oracles,  $\text{coll}_{\text{real}}$  happens with the same probability as in the case where  $\mathcal{A}$  interacts with  $(\mathcal{O}, \text{Real}_{H,S,\Delta}^{\mathcal{O}})$ . Although  $\mathcal{A}$  does not know the key  $S$  in the emulation (in contrast to a public  $S$  in  $\text{Real}_{H,S,\Delta}^{\mathcal{O}}$ ), this does not affect the probability of  $\text{coll}_{\text{real}}$  since every NNQ only depends on the previous NNQ responses. One can consider the event  $\text{coll}_{\text{real}}$  in the emulated and the real cases under the same probability space, i.e., the identical literal values of  $\Delta$  and (unknown)  $S$  in both two cases. Since  $\mathcal{A}$  is deterministic, an induction from the first NNQ queried by  $\mathcal{A}$  shows that the responses are the *same* in the two cases given the same  $\Delta$  and  $S$ . Depending on these responses, the occurrence of  $\text{coll}_{\text{real}}$  is the same. Thus,

$$\Pr_{S \leftarrow \mathbb{S} \chi} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{H,S}^{\mathcal{O},\text{ccr}}} () = 1 \right] = \Pr_{\Delta, S \leftarrow \mathbb{S} \chi} \left[ \text{coll}_{\text{real}} \right].$$

If  $\mathcal{A}_{\text{CCR}}$  is given  $f^* \leftarrow \mathbb{S} \mathcal{F}_{\ell+1,\ell}$  as the second oracle,  $\mathcal{A}$  will interact with an emulated second oracle that perfectly works as  $\text{Hyb}_{f',\Delta}^{\mathcal{O}}$  for the random function  $f'(\cdot) := f^*(\cdot, 0)$ . Thus,

$$\Pr_{f^* \leftarrow \mathbb{S} \mathcal{F}_{\ell+1,\ell}} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, f^*} () = 1 \right] = \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}} \right].$$

By the contradiction assumption,  $\mathcal{A}_{\text{CCR}}$  breaks the CCR property of  $H^{\mathcal{O}}$ .

Second, we prove in the following induction that

$$\Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}} \right] \leq \frac{Q^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}}. \quad (53)$$

As the base case, [Eq. \(53\)](#) trivially holds for only one NNQ. Then, assume that [Eq. \(53\)](#) holds for exact  $1 \leq Q' - 1 \leq Q - 1$  NNQs and consider the  $Q'$ -th NNQ that defines a result  $x \in \{0, 1\}^{\ell}$ . Let  $\text{coll}_{\text{hyb}}(i)$  be the event that, during the interaction between  $\mathcal{A}$  and its given oracles, there exist two results  $x, x' \in \{0, 1\}^{\ell}$  such that  $x = x'$  in the first  $i$  NNQs.  $\text{coll}_{\text{hyb}} = \text{coll}_{\text{hyb}}(Q)$  and

$$\begin{aligned} & \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q') \right] \\ &= \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q') \mid \text{coll}_{\text{hyb}}(Q' - 1) \right] \cdot \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q' - 1) \right] \\ & \quad + \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1) \right] \cdot \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \neg \text{coll}_{\text{hyb}}(Q' - 1) \right] \\ &= \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q' - 1) \right] \\ & \quad + \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1) \right] \cdot \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \neg \text{coll}_{\text{hyb}}(Q' - 1) \right] \\ &\leq \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q' - 1) \right] + \Pr_{\Delta \leftarrow \mathbb{S} \chi, f' \leftarrow \mathbb{S} \mathcal{F}_{\ell,\ell}} \left[ \text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1) \right]. \end{aligned}$$

For each result  $x' \in \{0, 1\}^{\ell}$  defined in the previous  $Q' - 1$  NNQs, it *syntactically* holds that

$$x \oplus x' = \left( \bigoplus_{j \in \mathcal{J}} f'(x_j) \right) \oplus \left( \bigoplus_{k \in \mathcal{K}} f'(x_k \oplus \Delta) \right) \oplus b \cdot \Delta \oplus \bigoplus_{\tau \in \mathcal{T}} b_{\tau} \cdot \tau$$

as  $x, x'$  can be written as a linear combination of NNQs. Note that the non-triviality of NNQ ensures that the case “ $\mathcal{J} = \mathcal{K} = \emptyset$  and  $b = 0$  and each  $b_{\tau} = 0$ ” is impossible. Conditioned on  $\neg \text{coll}_{\text{hyb}}(Q' - 1)$ , there are two cases for such  $x$  and  $x'$ :

- $\mathcal{J} = \mathcal{K} = \emptyset$  and  $b = 0$  and at least one  $b_{\tau} \neq 0$ . By the definition of  $\mathcal{T}$ ,  $x \oplus x' \neq 0$ .

- At least one  $\mathcal{J}, \mathcal{K} \neq \emptyset$  or  $b \neq 0$ . This case is essentially identical to the case that at least one  $\mathcal{J}, \mathcal{K} \neq \emptyset$  since  $b \neq 0$  implies that  $\mathcal{K}$  is not trivially empty, given that (i)  $b \neq 0$  implies at least one syntactical term of the form  $f'(x_k \oplus \Delta)$  for some  $k \in \mathcal{K}$ , and (ii) any  $x_k$  for  $k \in \mathcal{K}$  is distinct conditioned on  $\neg \text{coll}_{\text{hyb}}(Q' - 1)$  (i.e., any additive term  $f'(x_k \oplus \Delta)$  in  $\mathcal{K}$  cannot be canceled due to previous collisions).

The probability of  $x = x'$  in this case equals the probability of that a random string (from at least one non-empty  $\mathcal{J}, \mathcal{K}$ ) lies in the space spanned by linearly independent strings in  $\mathcal{T}$  (for any Boolean coefficients  $\{b_\tau\}_{\tau \in \mathcal{T}}$ ). This probability is bounded by  $2^{|\mathcal{T}|}/2^\ell$ .

Combining the two cases, we can see that  $x = x'$  occurs with the probability at most  $2^{|\mathcal{T}|}/2^\ell$ . Taking a union bound over all  $x'$ 's in the previous  $Q' - 1$  NNQs,

$$\Pr_{\Delta \leftarrow \$\chi, f' \leftarrow \$\mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1)] \leq \frac{(Q' - 1) \cdot 2^{|\mathcal{T}|}}{2^\ell}.$$

Using the induction assumption, we have that

$$\begin{aligned} \Pr_{\Delta \leftarrow \$\chi, f' \leftarrow \$\mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q')] &\leq \Pr_{\Delta \leftarrow \$\chi, f' \leftarrow \$\mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q' - 1)] \\ &\quad + \Pr_{\Delta \leftarrow \$\chi, f' \leftarrow \$\mathcal{F}_{\ell, \ell}} [\text{coll}_{\text{hyb}}(Q') \mid \neg \text{coll}_{\text{hyb}}(Q' - 1)] \\ &\leq \frac{(Q' - 1)^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}} + \frac{(Q' - 1) \cdot 2^{|\mathcal{T}|}}{2^\ell} < \frac{Q'^2 \cdot 2^{|\mathcal{T}|}}{2^{\ell+1}} \end{aligned}$$

This induction shows that Eq. (53) holds. This lemma follows from Eq. (52) and Eq. (53).  $\square$

**Lemma 8.14.** *Let  $\mathcal{U}$  and  $c$  be defined as per Lemma 3.1. Let  $\text{coll}_{\text{real}}$  be the event defined as per Lemma 8.13. It holds that*

$$\begin{aligned} &\left| \Pr_{\Delta, S \leftarrow \$\chi} [\mathcal{A}^{\mathcal{O}, \text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}}() = 1] - \Pr_{S \leftarrow \$\chi, c \leftarrow \$\mathcal{U}} [\mathcal{A}^{\mathcal{O}, \text{Ideal}_{\text{H}_S}^{\mathcal{O}}}() = 1] \right| \\ &\leq \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, \text{H}}(Q'_{\mathcal{O}}, Q, \chi) + \Pr_{\Delta, S \leftarrow \$\chi} [\text{coll}_{\text{real}}]. \end{aligned}$$

*Proof.* Assume that this lemma does not hold for some deterministic polynomial-time  $\mathcal{A}$ . The following adversary  $\mathcal{A}_{\text{CCR}}$  can use  $\mathcal{A}$  to break the CCR security of  $\text{H}^{\mathcal{O}}$ .

1. Given the quantum oracle access to  $\mathcal{O}$ ,  $\mathcal{A}_{\text{CCR}}$  uses  $\mathcal{O}$  for  $\mathcal{A}$ 's quantum queries to  $\mathcal{O}$ .
2.  $\mathcal{A}_{\text{CCR}}$  samples  $S \leftarrow \$\chi$ . Upon receiving an NNQ  $q_i$  from  $\mathcal{A}$ ,  $\mathcal{A}_{\text{CCR}}$  proceeds as follows:
  - If  $q_i$  is a Type 1/2/3 NNQ,  $\mathcal{A}_{\text{CCR}}$  executes it as required.
  - If  $q_i$  is a Type 4 NNQ with input  $(x_j, b_j)$  and  $x_j$  was not shown in previous NNQs,  $\mathcal{A}_{\text{CCR}}$  queries the CCR oracle with  $(x_j \oplus S, b_j)$ , receives a response  $r$ , and defines the result of  $q_i$  to  $r$ . If  $q_i$  is a Type 4 NNQ with input  $(x_j, b_j)$  and  $x_j$  was shown in some previous NNQ,  $\mathcal{A}_{\text{CCR}}$  samples  $s \leftarrow \$\{0, 1\}^\ell$  and defines the result of  $q_i$  to  $s$ .
3.  $\mathcal{A}_{\text{CCR}}$  terminates when  $\mathcal{A}$  terminates and outputs whatever  $\mathcal{A}$  outputs.

$\mathcal{A}_{\text{CCR}}$  makes at most  $Q'_{\mathcal{O}}$  queries respectively to the oracles in  $\mathcal{O}$  due to at most  $Q_{\mathcal{O}}$  queries of  $\mathcal{A}$  plus at most  $Q$  queries per oracle in  $\mathcal{O}$  resulting from Type 3 NNQs, and at most  $Q$  queries to the CCR oracle due to at most  $Q$  NNQs.  $\mathcal{A}_{\text{CCR}}$  is polynomial-time.

Consider that  $\mathcal{A}_{\text{CCR}}$  is given  $\mathcal{O}_{\text{H}, \Delta}^{\mathcal{O}, \text{CCR}}$  as the second oracle for some  $\Delta \leftarrow \$\chi$ . We analyze the event  $\text{coll}_{\text{real}}$  during the interaction between  $\mathcal{A}$  and  $\mathcal{O}$  plus the second oracle either emulated by  $\mathcal{A}_{\text{CCR}}$  or the oracle  $\text{Real}_{\text{H}_S, \Delta}^{\mathcal{O}}$  under the same probability space. That is, the randomness  $\Delta$  and  $S$  take the *same* values in both cases, respectively (note that, in the emulated case,  $\text{coll}_{\text{real}}$  does

not depend on the randomness  $s$  in the presence of any colliding  $x_j \in \{0, 1\}^\ell$ ; otherwise, there will be a circularity).

For any given deterministic  $\mathcal{A}$ , the first NNQ is the same regardless of the oracle given to it. Assume that the event  $\text{coll}_{\text{real}}$  does not occur. By the assumption, an induction based on the same values of  $\Delta$  and  $S$  shows that every defined result and every queried NNQ are the *same* in the two cases. Thus, for any fixed  $\Delta$  and  $S$ , the event  $(\mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$  occurs if and only if the event  $(\mathcal{A}^{\mathcal{A}_{\text{CCR}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$  occurs. Since, in the emulation,  $\mathcal{A}_{\text{CCR}}$  outputs 1 if and only if  $\mathcal{A}$  outputs 1, the event  $(\mathcal{A}^{\mathcal{A}_{\text{CCR}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$  occurs if and only if  $(\mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{H_S, \Delta}^{\mathcal{O}, \text{CCR}}}() = 1) \wedge \neg \text{coll}_{\text{real}}$  occurs. It follows from the difference lemma [Sho04] that

$$\left| \Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{H_S, \Delta}^{\mathcal{O}, \text{CCR}}}() = 1 \right] - \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1 \right] \right| \leq \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} [\text{coll}_{\text{real}}]. \quad (54)$$

If  $\mathcal{A}_{\text{CCR}}$  is given  $f^* \leftarrow \mathcal{S}_{\ell+1, \ell}$ , we have

$$\Pr_{f^* \leftarrow \mathcal{S}_{\ell+1, \ell}} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, f^*}() = 1 \right] = \Pr_{S \leftarrow \mathcal{S}_\chi, c \leftarrow \mathcal{U}} \left[ \mathcal{A}^{\mathcal{O}, \text{Ideal}_{H_S}^{\mathcal{O}}}() = 1 \right]. \quad (55)$$

for the following reasons. First, if there is no collision in the execution of Type 4 NNQs, the unique  $x_j$ 's ensure that the outputs of the random function  $f^*$  (when  $\mathcal{A}$  is given the second oracle emulated by  $\mathcal{A}_{\text{CCR}}$ ) are uniform and pairwise independent as those uniformly sampled (when  $\mathcal{A}$  is given  $\text{Ideal}_{H_S}^{\mathcal{O}}$  as the second oracle). Second, if there exists at least one collision, the result defined by the first NNQ with the conflicting  $x_j$  is still uniform as per  $f^*$ . For the subsequent results with respect to the same  $x_j$ , their uniformness and pairwise independence are ensured by the fresh sampling of  $\mathcal{A}_{\text{CCR}}$ .

Using the contradiction assumption and Eq. (54), Eq. (55), we have

$$\begin{aligned} & \left| \Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{H_S, \Delta}^{\mathcal{O}, \text{CCR}}}() = 1 \right] - \Pr_{f^* \leftarrow \mathcal{S}_{\ell+1, \ell}} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, f^*}() = 1 \right] \right| \\ &= \left| \left( \Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{H_S, \Delta}^{\mathcal{O}, \text{CCR}}}() = 1 \right] - \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1 \right] \right) \right. \\ & \quad \left. + \left( \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \mathcal{S}_\chi, c \leftarrow \mathcal{U}} \left[ \mathcal{A}^{\mathcal{O}, \text{Ideal}_{H_S}^{\mathcal{O}}}() = 1 \right] \right) \right| \\ &\geq \left| \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1 \right] - \Pr_{S \leftarrow \mathcal{S}_\chi, c \leftarrow \mathcal{U}} \left[ \mathcal{A}^{\mathcal{O}, \text{Ideal}_{H_S}^{\mathcal{O}}}() = 1 \right] \right| \\ & \quad - \left| \Pr_{\Delta \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}_{\text{CCR}}^{\mathcal{O}, \mathcal{O}_{H_S, \Delta}^{\mathcal{O}, \text{CCR}}}() = 1 \right] - \Pr_{\Delta, S \leftarrow \mathcal{S}_\chi} \left[ \mathcal{A}^{\mathcal{O}, \text{Real}_{H_S, \Delta}^{\mathcal{O}}}() = 1 \right] \right| > \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q'_\mathcal{O}, Q, \chi), \end{aligned}$$

which contradicts the CCR security of  $H^{\mathcal{O}}$ .  $\square$

This lemma follows from Lemma 8.13, Lemma 8.14, and the trivial fact that

$$\text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q'_\mathcal{O}, Q, \chi) \geq \text{AdvCCR}_{\mathcal{A}_{\text{CCR}}}^{\mathcal{O}, H}(Q_\mathcal{O}, Q, \chi)$$

since  $\mathcal{A}_{\text{CCR}}$  can make more queries to the oracles in  $\mathcal{O}$ .  $\square$

### 8.4.3 (Quantum) Security of the CTR-based PRG

**Quantum pseudorandomness of CTR-based PRG.** We first show the quantum pseudorandomness security bound of  $G^E(k, \text{iv}, t) := E(k, \text{iv} \| t) \| E(k, (\text{iv} \| t) + 1) \| \dots \| E(k, (\text{iv} \| t) + \ell - 1)$ , in the quantum ideal cipher model.

**Theorem 8.8** (Quantum pseudorandomness security bound of CTR-based PRG). *Fix  $\text{iv}$  and  $t$  in arbitrary. For any quantum adversary  $\mathcal{A}^E$  making  $T$  queries to the quantum ideal cipher  $E : \{0, 1\}^\lambda \times \{0, 1\}^n \mapsto \{0, 1\}^n$ , its advantage on distinguishing  $G^E(k, \text{iv}, t)$  output and random strings is bounded by*

$$\left| \Pr[k \leftarrow_s \{0, 1\}^\lambda : \mathcal{A}^E(G^E(k, \text{iv}, t)) = 1] - \Pr[z \leftarrow_s \{0, 1\}^{\ell n} : \mathcal{A}^E(y) = 1] \right| \leq O\left(\frac{T}{2^{\lambda/2}} + \frac{\ell^3}{2^n}\right). \quad (56)$$

*Proof.* We introduce a random permutation  $\mathbf{RP} : \{0, 1\}^n \mapsto \{0, 1\}^n$  and define  $\mathbf{RP}$ -based “idealized” PRG variant  $G^{\mathbf{RP}}(k, \text{iv}, t) := \mathbf{RP}(\text{iv}||t) || \mathbf{RP}((\text{iv}||t) + 1) || \dots || \mathbf{RP}((\text{iv}||t) + \ell - 1)$ . It is well-known that

$$\left| \Pr[k \leftarrow_s \{0, 1\}^\lambda : \mathcal{A}^E(G^E(k, \text{iv}, t)) = 1] - \Pr[k \leftarrow_s \{0, 1\}^\lambda : \mathcal{A}^E(G^{\mathbf{RP}}(k, \text{iv}, t)) = 1] \right| \leq O\left(\frac{T}{2^{\lambda/2}}\right). \quad (57)$$

Further introduce a random function  $\mathbf{RF} : \{0, 1\}^n \mapsto \{0, 1\}^n$  and consider  $\mathbf{RF}$ -based PRG  $G^{\mathbf{RF}}(k, \text{iv}, t) := \mathbf{RF}(\text{iv}||t) || \mathbf{RF}((\text{iv}||t) + 1) || \dots || \mathbf{RF}((\text{iv}||t) + \ell - 1)$ . Since  $G^{\mathbf{RP}}(k, \text{iv}, t)$ , resp.  $G^{\mathbf{RF}}(k, \text{iv}, t)$ , makes  $\ell$  calls to  $\mathbf{RP}$ , resp.  $\mathbf{RF}$ , the quantum PRF-PRP switch lemma indicates that

$$\left| \Pr[k \leftarrow_s \{0, 1\}^\lambda : \mathcal{A}^E(G^{\mathbf{RP}}(k, \text{iv}, t)) = 1] - \Pr[k \leftarrow_s \{0, 1\}^\lambda : \mathcal{A}^E(G^{\mathbf{RF}}(k, \text{iv}, t)) = 1] \right| \leq O\left(\frac{\ell^3}{2^n}\right). \quad (58)$$

Summing over Eqs. (57) and (58) yields the claim.  $\square$

**Classical and quantum almost injectivity of CTR-based PRG.** The security of the scheme relies on the so-called almost injectivity of the PRG, meaning informally that an adversary cannot produce a PRG output that has multiple preimages. In this section, we formalize this notion and prove it in the ideal cipher model. Since the proof relies on the quantum ideal cipher model, the definition is also presented only in this model.

**Proof for classical almost injectivity.** The security of the scheme relies on the so-called almost injectivity of the PRG, meaning informally that an adversary cannot produce a PRG output that has multiple preimages. In this section, we formalize this notion and prove it in the ideal cipher model. Since the proof relies on the quantum ideal cipher model, the definition is also presented only in this model.

**Definition 8.4.** Consider an IV-based tweakable PRG  $G^E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \times \{0, 1\}^\lambda \mapsto \{0, 1\}^{\ell\lambda}$ . For any adversary  $\mathcal{A}^E$ , we define the injectivity game  $\text{Inj}_{G, \mathcal{A}}$ , which runs in two phases:

1. The adversary  $\mathcal{A}^E$  interacts with the ideal cipher  $E$ , and outputs an IV  $\text{iv} \in \{0, 1\}^\lambda$  and  $D$  tuples  $(t_1, y_1), \dots, (t_D, y_D)$ .
2. The game  $\text{Inj}_{G, \mathcal{A}}$  outputs 1 if there exists at least one tuple  $(t_j, y_j)$  among the  $D$  tuples such that there exist two distinct PRG seeds  $k, k' \in \{0, 1\}^\lambda$  both producing  $y_j$  under  $t_j$ , i.e.,  $G^E(k, \text{iv}, t_j) = G^E(k', \text{iv}, t_j) = y_j$ . Otherwise, the game  $\text{Inj}_{G, \mathcal{A}}$  outputs 0.

The advantage of adversary  $\mathcal{A}$  in breaking the almost injectivity of  $G$  is defined as

$$\text{Adv}_G^{\text{Inj}}(\mathcal{A}) := \Pr[\text{Inj}_{G, \mathcal{A}} = 1]. \quad (59)$$

We then prove that  $G^E(k, iv, t) := E(k, iv||t)||E(k, (iv||t) + 1)||\dots||E(k, (iv||t) + \ell - 1)$  is quantum almost injective in the quantum ideal cipher model.

**Theorem 8.9** (Almost Injectivity Bound for CTR-based PRG). *Consider any adversary  $\mathcal{A}^E$  that makes  $T$  queries to the ideal cipher  $E : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^n$  and outputs  $D$  tuples  $(t_1, y_1), \dots, (t_D, y_D)$  in the first phase of  $\text{Inj}_{G, \mathcal{A}}$ . Then*

$$\text{Adv}_G^{\text{Inj}}(\mathcal{A}) \leq \frac{2T^2(\ell^\ell + 1) + \ell T 2^\lambda + D 2^\lambda}{2^{\ell(n-1)}}. \quad (60)$$

For the case  $\ell n = 2\lambda$ , the above bound simplifies to

$$\text{Adv}_G^{\text{Inj}}(\mathcal{A}) \leq \frac{2^{\ell+1} T^2 (\ell^\ell + 1)}{2^{2\lambda}} + \frac{2^\ell (\ell T + D)}{2^\lambda}. \quad (61)$$

*Proof.* Given an adversary  $\mathcal{A}$  for the game  $\text{Inj}_{G, \mathcal{A}}$ , we construct an adversary  $\mathcal{B}^E$  that maintains a query list  $\mathcal{Q}_E$  and runs  $\mathcal{A}$  as follows. When  $\mathcal{A}$  queries  $E(L, u)$ ,  $\mathcal{B}^E$  queries its own oracle  $E(L, u) \rightarrow v$ , forwards the answer  $v$  to  $\mathcal{A}$ , and adds the query record  $(L, u, v)$  to  $\mathcal{Q}_E$ .  $\square$

*Proof.* Given an adversary  $\mathcal{A}$  for the game  $\text{Inj}_{G, \mathcal{A}}$ , we construct an adversary  $\mathcal{B}^E$  that maintains a query list  $\mathcal{Q}_E$  and runs  $\mathcal{A}$  as follows. When  $\mathcal{A}$  queries  $E(L, u)$ ,  $\mathcal{B}^E$  queries its oracle  $E(L, u) \rightarrow v$ , forwards the reply  $v$  to  $\mathcal{A}$ , and adds the record  $(L, u, v)$  to  $\mathcal{Q}_E$ .

When  $\mathcal{A}$  queries  $E^{-1}(L, v)$ ,  $\mathcal{B}^E$  queries its oracle  $E^{-1}(L, v) \rightarrow u$ , forwards the reply  $u$  to  $\mathcal{A}$ , and adds the record  $(L, u, v)$  to  $\mathcal{Q}_E$ .

Then, if the number of records under key  $L$  in  $\mathcal{Q}_E$  reaches  $2^\lambda/2$ ,  $\mathcal{B}$  issues all remaining  $2^\lambda/2$  unseen queries under  $L$ . That is, for all  $(L, u) \in \{0, 1\}^\lambda \times \{0, 1\}^n$ , if  $\mathcal{Q}_E$  does not already contain a record of the form  $(L, u, \star)$ ,  $\mathcal{B}$  makes an additional query  $E(L, u) \rightarrow v$  and appends  $(L, u, v)$  to  $\mathcal{Q}_E$ .

We now analyze the execution of the game  $\text{Inj}_{G, \mathcal{B}}$ . It is easy to see that if  $\mathcal{A}$  makes  $T$  queries to  $E$  during phase one, then  $\mathcal{B}$  makes at most  $2T$  queries to  $E$  in phase one.<sup>3</sup>

An execution of  $\text{Inj}_{G, \mathcal{A}}$  is called *bad* if, after completion, there exist  $2\ell$  triples in  $\mathcal{Q}_E$  with identical plaintexts and ciphertexts but distinct keys, i.e., of the form  $(L, u, v_1), (L', u, v_1), (L, u+1, v_2), (L', u+1, v_2), \dots, (L, u+\ell-1, v_\ell), (L', u+\ell-1, v_\ell)$  with  $L \neq L'$ .

Given two such starting records  $(L, u, v_1)$  and  $(L', u, v_1)$ , of which there are  $\binom{2T}{2} \leq 2T^2$  possible pairs, the subsequent  $2\ell - 2$  records are uniquely determined. For any pair  $((L, u + i, v_i), (L', u + i, v_i))$ , we consider the following cases:

- Case where  $(L, u + i, v_i)$  appears later than  $(L', u + i, v_i)$ :
  - If  $(L, u + i, v_i)$  is generated by  $\mathcal{B}$  forwarding a query  $E(L, u + i)$  from  $\mathcal{A}$ , then by construction of  $\mathcal{B}$ , at the time of the query, the number of records under key  $L$  in  $\mathcal{Q}_E$  is at most  $2^n/2$ . Thus the response  $E(L, u + i)$  is chosen uniformly from at least  $2^n - 2^n/2 \geq 2^n/2$  values, so the probability that  $E(L, u + i) = v_i$  is at most  $2/2^n$ .
  - If  $(L, u + i, v_i)$  is generated by an additional query  $E(L, u + i)$  from  $\mathcal{B}$ , then the response is again uniform over  $2^n/2$  values, so the probability that  $E(L, u + i) = v_i$  is at most  $2/2^n$ .
  - If  $(L, u + i, v_i)$  is generated by  $\mathcal{B}$  forwarding an inverse query  $E^{-1}(L, v_i)$  from  $\mathcal{A}$ , then similarly, the number of records under  $L$  is at most  $2^n/2$ , so the response  $E^{-1}(L, v_i)$  is uniform over at least  $2^n/2$  values, and the probability that  $E^{-1}(L, v_i) = u + i$  is at most  $2/2^n$ .
- The case where  $(L', u + i, v_i)$  appears later than  $(L, u + i, v_i)$  is symmetric.

<sup>3</sup>The reason is that  $T$  queries by  $\mathcal{A}$  can “fill” at most  $\frac{T}{2^{\lambda-1}} = \frac{2T}{2^\lambda}$  keys  $L$  to the point where  $\mathcal{B}$  must complete  $2^\lambda/2$  queries. Thus  $\mathcal{B}$  completes at most  $\frac{2T}{2^\lambda} \times \frac{2^\lambda}{2} = T$  queries, which together with  $\mathcal{A}$ ’s  $T$  queries sum to  $2T$ .

In all cases, the probability that  $\mathcal{B}$  successfully forms such a pair  $((L, u + i, v_i), (L', u + i, v_i))$  is bounded by  $1/2^{n-1}$ . Taking the union bound over  $\ell$  positions and summing over the  $2T^2$  initial pairs yields

$$\Pr [\text{Inj}_{G,\mathcal{B}} \text{ is bad}] \leq \frac{2T^2}{2^{\ell(n-1)}}. \quad (62)$$

An execution of  $\text{Inj}_{G,\mathcal{B}}$  is called *good* if  $\mathcal{Q}_E$  contains no such families of triples. It follows that

$$\begin{aligned} \Pr [\text{Inj}_{G,\mathcal{B}} = 1] &\leq \Pr [\text{Inj}_{G,\mathcal{B}} \text{ is bad}] \\ &\quad + \Pr [\text{Inj}_{G,\mathcal{B}} = 1 \mid \text{Inj}_{G,\mathcal{B}} \text{ is good}]. \end{aligned} \quad (63)$$

In a good execution of  $\text{Inj}_{G,\mathcal{B}}$ , let  $\mathcal{Q}_E = ((L_1, u_1, v_1), \dots, (L_T, u_T, v_T))$  be the list of queries after phase one. Under this condition, the ideal cipher  $E$  satisfies

$$E(L_1, u_1) = v_1 \wedge \dots \wedge E(L_T, u_T) = v_T.$$

We write  $E \vdash \mathcal{Q}_E$  for this condition.

Let  $\mathcal{S} = \{(t_1, y_1), \dots, (t_D, y_D)\}$  be the set output by  $\mathcal{A}$ . Then

$$\begin{aligned} &\Pr [\text{Inj}_{G,\mathcal{B}} = 1 \mid \text{Inj}_{G,\mathcal{B}} \text{ is good}] \\ &\leq \sum_{\substack{(k,k',(t,y)) \in \{0,1\}^\lambda \times \{0,1\}^\lambda \times \mathcal{S} \\ k \neq k'}} \Pr [G(k, \text{iv}, t) = G(k', \text{iv}, t) = y \mid \text{Inj}_{G,\mathcal{B}} \text{ is good} \wedge E \vdash \mathcal{Q}_E]. \end{aligned} \quad (64)$$

The sum contains at most  $(D2^\lambda)^2$  distinct tuples  $(k, k', (t, y))$ .

For any such tuple, the event  $G(k, \text{iv}, t) = G(k', \text{iv}, t) = y$  depends on  $2\ell$  entries of  $E$ :

$$\begin{aligned} &E(k, \text{iv} \parallel t) = E(k', \text{iv} \parallel t) = y[1] \\ &\wedge E(k, (\text{iv} \parallel t) + 1) = E(k', (\text{iv} \parallel t) + 1) = y[2] \\ &\wedge \dots \\ &\wedge E(k, (\text{iv} \parallel t) + \ell - 1) = E(k', (\text{iv} \parallel t) + \ell - 1) = y[\ell]. \end{aligned}$$

Under  $E \vdash \mathcal{Q}_E$ , some of these entries are fixed by  $\mathcal{Q}_E$ . However, under the goodness condition, the  $\ell$  equations

$$\begin{aligned} &E(k, \text{iv} \parallel t) = E(k', \text{iv} \parallel t), \\ &E(k, (\text{iv} \parallel t) + 1) = E(k', (\text{iv} \parallel t) + 1), \\ &\dots, \\ &E(k, (\text{iv} \parallel t) + \ell - 1) = E(k', (\text{iv} \parallel t) + \ell - 1) \end{aligned} \quad (65)$$

cannot all hold simultaneously.

We classify tuples  $(k, k', (\text{iv}, t, y))$  into four types based on which entries are fixed:

- **Type 1 (Pair Conflict):** There exists an index  $i \in \{0, \dots, \ell - 1\}$  such that  $\mathcal{Q}_E$  contains two records  $(k, (\text{iv} \parallel t) + i, v)$  and  $(k', (\text{iv} \parallel t) + i, v')$  with  $v \neq v'$ . Then under  $E \vdash \mathcal{Q}_E$ , we have  $E(k, (\text{iv} \parallel t) + i) \neq E(k', (\text{iv} \parallel t) + i)$ , so (65) cannot hold. Thus

$$\Pr [\exists \text{ Type 1 tuple } (k, k', (t, y)) : G(k, \text{iv}, t) = G(k', \text{iv}, t) = y] = 0. \quad (66)$$

- **Type 2 ( $\alpha$ -Paired),  $\alpha = 1, \dots, \ell - 1$ :** There exist  $\alpha$  indices  $i_1, \dots, i_\alpha \in \{0, \dots, \ell - 1\}$  such that:

- For each  $i \in \{i_1, \dots, i_\alpha\}$ ,  $\mathcal{Q}_E$  contains both  $(k, (\text{iv}\|t) + i, y[i+1])$  and  $(k', (\text{iv}\|t) + i, y[i+1])$ , so under  $E \vdash \mathcal{Q}_E$ ,  $E(k, (\text{iv}\|t) + i) = E(k', (\text{iv}\|t) + i) = y[i+1]$ .
- For each  $i \in \{0, \dots, \ell-1\} \setminus \{i_1, \dots, i_\alpha\}$ ,  $\mathcal{Q}_E$  is missing at least one of  $(k, (\text{iv}\|t) + i, \star)$  or  $(k', (\text{iv}\|t) + i, \star)$ , so at least one of  $E(k, (\text{iv}\|t) + i)$  or  $E(k', (\text{iv}\|t) + i)$  is unassigned.

Since at least  $\ell - \alpha$  positions are unassigned,

$$\Pr \left[ \bigwedge_{\substack{i \in \{0, \dots, \ell-1\} \\ i \notin \{i_1, \dots, i_\alpha\}}} E(k, (\text{iv}\|t) + i) = y[i+1] \wedge E(k', (\text{iv}\|t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E \right] \leq \left( \frac{1}{2^{n-1}} \right)^{\ell-\alpha}. \quad (67)$$

To count such  $\alpha$ -paired tuples, we define an  $\alpha$ -paired query sequence for a set  $\{i_1, \dots, i_\alpha\}$  as  $2\alpha$  records of the form

$$\begin{aligned} & (L, u + i_1, v_1), (L', u + i_1, v_1), \\ & (L, u + i_2, v_2), (L', u + i_2, v_2), \\ & \dots, \\ & (L, u + i_\alpha, v_\alpha), (L', u + i_\alpha, v_\alpha). \end{aligned}$$

For any such index set, there are at most  $\binom{2T}{2} \leq 2T^2$  pairs of queries among  $\mathcal{B}$ 's  $2T$  queries that could form such a sequence. The probability that any pair actually forms such a sequence is at most  $1/2^{\alpha(n-1)}$ . By linearity of expectation,

$$\mathbb{E}[\text{number of } \{i_1, \dots, i_\alpha\}\text{-}\alpha\text{-paired sequences}] \leq \frac{2T^2}{2^{\alpha(n-1)}}. \quad (68)$$

There are at most  $\binom{\ell}{\alpha} \leq \ell^\alpha$  index sets, so

$$\mathbb{E}[\text{number of } \alpha\text{-paired sequences}] \leq \frac{2T^2 \ell^\alpha}{2^{\alpha(n-1)}}. \quad (69)$$

The number of  $\alpha$ -paired tuples is at most the number of such sequences. Thus

$$\begin{aligned} & \Pr [\exists \alpha\text{-paired tuple } (k, k', (t, y)) : G(k, \text{iv}, t) = G(k', \text{iv}, t) = y] \\ & \leq \sum_M \Pr [\text{number of } \alpha\text{-paired sequences} = M] \cdot M \cdot \left( \frac{1}{2^{n-1}} \right)^{\ell-\alpha} \\ & \leq \mathbb{E}[\text{number of } \alpha\text{-paired sequences}] \cdot \frac{1}{2^{(\ell-\alpha)(n-1)}} \\ & \leq \frac{2T^2 \ell^\alpha}{2^{\ell(n-1)}}. \end{aligned} \quad (70)$$

Summing over  $\alpha$  yields

$$\begin{aligned} & \Pr [\exists \text{ Type 2 tuple } (k, k', (t, y)) : G(k, \text{iv}, t) = G(k', \text{iv}, t) = y] \\ & \leq \sum_{\alpha=1}^{\ell-1} \frac{2T^2 \ell^\alpha}{2^{\ell(n-1)}} \leq \frac{2T^2 \ell^\ell}{2^{\ell(n-1)}}. \end{aligned} \quad (71)$$

- **Type 3 ( $\beta$ -Partial),  $\beta = 1, \dots, \ell-1$ :** There exist  $\beta$  indices  $i_1, \dots, i_\beta \in \{0, \dots, \ell-1\}$  such that:

- For each  $i \in \{i_1, \dots, i_\beta\}$ ,  $\mathcal{Q}_E$  contains exactly one of  $(k, (\text{iv}\|t) + i, y[i+1])$  or  $(k', (\text{iv}\|t) + i, y[i+1])$ , so one value is fixed to  $y[i+1]$  and the other is unassigned.
- For all other  $i$ , neither entry is in  $\mathcal{Q}_E$ , so both are unassigned.

There are  $2\ell - \beta$  unassigned positions, so

$$\Pr \left[ \bigwedge_{i=0}^{\ell-1} E(k, (\text{iv}\|t) + i) = y[i+1] \wedge E(k', (\text{iv}\|t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E \right] \leq \left( \frac{1}{2^{n-1}} \right)^{2\ell-\beta} \leq \frac{1}{2^{\ell(n-1)}}. \quad (72)$$

Each Type 3 tuple corresponds to at least one query in  $\mathcal{Q}_E$ , with up to  $\ell$  possible positions and at most  $2^\lambda$  choices for  $k'$ . Thus the number of Type 3 tuples is at most  $\ell T 2^\lambda$ , and

$$\Pr [\exists \text{ Type 3 tuple } (k, k', (t, y)) : G(k, \text{iv}, t) = G(k', \text{iv}, t) = y] \leq \frac{\ell T 2^\lambda}{2^{\ell(n-1)}}. \quad (73)$$

- **Type 4 (Blank):** For all  $i \in \{0, \dots, \ell-1\}$ , neither  $(k, (\text{iv}\|t) + i, \star)$  nor  $(k', (\text{iv}\|t) + i, \star)$  appears in  $\mathcal{Q}_E$ . For each such tuple and each  $i$ ,

$$\begin{aligned} \Pr [E(k, (\text{iv}\|t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E] &\leq \frac{1}{2^{n-1}}, \\ \Pr [E(k', (\text{iv}\|t) + i) = y[i+1] \mid E \vdash \mathcal{Q}_E] &\leq \frac{1}{2^{n-1}}. \end{aligned}$$

Thus the probability that  $G(k, \text{iv}, t) = G(k', \text{iv}, t) = y$  is at most  $1/2^{2\ell(n-1)}$ . There are at most  $(D 2^\lambda)^2$  such tuples, so

$$\begin{aligned} &\Pr [\exists \text{ Type 4 tuple } (k, k', (t, y)) : G(k, \text{iv}, t) = G(k', \text{iv}, t) = y] \\ &\leq \left( \frac{D 2^\lambda}{2^{\ell(n-1)}} \right)^2. \end{aligned} \quad (74)$$

Summing (66), (71), (73), and (74) gives

$$\begin{aligned} &\Pr [\exists (k, k', (t, y)) : G(k, \text{iv}, t) = G(k', \text{iv}, t) = y \mid E \vdash \mathcal{Q}_E] \\ &\leq 0 + \frac{2T^2 \ell^\ell}{2^{\ell(n-1)}} + \frac{\ell T 2^\lambda}{2^{\ell(n-1)}} + \frac{D^2 2^{2\lambda}}{2^{2\ell(n-1)}} \\ &\leq \frac{2T^2 \ell^\ell + \ell T 2^\lambda + D^2 2^{2\lambda}}{2^{\ell(n-1)}}. \end{aligned} \quad (75)$$

Combining with the bad-event bound (62) yields the final bound (60).  $\square$

$$\Pr [\text{Inj}_{G, \mathcal{B}} \text{ is bad}] \leq \frac{2T^2}{2^{\ell(n-1)}}. \quad (76)$$

Equation (76) bounds the probability that  $\mathcal{A}$  can find records in its query list that violate injectivity. We thus obtain the following corollary.

**Theorem 8.10.** *Let  $\mathcal{A}^E$  be any adversary that makes  $T$  queries to the ideal cipher  $E : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^n$  and produces a query list  $\mathcal{Q}_E$ . Then*

$$\Pr [\mathcal{A}^E \Rightarrow \mathcal{Q}_E, \exists ((L, u + i, v_i), (L', u + i, v_i))_{i=0}^{\ell-1} \in \mathcal{Q}_E^{2\ell}] \leq \frac{2T^2}{2^{\ell(n-1)}}. \quad (77)$$

For the case  $\ell n = 2\lambda$ , the bound simplifies to

$$\text{Adv}_G^{\text{Inj}}(\mathcal{A}) \leq \frac{2^{\ell+1} T^2}{2^{2\lambda}}. \quad (78)$$

**Intuition for quantum almost injectivity.** The quantum analogue of almost injectivity can be obtained by modifying Definition 8.4 to allow the adversary  $\mathcal{A}^{[E]}$  having access to the ideal cipher implemented as a quantum oracle—but  $\mathcal{A}^{[E]}$  still produce classical outputs  $\text{iv} \in \{0,1\}^\lambda$  and  $D$  tuples  $(t_1, y_1), \dots, (t_D, y_D)$ . Denote this security game by  $\text{QInj}_{G,\mathcal{A}}$ , and define

$$\text{Adv}_G^{\text{QInj}}(\mathcal{A}) := \Pr[\text{QInj}_{G,\mathcal{A}} = 1]. \quad (79)$$

Regarding the PRG construction  $G^E(k, \text{iv}, t) := E(k, \text{iv}||t) \| E(k, (\text{iv}||t)+1) \| \dots \| E(k, (\text{iv}||t)+\ell-1)$ , we have the following conjecture on its quantum almost injective bound.

**Conjecture 8.1** (Quantum Almost Injectivity Bound for CTR-based PRG). *Consider any adversary  $\mathcal{A}^{[E]}$  that makes  $T$  superposition queries to the quantum ideal cipher  $E : \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^n$  and outputs  $D$  tuples  $(t_1, y_1), \dots, (t_D, y_D)$  in the first phase of  $\text{QInj}_{G,\mathcal{A}}$ . Then*

$$\text{Adv}_G^{\text{QInj}}(\mathcal{A}) \leq O\left(\frac{T^4}{2^n} + \frac{\ell T 2^\lambda + D 2^\lambda}{2^{\ell(n-1)}}\right). \quad (80)$$

For the case  $\ell n = c = 2\lambda$ , the above bound simplifies to

$$\text{Adv}_G^{\text{QInj}}(\mathcal{A}) \leq O\left(\frac{T^4}{2^n} + \frac{2^\ell(\ell T + D)}{2^\lambda}\right). \quad (81)$$

A similar conjecture has been made in FAESTv2 [BBB+26]. However, in FAESTv2, the authors heuristically treat ideal-cipher queries and PRG queries as the same and ignore backward queries, which may potentially be used by the adversary to break the almost injective PRG. Based on this treatment, they give a concrete security bound for a specific attack (see [BBB+26, page 71]). This bound does not explicitly consider the quantum setting. In contrast, we provide more intuitions to justify this almost injectiveness in the QICM as follows.

We serve intuitions for this conjectured bound below. The classical bound given in Eq. (60) consists of two terms  $\frac{2T^2(\ell+1)}{2^{\ell(n-1)}}$  and  $\frac{\ell T 2^\lambda + D 2^\lambda}{2^{\ell(n-1)}}$ : the former reflects the probability that the classical adversary  $\mathcal{A}^E$  “finds” (and thus can output) such two keys  $k, k'$  using his  $T$  (classical) ideal cipher queries, while the latter reflects the probability that without enough knowledge,  $\mathcal{A}^E$  succeeds in “guessing” such two keys  $k, k'$ . Intuitively, the latter only depends on the randomness of the ideal cipher and thus should be the same in the quantum setting.

To bound the former in the quantum setting, note that  $\mathcal{A}^{[E]}$  outputs such two keys  $k, k'$  (using his  $T$  quantum ideal cipher queries) only if  $\mathcal{A}^{[E]}$  can output two distinct inputs  $(k, x)$  and  $(k', x')$  such that  $k \neq k', x = x'$  and  $E(k, x) = E(k', x')$ . The probability is easy to bound in the classical setting, and a recent work of Cojocaru et al. [CHL+25] proved theorem lifting evasiveness w.r.t. classical ideal ciphers to that w.r.t. quantum ideal ciphers.

**Theorem 8.11** ([CHL+25], Theorem 4.1). *Let  $\mathcal{A}^{E, E^{-1}}$  be a quantum algorithm that makes  $T$  quantum queries to an (invertible) ideal cipher oracle  $E$  and  $R$  is a relation on  $(\{0,1\}^b)^m \times (\{0,1\}^b)^m$ . Then there exists an algorithm  $\mathcal{B}^{E, E^{-1}}$  making at most  $m$  classical queries such that*

$$\begin{aligned} \Pr_E[\mathcal{B}^E \Rightarrow ((k_1, x_1), \dots, (k_m, x_m)) : (((k_1, x_1), \dots, (k_m, x_m)), (E(k_1, x_1), \dots, E(k_m, x_m))) \in R] \\ \geq \frac{\left(1 - \frac{m^2}{2^b}\right)}{(8T+1)^{2m}} \cdot \Pr_E[\mathcal{A}^{[E]} \Rightarrow ((k_1, x_1), \dots, (k_m, x_m)) : \\ (((k_1, x_1), \dots, (k_m, x_m)), (E(k_1, x_1), \dots, E(k_m, x_m))) \in R]. \end{aligned} \quad (82)$$

By these, let  $R_{\text{pair}}$  be the relation such that

$$(((k_1, x_1), (k_2, x_2)), (E(k_1, x_1), E(k_2, x_2))) \in R_{\text{pair}} \text{ iff. } k_1 \neq k_2 \wedge x_1 = x_2 \wedge E(k_1, x_1) = E(k_2, x_2).$$

With 2 queries, it clearly holds

$$\Pr_E[\mathcal{B}^E \Rightarrow ((k_1, x_1), (k_2, x_2)) : (((k_1, x_1), (k_2, x_2)), (E(k_1, x_1), E(k_2, x_2))) \in R_{\text{pair}}] = O\left(\frac{1}{2^n}\right).$$

It thus holds

$$\Pr_E[\mathcal{A}^{[E]} \Rightarrow ((k_1, x_1), (k_2, x_2)) : (((k_1, x_1), (k_2, x_2)), (E(k_1, x_1), E(k_2, x_2))) \in R_{\text{pair}}] = O\left(\frac{T^4}{2^n}\right).$$

Using this to replace the first term in Eq. (60) yields Eq. (80).

#### 8.4.4 TCCR Security

**Non-quantum TCCR security.** We prove the following theorem.

**Theorem 8.12** (Non-Quantum Single-User TCCR Security of TCCR). *In the ideal cipher model, the hash function  $H^E$  defined above is  $(q_E, q_C, \rho, \varepsilon)$ -tweakable circular correlation robust (single-user), where*

$$\varepsilon \leq \frac{4q_E\mu}{2^\rho} + \frac{q_C(\mu - 1)}{2^n}, \quad (83)$$

and  $\mu$  is the maximum number of construction queries on any fixed tweak (so  $\mu \leq q_C$ ).

*Proof.* Fix a deterministic distinguisher  $\mathcal{D}$  that makes at most  $q_E$  queries to the ideal cipher  $E/E^{-1}$  and at most  $q_C$  queries to the construction oracle. In the real world, the construction oracle is  $\hat{\mathcal{O}}_\Gamma^{\text{tccr}}$  for  $\Gamma \leftarrow \mathcal{R}$ . In the ideal world, the construction oracle is a uniform random function  $\hat{f} : \{0, 1\}^\lambda \times \{0, 1\}^{n-8} \times \{0, 1\} \rightarrow \{0, 1\}^{2n}$ .

**Transcript.** Let  $Q_E$  be the transcript of ideal cipher queries, consisting of entries  $(k, x, y)$  recording  $E(k, x) = y$ . Let  $Q_O$  be the transcript of construction queries, consisting of entries  $(w, t, b, z)$  recording  $\hat{\mathcal{O}}(w, t, b) = z$ , where  $z = (z_L, z_R) \in \{0, 1\}^{2n}$ . Let  $\Gamma$  denote the key. The full transcript is  $\tau = (Q_E, Q_O, \Gamma)$ .

**Bad Transcripts.** We say a transcript  $\tau = (Q_E, Q_O, \Gamma)$  is *bad* if either of the following conditions holds:

- **(B-1)** There exists  $(w, t, b, z) \in Q_O$  and  $(k, x, y) \in Q_E$  such that one of the following four sub-cases occurs:
  1. (a)  $k = k(w_R, t)$  and  $x = p_0(w_L)$ ;
  2. (b)  $k = k(w_R, t)$  and  $x = p_1(w_L)$ ;
  3. (c)  $k = k(w_R, t)$  and  $y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_L$ ;
  4. (d)  $k = k(w_R, t)$  and  $y = z_R \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_R$ .
- **(B-2)** There exist *distinct*  $(w, t, b, z), (w', t', b', z') \in Q_O$  with  $w_R = w'_R$  and  $t = t'$  such that either:
  1. (a)  $w_L \oplus b \cdot \tilde{\Gamma}_L \oplus z_L = w'_L \oplus b' \cdot \tilde{\Gamma}_L \oplus z'_L$ ;
  2. (b)  $w_L \oplus b \cdot \tilde{\Gamma}_R \oplus z_R = w'_L \oplus b' \cdot \tilde{\Gamma}_R \oplus z'_R$  (in the  $n < \lambda \leq 2n$  case).

**Bounding  $\Pr[(\text{B-1})]$  in the ideal world.** We bound each case of (B-1) for every pair  $((w, t, b, z), (k, x, y)) \in Q_O \times Q_E$ .

**Case (a):**  $k = k(w_R, t) \wedge x = p_0(w_L)$ . This is equivalent to

$$(w_R \oplus \Gamma_R \parallel t \parallel 0^{2n-\lambda-8} = k) \wedge ([0]_8 \parallel (w_L \oplus \Gamma_L) = x). \quad (84)$$

From the first condition we recover  $\Gamma_R = \text{left}_{\lambda-(n-8)}(k) \oplus w_R$ . From the second condition,  $w_L \oplus \Gamma_L = \text{right}_{n-8}(x)$ , hence  $\Gamma_L = \text{right}_{n-8}(x) \oplus w_L$ . Thus  $\Gamma$  is fully determined by  $(k, x)$ , and the probability over  $\Gamma \leftarrow \mathcal{R}$  is at most  $2^{-\rho}$ .

**Case (b):**  $k = k(w_R, t) \wedge x = p_1(w_L)$ . By an identical argument, substituting  $[1]_8$  for  $[0]_8$ :  $\Gamma_R$  is recovered from the key equation, and  $\Gamma_L = \text{right}_{n-8}(x) \oplus w_L$  from the plaintext equation. The probability is bounded by  $2^{-\rho}$ .

**Case (c):**  $k = k(w_R, t) \wedge y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_L$ . We consider two sub-cases based on  $b$ .

*Sub-case  $b = 0$ :* The condition becomes

$$(w_R \oplus \Gamma_R \parallel t \parallel 0^{2n-\lambda-8} = k) \wedge (y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L))).$$

From the first equation we get  $\Gamma_R$ . From the second:

$$(0^8 \parallel (w_L \oplus \Gamma_L)) = z_L \oplus y,$$

so  $\Gamma_L = \text{right}_{n-8}(z_L \oplus y) \oplus w_L$ . Hence  $\Gamma$  is fully determined and the probability is  $\leq 2^{-\rho}$ .

*Sub-case  $b = 1$ :* The condition becomes

$$(w_R \oplus \Gamma_R \parallel t \parallel 0^{2n-\lambda-8} = k) \wedge (y = z_L \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus \tilde{\Gamma}_L).$$

From the first equation we get  $\Gamma_R = \text{left}_{\lambda-(n-8)}(k) \oplus w_R$ . From the second:

$$(0^8 \parallel w_L) \oplus (0^8 \parallel \Gamma_L) \oplus \tilde{\Gamma}_L = z_L \oplus y.$$

Recall that  $\tilde{\Gamma}_L = \text{left}_n(\Gamma \parallel 0^{2n-\lambda})$ , which equals the first  $n$  bits of  $\Gamma$ . Whereas  $\Gamma_L$  provides the first  $n-8$  bits of  $\Gamma$ . Therefore, the first  $n-8$  bits of  $\Gamma$  can be recovered from this equation, by which  $\Gamma$  is fully determined as well. This case thus also holds with probability  $\leq 2^{-\rho}$ .

**Case (d):**  $k = k(w_R, t) \wedge y = z_R \oplus (0^8 \parallel (w_L \oplus \Gamma_L)) \oplus b \cdot \tilde{\Gamma}_R$ .

The analysis is similar to Case (c) (yet simpler). When  $b = 0$ ,  $\Gamma_R$  is determined from the key equation, and  $\Gamma_L = \text{right}_{n-8}(z_R \oplus y) \oplus w_L$  from the output equation. When  $b = 1$ ,  $\Gamma_R$  is again determined from the key equation, and the output equation

$$(0^8 \parallel (w_L \oplus \Gamma_L)) \oplus \tilde{\Gamma}_R = z_R \oplus y$$

uniquely determines  $\Gamma_L$ , because  $\tilde{\Gamma}_R = \text{right}_n(\Gamma \parallel 0^{2n-\lambda})$  contains  $\Gamma_L$ -dependent bits as well as  $\Gamma_R$ -dependent bits (which are already known). In all sub-cases, the probability is  $\leq 2^{-\rho}$ .

We now sum over all pairs. For  $t \in \{0, 1\}^{n-8}$ , define

$$Q_E[t] := \{(k, x, y) \in Q_E \mid \text{the tweak field of } k \text{ equals } t\}.$$

Then by the union bound:

$$\begin{aligned} \Pr[(B-1)] &\leq \sum_{(w,t,b,z) \in Q_O} \frac{4 \cdot |Q_E[t]|}{2^\rho} \\ &= \sum_{t \in \{0,1\}^{n-8}} \sum_{(w,t,b,z) \in Q_O} \frac{4 \cdot |Q_E[t]|}{2^\rho} \\ &\leq \mu \sum_{t \in \{0,1\}^{n-8}} \frac{4 \cdot |Q_E[t]|}{2^\rho} = \frac{4\mu q_E}{2^\rho}. \end{aligned} \tag{85}$$

**Bounding  $\Pr[(\mathbf{B-2})]$  in the ideal world.** Consider distinct  $(w, t, b, z), (w', t', b', z') \in Q_O$  with  $w_R = w'_R$  and  $t = t'$ . In the ideal world,  $z_L, z_R, z'_L, z'_R$  are uniform and independent random values.

*Sub-case (a):* We need

$$w_L \oplus b \cdot \tilde{\Gamma}_L \oplus z_L = w'_L \oplus b' \cdot \tilde{\Gamma}_L \oplus z'_L,$$

or equivalently  $z_L \oplus z'_L = w_L \oplus w'_L \oplus (b \oplus b') \cdot \tilde{\Gamma}_L$ . Conditioned on  $\Gamma$ , the values  $z_L$  and  $z'_L$  are independent and uniform over  $\{0, 1\}^n$ . Hence the probability that this equation holds is exactly  $2^{-n}$ .

*Sub-case (b):* Similarly, for the  $R$  side,

$$w_L \oplus b \cdot \tilde{\Gamma}_R \oplus z_R = w'_L \oplus b' \cdot \tilde{\Gamma}_R \oplus z'_R.$$

Since  $z_R, z'_R$  are independent and uniform, the probability is  $2^{-n}$ .

Let  $C_t$  denote the number of construction queries with fixed tweak  $t$ . By union bound:

$$\begin{aligned} \Pr[(\mathbf{B-2})] &\leq \sum_{t \in \{0,1\}^{n-8}} \binom{C_t}{2} \cdot \frac{2}{2^n} \\ &\leq \sum_{t \in \{0,1\}^{n-8}} \frac{C_t(\mu-1)}{2} \cdot \frac{2}{2^n} = \frac{q_C(\mu-1)}{2^n}. \end{aligned} \quad (86)$$

**Analysis of Good Transcripts.** Fix a good transcript  $\tau = (Q_E, Q_O, \Gamma)$ . We compare the probabilities that  $\tau$  occurs in the real and ideal worlds.

*Ideal world.* The probability of obtaining  $\tau$  is:

$$\Pr_{\text{id}}[\tau] = \frac{1}{2^\rho} \cdot \prod_{j=1}^m \prod_{\ell=1}^{q_{E,j}} \frac{1}{2^n - \ell + 1} \cdot \frac{1}{2^{2n \cdot q_C}},$$

where  $m$  is the number of distinct keys appearing in  $Q_E$ , and  $q_{E,j}$  is the number of entries under the  $j$ -th key.

*Real world.* The probability is:

$$\Pr_{\text{re}}[\tau] = \frac{1}{2^\rho} \cdot \prod_{j=1}^m \prod_{\ell=1}^{q_{E,j}} \frac{1}{2^n - \ell + 1} \cdot p_O,$$

where

$$p_O := \Pr[\forall (w, t, b, z) \in Q_O : \hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w, t, b) = z \mid E \vdash Q_E].$$

For each construction query  $(w_i, t_i, b_i, z_i)$ , define:

$$p_i := \Pr[\hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w_i, t_i, b_i) = z_i \mid E \vdash Q_E \wedge \forall j < i : \hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w_j, t_j, b_j) = z_j].$$

Recall that  $\hat{\mathcal{O}}_\Gamma^{\text{tccr}}(w_i, t_i, b_i) = z_i$  requires:

$$z_{i,L} = E(k(w_{i,R}, t_i), p_0(w_{i,L})) \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_L, \quad (87)$$

$$z_{i,R} = E(k(w_{i,R}, t_i), p_1(w_{i,L})) \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_R. \quad (88)$$

Since the transcript is good, (B-1) does not occur, so neither  $E(k(w_{i,R}, t_i), p_0(w_{i,L}))$  nor  $E(k(w_{i,R}, t_i), p_1(w_{i,L}))$  is already in  $Q_E$ , nor are their inverses. Moreover, (B-2) does not occur, so these values are not determined by previous construction queries.

The ideal cipher  $E$  is a random permutation for each key. At the time of the  $i$ -th construction query, for each of the two  $E$  calls under  $k_i$ , there are at most  $q_E + i - 1$  already-determined

input-output pairs. Hence each evaluation is uniform over at least  $2^n - q_E - i + 1$  possible values, giving

$$p_i \geq \frac{1}{(2^n - q_E - i + 1)^2} \geq \frac{1}{2^{2n}}.$$

Thus  $p_O = \prod_{i=1}^{q_C} p_i \geq 2^{-2n \cdot q_C}$ , and we obtain

$$\frac{\Pr_{\text{re}}[\tau]}{\Pr_{\text{id}}[\tau]} \geq 1.$$

By the H-coefficient lemma [Pat09, CS14], the distinguishing advantage is bounded by

$$\text{Adv} \leq \Pr[(\text{B-1})] + \Pr[(\text{B-2})] \leq \frac{4\mu q_E}{2^\rho} + \frac{q_C(\mu - 1)}{2^n}.$$

This completes the proof.  $\square$

**Subroutine lemmas for the non-quantum proof.** For completeness, we provide self-contained statements and proofs of all supporting lemmas that verify the recovery arguments used in the main proof.

**Lemma 8.15** (H-Coefficient Lemma [Pat09, CS14]). *Let  $\mathcal{D}$  be a deterministic distinguisher interacting with either the real world or the ideal world. Let  $T_{\text{re}}$  and  $T_{\text{id}}$  denote the probability distributions on transcripts induced by the real and ideal worlds, respectively. Partition the set of attainable transcripts into  $\mathcal{T}_1$  (good) and  $\mathcal{T}_2$  (bad). If there exists  $\varepsilon_1 \in [0, 1)$  such that for all  $\tau \in \mathcal{T}_1$ ,*

$$\frac{\Pr[T_{\text{re}} = \tau]}{\Pr[T_{\text{id}} = \tau]} \geq 1 - \varepsilon_1,$$

*then the distinguishing advantage is bounded by*

$$\Delta(T_{\text{re}}, T_{\text{id}}) \leq \varepsilon_1 + \Pr[T_{\text{id}} \in \mathcal{T}_2].$$

*Proof.* See [CS14, Section 3.3] for the standard proof.  $\square$

**Lemma 8.16** (Real-World Conditional Probability for Good Transcripts). *For a good transcript  $\tau$  in the  $n < \lambda \leq 2n$  case, for each construction query  $i$ ,*

$$p_i := \Pr[\hat{\mathcal{O}}_{\Gamma}^{\text{tccr}}(w_i, t_i, b_i) = z_i \mid E \vdash Q_E \wedge \forall j < i : \hat{\mathcal{O}}_{\Gamma}^{\text{tccr}}(w_j, t_j, b_j) = z_j] \geq \frac{1}{(2^n - q_E - i + 1)^2}.$$

*Proof.* The event  $\hat{\mathcal{O}}_{\Gamma}^{\text{tccr}}(w_i, t_i, b_i) = z_i$  is equivalent to:

$$E(k_i, p_{0,i}) = z_{i,L} \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_L, \quad (89)$$

$$E(k_i, p_{1,i}) = z_{i,R} \oplus (0^8 \parallel (w_{i,L} \oplus \Gamma_L)) \oplus b_i \cdot \tilde{\Gamma}_R, \quad (90)$$

where  $k_i := k(w_{i,R}, t_i)$ ,  $p_{0,i} := p_0(w_{i,L})$ ,  $p_{1,i} := p_1(w_{i,L})$ .

Since the transcript is good, (B-1) does not occur. Hence neither  $(k_i, p_{0,i})$  nor  $(k_i, p_{1,i})$  appears as a forward query in  $Q_E$ , nor do the corresponding inverse entries appear. Furthermore, (B-2) does not occur, so these  $E$  values are not forced by previous construction queries.

The ideal cipher  $E$  is a random permutation for each key. At the time of the  $i$ -th construction query, for each of the two  $E$  calls, there are at most  $q_E + i - 1$  already-determined input-output pairs under  $k_i$ . Hence each evaluation is uniform over at least  $2^n - q_E - i + 1$  possible values, giving probability at least  $1/(2^n - q_E - i + 1)$  for each  $E$  call. Since the two calls are on the same key  $k_i$  but with distinct inputs  $p_{0,i} \neq p_{1,i}$  (they differ in the first 8 bits), they are independent uniform choices from the remaining permutation domain. Therefore,

$$p_i \geq \frac{1}{(2^n - q_E - i + 1)^2} \geq \frac{1}{2^{2n}} \quad (\text{assuming } q_E \ll 2^n).$$

$\square$

**Corollary 8.1** (Overall Advantage Bound). *The overall distinguishing advantage for the single-user TCCR security of TCCR in the non-quantum ideal cipher model is*

$$\text{Adv}_{H^E}^{\text{tccr}}(q_E, q_C) \leq \frac{4\mu q_E}{2^\rho} + \frac{q_C(\mu - 1)}{2^n}.$$

**Quantum Q1 TCCR security.** We will prove the following theorem step-by-step in the subsequent paragraphs.

**Theorem 8.13** (Quantum Q1 TCCR Security Bound for TCCR). *The hash function TCCR is  $(T, D, \chi, \varepsilon)$ -quantum TCCR, where*

$$\text{AdvTCCR}_{\mathcal{A}^{\text{TCCR}}}^{E, \text{TCCR}}(T, D, \chi) \leq O\left(\frac{D\sqrt{T}}{2^\rho} + \frac{T\sqrt{D}}{2^\rho} + \frac{D}{2^{\lambda-n}}\right), \quad (91)$$

where  $\rho$  is the min-entropy of the key distribution  $\chi$ ,  $D$  is the number of classical construction queries, and  $T$  is the total number of quantum ideal cipher queries.

**Toolkit for quantum proofs.** We recall the necessary quantum cryptography primitives, following the same presentation as in the original quantum proof.

**Notations and definitions.** We define the swap operator  $\text{swap}_{a,b} : \mathcal{X} \rightarrow \mathcal{X}$  as

$$\text{swap}_{a,b}(x) = \begin{cases} a, & \text{if } x = b, \\ b, & \text{if } x = a, \\ x, & \text{otherwise.} \end{cases}$$

Given a function  $F : \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ , we let  $|F\rangle$  denote the appropriate quantum oracle  $|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus F(x)\rangle$  for making quantum queries to  $F$ .

**A reprogramming lemma.** For a function  $F : \{0, 1\}^\ell \rightarrow \{0, 1\}^n$  and a set  $B \subset \{0, 1\}^\ell \times \{0, 1\}^n$  such that each  $x \in \{0, 1\}^\ell$  is the first element of at most one tuple in  $B$ , define

$$F^{(B)}(x) := \begin{cases} y & \text{if } (x, y) \in B, \\ F(x) & \text{otherwise.} \end{cases}$$

**Lemma 8.17** (Reprogramming Lemma [ABKM22]). *Let  $\mathcal{D}$  be a quantum distinguisher in the following experiment:*

**Phase 1:**  $\mathcal{D}$  outputs descriptions of a function  $F_0 = F : \{0, 1\}^\ell \rightarrow \{0, 1\}^n$  and a randomized algorithm  $\mathcal{B}$  whose output is a set  $B \subset \{0, 1\}^\ell \times \{0, 1\}^n$  where each  $x \in \{0, 1\}^\ell$  is the first element of at most one tuple in  $B$ . Let  $B_1 = \{x \mid \exists y : (x, y) \in B\}$  and  $\varepsilon = \max_{x \in \{0, 1\}^\ell} \{\Pr_{B \leftarrow \mathcal{B}}[x \in B_1]\}$ .

**Phase 2:**  $\mathcal{B}$  is run to obtain  $B$ . Let  $F_1 = F^{(B)}$ . A uniform bit  $b$  is chosen, and  $\mathcal{D}$  is given quantum access to  $F_b$ .

**Phase 3:**  $\mathcal{D}$  loses access to  $F_b$ , and receives the randomness  $r$  used to invoke  $\mathcal{B}$  in phase 2. Then  $\mathcal{D}$  outputs a guess  $b'$ .

For any  $\mathcal{D}$  making  $q$  queries in expectation when its oracle is  $F_0$ , it holds that

$$|\Pr[\mathcal{D} \text{ outputs } 1 \mid b = 1] - \Pr[\mathcal{D} \text{ outputs } 1 \mid b = 0]| \leq 2q \cdot \sqrt{\varepsilon}.$$

**Quantum ideal cipher and resampling.** Define  $\mathcal{E}(m, n)$  to be the space of all keyed permutations where the key  $k \in \{0, 1\}^m$  and the input  $x \in \{0, 1\}^n$ . In the quantum ideal cipher model (QICM), we treat the underlying block cipher as an ideal cipher  $E \leftarrow \mathcal{E}(m, n)$ . A quantum distinguisher in the QICM can apply the unitary operators

$$|k\rangle|x\rangle|y\rangle \mapsto |k\rangle|x\rangle|E_k(x) \oplus y\rangle, \quad (92)$$

$$|k\rangle|x\rangle|y\rangle \mapsto |k\rangle|x\rangle|E_k^{-1}(x) \oplus y\rangle. \quad (93)$$

**Lemma 8.18** (Resampling Lemma [ABMS26]). *Let  $\mathcal{D} = (\mathcal{D}_0, \mathcal{D}_1)$  be a quantum distinguisher interacting with the following experiment:*

**Phase 1:** *Choose a uniform ideal cipher  $E \in \mathcal{E}(m, n)$  and give  $\mathcal{D}$  quantum access to  $E$  and  $E^{-1}$ . Then  $\mathcal{D}_0$  outputs a distribution  $D$  over  $\{0, 1\}^{m+2n}$ .*

**Phase 2:** *Sample  $(k_0, s_0, s_1) \in \{0, 1\}^m \times \{0, 1\}^n \times \{0, 1\}^n$  according to  $D$ . Define  $E^{(0)} = E$  and  $E^{(1)}$  as*

$$E_{k^*}^{(1)}(x) = \begin{cases} E_{k^*}(x), & \text{if } k^* \neq k_0, \\ E_{k^*} \circ \text{swap}_{s_0, s_1}(x), & \text{if } k^* = k_0, \end{cases}$$

where  $\text{swap}_{s_0, s_1}$  is the transposition swapping  $s_0$  and  $s_1$ . A uniform bit  $b \in \{0, 1\}$  is chosen, and  $\mathcal{D}$  is given  $k_0, s_0, s_1$  along with quantum access to  $E^{(b)}$ . Finally,  $\mathcal{D}$  outputs a guess  $b'$ .

For a distribution  $D$  on  $\{0, 1\}^{m+2n}$ , define

$$\varepsilon = \max_{(k_0^*, s_0^*, s_1^*) \in \{0, 1\}^{m+2n}} D(k_0^*, s_0^*, s_1^*).$$

Then for any distinguisher  $\mathcal{D}$  making at most  $q$  queries to  $E$  in Phase 1, it holds that

$$|\Pr[\mathcal{D} \text{ outputs } 1 \mid b = 1] - \Pr[\mathcal{D} \text{ outputs } 1 \mid b = 0]| \leq 4\sqrt{2^n \cdot q \cdot \varepsilon}.$$

**Abstract model for the quantum proof.** We consider the case  $n < \lambda \leq 2n$  (the case  $\lambda = n$  follows analogously with a single  $E$ -call per hash evaluation). For a key  $\Delta \in \{0, 1\}^\lambda$ , define  $\Delta_L := \text{left}_{n-8}(\Delta)$  and  $\Delta_R := \text{right}_{\lambda-(n-8)}(\Delta)$ .

The TCCR construction oracle for TCCR returns:

$$\mathcal{O}_{H, \Delta}^{\text{tccr}}(w, \text{iv}, b) = z_L \parallel z_R,$$

where, using the expanded  $2n$ -bit formulation:

$$\begin{aligned} z_L &= E\left((w_R \oplus \Delta_R) \parallel \text{iv} \parallel 0^{2n-\lambda-8}, [0]_8 \parallel (w_L \oplus \Delta_L)\right) \oplus (0^8 \parallel (w_L \oplus \Delta_L)) \oplus b \cdot \tilde{\Delta}_L, \\ z_R &= E\left((w_R \oplus \Delta_R) \parallel \text{iv} \parallel 0^{2n-\lambda-8}, [1]_8 \parallel (w_L \oplus \Delta_L)\right) \oplus (0^8 \parallel (w_L \oplus \Delta_L)) \oplus b \cdot \tilde{\Delta}_R. \end{aligned} \quad (94)$$

Here  $\tilde{\Delta}$  is the  $2n$ -bit expansion of  $\Delta$ , and  $\tilde{\Delta}_L, \tilde{\Delta}_R$  are the left and right  $n$ -bit halves.

The two  $E$  calls share the *same* key:

$$k(w, \text{iv}, \Delta) := (w_R \oplus \Delta_R) \parallel \text{iv} \parallel 0^{2n-\lambda-8}.$$

They are differentiated by their *plaintexts*:

$$p_0(w_L, \Delta_L) := [0]_8 \parallel (w_L \oplus \Delta_L), \quad (95)$$

$$p_1(w_L, \Delta_L) := [1]_8 \parallel (w_L \oplus \Delta_L). \quad (96)$$

**The modified cipher.** For any list  $T_j = \{(w_i, \text{iv}_i, b_i, z_i)\}_{i=1}^j$  of classical construction query/answer pairs and key  $\Delta$ , we define the *modified cipher*  $E_\Delta^{T_j}$  as follows.

Set  $E_\Delta^{T_0} = E$ . For  $j \geq 1$ :

$$E_\Delta^{T_j}(x) = \begin{cases} E_k(x) & \text{if } k \neq \mathbf{k}(w_i, \text{iv}_i, \Delta) \text{ for all } i \in [j], \\ E_{\Delta, k}^{T_j}(x) & \text{if } k = \mathbf{k}(w_i, \text{iv}_i, \Delta) \text{ for some } i \in [j], \end{cases} \quad (97)$$

where  $E_{\Delta, k}^{T_j}$  is constructed recursively:  $E_\Delta^{T_j}$  is obtained from  $E_\Delta^{T_{j-1}}$  by performing two swap operations on the relevant key  $k = \mathbf{k}(w_j, \text{iv}_j, \Delta)$  to ensure consistency with the  $j$ -th construction query/answer pair. Concretely,

$$E_\Delta^{T_j} = \text{swap}_{s_{j,0}, s'_{j,0}} \circ \text{swap}_{s_{j,1}, s'_{j,1}} \circ E_\Delta^{T_{j-1}}, \quad (98)$$

where the swap points  $s_{j,0} = \mathbf{p}_0(w_{j,L}, \Delta_L)$  and  $s_{j,1} = \mathbf{p}_1(w_{j,L}, \Delta_L)$  are the two plaintext inputs, and  $s'_{j,0}, s'_{j,1}$  are chosen so that the modified cipher is consistent with the construction output  $z_j$ . Specifically:

$$s'_{j,0} := z_{j,L} \oplus (0^8 \parallel (w_{j,L} \oplus \Delta_L)) \oplus b_j \cdot \tilde{\Delta}_L, \quad (99)$$

$$s'_{j,1} := z_{j,R} \oplus (0^8 \parallel (w_{j,L} \oplus \Delta_L)) \oplus b_j \cdot \tilde{\Delta}_R. \quad (100)$$

Since  $\mathbf{p}_0$  and  $\mathbf{p}_1$  are distinct (they differ in the first 8 bits), the two swap operations are on distinct domain elements under the same key, and they commute. For compactness, we write  $E^j$  in place of  $E^{T_j, \Delta}$  when  $T_j$  and  $\Delta$  are clear from context.

**Hybrid experiments.** We define a sequence of hybrid experiments

$$H_0, H_0^1, H_0^2, H_0^3, H_0^4, H_1, H_1^1, \dots, H_{D-1}^4, H_D, \quad (101)$$

where each experiment begins by sampling uniform  $E \in \mathcal{E}(\lambda, n)$  and key  $\Delta \leftarrow \chi$ .

**Experiment  $H_j$**  (for  $j = 0, \dots, D$ ):

1. Run  $\mathcal{A}$ , answering its first  $j$  classical construction queries using  $f$  (a random function) and its quantum queries using  $|E\rangle$ . Stop immediately before the  $(j+1)$ -st classical query. Let  $T_j$  be the transcript of the first  $j$  classical construction queries.
2. For the remainder of the execution, answer classical construction queries using  $\mathcal{O}_{H^{E^{T_j, \Delta}}, \Delta}^{\text{tccr}}$ , and quantum queries using  $|E^{T_j, \Delta}\rangle$ .

**Experiment  $H_j^1$ :**

1. Run  $\mathcal{A}$ , answering its classical construction queries using  $f$  and quantum queries using  $|E\rangle$ , until  $\mathcal{A}$  makes its  $(j+1)$ -st classical construction query (assumed forward without loss of generality). Let  $T_j$  be the transcript so far.
2. Define

$$S := \{\mathbf{p}_0(w_{\ell,L}, \Delta_L), \mathbf{p}_1(w_{\ell,L}, \Delta_L) : \ell \in \{1, \dots, j\} \wedge (w_{\ell,R}, \text{iv}_\ell) = (w_{j+1,R}, \text{iv}_{j+1})\}. \quad (102)$$

Choose uniform  $s \leftarrow_{\$} \{0, 1\}^n \setminus S$ , and define  $E^{(1)}$  as

$$E_{k^*}^{(1)}(x) := \begin{cases} E_{k^*}(x) & \text{if } k^* \neq \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta), \\ E_{k^*} \circ \text{swap}_{\mathbf{p}_0(w_{j+1,L}, \Delta_L), s}(x) & \text{if } k^* = \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta). \end{cases} \quad (103)$$

Continue running  $\mathcal{A}$ , answering its remaining classical queries (including the  $(j+1)$ -st) using  $\mathcal{O}_{H^{(E^{(1)})^{T_j}, \Delta}}^{\text{tccr}}$ , and its quantum queries using  $|E^{(1)T_j}\rangle$ .

**Experiment  $H_j^2$ :**

1. Run  $\mathcal{A}$  as in  $H_j^1$  step 1, up to the  $(j+1)$ -st classical query.
2. Recall  $S$  from  $H_j^1$ . Choose uniform  $s \leftarrow \{0, 1\}^n \setminus S$ , and define  $E^{(2)}$  as

$$E_{k^*}^{(2)}(x) := \begin{cases} E_{k^*}(x) & \text{if } k^* \neq k(w_{j+1}, \text{iv}_{j+1}, \Delta), \\ E_{k^*} \circ \text{swap}_{\mathbf{p}_1(w_{j+1}, L, \Delta_L), s}(x) & \text{if } k^* = k(w_{j+1}, \text{iv}_{j+1}, \Delta). \end{cases} \quad (104)$$

Continue running  $\mathcal{A}$ , answering its remaining classical queries using  $\mathcal{O}_{\mathbf{H}(E^{(2)})^{T_j}, \Delta}^{\text{tccr}}$ , and its quantum queries using  $|E^{(2)}\rangle^{T_j}$ .

**Experiment  $H_j^3$ :**

1. Run  $\mathcal{A}$  as above. Let  $T_{j+1}$  be the list of classical queries through the  $(j+1)$ -st, and  $T_j$  the subset of the first  $j$ .
2. Construct  $E_{j+1} := E_{\Delta}^{T_{j+1}}$  (the modified cipher consistent with  $T_{j+1}$ ). Continue running  $\mathcal{A}$ , answering its remaining classical construction queries using  $\mathcal{O}_{\mathbf{H}^{E_{j+1}}, \Delta}^{\text{tccr}}$  and quantum queries using  $|E_{j+1}\rangle$ .

**Experiment  $H_j^4$ :**

1. Run  $\mathcal{A}$  as above, through the  $(j+1)$ -st classical query. Let  $T_{j+1}$  be the transcript.
2. For the remainder, answer classical queries using  $\mathcal{O}_{\mathbf{H}^{E^{T_{j+1}}}, \Delta}^{\text{tccr}}$  and quantum queries using  $|E^{T_{j+1}}\rangle$ .

**Distinguishability lemmas.** We establish bounds on the distinguishability of consecutive hybrids.

**Lemma 8.19.**  $|\Pr[\mathcal{A}(H_j) = 1] - \Pr[\mathcal{A}(H_j^1) = 1]| \leq 4\sqrt{\frac{T}{2^p}}.$

**Lemma 8.20.**  $|\Pr[\mathcal{A}(H_j^1) = 1] - \Pr[\mathcal{A}(H_j^2) = 1]| \leq 4\sqrt{\frac{T}{2^p}}.$

**Lemma 8.21.**  $H_j^2 = H_j^3.$

**Lemma 8.22.**  $|\Pr[\mathcal{A}(H_j^3) = 1] - \Pr[\mathcal{A}(H_j^4) = 1]| \leq \frac{2}{2^{\lambda-n}}.$

**Lemma 8.23.**  $|\Pr[\mathcal{A}(H_j^4) = 1] - \Pr[\mathcal{A}(H_{j+1}) = 1]| \leq 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j+1}{2^p}}$ , where  $T_{j+1}$  is the expected number of quantum queries  $\mathcal{A}$  makes between the  $(j+1)$ -st and  $(j+2)$ -nd classical queries.

**Proofs of the lemmas.** We now provide *complete* proofs for all five lemmas.

**Proof of Lemma 8.19.**

*Proof.* The hybrids  $H_j$  and  $H_j^1$  differ only in the quantum oracle provided after the  $j$ -th classical query: in  $H_j$  it is  $E$ , while in  $H_j^1$  it is  $E^{(1)}$ , which resamples the first plaintext point ( $\mathbf{p}_0$ ) relevant to the  $(j+1)$ -st construction query by swapping values.

We construct from  $\mathcal{A}$  a distinguisher  $\mathcal{D}$  for the resampling experiment of Lemma 8.18.  $\mathcal{D}$  operates as follows:

**Phase 1:**  $\mathcal{D}$  is given quantum access to an ideal cipher  $E$ . It samples key  $\Delta \leftarrow \chi$  and runs  $\mathcal{A}$ , answering its quantum queries with  $|E\rangle$  and its classical construction queries using  $\mathcal{O}_{\mathbf{H}^E, \Delta}^{\text{tccr}}$ .

(computed via the ideal cipher), until  $\mathcal{A}$  submits its  $(j+1)$ -st classical construction query  $(w_{j+1}, \text{iv}_{j+1}, b_{j+1})$ .

At that point,  $\mathcal{D}$  constructs a distribution  $D$  on  $\{0, 1\}^\lambda \times \{0, 1\}^n \times \{0, 1\}^n$  as follows. To sample  $(k, s_0, s_1) \leftarrow D$ :

1. Compute  $s_0 := \mathbf{p}_0(w_{j+1,L}, \Delta_L) = [0]_8 \parallel (w_{j+1,L} \oplus \Delta_L)$  and  $k := \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta) = (w_{j+1,R} \oplus \Delta_R) \parallel \text{iv}_{j+1} \parallel 0^{2n-\lambda-8}$ .
2. Construct

$$S := \{\mathbf{p}_0(w_{\ell,L}, \Delta_L), \mathbf{p}_1(w_{\ell,L}, \Delta_L) : \ell \in \{1, \dots, j\} \wedge (w_{\ell,R}, \text{iv}_\ell) = (w_{j+1,R}, \text{iv}_{j+1})\}. \quad (105)$$

3. Sample uniform  $s_1 \leftarrow \{0, 1\}^n \setminus S$ , and output  $(k, s_0, s_1)$ .

**Phase 2:**  $\mathcal{D}$  is given  $(k, s_0, s_1) \leftarrow D$  and quantum oracle access to  $E^{(b)}$ ,  $b \in \{0, 1\}$ .  $\mathcal{D}$  continues running  $\mathcal{A}$ , answering its remaining classical construction queries using  $\mathcal{O}^{\text{tccr}}_{\mathbf{H}^{(E^{(b)})}_{\Delta}, \Delta}^{T_j}$  and quantum queries using  $|(E^{(b)})_{\Delta}^{T_j}\rangle$ .  $\mathcal{D}$  outputs whatever  $\mathcal{A}$  does.

*Simulation correctness.* In Phase 1,  $\mathcal{D}$  perfectly simulates  $H_j$  and  $H_j^1$  for  $\mathcal{A}$  until the  $(j+1)$ -st classical query. If  $b = 0$ ,  $\mathcal{D}$  receives  $E^{(0)} = E$  (the original ideal cipher) and simulates  $H_j$ . If  $b = 1$ ,  $\mathcal{D}$  receives  $E^{(1)}$  with the swap on  $\mathbf{p}_0(w_{j+1,L}, \Delta_L)$  and simulates  $H_j^1$ .

*Parameter bound.* The key  $\Delta$  has min-entropy at least  $\rho$ , and  $s_1$  is sampled from a set of size at least  $2^n - 2j$  (since each of up to  $j$  prior queries with matching  $(w_R, \text{iv})$  contributes at most 2 elements to  $S$ ). When  $2j \leq 2^n/2$ , the quantity  $\varepsilon$  for Lemma 8.18 satisfies

$$\varepsilon = \max_{(k^*, s_0^*, s_1^*)} \Pr_{(k, s_0, s_1) \leftarrow D} [(k, s_0, s_1) = (k^*, s_0^*, s_1^*)] \leq \frac{1}{2^\rho \cdot (2^n - 2j)} \leq \frac{2}{2^{\rho+n}}. \quad (106)$$

Applying Lemma 8.18 yields

$$|\Pr[\mathcal{A}(H_j) = 1] - \Pr[\mathcal{A}(H_j^1) = 1]| \leq 4\sqrt{2^n \cdot T_{j+1} \cdot \frac{2}{2^{\rho+n}}} = 4\sqrt{\frac{2 \cdot T_{j+1}}{2^\rho}} \leq 4\sqrt{\frac{T}{2^\rho}}. \quad (107)$$

This completes the proof.  $\square$

### Proof of Lemma 8.20.

*Proof.* This lemma is the symmetric counterpart of Lemma 8.19, where the second plaintext point  $\mathbf{p}_1$  is resampled instead of  $\mathbf{p}_0$ . The proof follows the identical structure:

We construct a distinguisher  $\mathcal{D}$  for Lemma 8.18 where:

1. In Phase 1,  $\mathcal{D}$  runs  $\mathcal{A}$  until the  $(j+1)$ -st classical query  $(w_{j+1}, \text{iv}_{j+1}, b_{j+1})$ .
2.  $\mathcal{D}$  sets  $s_0 := \mathbf{p}_1(w_{j+1,L}, \Delta_L) = [1]_8 \parallel (w_{j+1,L} \oplus \Delta_L)$  and  $k := \mathbf{k}(w_{j+1}, \text{iv}_{j+1}, \Delta)$ .
3. The set  $S$  is identical to that in the previous proof.
4.  $\mathcal{D}$  samples  $s_1 \leftarrow \{0, 1\}^n \setminus S$  and outputs  $(k, s_0, s_1)$ .
5. In Phase 2,  $\mathcal{D}$  simulates  $\mathcal{A}$  with the modified cipher.

The same bound applies: the key  $\Delta$  has min-entropy  $\rho$ , and  $s_1$  is sampled from a set of size at least  $2^n - 2j$ . Therefore

$$\varepsilon \leq \frac{2}{2^{\rho+n}}, \quad (108)$$

and applying Lemma 8.18 gives

$$|\Pr[\mathcal{A}(H_j^1) = 1] - \Pr[\mathcal{A}(H_j^2) = 1]| \leq 4\sqrt{2^n \cdot T_{j+1} \cdot \frac{2}{2^{\rho+n}}} \leq 4\sqrt{\frac{T}{2^\rho}}. \quad (109)$$

$\square$

### Proof of Lemma 8.21.

*Proof.* We show that  $H_j^2$  and  $H_j^3$  are identical experiments. In  $H_j^2$ , the modified cipher  $(E^{(2)})^{T_j}$  incorporates a single swap on  $\mathbf{p}_1(w_{j+1,L}, \Delta_L)$  under key  $\mathbf{k}(w_{j+1}, \mathbf{iv}_{j+1}, \Delta)$ , and then the classical construction queries are answered using  $\mathcal{O}_{\mathbf{H}^{(E^{(2)})^{T_j}, \Delta}}^{\text{tccr}}$ .

In  $H_j^3$ , we directly construct  $E_{\Delta}^{T_{j+1}}$ , which incorporates both swaps (for  $\mathbf{p}_0$  and  $\mathbf{p}_1$ ) into the cipher. Crucially, in  $H_j^2$ , the swap corresponding to  $\mathbf{p}_1$  was already applied to obtain  $(E^{(2)})^{T_j}$  in Phase 2, and the swap for  $\mathbf{p}_0$  was applied in  $H_j^1$  (which is already accounted for). The construction oracle in  $H_j^2$ , when answering the  $(j+1)$ -st query, effectively completes the remaining consistency requirements, which is equivalent to directly computing  $E_{\Delta}^{T_{j+1}}$ .

Formally, let  $E' := (E^{(2)})^{T_j}$  be the cipher in  $H_j^2$  after Phase 2. Then  $E'$  has swaps consistent with all entries in  $T_j$  plus the  $\mathbf{p}_1$  swap from  $H_j^2$ . When the  $(j+1)$ -st construction query is answered in  $H_j^2$ , the response  $z_{j+1}$  is computed using  $\mathcal{O}_{\mathbf{H}^{E'}, \Delta}^{\text{tccr}}$ , which internally evaluates  $E'(k, \mathbf{p}_0)$  and  $E'(k, \mathbf{p}_1)$ . Since  $E'$  already has the  $\mathbf{p}_1$ -swap applied, both evaluations are consistent with the full transcript  $T_{j+1}$ . This is precisely the same state achieved by  $E_{\Delta}^{T_{j+1}}$  in  $H_j^3$ .

Therefore, for all subsequent classical and quantum queries, the oracle distributions are identical, establishing  $H_j^2 = H_j^3$ .  $\square$

### Proof of Lemma 8.22.

*Proof.*  $H_j^3$  and  $H_j^4$  differ only in the response to the  $(j+1)$ -st classical construction query. In  $H_j^4$ , the  $(j+1)$ -st response  $z_{j+1}$  is uniformly distributed in  $\{0, 1\}^{2n}$  (expanded output). In  $H_j^3$ , the response  $z_{j+1} = (z_{j+1,L}, z_{j+1,R})$  satisfies:

$$z_{j+1,L} = y_0 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_L, \quad (110)$$

$$z_{j+1,R} = y_1 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_R, \quad (111)$$

where  $y_0$  and  $y_1$  are obtained from the modified cipher:  $y_0 = (E^{(2)})^{T_j}(k, \mathbf{p}_0)$  and  $y_1 = (E^{(2)})^{T_j}(k, \mathbf{p}_1)$  for  $k = \mathbf{k}(w_{j+1}, \mathbf{iv}_{j+1}, \Delta)$ .

In  $H_j^2$  (and thus  $H_j^3$ ),  $y_0$  is uniformly distributed over a set of size at least  $2^n - 2j$  (the  $\mathbf{p}_0$  swap was applied in  $H_j^1$ , and consistency with prior queries removes at most  $2j$  values). Similarly,  $y_1$  is uniform over a set of size at least  $2^n - 2j$  after the  $H_j^2$  swap. Since XOR with a constant is a bijection, each  $z$ -component is uniform over at least  $2^n - 2j$  values.

The true uniform distribution in  $H_j^4$  is over all  $2^{2n}$  values. However, since the TCCR oracle returns  $\lambda$  bits (truncated from  $2n$ ), the relevant statistical distance per query is between  $\lambda$ -bit strings. When  $\lambda > n$ , the left  $n$  bits contribute fully and the right  $\lambda - n$  bits contribute partially. The worst-case distance is bounded by:

$$|\Pr[\mathcal{A}(H_j^3) = 1] - \Pr[\mathcal{A}(H_j^4) = 1]| \leq \frac{2}{2^{\lambda-n}}, \quad (112)$$

following the standard truncation analysis: the left  $n$  bits are nearly uniform for large  $n$ , and the truncated right  $\lambda - n$  bits contribute at most  $2^{-(\lambda-n)}$  distinguishing power from uniformity.  $\square$

### Proof of Lemma 8.23.

*Proof.*  $H_j^4$  and  $H_{j+1}$  differ in the quantum oracle:  $H_j^4$  uses  $E_{j+1}$  (the modified cipher after  $j+1$  queries), while  $H_{j+1}$  uses  $E$  (the original ideal cipher) for quantum queries between the  $(j+1)$ -st and  $(j+2)$ -nd classical queries.

We construct a distinguisher  $\mathcal{D}$  for the reprogramming lemma (Lemma 8.17):

**Phase 1:**  $\mathcal{D}$  samples uniform  $E \in \mathcal{E}(\lambda, n)$  and key  $\Delta \leftarrow \chi$ . It runs  $\mathcal{A}$ , answering its quantum queries using  $|E\rangle$  and its classical construction queries using random responses, until after it responds to  $\mathcal{A}$ 's  $(j+1)$ -st classical query.

$\mathcal{D}$  defines  $F : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^n$  by  $F(k, x) = E_k(x)$ . It also defines a randomized algorithm  $\mathcal{B}$  that, on input  $\Delta$  (sampled afresh from  $\chi$ ), computes the set  $B$  of input/output pairs to be reprogrammed such that  $F^{(B)}(k, x) = (E_{j+1})_k(x)$  for all keys relevant to the first  $j + 1$  construction queries.

For the  $i$ -th construction query ( $i \leq j + 1$ ), the key is

$$k_i := \mathbf{k}(w_i, \mathbf{iv}_i, \Delta) = (w_{i,R} \oplus \Delta_R) \parallel \mathbf{iv}_i \parallel 0^{2n-\lambda-8}.$$

The two plaintexts are  $\mathbf{p}_0(w_{i,L}, \Delta_L)$  and  $\mathbf{p}_1(w_{i,L}, \Delta_L)$ . For each such pair  $(k_i, \mathbf{p}_a)$  ( $a \in \{0, 1\}$ ), reprogramming is needed to swap its image to the value consistent with the construction output  $z_i$ .

**Phase 2:**  $\mathcal{B}$  is run to generate  $B$ , and  $\mathcal{D}$  is given quantum access to  $F_b$ .  $\mathcal{D}$  resumes running  $\mathcal{A}$ , answering its quantum queries using  $F_b$ . Phase 2 ends before  $\mathcal{A}$  makes its  $(j + 2)$ -nd classical query.

If  $b = 0$ ,  $\mathcal{D}$  provides  $F_0 = F = E$ , simulating  $H_{j+1}$ . If  $b = 1$ ,  $\mathcal{D}$  provides  $F_1 = F^{(B)} = E_{j+1}$ , simulating  $H_j^4$ .

*Computing  $\varepsilon$ .* The reprogramming set  $B_1 = \{x \mid \exists y : (x, y) \in B\}$  contains at most  $2(j + 1)$  points (two per construction query). For a fixed  $(k^*, x^*)$ , the event  $(k^*, x^*) \in B_1$  means there exists  $i \in [j + 1]$  such that:

$$k^* = (w_{i,R} \oplus \Delta_R) \parallel \mathbf{iv}_i \parallel 0^{2n-\lambda-8} \wedge x^* \in \{\mathbf{p}_0(w_{i,L}, \Delta_L), \mathbf{p}_1(w_{i,L}, \Delta_L)\}. \quad (113)$$

From the first equation,  $\Delta_R$  is uniquely determined ( $\Delta_R = \text{left}_{\lambda-(n-8)}(k^*) \oplus w_{i,R}$ ). From the second equation,  $\Delta_L$  is uniquely determined ( $\Delta_L = \text{right}_{n-8}(x^*) \oplus w_{i,L}$ ). Since  $\Delta$  has min-entropy  $\rho$ , the probability that this occurs for a specific  $i$  is at most  $2^{-\rho}$ . By the union bound over  $i \in [j + 1]$ :

$$\varepsilon = \max_{(k^*, x^*)} \Pr_{\Delta \leftarrow \chi} [(k^*, x^*) \in B_1] \leq \frac{2(j + 1)}{2^\rho}. \quad (114)$$

Applying Lemma 8.17 yields:

$$|\Pr[\mathcal{A}(H_j^4) = 1] - \Pr[\mathcal{A}(H_{j+1}) = 1]| \leq 2T_{j+1} \cdot \sqrt{\frac{2(j + 1)}{2^\rho}} = 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j + 1}{2^\rho}}. \quad (115)$$

This completes the proof.  $\square$

**Assembling the hybrid argument.** Using the five lemmas above, we bound the total distinguishing advantage between  $H_0$  and  $H_D$ :

$$\begin{aligned} & |\Pr[\mathcal{A}(H_0) = 1] - \Pr[\mathcal{A}(H_D) = 1]| \\ & \leq \sum_{j=0}^{D-1} \left( 4\sqrt{\frac{T}{2^\rho}} + 4\sqrt{\frac{T}{2^\rho}} + 0 + \frac{2}{2^{\lambda-n}} + 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j + 1}{2^\rho}} \right) \\ & \leq \sum_{j=0}^{D-1} \left( 8\sqrt{\frac{T_{j+1}}{2^\rho}} + \frac{2}{2^{\lambda-n}} + 2\sqrt{2} \cdot T_{j+1} \cdot \sqrt{\frac{j + 1}{2^\rho}} \right) \\ & \leq O\left( \frac{D\sqrt{T}}{2^{\rho/2}} + \frac{T\sqrt{D}}{2^{\rho/2}} + \frac{D}{2^{\lambda-n}} \right), \end{aligned} \quad (116)$$

where  $T = \sum_{j=0}^{D-1} T_{j+1}$  is the total number of quantum queries.

Simplifying to the standard asymptotic form, we obtain the final bound:

$$\text{AdvTCCR}_{\mathcal{A}_{\text{TCCR}}}^{E, \text{TCCR}}(T, D, \chi) \leq O\left(\frac{D\sqrt{T}}{2^\rho} + \frac{T\sqrt{D}}{2^\rho} + \frac{D}{2^{\lambda-n}}\right).$$

This completes the proof of Theorem 8.13.

### Additional lemma: classical indistinguishability detail.

**Lemma 8.24** (Classical Indistinguishability for the New Construction). *For  $j = 0, \dots, D$ , the statistical distance between the  $(j + 1)$ -st construction query response in  $H_j^3$  and  $H_j^4$  is bounded by  $2/2^{\lambda-n}$ .*

*Proof.* In  $H_j^3$ , after the two resampling steps ( $H_j^1$  and  $H_j^2$ ) and the swap consistency step, the  $(j + 1)$ -st response  $z_{j+1}$  has its left component  $z_{j+1,L}$  derived from a value  $y_0$  that is uniform over a set of size at least  $2^n - 2j$ , and its right component  $z_{j+1,R}$  derived from  $y_1$  that is uniform over a set of size at least  $2^n - 2j$ .

Specifically,  $y_0 = (E^{(1)})^{T_j}(k, \mathbf{p}_0(w_{j+1,L}, \Delta_L))$  and  $y_1 = (E^{(2)})^{T_j}(k, \mathbf{p}_1(w_{j+1,L}, \Delta_L))$ . After the two swaps:

- $y_0$  is uniform over  $\{0, 1\}^n \setminus S_0$ , where  $|S_0| \leq 2j$ .
- $y_1$  is uniform over  $\{0, 1\}^n \setminus S_1$ , where  $|S_1| \leq 2j$ .

The construction response is then:

$$z_{j+1,L} = y_0 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_L, \quad (117)$$

$$z_{j+1,R} = y_1 \oplus (0^8 \parallel (w_{j+1,L} \oplus \Delta_L)) \oplus b_{j+1} \cdot \tilde{\Delta}_R. \quad (118)$$

In  $H_j^4$ ,  $z_{j+1}$  is uniform over  $\{0, 1\}^{2n}$  (before truncation to  $\lambda$  bits). The statistical distance per component from uniformity is at most  $2j/2^n$ . Since we ultimately truncate to  $\lambda$  bits, the effective distinguishing power from the deviation in the right  $\lambda - n$  bits is bounded by  $2/2^{\lambda-n}$ .  $\square$

## References

- [ABB<sup>+</sup>26] Carlos Aguilar Melchor, Slim Bettaleb, Loïc Bidoux, Thibault Feneuil, Philippe Gaborit, Nicolas Gama, Shay Gueron, James Howe, Andreas Hülsing, David Joseph, Antoine Joux, Mukul Kulkarni, Edoardo Persichetti, Tovahery H. Randrianarisoa, Matthieu Rivain, and Dongze Yue. SDitHv2 — Syndrome Decoding in the Head. Technical report, National Institute of Standards and Technology, 2026. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-3-additional-signatures>.
- [ABBS25] Amit Agarwal, Carsten Baum, Lennart Braun, and Peter Scholl. Low-bandwidth mixed arithmetic in VOLE-based ZK from low-degree PRGs. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 396–426, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- [ABC<sup>+</sup>26] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, Quynh Dang, Thinh Hung Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status report on the second round of the additional digital signature schemes for the NIST post-quantum cryptography standardization process. NIST Internal Report NIST IR 8610, National Institute of Standards and Technology, Gaithersburg, MD, May 2026.

- [ABKM22] Gorjan Alagic, Chen Bai, Jonathan Katz, and Christian Majenz. Post-quantum security of the Even-Mansour cipher. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 458–487, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [ABMS26] Gorjan Alagic, Chen Bai, Christian Majenz, and Kaiyan Shi. Post-quantum security of block cipher constructions. Cryptology ePrint Archive, Paper 2026/209, 2026.
- [ADI<sup>+</sup>17] Benny Applebaum, Ivan Damgård, Yuval Ishai, Michael Nielsen, and Lior Zichron. Secure arithmetic computation with constant computational overhead. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 223–254, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland.
- [AFS03] Daniel Augot, Matthieu Finiasz, and Nicolas Sendrier. A fast provably secure cryptographic hash function. Cryptology ePrint Archive, Report 2003/230, 2003.
- [AFS05] Daniel Augot, Matthieu Finiasz, and Nicolas Sendrier. A family of fast syndrome based cryptographic hash functions. In *International Conference on Cryptology in Malaysia*, pages 64–83. Springer, 2005.
- [Bar94] Sergey Barg. Some new NP-complete coding problems. *Problems of Information Transmission*, 30(3):209–214, 1994.
- [BBB<sup>+</sup>25] Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. Shorter, tighter, FAESTer: Optimizations and improved (QROM) analysis for VOLE-in-the-head signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part VI*, volume 16005 of *Lecture Notes in Computer Science*, pages 124–156, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.
- [BBB<sup>+</sup>26] Carsten Baum, Lennart Braun, Ward Beullens, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAESTv2. Technical report, National Institute of Standards and Technology, 2026. available at <https://csrc.nist.gov/projects/pqc-dig-sig/round-3-additional-signatures>.
- [BBD<sup>+</sup>23a] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAEST. Technical report, National Institute of Standards and Technology, 2023. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>.
- [BBD<sup>+</sup>23b] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 581–615, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

- [BBM<sup>+</sup>24] Carsten Baum, Ward Beullens, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 463–493, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
- [BCC<sup>+</sup>24] Dung Bui, Eliana Carozza, Geoffroy Couteau, Dahmun Goudarzi, and Antoine Joux. Faster signatures from MPC-in-the-head. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 396–428, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
- [BCG<sup>+</sup>19a] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 291–308, London, UK, November 11–15, 2019. ACM Press.
- [BCG<sup>+</sup>19b] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 489–518, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
- [BCG<sup>+</sup>22] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Nicolas Resch, and Peter Scholl. Correlated pseudorandomness from expand-accumulate codes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 603–633, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 896–912, Toronto, ON, Canada, October 15–19, 2018. ACM Press.
- [Ber10] Daniel J. Bernstein. Grover vs. McEliece. In Nicolas Sendrier, editor, *The Third International Workshop on Post-Quantum Cryptography, PQCRYPTO 2010*, pages 73–80, Darmstadt, Germany, May 25–28, 2010. Springer Berlin Heidelberg, Germany.
- [BFKL94] Avrim Blum, Merrick L. Furst, Michael J. Kearns, and Richard J. Lipton. Cryptographic primitives based on hard learning problems. In Douglas R. Stinson, editor, *Advances in Cryptology – CRYPTO’93*, volume 773 of *Lecture Notes in Computer Science*, pages 278–291, Santa Barbara, CA, USA, August 22–26, 1994. Springer Berlin Heidelberg, Germany.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press.

- [BJMM12] Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in  $2^{n/20}$ : How  $1 + 1 = 0$  improves information set decoding. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 520–536, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [BMvT78] Elwyn R. Berlekamp, Robert J. McEliece, and Henk C. A. van Tilborg. On the inherent intractability of certain coding problems (corresp.). *IEEE Trans. Inf. Theory*, 24(3):384–386, 1978.
- [BØ23] Pierre Briaud and Morten Øygarden. A new algebraic approach to the regular syndrome decoding problem and implications for PCG constructions. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 391–422, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [CCH<sup>+</sup>19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-Shamir: from practice to theory. In Moses Charikar and Edith Cohen, editors, *51st Annual ACM Symposium on Theory of Computing*, pages 1082–1090, Phoenix, AZ, USA, June 23–26, 2019. ACM Press.
- [CCJ23] Eliana Carozza, Geoffroy Couteau, and Antoine Joux. Short signatures from regular syndrome decoding in the head. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 532–563, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [CGW<sup>+</sup>24] Hongrui Cui, Chun Guo, Xiao Wang, Chenkai Weng, Kang Yang, and Yu Yu. AES-based CCR hash with high security and its application to zero-knowledge proofs. Cryptology ePrint Archive, Report 2024/1271, 2024.
- [CHL<sup>+</sup>25] Alexandru Cojocaru, Minki Hhan, Qipeng Liu, Takashi Yamakawa, and Aaram Yun. Quantum lifting for invertible permutations and ideal ciphers. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part II*, volume 16001 of *Lecture Notes in Computer Science*, pages 481–512, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.
- [CLY<sup>+</sup>24] Hongrui Cui, Hanlin Liu, Di Yan, Kang Yang, Yu Yu, and Kaiyi Zhang. ReSolveD: Shorter signatures from regular syndrome decoding and VOLE-in-the-head. In Qiang Tang and Vanessa Teague, editors, *PKC 2024: 27th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 14601 of *Lecture Notes in Computer Science*, pages 229–258, Sydney, NSW, Australia, April 15–17, 2024. Springer, Cham, Switzerland.
- [CS14] Shan Chen and John P. Steinberger. Tight security bounds for key-alternating ciphers. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 327–350, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [DI14] Erez Druk and Yuval Ishai. Linear-time encodable codes meeting the gilbert-varshamov bound and their cryptographic applications. In Moni Naor, editor, *ITCS 2014: 5th Conference on Innovations in Theoretical Computer Science*, pages 169–182, Princeton, NJ, USA, January 12–14, 2014. Association for Computing Machinery.

- [DR00] Joan Daemen and Vincent Rijmen. The block cipher rijndael. In Jean-Jacques Quisquater and Bruce Schneier, editors, *Smart Card Research and Applications*, volume 1820 of *Lecture Notes in Computer Science*, pages 277–284. Springer, 2000.
- [ES24] Andre Esser and Paolo Santini. Not just regular decoding: Asymptotics and improvements of regular syndrome decoding attacks. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VI*, volume 14925 of *Lecture Notes in Computer Science*, pages 183–217, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- [Eur03] European NESSIE cryptography evaluation project. New european schemes for signatures, integrity, and encryption. <https://www.cryptoscam.info/old-projects/nessie/>, (2000–2003).
- [FJR22] Thibault Feneuil, Antoine Joux, and Matthieu Rivain. Syndrome decoding in the head: Shorter signatures from zero-knowledge proofs. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 541–572, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. How to enhance the security of public-key encryption at minimum cost. In Hideki Imai and Yuliang Zheng, editors, *PKC’99: 2nd International Workshop on Theory and Practice in Public Key Cryptography*, volume 1560 of *Lecture Notes in Computer Science*, pages 53–68, Kamakura, Japan, March 1–3, 1999. Springer Berlin Heidelberg, Germany.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany.
- [GKW<sup>+</sup>20] Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 793–822, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
- [GKWY20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press.
- [GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th Annual ACM Symposium on Theory of Computing*, pages 291–304, Providence, RI, USA, May 6–8, 1985. ACM Press.
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 218–229, New York City, NY, USA, May 25–27, 1987. ACM Press.
- [GMW91] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM*, 38(3):691–729, July 1991.

- [Gol01] Oded Goldreich. *Foundations of Cryptography: Basic Tools*, volume 1. Cambridge University Press, Cambridge, UK, 2001.
- [Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *28th Annual ACM Symposium on Theory of Computing*, pages 212–219, Philadelphia, PA, USA, May 22–24, 1996. ACM Press.
- [GWYY25] Chun Guo, Xiao Wang, Kang Yang, and Yu Yu. On tweakable correlation robust hashing against key leakages. *Designs, Codes and Cryptography*, 93(8):3375–3412, 2025.
- [GYW<sup>+</sup>23] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part I*, volume 14004 of *Lecture Notes in Computer Science*, pages 330–362, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [HOSS18] Carmit Hazay, Emmanuela Orsini, Peter Scholl, and Eduardo Soria-Vazquez. TinyKeys: A new approach to efficient multi-party computation. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 3–33, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [Ins25] Institute of Commercial Cryptography Standards (ICCS). Call for proposals for the next-generation public-key cryptographic algorithms. Official Announcement of the Next-generation Commercial Cryptographic Algorithms Program (NGCC), October 2025. Accessed: 2026-04-09.
- [KKL<sup>+</sup>05] Jongsung Kim, Guil Kim, Sangjin Lee, Jongin Lim, and Junghwan Song. Related-key attacks on reduced rounds of shacal-2. In Anne Canteaut and Kapaleeswaran Viswanathan, editors, *Progress in Cryptology - INDOCRYPT 2004*, pages 175–190, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [KSS10] Jonathan Katz, Ji Sun Shin, and Adam Smith. Parallel and concurrent security of the HB and HB+ protocols. *Journal of Cryptology*, 23(3):402–421, July 2010.
- [KW03] Jonathan Katz and Nan Wang. Efficiency improvements for signature schemes with tight security reductions. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent Jaeger, editors, *ACM CCS 2003: 10th Conference on Computer and Communications Security*, pages 155–164, Washington, DC, USA, October 27–30, 2003. ACM Press.
- [LB88] Pil Joong Lee and Ernest F. Brickell. An observation on the security of McEliece’s public-key cryptosystem. In C. G. Günther, editor, *Advances in Cryptology – EUROCRYPT’88*, volume 330 of *Lecture Notes in Computer Science*, pages 275–280, Davos, Switzerland, May 25–27, 1988. Springer Berlin Heidelberg, Germany.
- [LGX25] Heming Liao, Jiangxia Ge, and Rui Xue. Non-adaptive one-way to hiding not only implies adaptive quantum reprogramming, but also does better. Cryptology ePrint Archive, Report 2025/1998, 2025.
- [LWY<sup>+</sup>25] Hanlin Liu, Xiao Wang, Kang Yang, Longhui Yin, and Yu Yu. AGB 2.0: Refined algebraic attack against regular syndrome decoding for PCG applications. Cryptology ePrint Archive, Report 2025/1476, 2025.

- [LWYY24] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. The hardness of LPN over any integer ring and field for PCG applications. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part VI*, volume 14656 of *Lecture Notes in Computer Science*, pages 149–179, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- [MM11] Daniele Micciancio and Petros Mol. Pseudorandom knapsacks and the sample complexity of LWE search-to-decision reductions. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 465–484, Santa Barbara, CA, USA, August 14–18, 2011. Springer Berlin Heidelberg, Germany.
- [MO15] Alexander May and Ilya Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 203–228, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.
- [MR14] Ben Morris and Phillip Rogaway. Sometimes-recurse shuffle - almost-random permutations in logarithmic expected time. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 311–326, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [MS04] Alfred Menezes and Nigel P. Smart. Security of signature schemes in a multi-user setting. *Designs, Codes and Cryptography*, 33(3):261–274, 2004.
- [Nat01] National Institute of Standards and Technology. Advanced Encryption Standard (AES). Federal Information Processing Standards Publication (FIPS) NIST FIPS 197-upd1, Department of Commerce, Washington, D.C., 2001. Updated May 9, 2023.
- [Nat15] National Institute of Standards and Technology. Secure Hash Standard (SHS). Federal Information Processing Standards Publication FIPS PUB 180-4, National Institute of Standards and Technology, Gaithersburg, MD, August 2015.
- [Nat24] National Institute of Standards and Technology. NIST proposes to standardize a wider variant of AES. Computer Security Resource Center, NIST, December 2024.
- [ND15] National Institute of Standards and Technology and Morris J. Dworkin. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Federal Information Processing Standards Publication 202, National Institute of Standards and Technology, Gaithersburg, MD, August 2015.
- [NIS22] NIST. Call for additional digital signature schemes for the post-quantum cryptography standardization process. Technical report, National Institute of Standards and Technology, 2022.
- [OTX24] Ying Ouyang, Deng Tang, and Yanhong Xu. Code-based zero-knowledge from VOLE-in-the-head and their applications: Simpler, faster, and smaller. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part V*, volume 15488 of *Lecture Notes in Computer Science*, pages 436–470, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.

- [Pat09] Jacques Patarin. The “coefficients H” technique (invited talk). In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *SAC 2008: 15th Annual International Workshop on Selected Areas in Cryptography*, volume 5381 of *Lecture Notes in Computer Science*, pages 328–345, Sackville, New Brunswick, Canada, August 14–15, 2009. Springer Berlin Heidelberg, Germany.
- [Pra62] Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Trans. Inf. Theory*, 8(5):5–9, 1962.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part I*, volume 13507 of *Lecture Notes in Computer Science*, pages 657–687, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [Saa07] Markku-Juhani Olavi Saarinen. Linearization attacks against syndrome based hashes. In K. Srinathan, C. Pandu Rangan, and Moti Yung, editors, *Progress in Cryptology - INDOCRYPT 2007: 8th International Conference in Cryptology in India*, volume 4859 of *Lecture Notes in Computer Science*, pages 1–9, Chennai, India, December 9–13, 2007. Springer Berlin Heidelberg, Germany.
- [SAB<sup>+</sup>20] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, and Damien Stehlé. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [Sho04] Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Report 2004/332, 2004.
- [Ste88] Jacques Stern. A method for finding codewords of small weight. In Gérard D. Cohen and Jacques Wolfmann, editors, *Coding Theory and Applications, 3rd International Colloquium, Toulon, France, November 2-4, 1988, Proceedings*, volume 388 of *Lecture Notes in Computer Science*, pages 106–113. Springer, 1988.
- [WWY<sup>+</sup>25] Tianrui Wang, Anyu Wang, Kang Yang, Hanlin Liu, Yu Yu, Jun Zhang, and Xiaoyun Wang. A hybrid algorithm for the regular syndrome decoding problem. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025, Part IV*, volume 16248 of *Lecture Notes in Computer Science*, pages 466–497, Melbourne, VIC, Australia, December 8–12, 2025. Springer, Singapore.
- [YSWW21] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2986–3001, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.
- [YWZ20] Kang Yang, Xiao Wang, and Jiang Zhang. More efficient MPC from improved triple generation and authenticated garbling. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 1627–1646, Virtual Event, USA, November 9–13, 2020. ACM Press.

- [YYW<sup>+</sup>25] Yu Yu, Kang Yang, Xiao Wang, Anyu Wang, Tianrui Wang, Hanlin Liu, Xinpeng Hao, and Juanru Li. Estimator of LPN problems over any finite fields and power-of-two rings for PCG and MPC applications. <https://lpnestimator.com>, 2025. Accessed: 2026-06-18.
- [ZFY23] Jiang Zhang, Dengguo Feng, and Di Yan. NEV: Faster and smaller NTRU encryption using vector decoding. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part VII*, volume 14444 of *Lecture Notes in Computer Science*, pages 157–189, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- [Zha12] Mark Zhandry. How to construct quantum random functions. In *53rd Annual Symposium on Foundations of Computer Science*, pages 679–687, New Brunswick, NJ, USA, October 20–23, 2012. IEEE Computer Society Press.
- [Zha19] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
- [Zha21] Mark Zhandry. Redeeming reset indistinguishability and applications to post-quantum security. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part I*, volume 13090 of *Lecture Notes in Computer Science*, pages 518–548, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.
- [Zha25] Mark Zhandry. A Note on Quantum-Secure PRPs. *Quantum*, 9:1696, 2025.
- [ZYF<sup>+</sup>20] Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: KEMs and signatures of smaller sizes. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 37–65, Edinburgh, UK, May 4–7, 2020. Springer, Cham, Switzerland.

## A Proving Knowledge of Secret Keys for NEV-KEM

NEV is a lattice-based key encapsulation mechanism [ZFY23]. In this section, we describe a method to prove the knowledge of secret key that corresponds to a given public key in NEV. We first recall the syntax and definition of the public/secret key pair in NEV, then demonstrate how to prove it using VOLEitH. This section serves as an example of the extensibility of VOLEitH when proving relations expressible as systems of low-degree multivariate polynomials.

### A.1 Definition of NEV

We recall only the part of NEV that is needed for proving knowledge of a secret key corresponding to a public key. The full NEV-KEM construction is obtained from the passively secure NEV-PKE by applying the Fujisaki-Okamoto transform with implicit rejection [FO99, ZFY23]. The same transform is also applied to the optimized variant NEV-PKE' to obtain NEV-KEM'. In particular, the KEM key generation first runs the corresponding PKE key generation algorithm to obtain a pair  $(pk', sk')$ , and then stores additional KEM material in the secret key. These

additional values do not change the algebraic public/secret key relation. Thus, for the purpose of proving the knowledge of secret key, it suffices to prove the public/secret key relation of the underlying IND-CPA encryption scheme.

Let

$$R = \mathbb{Z}[x]/(x^n + 1), \quad R_q = \mathbb{Z}_q[x]/(x^n + 1),$$

where  $n$  is a power of two and  $q$  is an odd prime. The plaintext space is  $\{0, 1\}^\ell$ . Let  $k$  be the largest integer such that  $k \mid n$  and  $n/k \geq \ell$ . NEV sets

$$v = 1 - x^{n/k} \in R_q^*, \quad v^{-1} = \frac{q+1}{2} (1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*.$$

The distribution  $B_\eta$  is the centered binomial distribution with parameter  $\eta$ , sampled coefficient-wise for ring elements. The distribution  $T_\sigma$  is the ternary distribution over  $\{-1, 0, 1\}$  with probabilities  $\Pr[1] = \Pr[-1] = \sigma$  and  $\Pr[0] = 1 - 2\sigma$ . In particular,  $B_1 = T_{1/4}$ . We describe the key generation algorithms of NEV-PKE and NEV-PKE' in [Algorithms 1](#) and [2](#).

---

**Algorithm 1** NEV-PKE.KeyGen

---

**Input:** Parameters  $(n, q, k, \ell, \chi_f, \chi_g)$  and  $v = 1 - x^{n/k} \in R_q^*$ .

**Output:** A public key  $pk = h$  and a secret key  $sk = f$ .

- 1: Sample  $f' \leftarrow \chi_f$  and  $g \leftarrow \chi_g$  coefficient-wise such that  $f := vf' + 1$  is invertible in  $R_q$ .
  - 2: Compute  $h := g/f = g \cdot f^{-1}$  in  $R_q$ .
  - 3: **return**  $(pk, sk) := (h, f) \in R_q \times R_q$ .
- 

---

**Algorithm 2** NEV-PKE'.KeyGen

---

**Input:** Parameters  $(n, q, k, \ell, \chi_f, \chi_g)$  and  $v = 1 - x^{n/k} \in R_q^*$ .

**Output:** A public key  $pk = h$  and a secret key  $sk = f$ .

- 1: Sample  $f' \leftarrow \chi_f$  and  $g \leftarrow \chi_g$  coefficient-wise such that  $f := f' + v^{-1}$  is invertible in  $R_q$ .
  - 2: Compute  $h := g/f = g \cdot f^{-1}$  in  $R_q$ .
  - 3: **return**  $(pk, sk) := (h, f) \in R_q \times R_q$ .
- 

We recall the selected parameter sets of [\[ZFY23\]](#) in [Table 22](#).

Table 22: Recommended parameters for the NEV parameter sets used in the public/secret key relation.

| Parameter set | $n$  | $q$ | $\ell$ | $k$ | $v$           | $(\chi_f, \chi_g)$ | $(\chi_r, \chi_e)$ | $( pk ,  ct )$ | Dec. failure | LWE est. $(sk, ct)$ |
|---------------|------|-----|--------|-----|---------------|--------------------|--------------------|----------------|--------------|---------------------|
| NEV-512       | 512  | 769 | 256    | 2   | $1 - x^{256}$ | $(B_1, B_1)$       | $(B_1, T_{1/6})$   | (615, 615)     | $2^{-138}$   | (145, 141)          |
| NEV'-512      | 512  | 769 | 256    | 2   | $1 - x^{256}$ | $(B_1, B_1)$       | $(B_1, B_1)$       | (615, 615)     | $2^{-200}$   | (145, 145)          |
| NEV-1024      | 1024 | 769 | 256    | 4   | $1 - x^{256}$ | $(B_1, B_1)$       | $(B_1, T_{1/6})$   | (1229, 1229)   | $2^{-152}$   | (292, 281)          |
| NEV'-1024     | 1024 | 769 | 256    | 4   | $1 - x^{256}$ | $(B_1, B_1)$       | $(B_1, B_1)$       | (1229, 1229)   | $2^{-200}$   | (292, 292)          |

## A.2 Expressing Public/Secret Key Relation as Polynomials

As VOLEitH utilizes QuickSilver as its underlying proof system and QuickSilver excels at proving systems of low-degree polynomials, we first express the public/secret key relation of NEV-PKE and NEV-PKE' in this form.

**NEV-PKE.** In [Algorithm 1](#), the secret-key multiplier is  $f = vf' + 1$ , and the public key satisfies  $h = g/f$  in  $R_q$ . Equivalently, a prover who knows the secret key can prove that there exist small polynomials  $f', g \in R$  such that

$$h(vf' + 1) = g \pmod{q}$$

in  $R_q$ . Since  $h$  and  $v$  are public, this is a system of linear equations over  $\mathbb{Z}_q$  in the coefficient variables of  $f'$  and  $g$ .

**NEV-PKE'.** In [Algorithm 2](#), the optimized variant instead sets  $f = f' + v^{-1}$ . Thus the corresponding public/secret key relation is the existence of small polynomials  $f', g \in R$  such that

$$h(f' + v^{-1}) = g \pmod{q}$$

in  $R_q$ . Again, because  $h$  and  $v^{-1}$  are public constants, this gives only linear constraints over  $\mathbb{Z}_q$  on the coefficients of  $f'$  and  $g$ .

Therefore, both variants require committing to the coefficients of the same two witness polynomials  $(f', g)$ . For all parameter sets in [Table 22](#), the key distributions are  $(\chi_f, \chi_g) = (B_1, B_1)$ , so every committed coefficient must lie in  $\{-1, 0, 1\}$ . Over  $\mathbb{Z}_q$ , this support condition for each coefficient  $z$  is the degree-3 equation

$$z(z-1)(z+1) = 0.$$

Consequently, the public/secret key relation for both NEV-PKE and NEV-PKE' is expressed as a combination of linear equations and degree-3 coefficient-range equations, which is well suited to the VOLEitH proof system.

### A.3 Estimating the Proof Size

Although in ReSolveD- $\alpha$ , we only utilize VOLEitH to prove degree-2 relations over  $\mathbb{F}_2$ , VOLEitH can be generalized to proving arbitrary low-degree polynomials over prime fields. Only the QuickSilver component needs to be tweaked. Here we provide estimations of proving the knowledge of secret keys for NEV, using the formalization described above.

Since we only need to commit to the coefficients of  $f', g$ , we have  $\ell_{\text{wit}} = 2n$ . By working in a larger prime field  $\mathbb{F}_q$ , we can choose a smaller field extension degree  $d_{\text{ext}}$  such that  $|\mathbb{F}_{q^{d_{\text{ext}}}}| \geq 2^\lambda$ . Moreover, since the polynomial system has maximum degree of 3, we need two random  $\mathbb{F}_{q^{d_{\text{ext}}}}$  IT-MACs as masks. Moreover, to minimize proof size, we can use a random vector over  $\mathbb{F}_{q^{d_{\text{ext}}}}$  as the linear universal hash function instead of [VOLEHash](#). Therefore, we can tweak [Equation \(6\)](#) to get the proof size estimate as shown in [Equation \(119\)](#).

$$\begin{aligned} \text{Size} = & \underbrace{2n \cdot \log(q)}_{\text{Fixing witness}} + \underbrace{(\tau-1)(2n+3d_{\text{ext}}) \cdot \log(q)}_{c_1, \dots, c_{\tau-1}} + \underbrace{(T_{\text{open}} + 2\tau)\lambda}_{\text{BAVC Opening}} \\ & + \underbrace{2d_{\text{ext}} \cdot \log(q)}_{\text{QuickSilver}} + \underbrace{d_{\text{ext}} \cdot \log(q)}_{\text{SoftSpoken}} + \underbrace{\lambda}_{\text{chall}_3} + \underbrace{120}_{\text{iv}} + \underbrace{32}_{\text{Counter}} \end{aligned} \quad (119)$$

Using [Equation \(119\)](#), we estimate the concrete proof sizes for the NEV parameter sets in [Table 23](#). The NEV-512 parameter sets correspond to 128-bit security, so we use the FAEST-128s VOLEitH parameters  $(\lambda, \tau, w_{\text{grind}}, T_{\text{open}}) = (128, 11, 7, 102)$ . The NEV-1024 parameter sets correspond to 256-bit security, so we use our ReSolveD- $\alpha$ -256s VOLEitH parameters from [Table 4](#), namely  $(\lambda, \tau, w_{\text{grind}}, T_{\text{open}}) = (256, 22, 12, 224)$ . In the estimate,  $\log(q)$  is instantiated as  $\lceil \log_2(q) \rceil$  bits per field element.

Table 23: Estimated VOLEitH proof sizes for proving knowledge of an NEV secret key.

| Parameter Set | $\lambda$ | $\tau$ | $w_{\text{grind}}$ | $T_{\text{open}}$ | $d_{\text{ext}}$ | Size (bits) | Size (bytes) |
|---------------|-----------|--------|--------------------|-------------------|------------------|-------------|--------------|
| NEV-512       | 128       | 11     | 7                  | 102               | 14               | 133,412     | 16,677       |
| NEV'-512      | 128       | 11     | 7                  | 102               | 14               | 133,412     | 16,677       |
| NEV-1024      | 256       | 22     | 12                 | 224               | 27               | 537,396     | 67,175       |
| NEV'-1024     | 256       | 22     | 12                 | 224               | 27               | 537,396     | 67,175       |